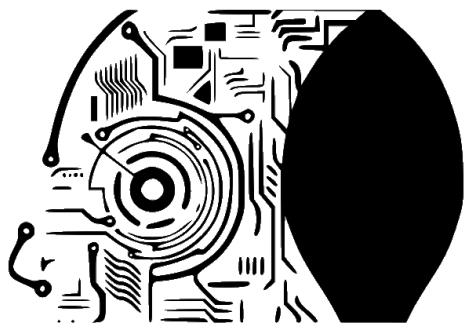




MODELOS DE I.A.

<https://martinezpenya.es/ModelosIA/>

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Modelos de IA (CE Inteligencia Artificial y Big Data)

IES Eduardo Primo Marques (Carlet)

David Martínez Peña

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Índice de contenidos

1. 🚀 Información importante	8
1.1 🚫 Denominación del curso	8
1.2 📄 Contenidos y temporalización	8
1.3 🎯 Resultados de aprendizaje, pesos y calificación	9
1.4 📝 Evaluación	10
1.5 📖 Legislación vigente	10
2. UD00	11
2.1 UD00 — Presentación del módulo y curso rápido de Docker	11
1. Introducción	11
2. Resultados de aprendizaje y objetivos	11
3. Presentación del curso y del módulo (RA7-i)	12
4. Cómo se evalúa el módulo	13
5. Entorno de trabajo del curso	13
6. Curso rápido de Docker: conceptos (RA7-i)	17
7. Curso rápido de Docker: uso básico (RA7-i)	21
8. El Dockerfile: construir imágenes reproducibles (RA7-i)	22
9. Docker Compose: varios servicios a la vez (RA7-i)	24
10. El contenedor de prácticas de IA (RA7-i)	25
11. Copias de seguridad de imágenes y contenedores (RA7-i)	28
12. Puntos clave de la unidad	28
13. Glosario	29
14. FAQ	29
15. Sesiones	30
A. Anexo · chuleta de comandos	30
B. Anexo · catálogo de contenedores para experimentar	31
16. Recursos	32
17. Evaluación	33
18. Recuperación	33
2.2 Diapositivas de la UD00	34
3. UD01	35
3.1 UD01 — Caracterización de sistemas de IA	35
1. Resultado de aprendizaje y criterios de evaluación	35
2. Objetivos de la unidad	35
3. Fundamentos de los sistemas inteligentes (RA1-a)	35
4. IA, machine learning, deep learning e IA generativa (RA1-c)	38

5. Campos de aplicación de la IA (RA1-b)	41
6. Nuevas formas de interacción y eficiencia operativa (RA1-d)	42
7. Beneficios, riesgos y ética (visión general)	45
8. Puntos clave de la unidad	45
9. Glosario	46
10. FAQ	46
11. Planificación sesión a sesión	47
12. Tabla final RA/CE	47
13. Recursos	48
14. Evaluación	48
15. Recuperación	48
3.2 Diapositivas de la UD01	49
3.3 UD01 — Actividades guiadas	50
Notebook 1 · Técnicas de IA en acción	50
4. UD02	51
4.1 UD02 — Modelos de IA y resolución de problemas	51
1. Introducción	51
2. Resultado de aprendizaje y criterios de evaluación	51
3. Objetivos de la unidad	52
4. Sistema de resolución de problemas (RA2-a)	52
5. Clasificación de modelos de IA (RA2-b)	54
6. Modelos de automatización de tareas (RA2-c)	55
7. Razonamiento impreciso: lógica difusa (RA2-d)	56
8. Sistemas basados en reglas (RA2-e)	58
9. Adecuación del modelo (RA2-f)	60
10. Caso de estudio: Robocode como sistema de resolución de problemas completo	60
11. Puntos clave de la unidad	61
12. Glosario	61
13. FAQ	62
14. Planificación sesión a sesión	63
15. Tabla final RA/CE	63
16. Recursos	63
17. Evaluación	64
18. Recuperación	64
4.2 Diapositivas de la UD02	65
4.3 Actividades Entregables	66
UD02 — Actividad entregable: Robocode Tank Royale	66
UD02 · Comparativa Java vs Python para Robocode	69

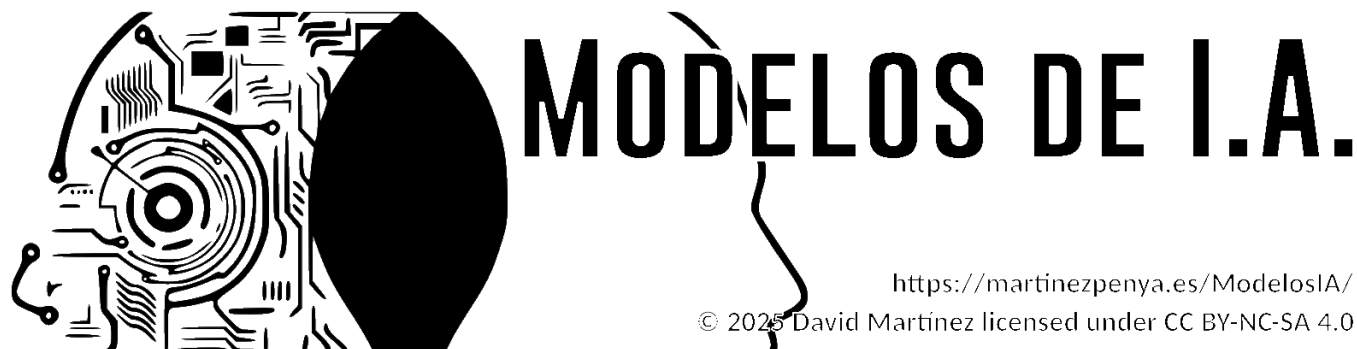
UD02 · Robocode Tank Royale en Java	73
UD02 · Robocode Tank Royale en Python	84
5. UD05	101
5.1 UD05 — Sistemas expertos y controladores inteligentes	101
1. Introducción	101
2. Resultado de aprendizaje y criterios de evaluación	101
3. Objetivos de la unidad	102
4. Arquitectura y dinámica de los sistemas expertos (RA5-a)	102
5. Estructuras elementales de representación (RA5-a/b)	105
6. Representar y simular comportamientos con experta (RA5-b)	106
7. Sistemas híbridos reglas/datos (RA5-b)	108
8. Sistemas de razonamiento impreciso: lógica difusa (RA5-b/d)	109
9. Variación de características y dinámica (RA5-c)	115
10. Estrategias de control de un sistema experto (RA5-d)	116
11. Controladores inteligentes (RA5-e)	117
12. Aplicaciones y tendencias (contenidos del RA5)	119
13. Puntos clave de la unidad	120
14. Glosario	120
15. FAQ	121
16. Sesiones	122
17. Recursos	122
18. Evaluación	123
19. Recuperación	123
5.2 Diapositivas de la UD05	124
5.3 UD05 · Actividades guiadas	125
Referencia: un sistema experto grande, resuelto	125
5.4 UD05 · Actividades entregables	126
Rúbricas	126
6. UD03	129
6.1 UD03 — Procesamiento del Lenguaje Natural	129
1. Introducción	129
2. Resultado de aprendizaje y criterios de evaluación	129
3. Objetivos de la unidad	130
4. Qué es el PLN (RA3-a)	130
5. El potencial (RA3-c)	132
6. La ambigüedad, el problema de fondo (RA3-c)	132
7. Desambiguación y etiquetado morfológico (RA3-a, RA3-c)	133
8. Las demás limitaciones (RA3-c)	134

9. Cuándo es factible aplicar PLN (RA3-d)	135
10. Quién hace un proyecto de PLN (RA3-b, RA3-e, RA3-f)	136
11. Las herramientas en Python (RA3-c, RA3-g)	137
12. Construir un sistema orientado a una tarea (RA3-g)	139
13. Puntos clave de la unidad	141
14. Glosario	142
15. FAQ	143
16. Sesiones	144
17. Recursos	144
18. Evaluación	145
19. Recuperación	145
6.2 Diapositivas de la UD03	146
6.3 UD03 · Actividades guiadas	147
N01 · Introducción al PLN	147
N02 · Clasificación de texto con PyTorch	147
N03 · Modelos de lenguaje: afinar DistilBERT	147
N04 · spaCy, primeros pasos	147
N05 · Ampliación: clasificador de géneros musicales	148
6.4 UD03 · Actividades entregables	149
EX1 · Representación de texto, por dos caminos	149
EX2 · Clasificador de preguntas	149
EX3 · NLTK y Python: etiquetado con <code>cess_esp</code>	149
EX4 · Análisis de sentimiento en reseñas de cine	150
EX5 · Asistente virtual por voz	150
Rúbricas	150
7. UD04	152
7.1 UD04 — Análisis de sistemas robotizados	152
1. Introducción	152
2. Resultado de aprendizaje y criterios de evaluación	152
3. Objetivos de la unidad	153
4. Métodos y aplicaciones de la robótica (RA4-a)	153
5. Qué clase de problema resuelve la robótica	156
6. Modelado y control cinemático (RA4-a)	157
7. Los problemas y sus soluciones (RA4-b)	159
8. Percepción robótica (RA4-b)	161
9. Incertidumbre y aprendizaje (RA4-b)	162
10. Técnicas de programación de robots (RA4-c)	163
11. Humanos y robots (RA4-c, RA4-d)	0

12. Diseño e implementación de sistemas robotizados (RA4-d)	0
13. Practicar sin robot: los dos simuladores de la unidad	0
14. Puntos clave de la unidad	0
15. Glosario	0
16. FAQ	0
17. Sesiones	0
18. Recursos	0
19. Evaluación	0
20. Recuperación	0
7.2 Diapositivas de la UD04	0
7.3 UD04 · Actividades guiadas	0
1 · Introducción a OpenCV	0
2 · Vehículos de Braitenberg	0
3 · Ejemplos de robots en AITK	0
7.4 UD04 · Actividades entregables	0
EX1 · Ejercicios de OpenCV	0
EX2 · Navegar con cámara, por reglas	0
EX3 · Navegar con cámara, con lógica difusa	0
EX4 · Generar los datos de entrenamiento	0
EX5 · Controlar el robot con una red neuronal	0
EX6 · Aprendizaje por refuerzo con NEAT	0
Rúbricas	0
8. UD06	0
8.1 UD06 — Aplicación de principios legales y éticos de la IA	0
1. Introducción	0
2. Resultado de aprendizaje y criterios de evaluación	0
3. Objetivos de la unidad	0
4. Riesgos y deontología de la IA (RA6-a)	0
5. Privacidad y protección de datos (RA6-b)	0
6. Cumplimiento estricto de la legalidad (RA6-c)	0
7. Security by design (RA6-d)	0
8. Privacy by design (RA6-e)	0
9. Sesgos de género y algorítmicos (RA6-f)	0
10. Puntos clave de la unidad	0
11. Glosario	0
12. FAQ	0
13. Sesiones	0
14. Recursos	0

15. Evaluación	0
16. Recuperación	0
8.2 Diapositivas de la UD06	0
8.3 Debates	0
UD06 · Debate 1 — Límites éticos de la Inteligencia Artificial	0
UD06 · Debate 2 — El algoritmo contra el crimen	0
8.4 UD06 · Actividades guiadas	0
Antes de cada debate	0
Para ampliar	0
Actualidad: veinticuatro noticias para el debate	0
8.5 UD06 · Actividades entregables	0
Resumen de entregas	0
Rúbrica de los debates	0
Tertulia de ciencia-ficción	0
9. 🖐 Sobre mí	0
9.1 David Martínez Peña	0
9.2 📧 Contacto	0
10. Fuentes de información	0
10.1 Generales del curso	0
10.2 UD00 — Presentación y Docker	0
10.3 UD01 — Caracterización de sistemas de IA	0
10.4 UD02 — Modelos de IA y resolución de problemas	0
10.5 UD03 — Procesamiento del lenguaje natural	0
10.6 UD04 — Robótica	0
10.7 UD05 — Sistemas expertos y lógica difusa	0
10.8 UD06 — Riesgos, ética y legalidad de la IA	0

1. 🚀 Información importante



1.1 🏠 Denominación del curso

📖 **Curso de especialización de Inteligencia Artificial y Big Data**

🏠 Modelos de Inteligencia Artificial (MIA) · Código **5071** · **90 h** · 4 ECTS

i Curso 2026-2027

La docencia empieza el **1 de octubre de 2026** y el módulo termina el **28 de mayo de 2027**; junio se reserva a la **convocatoria ordinaria**. Las clases son de **lunes a jueves**: los viernes no hay clase y se aprovechan para las tareas y los talleres en casa.

Vacaciones: Navidad del 22 de diciembre al 6 de enero · Pascua del 25 de marzo al 5 de abril.

Festivos que caen en día de clase: 12 de octubre, 7 y 8 de diciembre, y **Fallas, 17 y 18 de marzo**. El 9 de octubre (Día de la Comunitat Valenciana) y el 19 de marzo caen en viernes, así que no afectan.

Entre Fallas y Pascua, **marzo es el mes más interrumpido del curso**: tenlo en cuenta para planificar entregas.

1.2 📋 Contenidos y temporalización

Las unidades se imparten en este orden. El identificador **UDxx** va ligado a su resultado de aprendizaje (UD05 es siempre RA5), no al orden en que se imparte.

UD	Título	RA	Horas	Semanas	Fechas
UD00	Presentación y curso rápido de Docker	—	6	1-2	1 oct - 8 oct
UD01	Caracterización de sistemas de IA	RA1	12	3-6	12 oct - 6 nov
UD02	Modelos de IA y resolución de problemas	RA2	12	7-10	9 nov - 3 dic
UD05	Sistemas expertos	RA5	12	11-15	7 dic - 14 ene
UD03	Procesamiento del Lenguaje Natural	RA3	12	16-19	18 ene - 11 feb
UD04		RA4	12	20-23	15 feb - 11 mar

UD	Título	RA	Horas	Semanas	Fechas
	Análisis de sistemas robotizados				
UD06	Principios legales y éticos de la IA	RA6	6	24-28	15 mar - 23 abr
UD07	Proyecto integrador (común al curso)	RA7	15	29-33	26 abr - 28 may
Cierre	Presentaciones RA7 y prueba ordinaria (solo RA no superados)	—	3	34-35	1 jun - 18 jun
Total			90	30 nominales 33 con clase	

**Por qué este orden**

La UD05 (sistemas expertos) se imparte justo después de la UD02 porque se construye sobre las reglas y la lógica difusa que se ven allí. Y la UD06 (ética) ocupa el tramo de Fallas y Pascua, el más fragmentado del curso, porque es la unidad con menos carga de laboratorio.

El **contenido del módulo acaba a finales de abril**. Después viene el **proyecto intermodular (RA7)**, común a todo el curso de especialización: durante unas cinco semanas tu equipo trabaja el proyecto en las horas de todos los módulos.

1.3 Resultados de aprendizaje, pesos y calificación

RA	CE	Descripción	Nota mínima	Peso
RA1	4	Caracteriza sistemas de Inteligencia Artificial relacionándolos con la mejora de la eficiencia operativa de las organizaciones y empresas.	5	13,33 %
RA2	6	Utiliza modelos de sistemas de Inteligencia Artificial implementando sistemas de resolución de problemas.	5	13,33 %
RA3	7	Relaciona el procesamiento de lenguaje natural con sus aplicaciones determinando su potencial e identificando sus limitaciones.	5	13,33 %
RA4	4	Analiza sistemas robotizados, evaluando opciones de diseño e implementación.	5	13,33 %
RA5	5	Aplica sistemas expertos evaluando la influencia de los controladores inteligentes en el comportamiento del sistema.	5	13,33 %
RA6	6	Aplica principios legales y éticos al desarrollo de la Inteligencia Artificial integrándolos como parte del proceso.	5	13,33 %
RA7	9	Desarrolla un proyecto integrador que combine técnicas de IA y análisis	5	20 %

RA	CE	Descripción	Nota mínima	Peso
		de datos masivos para resolver un problema real o simulado, gestionando todo el ciclo de vida del proyecto.		
				100 %

Cada RA se califica **1 a 10, sin decimales** (Orden 8/2025, art. 5.1): **40 %** tareas, talleres y ejercicios + **60 %** prueba escrita. Para superar el módulo hace falta **5 o más en cada RA**.

1.4 Evaluación

-  La evaluación del módulo se realiza sobre los **Resultados de Aprendizaje (RA)** del currículo. Cada RA tiene asociados sus **criterios de evaluación (CE)**, que son los que determinan el grado de adquisición de las competencias del módulo.
-  La nota final del módulo sale de la ponderación de los RA. Cada RA se evalúa de forma independiente, con **calificación de 1 a 10, sin decimales** (Orden 8/2025, art. 5.1).
-  Cada RA tiene **una prueba escrita en Moodle** al cerrar su unidad, con preguntas de test y de desarrollo sobre el contenido de la unidad.
-  Hay que obtener al menos un **5 en cada RA** para aprobar el módulo.
-  Si un RA queda por debajo de 5, se recupera con las actividades y pruebas diseñadas para **ese** resultado de aprendizaje. La convocatoria ordinaria de junio se dirige **solo a los RA no superados**: no hay prueba global del módulo.
-  La **UD00** (presentación y Docker) **no se califica**: el entorno de trabajo es un requisito. Sus tres entregables —los dos talleres y el notebook— se marcan como **hecho / no hecho**, sin nota pero **obligatorios**: son la prueba de que tienes el entorno operativo, y sin ellos no se pueden hacer las prácticas de la UD02.

Importante

-  Aprobar las evaluaciones parciales **no garantiza** aprobar el módulo.
-  Puedes aprobar las dos evaluaciones con buena nota, tener **un solo RA suspendido** y suspender el módulo.

1.5 Legislación vigente

-  [RD 279/2021, de 20 de abril](#) — establece el curso de especialización y fija los **RA y CE** del módulo.
-  [RD 497/2024, de 21 de mayo](#) — modifica determinados reales decretos de cursos de especialización.
-  [Decreto 95/2026, de 9 de junio \(DOGV\)](#) — currículo en la Comunitat Valenciana: fija el módulo en **90 h**.
-  [Horario y organización del curso \(GVA\)](#)

 23 de agosto de 2026

2. UD00

2.1 UD00 — Presentación del módulo y curso rápido de Docker

Unidad 0 · 6 h · semanas 1-2 (1 al 8 de octubre)

Unidad transversal de presentación y arranque del curso. **No tiene prueba escrita:** el entorno de trabajo es **requisito** para el resto del módulo, y el entregable del Taller 1 se marca como **hecho / no hecho**.

1. Introducción

Esta primera unidad tiene un doble objetivo. Por una parte, **situar el curso:** qué es el curso de especialización en Inteligencia Artificial y Big Data, qué se va a aprender en el módulo 5071 *Modelos de Inteligencia Artificial* y cómo se va a evaluar. Por otra, **dejar listo el entorno de trabajo:** sin herramientas fiables y reproducibles, cualquier práctica de IA se convierte en una lucha contra configuraciones que no funcionan. Por eso el núcleo técnico de la unidad es un **curso rápido de Docker**, la tecnología que nos permitirá ejecutar el mismo entorno de Python y Jupyter en cualquier equipo.

La idea que sostiene la unidad

La IA se hace con **entornos reproducibles:** si cada alumno tiene una configuración distinta, los resultados no son comparables. Docker resuelve ese problema empaquetando el entorno completo en un contenedor.

2. Resultados de aprendizaje y objetivos

La UD00 es **transversal** (no desarrolla un RA propio del módulo): sirve de nivelador tecnológico y de presentación. Las competencias que trabaja se asocian a la **ética y el cuidado del entorno de trabajo** (RA7-i) y a los **hábitos de trabajo** del curso.

Objetivo	Descripción
O1	Describir el curso de especialización IA y Big Data y el papel del módulo 5071 en él.
O2	Explicar cómo se evalúa el módulo (criterios, calificación, recuperación).
O3	Identificar el entorno de trabajo del curso: Aules, web, Python, Jupyter y Docker.
O4	Diferenciar contenedor, imagen, volumen y red, y explicar qué ventajas aporta Docker frente a las máquinas virtuales.
O5	Crear, ejecutar, detener y eliminar contenedores con <code>docker run</code> .
O6	Escribir un <code>Dockerfile</code> sencillo y levantarlo con <code>docker build</code> .
O7	Orquestar varios servicios con Docker Compose y conservar datos con volúmenes.
O8	Levantar un contenedor de prácticas con Jupyter y las bibliotecas del curso.

¿Por qué no tiene RA propio?

El currículo del RD 279/2021 desarrolla 6 resultados de aprendizaje para el módulo 5071 (RA1-RA6). La UD00 es una **unidad de presentación y puesta en marcha** que prepara el terreno para esas unidades: sin ella, los RA1-RA6 perderían horas en configurar entornos.

3. Presentación del curso y del módulo (RA7-i)

3.1 El curso de especialización en IA y Big Data

El **curso de especialización en Inteligencia Artificial y Big Data** es un título de **Grado Superior** (familia Informática y Comunicaciones) de **600 horas y 36 ECTS** establecido por el RD 279/2021. Su competencia general es *programar y aplicar sistemas inteligentes que optimizan la gestión de la información y la explotación de datos masivos, garantizando la seguridad y la ética*.

Se organiza en **cinco módulos**:

Código	Módulo	Horas en CV
5071	Modelos de Inteligencia Artificial	90
5072	Sistemas de aprendizaje automático	90
5073	Programación de Inteligencia Artificial	210
5074	Sistemas de Big Data	90
5075	Big Data aplicado	120

El acceso se realiza desde titulaciones de grado superior de la familia de informática y otras relacionadas (ASIR, DAM, DAW, telecomunicaciones, mecatrónica y robótica industrial).



Una frase para recordar

Este curso enseña a **construir sistemas inteligentes** (5071, 5072, 5073) y a **tratar datos a gran escala** (5074, 5075). El módulo 5071 es la base conceptual de los sistemas de IA.

3.2 El módulo 5071 *Modelos de Inteligencia Artificial*

El módulo 5071 se imparte a lo largo de todo el curso (**90 horas, 3 horas semanales**, 4 ECTS) y desarrolla **6 resultados de aprendizaje** (RA1-RA6), más el **RA7 de proyecto integrador transversal**:

RA	Qué aprenderás
RA1	Caracterizar sistemas de IA y relacionarlos con la eficiencia operativa
RA2	Utilizar modelos de IA para implementar sistemas de resolución de problemas
RA3	Relacionar el procesamiento del lenguaje natural (PLN) con sus aplicaciones
RA4	Analizar sistemas robotizados y evaluar su diseño e implementación
RA5	Aplicar sistemas expertos y valorar los controladores inteligentes
RA6	Aplicar principios legales y éticos (privacidad, sesgos, normativa)
RA7	Proyecto integrador transversal de todo el curso (definición → datos → modelos → Big Data → comunicación)

3.3 El proyecto integrador (RA7)

El **RA7** es un **proyecto común a todo el curso de especialización**: se desarrolla en **equipos** durante el bloque final (semanas 25-29) y en él **confluyen todos los módulos** del título. El alumnado define un problema, obtiene datos, aplica modelos de IA, integra soluciones de Big Data, comunica resultados y evalúa críticamente el trabajo realizado.

Es transversal por dos razones: 1. Lo **imparten todos los docentes** del curso, no solo el del 5071. 2. La **nota se consensúa** entre el equipo docente, valorando la contribución de cada miembro.

**Implicación para este módulo**

El contenido del 5071 (UD00-UD06) **finaliza a finales de abril** porque las últimas semanas del curso se dedican por completo al proyecto integrador. Desde el primer día, el proyecto te permitirá aplicar lo aprendido en este módulo.

4. Cómo se evalúa el módulo

La evaluación se rige por la **Orden 8/2025** de la Comunitat Valenciana (modificada por la **Orden 5/2026**). Lo que debes saber desde el primer día:

Los **pesos de cada RA, la nota mínima y el reparto 40/60** están en la [página de información importante](#), que es la referencia única: si algún día cambian, cambian allí. Aquí van las reglas que conviene tener claras desde el primer día y que no dependen de los pesos:

Aspecto	Regla
Calificación por RA	De 1 a 10, sin decimales (art. 5.1)
Evaluación continua	Se pierde al superar el 15 % de inasistencia (asistencia $\geq$ 85 %)
Evaluaciones parciales	Dos parciales informativos (fin del 1.er y 2.º trimestre)
Recuperación	Programa individual por cada RA no superado ; en junio, solo los RA pendientes
Esta unidad (UD00)	No se califica : es requisito. Sus tres entregables (los dos talleres y el notebook) se marcan hecho / no hecho

**Asistencia**

Si superas el **15 % de faltas** perderás la evaluación continua del módulo (art. 7.3 Orden 8/2025). Es un criterio objetivo, no una decisión del profesor.

**Evaluación inicial**

Durante el primer mes realizaremos una **evaluación inicial o diagnóstica** (obligatoria antes de finalizar el segundo mes lectivo). No cuenta para la nota: sirve para conocer tus competencias previas y adaptar el ritmo del curso.

5. Entorno de trabajo del curso

Herramienta	Para qué la usaremos
Aules (Moodle de la GVA)	Entrega de tareas, pruebas, avisos y calificaciones
Web del curso (mkdocs)	Teoría, ejercicios, talleres y notebooks en el navegador
Python + Jupyter	Notebooks de prácticas con las bibliotecas de IA
Docker	Entornos reproducibles y el contenedor de prácticas
Git (opcional)	Control de versiones de tus proyectos

**Sobre Aules**

Aules es la **plataforma oficial de e-learning de la Conselleria de Educación**, basada en **Moodle**. Accederás con tu usuario (NIA) y desde ahí se gestionan las tareas del módulo.

El stack de IA del curso

Las prácticas se basan en **Python** con estas bibliotecas:

numpy, pandas, matplotlib, scikit-learn, scikit-fuzzy, nltk, spaCy, torch y experta.

Este stack se ejecutará en un **contenedor de prácticas de IA** que montaremos en el taller 2 de esta unidad.

5.2 Instalar Docker

Antes de nada, Docker tiene que funcionar en tu máquina. Estos pasos están probados en el aula.

INSTALACIÓN EN UBUNTU

<https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-on-ubuntu-20-04-es>

Requisitos previos:

- **apt-transport-https:** permite que el administrador de paquetes transfiera datos a través de https
- **ca-certificates:** permite que el navegador web y el sistema verifiquen los certificados de seguridad
- **curl:** transfiere datos (similar a wget)
- **software-properties-common:** agrega scripts para administrar el software

```
1 sudo apt-get install curl apt-transport-https ca-certificates software-properties-common
```

Agregamos repositorio

```
1 # Primero clave GPG
2 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
3
4 sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
5
6 sudo apt update
7
8 sudo apt install docker-ce
9
10 sudo systemctl status docker
```

Por defecto, el comando docker solo puede ser ejecutado por el usuario root o un usuario del grupo docker, que se crea automáticamente durante el proceso de instalación de Docker.

Para evitar escribir sudo al ejecutar el comando docker, agregue su nombre de usuario al grupo docker:

```
1 sudo usermod -aG docker ${USER}
2
3 # Cerramos y abrimos sesión de nuevo o ejecutamos
4 su - ${USER}
5
6 # Confirmamos los grupos de nuestro usuario
7 id -nG
```

DOCKER DESKTOP

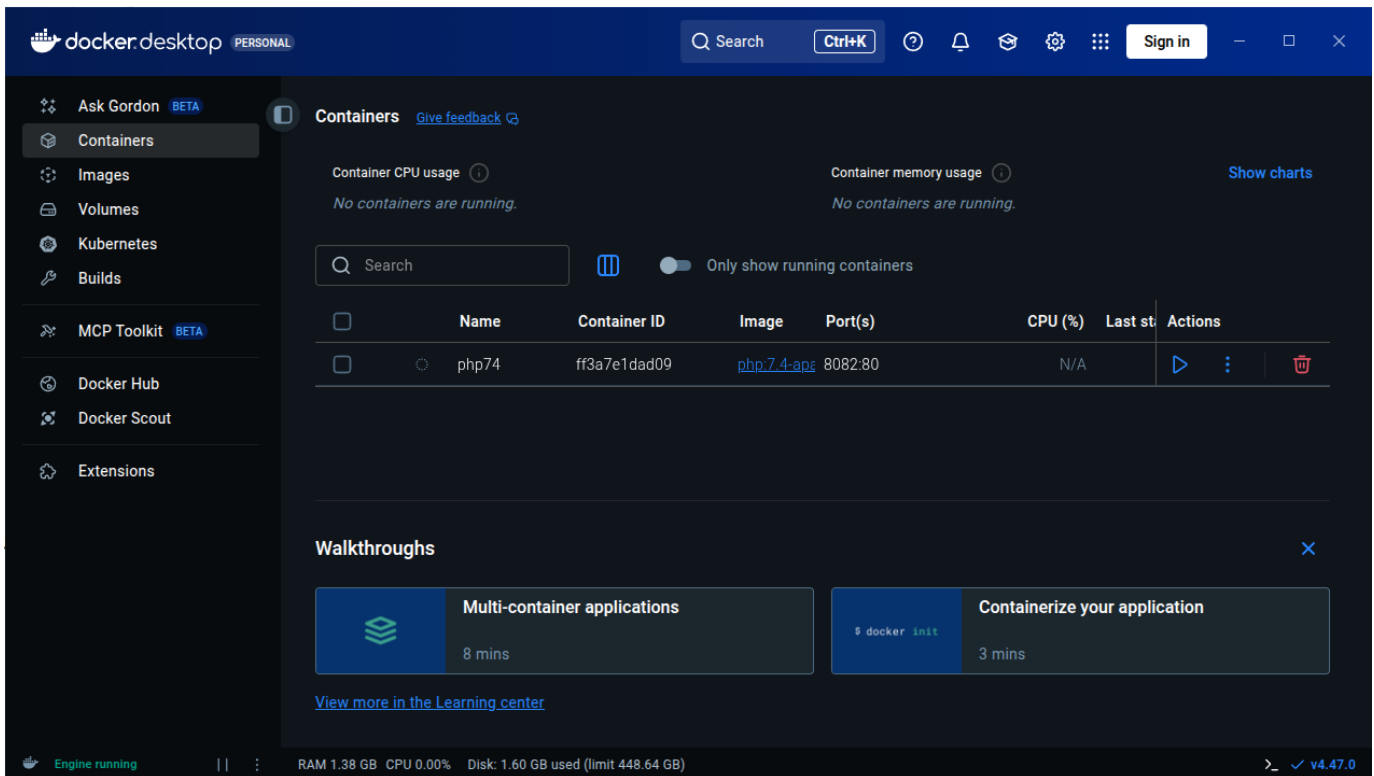
¿Qué es Docker Desktop?



Es la aplicación oficial de Docker que te da una **interfaz gráfica (GUI)** para manejar contenedores, además de la línea de comandos.

¿Para qué sirve?

- **Gestión visual:** Ver contenedores, imágenes y volúmenes de forma gráfica
- **Configuración fácil:** Ajustar recursos (CPU, RAM) con sliders
- **Monitorización:** Ver en tiempo tiempo real qué está pasando



Compatibilidad por Sistema Operativo

Windows

- **Windows 10/11** 64-bit (versiones Home, Pro, Enterprise, Education)
- **Requisitos importantes:**
 - Habilitar **WSL 2** (Windows Subsystem for Linux)
 - Virtualización activada en BIOS/UEFI
- **Windows Home** necesita WSL 2, **Pro/Enterprise** puede usar Hyper-V

macOS

- **macOS 12 Monterey** o superior
- **Tipos de chip:**
 - **Apple Silicon** (M1, M2, M3, etc.)
 - **Intel** con procesador de 2010 o más nuevo
- Necesita **macOS actualizado**

Linux (versión nativa)

- **Distribuciones compatibles:**
 - Ubuntu 20.04 LTS o superior
 - Debian 11 o superior
 - Fedora 36 o superior
 - Arch Linux (y derivados)
- **Requisitos:** kernel 5.10+, systemd, 64-bit

Guía Rápida de Instalación

Windows:

1. Descarga desde docker.com/products/docker-desktop
2. Ejecuta el instalador `.exe`
3. Sigue el asistente (marca "Use WSL 2" si tienes Windows Home)
4. Reinicia cuando termine

5. ¡Listo! Docker se inicia automáticamente

macOS:

1. Descarga desde la web oficial
2. Arrastra Docker.app a la carpeta Applications
3. Ejecuta desde Launchpad
4. Autoriza con contraseña del sistema
5. Espera a que configure todo (puede tardar unos minutos)

Linux (Ubuntu/Debian ejemplo):

```
1 # Paso 1: Descargar el .deb oficial (siempre la última versión)
2 wget https://desktop.docker.com/linux/main/amd64/docker-desktop-amd64.deb
3
4 # Paso 2: Instalar
5 sudo apt install ./docker-desktop-*.deb
6
7 # Paso 3: Iniciar
8 systemctl --user start docker-desktop
```

⚠ Problemas con Ubuntu 24.04 ▾

Si tienes problemas con Ubuntu 24.04, yo me he encontrado dos:

1. Problema de permisos

```
1 ...
2 S'estan processant els activadors per a desktop-file-utils (0.27-2build1)...
3 N: La baixada es duu a terme fora de l'entorn segur com a root ja que el fitxer «/home/ubuntu/docker-desktop-4.27.2-amd64.deb» no és accessible per l'usuari «_apt». - pkgAcquire::Run (13: Permission denied)
```

Lo he resuelto cambiando los permisos del deb:

```
1 # Change ownership of the file to make it accessible
2 sudo chown _apt:root /home/ubuntu/docker-desktop-4.27.2-amd64.deb
3 # Or alternatively, change permissions to make it readable
4 sudo chmod 644 /home/ubuntu/docker-desktop-4.27.2-amd64.deb
```

2. Lanzas Docker-desktop, aparece el icono, pero desaparece y la aplicación no inicia: Parece un problema con un cambio en Ubuntu 24.04 que he resuelto con la información de este [post](#):

```
1 sudo systemctl -w kernel.apparmor_restrict_unprivileged_userns=0
2 systemctl --user restart docker-desktop
```

En algunos casos parece que la solución anterior solo sirve hasta que reinicias, si es así, prueba esto también:

Crea un nuevo fichero:

```
1 sudo nano /etc/apparmor.d/opt.docker-desktop.bin.com.docker.backend
```

Escribe dentro el siguiente contenido:

```
1 abi <abi/4.0>,
2
3 include <tunables/global>
4
5 /opt/docker-desktop/bin/com.docker.backend flags=(default_allow) {
6   userns,
7
8   # Site-specific additions and overrides. See local/README for details.
9   include if exists <local/opt.docker-desktop.bin.com.docker.backend>
10 }
```

Reinicia el servicio `apparmor.service`:

```
1 sudo systemctl restart apparmor.service
```


Ventajas docker desktop (GUI) vs Línea de Comandos (CLI)

✓ Ventajas de Docker Desktop:

- **Más fácil para empezar** - Ideal para principiantes
- **Todo integrado** - No necesitas instalar nada más
- **Debugging visual** - Ves los logs y estados de un vistazo
- **Gestión de recursos** - Controlas CPU/RAM fácilmente

✗ Desventajas:

- **Más pesado** - Consume más recursos de tu PC
- **Menos flexible** - Algunas opciones avanzadas solo por comandos
- **Dependes de la GUI** - Si se cierra la app, pierdes la interfaz

🎯 Conclusión:

- **Empezad con Docker Desktop** para aprender sin frustraciones
- **Aprended también los comandos básicos** para ser más versátiles
- Usad **ambos**: la GUI para lo cotidiano y la terminal para lo avanzado

⚠ Comprueba la instalación antes de seguir

`docker run hello-world` debe terminar sin errores. Si te responde `permission denied while trying to connect to the Docker daemon socket`, te falta el paso de añadir tu usuario al grupo `docker` y **cerrar y abrir sesión**.

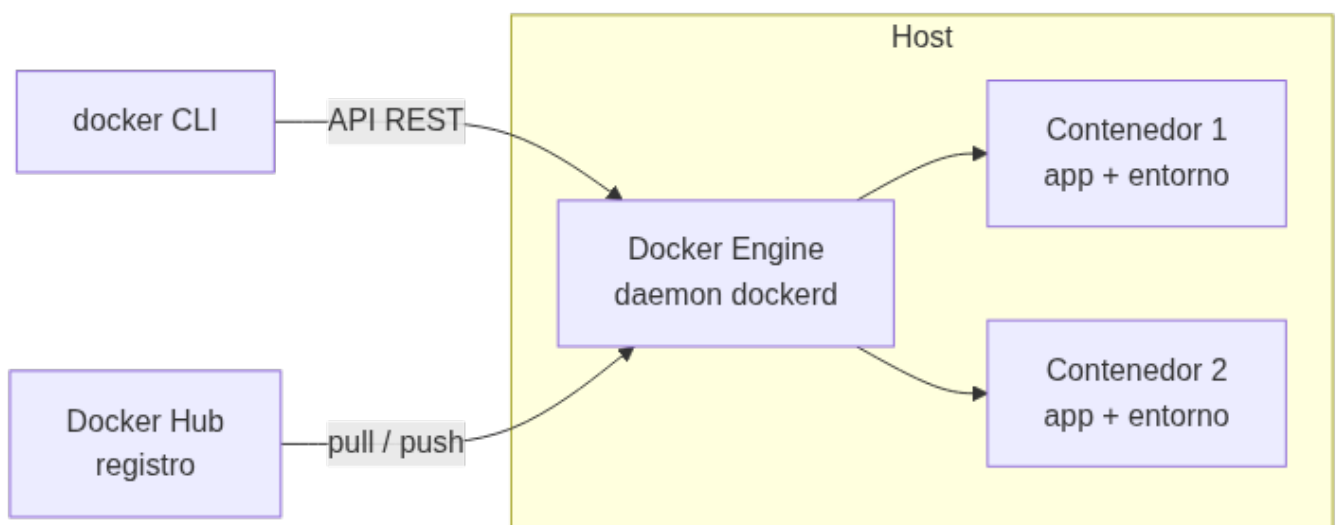
6. Curso rápido de Docker: conceptos (RA7-i)

6.1 El problema: entornos que no se reproducen

¿Cuántas veces funciona "en mi ordenador" y no en el del compañero? La causa es que los entornos difieren: versiones de Python, bibliotecas, sistema operativo, configuraciones. **Docker resuelve este problema** empaquetando la aplicación y todo lo que necesita en un **contenedor**.

✍ Definición

Docker es una plataforma abierta para desarrollar, distribuir y ejecutar aplicaciones. Permite separar la aplicación de la infraestructura mediante contenedores: entornos aislados y ligeros que incluyen todo lo necesario para ejecutar la aplicación.



6.2 Contenedores frente a máquinas virtuales

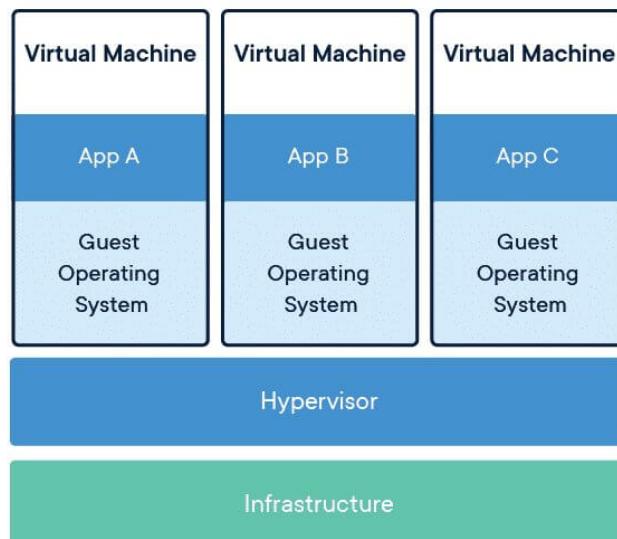
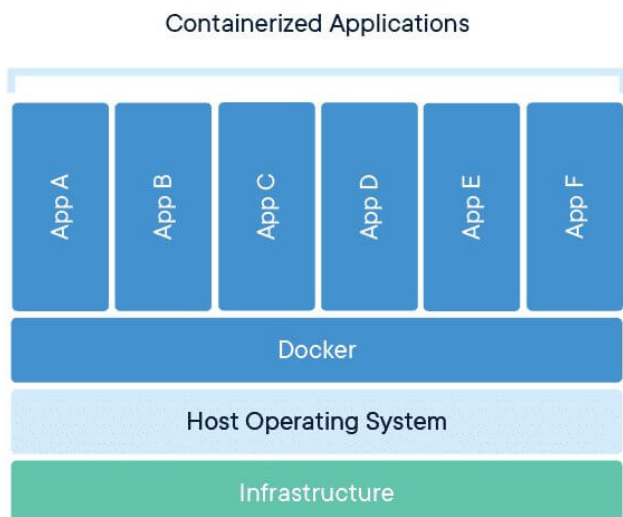
Característica	Máquina virtual	Contenedor
Aislamiento	Por hardware (hipervisor)	Por procesos (namespaces del kernel)
Sistema operativo	SO completo con su propio kernel	Comparte el kernel del host
Arranque	Minutos	Segundos
Tamaño	Gigabytes	Megabytes
Carga en un servidor	Decenas	Cientos/miles
Reproducibilidad	Media	Alta (imagen inmutable + capas)

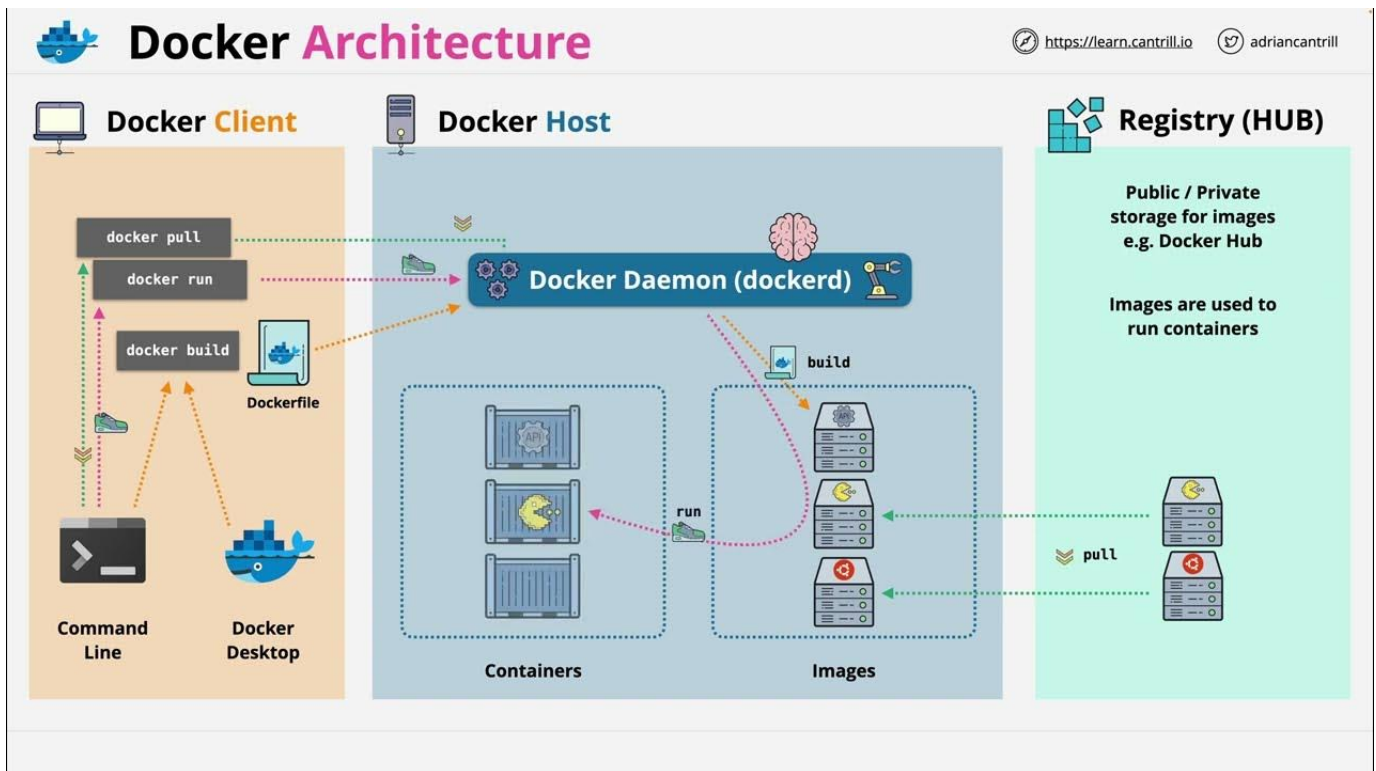


¿Y entonces las VMs son inútiles?

No. Las VMs siguen siendo necesarias cuando se requiere **aislamiento total de kernel** o se ejecutan sistemas operativos distintos. La práctica habitual es combinarlas: una VM con Docker encima. En este curso trabajaremos con contenedores porque son **ligeros y reproducibles**.

LA ARQUITECTURA DE DOCKER EN IMÁGENES





Imágenes

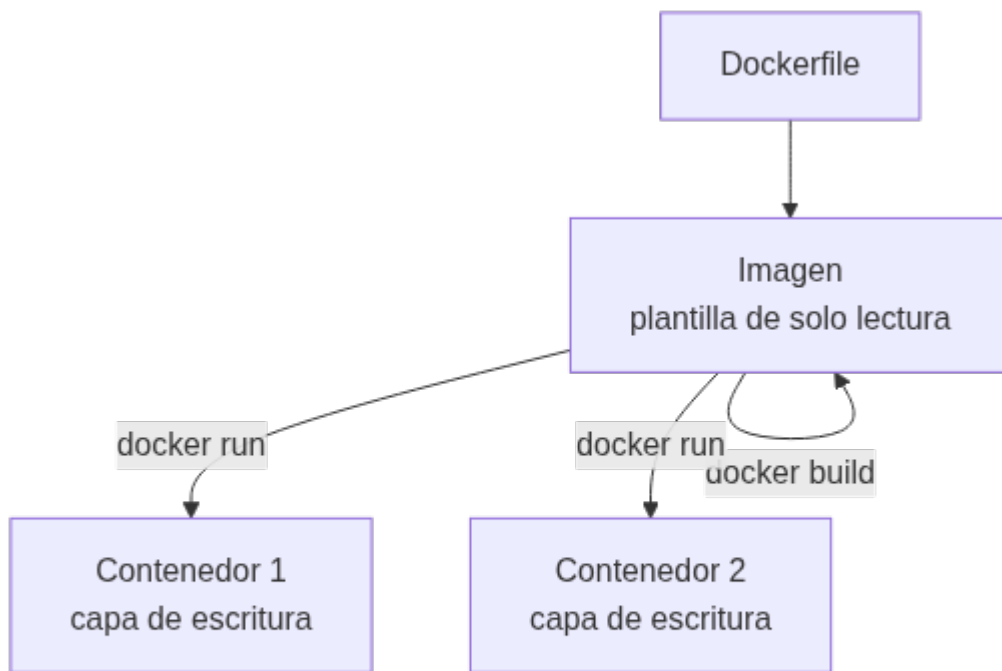
- ▶ Plantilla de solo lectura
- ▶ Sistema de ficheros y parámetros listos para ejecutar
- ▶ Basadas en sistemas operativos Linux

Contenedores

- ▶ Instancia de una imagen
- ▶ Es un directorio dentro del sistema, similar a los "jail root"
- ▶ Pueden ser ejecutados, reiniciados, parados...

6.3 Imágenes y contenedores

- **Imagen:** plantilla **de solo lectura** (un paquete inmutable con el SO base, la aplicación y las dependencias). Se construye por **capas**.
- **Contenedor:** **instancia ejecutable** creada a partir de una imagen. Añade una capa de escritura por encima de la imagen.



La regla de las capas

Cada instrucción del `Dockerfile` crea una **capa**. Al reconstruir una imagen, solo se regeneran las capas que han cambiado: por eso **el orden de las instrucciones importa** para aprovechar la caché.

6.4 El registro y el primer contenedor

Docker Hub es el registro público por defecto: se usa `pull` para **descargar** imágenes y `push` para **subirlas**. Nuestro primer comando será:

```
1 docker run hello-world
```

La primera vez, Docker descarga la imagen `hello-world`, crea un contenedor a partir de ella, ejecuta un mensaje de confirmación y lo detiene. Ese es exactamente el ciclo completo de Docker en una sola línea.



Comprueba tu instalación

Antes de seguir, verifica que Docker está instalado y el demonio en marcha:

```
1 docker --version
2 docker run hello-world
```

6.5 Mismo código, dos entornos: el caso de `experta`

El argumento definitivo a favor de los contenedores se ve mejor con una biblioteca que usaremos de verdad en este módulo: `experta`, el motor de reglas de la UD02 y la UD05.

Dato	Valor
Última versión	1.9.4 , del 16/11/2019

Dato	Valor
Python que declara soportar	3.5 a 3.8
Mantenimiento	abandonado ; fallo abierto como <i>issue</i> #34 desde julio de 2023
Causa	su dependencia <code>frozendict==1.2</code> usa <code>collections.Mapping</code> , que desapareció en Python 3.10

Monta dos contenedores con **el mismo código** y compara:

```

1 # Contenedor A: Python 3.9, experta funciona tal cual
2 docker run --rm python:3.9-slim bash -c \
3   "pip install -q experta && python -c 'from experta import KnowledgeEngine; print(1)'"
4
5 # Contenedor B: Python 3.12, el mismo código revienta
6 docker run --rm python:3.12-slim bash -c \
7   "pip install -q experta && python -c 'from experta import KnowledgeEngine; print(1)'"

```

El contenedor B falla con `AttributeError: module 'collections' has no attribute 'Mapping'`, y se arregla con tres líneas **antes** del `import`, las mismas que verás en los notebooks de la UD02:

```

1 import collections, collections.abc
2 for nombre in ("Mapping", "Iterable", "MutableMapping"):
3     if not hasattr(collections, nombre):
4         setattr(collections, nombre, getattr(collections.abc, nombre))
5
6 from experta import KnowledgeEngine # ahora sí

```



Tres lecciones en un solo ejemplo

1. **El entorno importa:** mismo código, misma biblioteca, dos resultados. Sin fijar la versión de Python, «en mi máquina funciona» no significa nada.
2. **Las dependencias se abandonan:** `experta` no publica nada desde 2019. Elegir una biblioteca es apostar también por quien la mantiene.
3. **Se puede convivir con ello:** un parche documentado y versionado, y sigues adelante. Lo que no vale es descubrirlo el día de la entrega.

7. Curso rápido de Docker: uso básico (RA7-i)

7.1 El ciclo de vida de un contenedor

Comando	Qué hace
<code>docker run</code>	Crea y arranca un contenedor nuevo (descarga la imagen si falta)
<code>docker ps</code>	Lista los contenedores en marcha
<code>docker ps -a</code>	Lista también los detenidos
<code>docker stop</code>	Detiene un contenedor (envía SIGTERM)
<code>docker start</code>	Reactiva un contenedor parado conservando sus cambios
<code>docker rm</code>	Elimina un contenedor (parado)
<code>docker rmi</code>	Elimina una imagen



`docker run` no borra el contenedor

Cuando el proceso principal termina, el contenedor **se detiene pero no se elimina**. Cada `docker run` crea un contenedor nuevo; por eso conviene usar `--rm` en los contenedores desechables y limpiar con `docker rm` los que ya no se usen.

7.2 Flags clave de `docker run`

Flag	Significado	Ejemplo
<code>-it</code>	Interactivo + terminal (para entrar a un shell)	<code>docker run -it ubuntu bash</code>
<code>-d</code>	En segundo plano (detached)	<code>docker run -d -p 8080:80 nginx</code>
<code>-p</code>	Publica un puerto (host:contenedor)	<code>docker run -p 8888:8888 ...</code>
<code>-v</code>	Monta un volumen o bind mount	<code>docker run -v "\$PWD":/app ...</code>
<code>--rm</code>	Elimina el contenedor al terminar	<code>docker run --rm hello-world</code>
<code>--name</code>	Asigna un nombre al contenedor	<code>docker run --name mi-python ...</code>

7.3 Qué ocurre dentro de `docker run`

Cuando ejecutas `docker run`, el demonio hace lo siguiente:

1. Comprueba si la imagen está en la caché local; si no, la **descarga** (`pull`).
2. **Crea el contenedor**: añade una capa de escritura sobre las capas de solo lectura.
3. **Monta el filesystem** (y los volúmenes o binds indicados) y crea la **interfaz de red**.
4. **Arranca el proceso principal** (el comando indicado, o el `CMD` de la imagen).
5. Cuando ese proceso termina, el contenedor **se detiene** (no se elimina).

7.4 Volúmenes y persistencia

Los contenedores son **efímeros**: al eliminarlos se pierden sus datos. Para conservarlos:

- **Volúmenes gestionados** (*named volumes*): los gestiona Docker (en `/var/lib/docker/volumes`). Son la opción recomendada para datos que no necesitas ver desde el host (cachés, datasets, bases de datos).
- **Bind mounts**: montan una **carpeta real del host** dentro del contenedor. Son ideales para el código y los notebooks del curso: editas en tu equipo y el contenedor ve los cambios al instante.

```
1 # bind mount: la carpeta practicas del host se ve en /app dentro del contenedor
2 docker run --rm -v "$PWD/practicas":/app -w /app python:3.12 python app.py
```



Bind mount vs volumen

Para **notebooks y código** del curso usa un **bind mount** (`-v ./practicas:/home/jovyan/work`): son ficheros reales de tu equipo. Los **volúmenes nombrados** quedan para datos que no necesitas tocar desde el host.

8. El Dockerfile: construir imágenes reproducibles (RA7-i)

8.1 Instrucciones esenciales

Un `Dockerfile` es un **guion** que describe cómo construir la imagen. Siempre empieza por `FROM` (la imagen base).

Instrucción	Qué hace
<code>FROM</code>	Define la imagen base (obligatoria)
<code>WORKDIR</code>	Establece el directorio de trabajo
<code>COPY</code>	Copia ficheros del contexto de build a la imagen
<code>RUN</code>	Ejecuta un comando durante el build (instalar, compilar)
<code>ENV</code>	Define variables de entorno
<code>EXPOSE</code>	Documenta el puerto (no lo publica; hace falta <code>-p</code>)
<code>CMD</code>	Comando por defecto en runtime (sobrescribible en <code>docker run</code>)

Instrucción	Qué hace
ENTRYPOINT	Ejecutable fijo; se puede combinar con <code>CMD</code> para los argumentos

```

1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY app.py .
6 CMD ["python", "app.py"]

```

8.2 La caché de capas

Cada instrucción crea una capa. Docker **reutiliza las capas en caché** mientras las instrucciones y su contenido no cambien. Por eso se copia `requirements.txt` **antes** que el código: así las dependencias se instalan una vez y se cachean.



Buenas prácticas para un curso rápido

- Anclar la versión de la imagen base (`FROM python:3.12-slim`, no `FROM python`).
- Usar `.dockerignore` para excluir del build context `.git`, `*.pyc`, `.env`, datasets grandes.
- Unir `apt-get update` && `apt-get install` en un solo `RUN`.
- Un **proceso por contenedor** y contenedores **efímeros**.

8.3 Construir y ejecutar

```

1 docker build -t mi-app . # construye la imagen desde el Dockerfile del directorio actual
2 docker run -p 8000:8000 mi-app # ejecuta la imagen publicando el puerto 8000

```

8.4 Gestionar las imágenes que acumulas

Es un instalador donde podemos incorporar nuestra aplicación. Es el punto de inicio para crear contenedores. Hay imágenes oficiales de por ejemplo Ubuntu, Apache, etc, que fueron creadas por sus creadores oficiales.

Página oficial para imágenes: <https://hub.docker.com>

Vamos a utilizar la siguiente imagen para pruebas:

https://hub.docker.com/_/hello-world

Para ejecutar este contenedor “hello-word” escribimos en la terminal:

```
1 docker run hello-world
```

Una vez ejecutado, ya dispondremos de la imagen descargada, podemos ver todas las imágenes que tenemos descargadas con:

```

1 docker images
2
3 REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
4 hello-world         latest      ee301c921b8a     9 months ago    9.14kB

```

Desde la página web de docker hub, podemos ver diferentes versiones de la misma imagen en la pestaña “TAGS”

Overview **Tags**

Sort by Newest Filter Tags

TAG: [latest](#)
Last pushed 4 days ago by [dolianky](#)
docker pull hello-world:latest

DIGEST	OS/ARCH	VULNERABILITIES	COMPRESSED SIZE
dbbd3cf66631	linux/386	None found	2.66 KB
245fe15fbb8f	windows/amd64	None found	115.14 MB
088bdbea94d5	windows/amd64	None found	99.78 MB

+8 more...

TAG: [nanoserver-ltsc2022](#)
Last pushed 4 days ago by [dolianky](#)
docker pull hello-world:nanoserv_

DIGEST	OS/ARCH	VULNERABILITIES	COMPRESSED SIZE
245fe15fbb8f	windows/amd64	None found	115.14 MB

Podemos descargar una imagen específica y ejecutarla:

```
1 docker run hello-world:linux
2
3 docker images
4
5 REPOSITORY TAG IMAGE ID CREATED SIZE
6 hello-world latest ee301c921b8a 9 months ago 9.14kB
7 hello-world linux ee301c921b8a 9 months ago 9.14kB
8
9 docker run hello-world:linux
```

Para eliminar una imagen utilizamos el parámetro `rmi` por ejemplo:

```
1 docker pull alpine
2
3 docker images
4
5 REPOSITORY TAG IMAGE ID CREATED SIZE
6 alpine latest ace17d5d883e 3 weeks ago 7.73MB
7 hello-world latest ee301c921b8a 9 months ago 9.14kB
8 hello-world linux ee301c921b8a 9 months ago 9.14kB
9
10 # Eliminamos, opción 1
11 docker rmi ace17d5d883e
12
13 # Eliminamos, opción 2
14 docker rmi alpine
15
16 # Eliminamos, opción 3
17 docker rmi ace1
```

También se pueden buscar imágenes desde consola.

```
1 docker search ubuntu
2
3 NAME DESCRIPTION STARS OFFICIAL
4 ubuntu Ubuntu is a Debian-based Linux operating sys... 16888 [OK]
5 websphere-liberty WebSphere Liberty multi-architecture images ... 298 [OK]
6 open-liberty Open Liberty multi-architecture images based... 64 [OK]
7 neurodebian NeuroDebian provides neuroscience research s... 106 [OK]
8 ubuntu-debootstrap DEPRECATED; use "ubuntu" instead 52 [OK]
9 ubuntu-upstart DEPRECATED, as is Upstart (find other proces... 115 [OK]
10 ubuntu/nginx Nginx, a high-performance reverse proxy & we... 112
11 ubuntu/squid Squid is a caching proxy for the Web. Long-t... 83
12 ubuntu/cortex Cortex provides storage for Prometheus. Long... 4
13 ubuntu/prometheus Prometheus is a systems and service monitori... 56
14 ubuntu/apache2 Apache, a secure & extensible open-source HT... 70
15 ...
```

9. Docker Compose: varios servicios a la vez (RA7-i)

Compose orquesta **múltiples contenedores** (p. ej. la aplicación + una base de datos) mediante un fichero `compose.yaml`.


```

1 services:
2   jupyter:
3     image: jupyter/scipy-notebook
4     ports:
5       - "8888:8888"
6     volumes:
7       - ./practicas:/home/jovyan/work
8     environment:
9       - JUPYTER_TOKEN=cursoia

```

Comando	Qué hace
<code>docker compose up -d</code>	Crea y arranca los servicios en segundo plano
<code>docker compose down</code>	Detiene y elimina contenedores (y la red por defecto)
<code>docker compose down -v</code>	Además borra los volúmenes nombrados (¡borra datos!)
<code>docker compose config</code>	Muestra la configuración resuelta sin arrancar nada
<code>docker compose logs -f</code>	Sigue los logs de los servicios
<code>docker compose exec</code>	Ejecuta un comando dentro de un contenedor en marcha

Para ordenar el arranque se usa `depends_on` (con `condition: service_healthy` y un `healthcheck` para esperar a que un servicio esté listo).

⚠️ `down -v` borra datos

`docker compose down -v` elimina también los **volúmenes nombrados**. Úsalo solo cuando quieras empezar de cero; tus notebooks se perderían.

10. El contenedor de prácticas de IA (RA7-i)

El entorno del curso se levantará con la imagen `jupyter/scipy-notebook`, que ya incluye `numpy`, `scipy`, `scikit-learn`, `pandas`, `matplotlib` y otras bibliotecas científicas.

🔥 ¿Por qué `python:3.12-slim` y no Alpine?

La variante `-slim` usa **Debian (glibc)**; la `-alpine` usa **musl libc**. Bibliotecas como `scikit-learn` y `torch` **solo publican wheels para glibc**: en Alpine habría que compilar desde fuente (`torch` puede tardar 30-60 min). Con `-slim` todo el stack se instala con wheels precompilados. Los Jupyter Docker Stacks científicos también se construyen sobre Ubuntu (glibc).

El contenedor de prácticas:

- Publica el puerto **8888** (Jupyter).
- Monta tu carpeta de prácticas en `/home/jovyan/work` (bind mount) para que los notebooks sean ficheros reales de tu equipo.
- Fija el token con `JUPYTER_TOKEN` para que la URL sea determinista.

```

1 docker compose up -d
2 # abre http://localhost:8888 y usa el token cursoia
3 docker compose exec jupyter bash # para entrar dentro
4 docker compose down             # cuando termines

```

Lo pondrás en marcha paso a paso en el **Taller 2** de esta unidad.

10.1 Construir la imagen con un Dockerfile propio

Otra forma de levantar un contenedor con `docker-compose` es utilizar un `Dockerfile` para generar la imagen (en lugar de usar una de DockerHub)

Necesitamos una carpeta con la siguiente estructura:

```

1  .
2  ├── docker-compose.yml
3  ├── Dockerfile
4  └── notebooks
5      └── rockpaperscissors.ipynb

```

El fichero `Dockerfile` tiene el siguiente contenido:

```

1  #versión de python compatible con experta
2  FROM python:3.6
3  #librería de python para sistemas expertos
4  RUN pip3 install experta
5  #librería para usar notebooks en python
6  RUN pip3 install notebook
7  #creamos la carpeta de trabajo
8  WORKDIR /home/MIA2526
9  #ejecutamos el jupyter notebook en el puerto 8888
10 CMD jupyter notebook --allow-root --ip=0.0.0.0 --port=8888 --no-browser

```

Ahora, para el `docker-compose.yml` tendremos:

```

1  services:
2    experta:
3      container_name: experta #bautizamos a nuestro contenedor
4      build: . #con esta línea le indicamos que busque el Dockerfile, en lugar de buscar una imagen en un repositorio
5      ports:
6        - "8888:8888" #publicamos el puerto para que sea accesible desde fuera
7      volumes:
8        - ./notebooks:/home/MIA2526 #aquí asignamos la carpeta local notebooks con la carpeta de trabajo del contenedor

```

Y por último necesitamos el notebook para hacer pruebas, en nuestro caso es un juego de piedra, papel o tijeras:

[rockpaperscissors.ipynb](#) este fichero debemos guardarlo en la carpeta `notebooks`

Ahora con toda la estructura lista y desde la raíz de la carpeta (donde están el `docker-compose.yml` y el `Dockerfile`) ejecutamos:

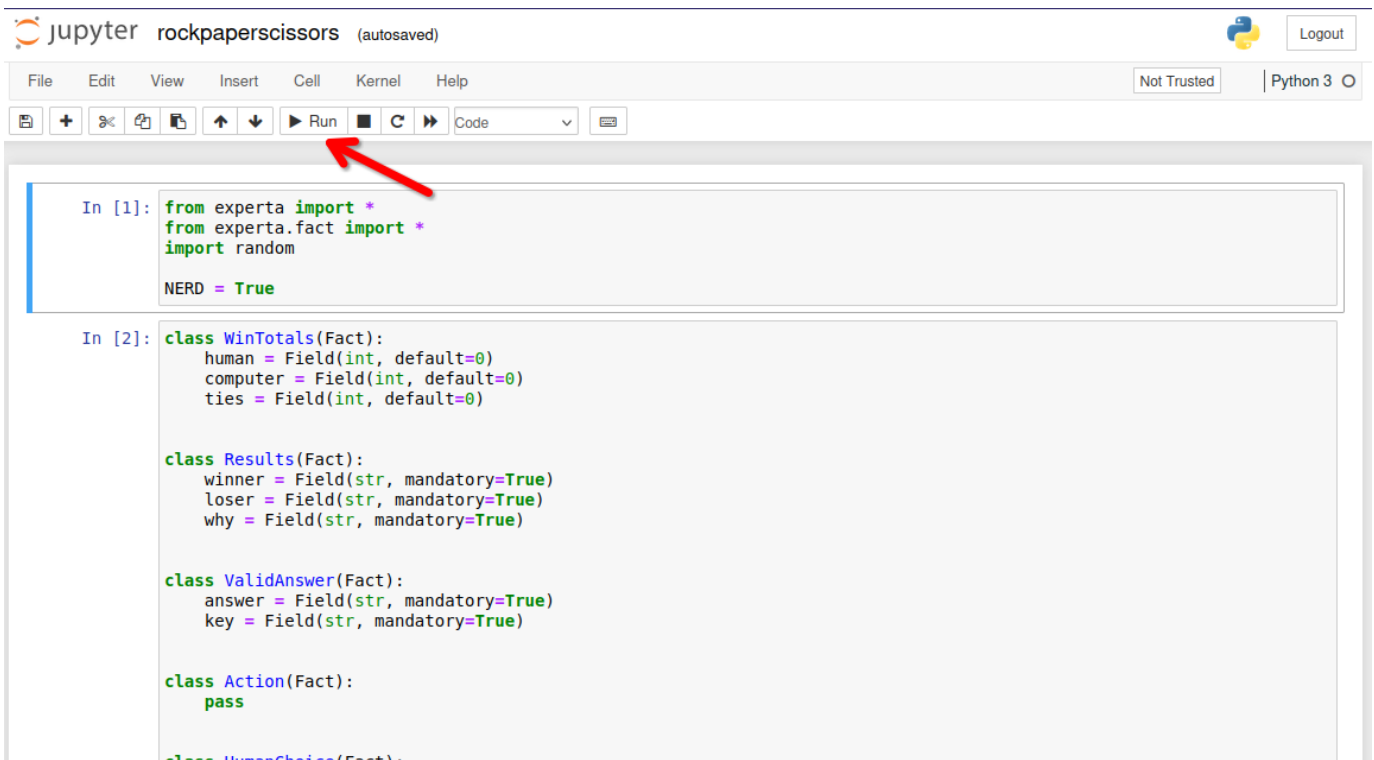
```

1  $ docker-compose up
2  [+] Running 1/1
3  ✓ Container experta Recreated                                0.1s
4  Attaching to experta
5  experta | [I 16:30:08.568 NotebookApp] Writing notebook server cookie secret to /root/.local/share/jupyter/runtime/notebook_cookie_secret
6  experta | [I 16:30:08.784 NotebookApp] Serving notebooks from local directory: /home/MIA2324
7  experta | [I 16:30:08.784 NotebookApp] Jupyter Notebook 6.4.0 is running at:
8  experta | [I 16:30:08.784 NotebookApp] http://e0ec65acd84:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b
9  experta | [I 16:30:08.784 NotebookApp] or http://127.0.0.1:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b
10 experta | [I 16:30:08.784 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
11 experta | [C 16:30:08.787 NotebookApp]
12 experta |
13 experta | To access the notebook, open this file in a browser:
14 experta | file:///root/.local/share/jupyter/runtime/nbserver-7-open.html
15 experta | Or copy and paste one of these URLs:
16 experta | http://e0ec65acd84:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b
17 experta | or http://127.0.0.1:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b

```

Ahora podemos hacer click directamente sobre el enlace a <http://127.0.0.1:8888/?token=beb660273e8e168c7fec90978ed56fb50db6b08d915cb14> donde veremos nuestro jupyter notebook:

Si hacemos click sobre el notebook `rockpaperscissors.ipynb` podremos ver:



JupyterLab interface showing two code cells. A red arrow points to the 'Run' button in the toolbar.

```
In [1]: from experta import *
        from experta.fact import *
        import random

        NERD = True
```

```
In [2]: class WinTotals(Fact):
        human = Field(int, default=0)
        computer = Field(int, default=0)
        ties = Field(int, default=0)

        class Results(Fact):
            winner = Field(str, mandatory=True)
            loser = Field(str, mandatory=True)
            why = Field(str, mandatory=True)

        class ValidAnswer(Fact):
            answer = Field(str, mandatory=True)
            key = Field(str, mandatory=True)

        class Action(Fact):
            pass

        class HumanChoice(Fact):
```

Después de pulsar varias veces (para ir ejecutando todas las celdas) podremos ver como evoluciona nuestro juego:

```
In [*]: rps.reset()
        rps.run()
```

Lets play a game!
 You choose rock, paper, or scissors,
 and I'll do the same.
 Scissors (S), Paper (P), Rock (R)? R
 Computer wins! Paper covers rock
 Play again?Y
 Scissors (S), Paper (P), Rock (R)? S
 Tie! Ha-ha!
 Play again?Y
 Scissors (S), Paper (P), Rock (R)? P
 You win! Paper covers rock
 Play again?

Para detener el contenedor que hemos lanzado con `docker-compose`, solo hemos de pulsar `^ Ctrl + C`

Actualizado respecto al material antiguo

El `Dockerfile` de partida fijaba `python:3.6`, fuera de soporte desde 2021. La versión del curso usa `python:3.12-slim` e instala las dependencias desde `requirements-ia.txt`. Los ficheros listos para usar están en `entorno/`, junto a esta unidad. Con Python 3.12, `experta` necesita el parche del apartado 6.5.

11. Copias de seguridad de imágenes y contenedores (RA7-i)

Copias de contenedores

Ya estén encendidos o apagados, podemos realizar respaldos de seguridad de los contenedores. Utilizando la opción “*export*” empaquetará el contenido, generando un fichero con extensión “*.tar*” de la siguiente manera:

```
1 docker export -o fichero-resultante.tar nombre-contenedor
```

O

```
1 docker export nombre-contenedor > fichero-resultante.tar
```

Restauración de copias de seguridad de contenedores

Hay que tener en cuenta, antes de nada, que no es posible restaurar el contenedor directamente, de forma automática. En cambio, sí podemos crear una imagen, a partir de un respaldo de un contenedor, mediante el parámetro “*import*” de la siguiente manera:

```
1 docker import fichero-backup.tar nombre-nueva-imagen
```

Copias de imágenes

Aunque no tiene mucho sentido por que se bajan muy rápido, también tenemos la posibilidad de realizar copias de seguridad de imágenes. El proceso se realiza al utilizar el parámetro ‘*save*’, que empaquetará el contenido y generará un fichero con extensión “*.tar*”, así:

```
1 docker save nombre_imagen > imagen.tar
```

O

```
1 docker save -o imagen.tar nombre_imagen
```

Restaurar copias de seguridad de imágenes

Con el parámetro ‘*load*’, podemos restaurar copias de seguridad en formato ‘*.tar*’ y de esta manera recuperar la imagen.

```
1 docker load -i fichero.tar
```

12. Puntos clave de la unidad

- El curso de especialización IA y Big Data dura **600 horas / 36 ECTS** y el módulo 5071 se imparte en **90 horas** en la Comunitat Valenciana.
- El módulo 5071 desarrolla **6 RA** más el **RA7 de proyecto integrador transversal** (común a todo el curso, semanas 25-29, nota consensuada).
- **La normativa** exige alcanzar **todos los RA** del módulo; el centro lo concreta en **cada RA ≥ 5** . La nota de cada RA combina **40 % tareas + 60 % prueba escrita**; se pierde la evaluación continua superando el **15 % de inasistencia**.
- El entorno de trabajo usa **Aules** (Moodle), la web del curso, **Python + Jupyter** y **Docker**.
- Un **contenedor** es una instancia ejecutable de una **imagen**; Docker aporta aislamiento por procesos, ligereza y reproducibilidad frente a las máquinas virtuales.
- Los contenedores son **efímeros**: los datos se conservan con **volúmenes** o **bind mounts**.
- Un **Dockerfile** define la imagen por capas; el **orden de las instrucciones** aprovecha la caché.
- **Compose** orquesta varios servicios con un solo fichero.

13. Glosario

Término	Definición
Contenedor	Instancia ejecutable de una imagen con su propia capa de escritura
Imagen	Plantilla de solo lectura que define el entorno (SO base + app + dependencias)
Capa	Cada instrucción del Dockerfile; las capas se reutilizan en caché
Dockerfile	Fichero que describe cómo construir una imagen
Registro	Repositorio de imágenes (p. ej. Docker Hub)
Daemon (dockerd)	Proceso de Docker que hace el trabajo pesado
CLI (docker)	Cliente de línea de comandos que envía órdenes al daemon
Bind mount	Montaje de una carpeta real del host dentro del contenedor
Volumen	Almacenamiento gestionado por Docker para persistir datos
Compose	Herramienta para orquestar varios contenedores con un fichero YAML
Servicio	Un contenedor definido dentro de un <code>compose.yaml</code>
Healthcheck	Comprobación del estado de un servicio para ordenar el arranque
RA (resultado de aprendizaje)	Capacidad que el alumnado debe demostrar al terminar un módulo
CE (criterio de evaluación)	Evidencia observable que permite comprobar un RA
FE (formación en empresa)	Fase en empresa del alumnado (opcional en este curso, según centro)

14. FAQ

? ¿Docker es una máquina virtual? ▾

No. Las VMs virtualizan **hardware** (cada una con su kernel); los contenedores virtualizan el **sistema operativo** (comparten el kernel del host) y aíslan por procesos. Por eso son más ligeros y arrancan en segundos.

? ¿Pierdo mis datos si elimino un contenedor? ▾

Sí, a menos que los guardes fuera: en un **volumen** o en un **bind mount**. Por eso en las prácticas montamos tu carpeta de trabajo dentro del contenedor.

? ¿Por qué `EXPOSE` no me deja abrir el navegador? ▾

Porque `EXPOSE` solo **documenta** el puerto. Para que sea accesible hay que **publicarlo** con `-p` (o `ports:` en Compose).

? ¿Qué pasa si `docker run` me pide descargar una imagen muy grande? ▾

Solo ocurre la **primera vez**. Después, Docker reutiliza la imagen desde la caché local. Si la imagen es demasiado grande, se puede empezar con variantes más ligeras (`-slim`) y anclar la versión.

? ¿El RA7 cuenta para la nota del módulo 5071? ▾

Sí. El RA7 aporta el **20 %** de la nota final del módulo (RA1-RA6 el 80 %). Además, como es transversal, se evalúa de forma **consensuada** por todo el equipo docente del curso.

¿Puedo instalar Docker en cualquier equipo? ▾

Docker funciona en Linux, Windows y macOS. En este curso lo usaremos en el aula y para los talleres; si tu equipo no lo soporta, el entorno gestionado del aula será el plan B.

15. Sesiones

La unidad son **6 h en dos semanas** (1-8 de octubre), a 3 h por semana.

Semana	Fechas	Horas	Contenido	Evidencia
1	1-2 oct	3	Presentación del curso, del módulo y de la evaluación · instalación de Docker · conceptos: imagen, contenedor, registro	Docker funcionando (<code>docker run hello-world</code>)
2	5-8 oct	3	<code>docker run</code> y volúmenes · Dockerfile y Compose · contenedor de prácticas de IA	Taller 1 (hecho / no hecho) y entorno del curso levantado

Si el horario del módulo son 2 h + 1 h

Cada semana se parte en dos sesiones. El **bloque de 2 h** es el único que admite trabajo con contenedores (descargas, construcción de imágenes, resolución de errores); la sesión de **1 h** se dedica a los conceptos, al repaso y a cerrar el taller. La primera semana es corta: el curso empieza el jueves 1 de octubre.

A. Anexo · chuleta de comandos

Referencia rápida para consultar en clase, no para memorizar.

GESTIÓN DE IMAGENES

```
1 docker image
2 docker history
3 docker inspect
4 docker save/load
5 docker rmi
```

GESTIÓN DE CONTENEDORES

```
1 docker attach
2 docker exec
3 docker inspect
4 docker kill
5 docker logs
6 docker pause/unpause
7 docker port
8 docker ps
9 docker rename
10 docker start/stop/restart
11 docker rm
12 docker run
13 docker stats
14 docker top
15 docker update
```

EJEMPLO

```

1 # Ver los contenedores que tenemos
2 docker ps
3 CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
4
5 # Ver las imagenes que tenemos
6 docker images
7 REPOSITORY          TAG   IMAGE ID   CREATED   SIZE
8
9 # Crear un contenedor con una imagen básica de debian
10 # Como no tenemos ninguna imagen de debian, la descarga y la ejecuta
11 docker run debian
12 # Intentamos ver el contenedor en ejecución, no aparece nada porque ya se ha cerrado
13 docker ps
14 # Podemos verlo con
15 docker ps -a
16 CONTAINER ID   IMAGE     COMMAND   CREATED        STATUS      PORTS   NAMES
17 09b14daab800   debian    "bash"    2 seconds ago  Exited (0) 1 second ago  pensive_wozniak
18
19 # Ejecutar un comando en un contenedor
20 docker run debian /bin/echo "Hello World"
21 Hello World
22
23 # Información
24 docker inspect debian

```

Crear contenedor interactivo y con nombre

<https://jolphgs.wordpress.com/2019/09/25/create-a-debian-container-in-docker-for-development/>

Para que docker no se invente un nombre como “pensive_wozniak” (comando anterior) podemos definir el nombre que queremos.

Utilizaremos una de las imágenes de: https://hub.docker.com/_/debian/tags

```

1 # Obtenemos la imagen, en el apartado anterior la hemos ejecutado directamente con "run", esto
2 # la obtiene implícitamente. En este caso la vamos a descargar.
3 $ docker pull debian:13-slim
4
5 # --name
6 # -h hostname que tendrá el contenedor
7 # -e codificación de caracteres
8 # -it modo interactivo
9 # /bin/bash -l la shell que se ejecutará
10 $ docker run --name debian-mini -h equipo1 -e LANG=C.UTF-8 -it debian:13-slim /bin/bash -l
11
12 --- Estamos dentro del contenedor ---
13 # Una vez dentro del contenedor podemos actualizarlo e instalar los paquetes que creamos necesarios
14 apt update && apt upgrade --yes && apt install sudo locales --yes
15 # Configurar timezone
16 dpkg-reconfigure tzdata
17
18 # Vamos a nuestra home y creamos un archivo
19 cd
20 echo "hola" > prueba.txt
21
22 # Salimos del contenedor
23 exit (o control + d)
24
25 --- Ahora volvemos a la consola de nuestro equipo ---
26 $ docker ps
27 CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
28
29 $ docker ps -a
30 CONTAINER ID   IMAGE     COMMAND   CREATED        STATUS      PORTS   NAMES
31 8c6b29c818ee   debian:13-slim    "/bin/bash -l"    44 seconds ago  Exited (0) 11 seconds ago  debian-mini

```

B. Anexo · catálogo de contenedores para experimentar

MONITORIZACIÓN Y GESTIÓN

- **Netdata** - Monitorización de sistemas en tiempo real con dashboard web completo
- **Glances** - Monitorización del sistema via web que muestra información de CPU, memoria, red y procesos
- **Portainer** - Interfaz web para gestionar y administrar contenedores Docker
- **Uptime Kuma** - Monitor de disponibilidad para verificar el estado de contenedores y servicios

PROXY Y RED

- **Nginx Proxy Manager** - Proxy inverso con interfaz web para gestionar hosts virtuales y certificados SSL/TLS
- **Acestream** - Motor P2P para streaming de video, útil para integrar canales en Jellyfin

- **Libreddit** - Interfaz alternativa para Reddit libre de publicidad y tracking (modo lectura)
- **Invidious** - Interfaz alternativa para YouTube que respeta la privacidad

MULTIMEDIA Y ENTRETENIMIENTO

- **Jellyfin** - Servidor multimedia libre para organizar y streaming de películas, series y música
- **Soulseek** - Cliente para la red P2P Soulseek especializada en compartir música
- **youtube-dl** - Herramienta para descargar videos y audio de YouTube y otros sitios
- **Photoprism** - Servicio de gestión y visualización de fotos personales con funciones de IA

PRODUCTIVIDAD Y ORGANIZACIÓN

- **Nextcloud** - Plataforma de colaboración y almacenamiento en la nube auto-alojado
- **Linkding** - Marcador social para guardar y organizar enlaces web
- **GitBucket** - Plataforma Git auto-alojada similar a GitHub
- **SearXNG** - Metabuscador privado que agrega resultados de múltiples motores de búsqueda

SEGURIDAD Y CONTRASEÑAS

- **Vaultwarden** - Implementación alternativa del gestor de contraseñas Bitwarden, más ligera y eficiente

AUTOMATIZACIÓN DEL HOGAR

- **Home Assistant** - Plataforma de domótica de código abierto para automatizar y controlar dispositivos del hogar
- **Homebridge** - Puente que permite integrar dispositivos no compatibles con el ecosistema Apple HomeKit
- **Camera.UI** - Aplicación para gestionar y visualizar cámaras de seguridad con integración HomeKit

UTILIDADES Y MANTENIMIENTO

- **Homepage** - Dashboard personalizable como página de inicio para acceder a todos los servicios
- **Watchtower** - Servicio que actualiza automáticamente los contenedores Docker cuando hay nuevas versiones disponibles



Uso de este catálogo

Sirve para elegir imagen en la fase 4 del Taller 1. Antes de proponer una, comprueba en Docker Hub que sigue mantenida: hay imágenes populares con años sin actualizar.

16. Recursos

- [Diapositivas](#)
- [Ejercicios de la unidad](#)
- Talleres:
 - [T01 · Verificación del entorno y primer contenedor](#) — entregable, se marca **hecho / no hecho**
 - [T02 · Contenedor de prácticas de IA](#) — entregable, se marca **hecho / no hecho**
- **Notebooks** — el N01 de la unidad, entregable que se marca **hecho / no hecho**; descarga y apertura en Colab desde el propio notebook, en el menú «Notebooks»

**Referencias de la unidad** ▾

Documentación oficial de Docker:

- [What is Docker?](#)
- [What is a container?](#)
- [Dockerfile reference](#)
- [Compose Quickstart](#)
- [Building best practices](#)

Normativa y plataforma:

- [RD 279/2021](#) — currículo del curso de especialización
- [Orden 8/2025 de la Comunitat Valenciana](#) — evaluación
- [Aules GVA](#) — plataforma del centro

17. Evaluación

- **Tarea de la unidad** (40 % de la nota del RA al que se asocia): entregar el informe de los talleres 1 y 2 en Moodle.
- **Evaluación inicial:** diagnóstica, sin nota, antes del segundo mes lectivo.
- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «*en función de la consecución de los RA*»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.

18. Recuperación

Si no superas la tarea de esta unidad, se activará un **programa de recuperación individual** (art. 14.4 Orden 8/2025) con actividades y criterios de evaluación específicos para el RA correspondiente.

[Volver al índice](#)

🕒 23 de agosto de 2026

2.2 Diapositivas de la UD00



Diapositivas PDF



Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

🕒 22 de agosto de 2026

3. UD01

3.1 UD01 — Caracterización de sistemas de IA

Unidad 1 · 12 h · semanas 3-6 (12 de octubre al 6 de noviembre)

1. Resultado de aprendizaje y criterios de evaluación

RA1 — Caracteriza sistemas de Inteligencia Artificial relacionándolos con la mejora de la eficiencia operativa de las organizaciones y empresas.

CE	Criterio de evaluación
RA1-a	Se han identificado los principios fundamentales de los sistemas inteligentes.
RA1-b	Se ha recopilado información sobre campos donde se aplica Inteligencia Artificial.
RA1-c	Se han identificado las técnicas básicas a utilizar en el entorno de la IA.
RA1-d	Se han identificado nuevas formas de interacciones en los negocios que mejoran la eficiencia operativa.

Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo para este RA dice textualmente:

- *Caracterización de sistemas de Inteligencia Artificial:*
- – *Principios de los sistemas inteligentes.*
- – *Campos de aplicación.*
- – *Técnicas de la Inteligencia Artificial.*
- – *Nuevas formas de interacción.*

2. Objetivos de la unidad

Objetivo	Descripción
O1	Explicar qué es un sistema inteligente y cuáles son sus principios fundamentales.
O2	Distinguir las escuelas de pensamiento y las clasificaciones de la IA (débil/fuerte, Russell-Norvig, Hintze).
O3	Distinguir IA, aprendizaje automático, aprendizaje profundo e IA generativa.
O4	Enumerar los principales campos de aplicación de la IA con ejemplos reales.
O5	Distinguir las técnicas básicas de la IA y saber qué problema resuelve cada una.
O6	Relacionar las nuevas formas de interacción (asistentes, chatbots, visión, voz) con la mejora de la eficiencia operativa.
O7	Identificar en casos prácticos qué técnica usar y qué mejora operativa aporta.

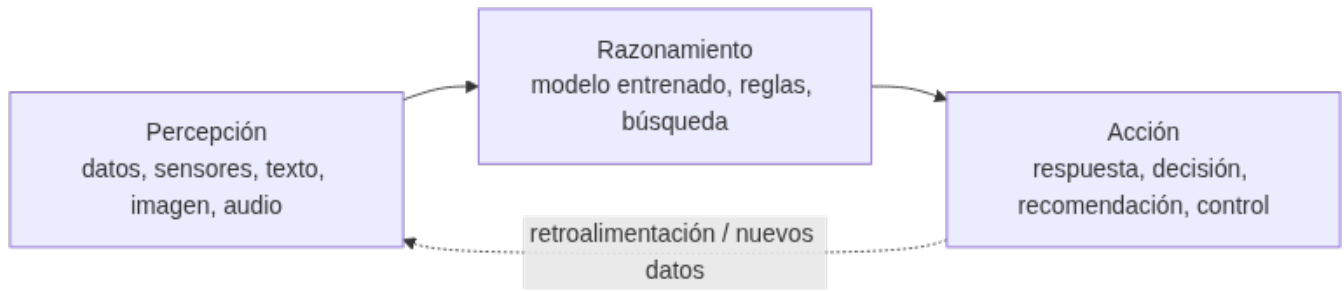
3. Fundamentos de los sistemas inteligentes (RA1-a)

3.1 ¿Qué es la inteligencia artificial?

La **inteligencia artificial (IA)** es la tecnología que permite a las máquinas **simular el aprendizaje, la comprensión, la resolución de problemas, la toma de decisiones y la creatividad** humanas. Las aplicaciones dotadas de IA pueden ver e identificar objetos, entender y responder al lenguaje, aprender de la experiencia, recomendar decisiones e incluso **actuar de forma autónoma** (p. ej. un coche autónomo).

3.2 El ciclo percepción → razonamiento → acción

Un **sistema inteligente** percibe su entorno, razona sobre esa información y actúa para conseguir un objetivo:



- **Percepción:** captar datos del mundo (texto, imagen, audio, sensores, registros de negocio).
- **Razonamiento:** procesar los datos para obtener conocimiento o tomar decisiones (un modelo entrenado, un conjunto de reglas, una búsqueda).
- **Acción:** actuar sobre el entorno o sobre las personas (responder, recomendar, controlar un proceso).

3.3 Características de un sistema inteligente

Característica	Descripción
Autonomía	Opera sin supervisión humana constante
Adaptación	Aprende de los datos y mejora con la experiencia
Toma de decisiones	Recomienda o actúa basándose en datos, no solo en reglas fijas



IA basada en reglas vs. IA que aprende

La IA **más elemental** puede ser un conjunto de reglas *si... entonces...* programadas a mano (un termostato que enciende la calefacción si baja de 18 °C es, formalmente, un sistema de IA basado en reglas). Pero cuando la tarea se complica, definir todas las reglas es imposible: ahí entra el **aprendizaje automático**, que deduce los patrones de los datos.

3.4 IA débil frente a IA fuerte

La clasificación más simple de la IA es **según la tarea que resuelve**: hay tareas concretas y acotadas (jugar al ajedrez) y tareas abiertas que exigen contexto, ética o creatividad (gestionar una cocina sin intervención humana). De ahí surgen dos categorías:

- **IA débil o estrecha (narrow/weak):** diseñada para **una tarea o un conjunto limitado de tareas**. Es **reactiva** (no actúa si no se la activa), **no flexible** (colapsa ante lo no previsto), queda **limitada por lo que programó una persona** y **no tiene conciencia**: computa, no razona en sentido humano. Los asistentes virtuales (Siri, Alexa) son el ejemplo típico — operan dentro de un rango de respuestas definido en su base de datos y, fuera de él, dan respuestas inadecuadas o directamente no responden. **Es toda la IA que existe hoy.**
- **IA general o fuerte (AGI/strong):** igualaría o superaría la inteligencia humana en **cualquier** tarea intelectual, sería **proactiva, flexible** (aprendería una tarea nueva a partir de una parecida) y se **autorregularía**. **Es puramente teórica**: ningún sistema actual se aproxima, más allá de la ficción (T-800, Wall-E, J.A.R.V.I.S.).



La IA débil también tiene riesgos

Precisamente por no considerar un contexto amplio ni reglas sociales o éticas, la IA débil ejecuta su tarea con eficacia y sin matices: no evalúa consecuencias como lo haría una persona. Es una tecnología incompleta y potencialmente peligrosa si se usa sin prudencia, o si quien la programa busca causar daño — el motivo de fondo de la UD06.

3.5 Escuelas de pensamiento y clasificaciones de la IA

Además de por la tarea, la IA se puede clasificar por **cómo llega a sus resultados** (la escuela de pensamiento) o por **qué capacidades tiene** (Russell-Norvig, Hintze). Ninguna de las tres es «la correcta»: son lentes distintas sobre el mismo campo.

DOS ESCUELAS: CONVENCIONAL Y COMPUTACIONAL

	IA convencional (simbólico-deductiva)	IA computacional (subsimbólico-inductiva)
Cómo razona	Análisis formal y estadístico explícito	Aprendizaje interactivo a partir de datos empíricos
Técnicas	Razonamiento basado en casos, sistemas expertos, redes bayesianas	Redes neuronales, máquinas de vectores soporte, sistemas difusos, computación evolutiva
Qué originó	La «automatización» clásica (reglas + estadística)	El aprendizaje automático actual

Con el auge del *machine learning*, buena parte de lo que hacía la escuela convencional se ha ido llevando al campo computacional — no son compartimentos estancos.

CLASIFICACIÓN DE RUSSELL Y NORVIG (1995)

En *Artificial Intelligence: A Modern Approach* — el libro de texto de IA más usado en el mundo —, Stuart Russell y Peter Norvig proponen cuatro categorías según el **origen del comportamiento inteligente**:

Categoría	Enfoque	Ejemplo de aplicación
Sistemas cognitivos	Piensan como humanos: emulan el proceso de decisión	Modelos cognitivos, aprendizaje
Test de Turing	Actúan como humanos, sin pasar por el razonamiento	Robótica, actuadores en el mundo físico
Leyes del pensamiento	Piensan con lógica formal, sin excepciones	Sistemas expertos (aproximaciones acotadas)
Agentes racionales	Actúan racionalmente, sin razonamiento lógico explícito	Agentes de software actuales

Las dos últimas exigen una capacidad de cómputo que, para el caso general, todavía es inalcanzable.

CLASIFICACIÓN DE HINTZE (2016)

Arend Hintze (Universidad de Michigan) propuso una clasificación por **capacidades**, de la más simple a la más avanzada — la más citada para explicar hacia dónde evoluciona la IA:



- **Máquinas reactivas:** sin memoria ni capacidad de usar experiencia pasada. **Deep Blue** (IBM, venció a Kasparov en 1997) es el ejemplo perfecto: evalúa el tablero en tiempo real, sin ningún concepto de lo ocurrido antes.
- **Memoria limitada:** usan observaciones recientes para decidir, sin guardarlas a largo plazo. Los **vehículos autónomos** actuales encajan aquí: memorizan velocidad y trayectoria de otros coches para decidir un cambio de carril, pero esa información es transitoria.
- **Teoría de la mente (teórica):** formaría representaciones sobre lo que piensan y sienten otros agentes — la base de la interacción social. Ningún sistema actual llega a esto.
- **Autoconciencia (teórica):** el sistema se representaría a sí mismo, conocería sus propios estados internos. Es el paso final, y el más lejano, del desarrollo de la IA.



Cómo no perderse entre tres clasificaciones

Si te preguntan «¿qué tipo de IA es esto?», identifica primero la **tarea** (débil casi siempre, hoy), después la **escuela** (¿reglas explícitas o aprendida de datos?) y por último, si aplica, el **nivel de Hintze** (¿tiene memoria de lo que ha visto?). Las tres respuestas pueden convivir para el mismo sistema.

3.6 Breve historia de la IA

Año	Hito
1950	Alan Turing publica <i>Computing Machinery and Intelligence</i> y propone el Test de Turing
1956	Dartmouth: John McCarthy acuña el término inteligencia artificial
1958	Frank Rosenblatt publica el perceptrón , primera red neuronal que aprende (concebido en 1957; máquina Mark I Perceptron construida entre 1958 y 1960)
1997	Deep Blue (IBM) vence al campeón de ajedrez Kasparov
2011	Watson (IBM) gana en <i>Jeopardy!</i> ; emerge la ciencia de datos
2016	AlphaGo (DeepMind) vence en Go (más de 14,5 billones de jugadas tras 4 movimientos)
2017	Vaswani et al. publican <i>Attention Is All You Need</i> y proponen la arquitectura Transformer , basada solo en mecanismos de atención: elimina la recurrencia y las convoluciones. Es la base técnica de los LLM y de la IA generativa posterior
2022	Los grandes modelos de lenguaje (LLM) , p. ej. ChatGPT, cambian la industria
2024-26	Modelos multimodales y agentes de IA autónomos

3.7 Tipos de IA según su forma

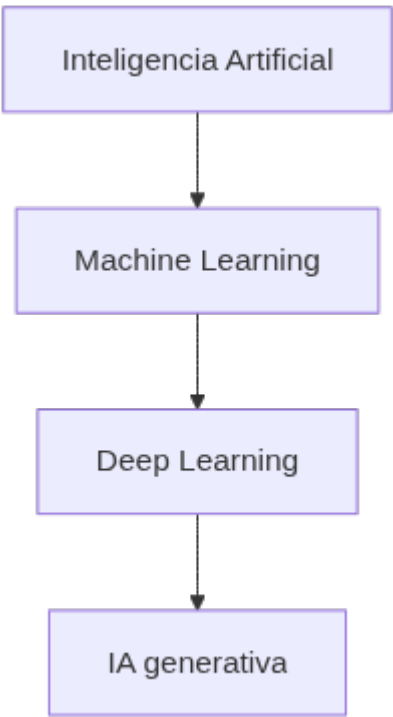
La IA se presenta en dos formas generales:

- **IA de software:** programas que procesan información sin cuerpo físico (asistentes virtuales, buscadores, análisis de imágenes, reconocimiento de voz y rostro, motores de traducción).
- **IA "encarnada" (embodied):** sistemas con presencia física que interactúan con el mundo (robots, coches autónomos, drones, internet de las cosas).

Muchas aplicaciones combinan ambas: un coche autónomo usa visión y decisiones de software dentro de un vehículo físico.

4. IA, machine learning, deep learning e IA generativa (RA1-c)

La relación entre estos términos se representa como cajas anidadas:





Regla para recordar

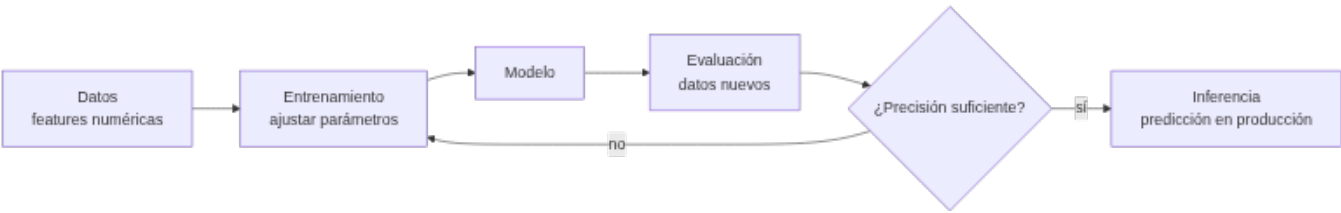
Todo machine learning es IA, pero no toda IA es machine learning. Y todo deep learning es machine learning, y la IA generativa es una parte del deep learning.

4.1 Aprendizaje automático (machine learning, ML)

El ML crea **modelos** entrenando un algoritmo sobre datos para hacer predicciones o tomar decisiones **sin ser programado explícitamente** para cada caso. Su objetivo central es la **generalización**: que el modelo acierte con datos **nuevos**, no solo con los de entrenamiento.

En lugar de programar reglas manuales (filtro de spam por criterios a mano), el modelo **aprende** de ejemplos etiquetados y deduce los criterios por sí mismo.

CÓMO FUNCIONA UN MODELO DE ML, PASO A PASO



- 1. **Representar los datos numéricamente**: cada ejemplo se convierte en un vector de *features* (características). P. ej., una casa se representa como [superficie, habitaciones, edad].
- 2. **Entrenar**: el algoritmo ajusta sus **parámetros** (los pesos de la fórmula) para que el error entre su salida y la respuesta correcta sea mínimo. Se usa una **función de pérdida** y un optimizador (gradiente descendente).
- 3. **Evaluar**: se comprueba el rendimiento con **datos que el modelo no ha visto** (test), no solo con los de entrenamiento, para asegurar que **generaliza** y no memoriza (sobreajuste).
- 4. **Inferir**: en producción, el modelo predice sobre datos nuevos (a esto se le llama *AI inference*).



Ejemplo con números

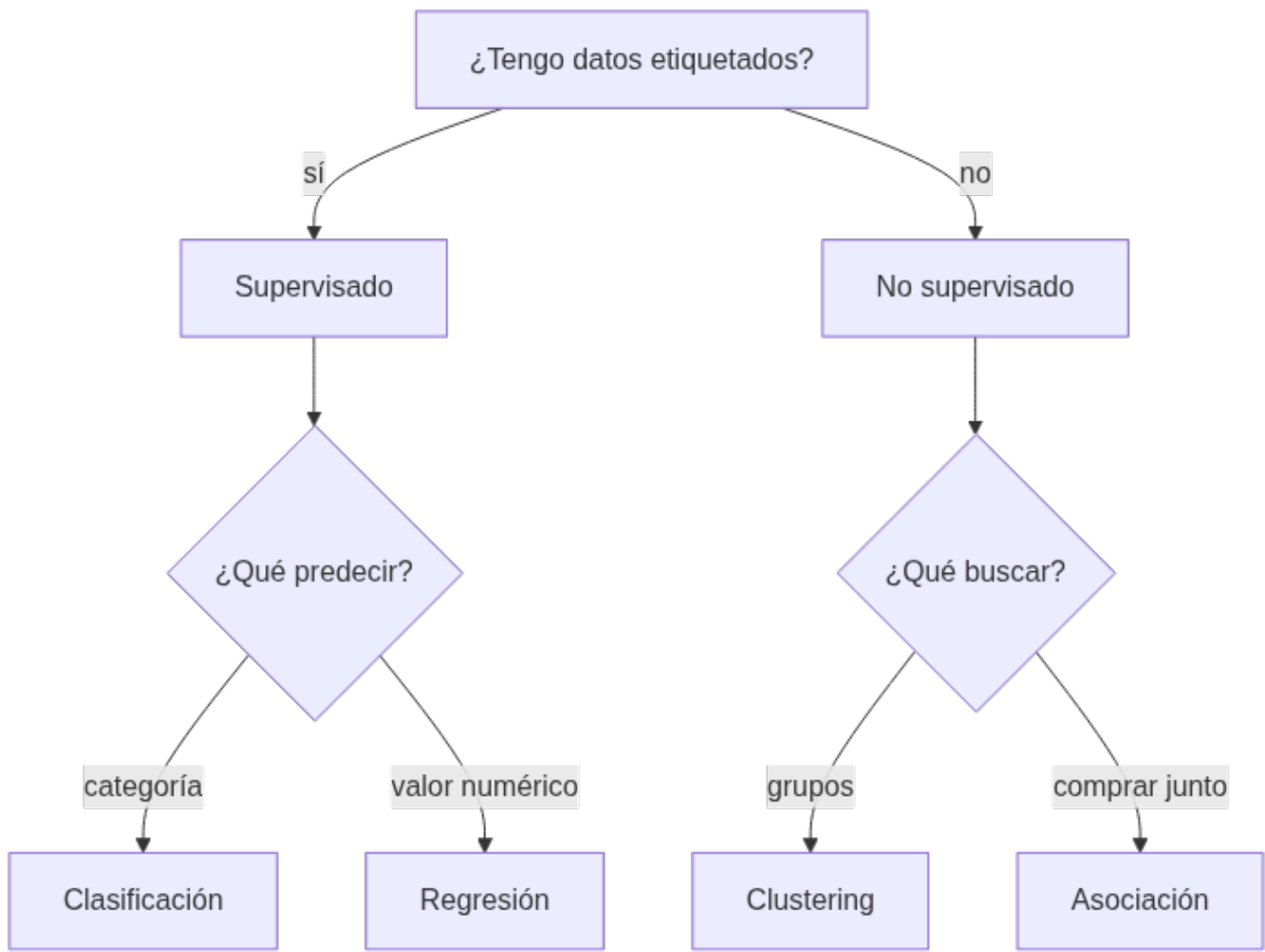
Para predecir el precio de una casa: $\text{Precio} = A \cdot \text{superficie} + B \cdot \text{habitaciones} - C \cdot \text{edad} + \text{base}$. El objetivo del ML es encontrar los valores de *A*, *B*, *C* y *base* que minimizan el error con las ventas conocidas.

4.2 Tipos de aprendizaje

Tipo	Datos	Objetivo	Ejemplos
Supervisado	Etiquetados (con respuesta)	Predecir la respuesta de datos nuevos	Clasificación (spam/no spam), regresión (precio)
No supervisado	Sin etiquetas	Encontrar estructura oculta	Clustering (segmentar clientes), asociación, reducción de dimensionalidad
Refuerzo (RL)	Interacción con el entorno	Maximizar recompensa por prueba-error	Robots, juegos, control
Semi-supervisado	Muchos sin etiquetar + pocos etiquetados	Combinar ambos	Cuando etiquetar es caro
Auto-supervisado	Sin etiquetas humanas	Aprender de la propia estructura (p. ej. ocultar palabras y predecirlas)	Entrenamiento de LLM

- **Supervisado** → clasificación (categorías) y regresión (valores continuos). Algoritmos típicos: árboles de decisión, k-vecinos (KNN), naive Bayes, SVM, regresión logística, random forest.
- **No supervisado** → clustering (k-means, DBSCAN), detección de anomalías, PCA.

4.2.1 ¿QUÉ ALGORITMO ELEGIR?



Un vistazo a los algoritmos más usados (todos disponibles en `scikit-learn`):

Algoritmo	Tipo	Cómo funciona (resumen)	Ejemplo de uso
k-vecinos (KNN)	Supervisado	Clasifica según los k ejemplos más cercanos	Recomendar producto similar
Árbol de decisión	Supervisado	Serie de preguntas <code>si/entonces</code> aprendidas de los datos	Decidir aprobar un préstamo
Naïve Bayes	Supervisado	Probabilidades con supuesto de independencia	Clasificar spam / análisis de sentimiento
Regresión logística	Supervisado	Probabilidad de pertenecer a una clase	Predecir abandono de cliente
Random forest	Supervisado	Combinación de muchos árboles	Mantenimiento predictivo
k-means	No supervisado	Agrupar en k grupos por cercanía al centro	Segmentar clientes
DBSCAN	No supervisado	Agrupar por densidad, detecta anomalías	Detectar transacciones atípicas

Primer contacto con scikit-learn

Todos estos algoritmos comparten la misma interfaz `fit / predict`, y se evalúan dividiendo los datos en **entrenamiento y prueba** (`train_test_split`). Por ejemplo, para clasificar especies con el dataset *Iris*:

```
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier
4
5 X, y = load_iris(return_X_y=True)
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
7
8 modelo = DecisionTreeClassifier()
9 modelo.fit(X_train, y_train) # entrena con los datos etiquetados
10 precision = modelo.score(X_test, y_test) # evalúa con datos nuevos
11 print("Precisión:", precision)
```

Este mismo patrón (`fit` → `score`) se repetirá en las prácticas de la UD02 y siguientes; en esta unidad solo necesitas reconocerlo.

4.3 Aprendizaje profundo (deep learning, DL)

El DL usa **redes neuronales artificiales con muchas capas** (entrada, capas ocultas, salida). Cada conexión tiene un **peso** y cada neurona una **activación no lineal**: la red puede modelar patrones muy complejos. Se entrena con **backpropagation + gradiente descendente** y necesita **muchos datos y GPU**.

- **CNN**: redes convolucionales, especializadas en imágenes (visión).
- **RNN/LSTM**: redes recurrentes, pensadas para secuencias (texto, series temporales).
- **Transformers** (2017): arquitectura con **mecanismo de atención**; base de los LLM y de casi toda la IA moderna.
- **Mamba** (2023): arquitectura alternativa basada en *state space models*.

Ventajas del DL: aprende representaciones complejas sin extraer características a mano. Inconvenientes: necesita muchos datos y cómputo, y es menos explicable.

4.4 IA generativa

La **IA generativa** (gen AI) crea contenido nuevo (texto, imágenes, vídeo, audio) a partir de un **prompt**. Funciona en tres fases:

1. **Entrenamiento**: se crea un **modelo de base** (foundation model) entrenado con enormes volúmenes de datos (texto de internet, imágenes...). Los **LLM** (ChatGPT, GPT-4, BERT, Llama, Gemini) son el ejemplo más común.
2. **Ajuste (tuning)**: se adapta a la tarea con *fine-tuning* o *RLHF* (refuerzo con feedback humano).
3. **Generación y evaluación**: se genera y se mejora de forma continua; técnicas como **RAG** conectan el modelo a fuentes de datos externas para mayor precisión.

Agentes de IA / agentic AI: programas autónomos que **diseñan su propio flujo de trabajo** y usan herramientas (apps, servicios) para lograr objetivos. Es el paso natural después de la IA generativa: no solo responden, **actúan**.

5. Campos de aplicación de la IA (RA1-b)

Campo	Ejemplos de aplicación	Beneficio típico
Industria y logística	Mantenimiento predictivo, optimización de rutas, control de calidad visual	Menos paradas, menos costes
Salud	Diagnóstico asistido por imagen, triaje de pacientes, descubrimiento de fármacos	Mayor precisión, menos errores
Finanzas y banca	Detección de fraude, scoring crediticio, atención al cliente	Menos pérdidas, más velocidad
Comercio y retail	Recomendación de productos, previsión de demanda, chatbots de venta	Más ventas, menos stock roto
Marketing	Segmentación de clientes, análisis de sentimiento, personalización	Campañas más rentables
Educación	Tutoría adaptativa, corrección automática, análisis de abandono	Menos abandono, más personalización

Campo	Ejemplos de aplicación	Beneficio típico
Administración	Automatización de trámites, asistentes virtuales	Menos tiempos y costes
Transporte	Conducción asistida, gestión del tráfico, mantenimiento de flotas	Seguridad y eficiencia
Recursos humanos	Cribado de CV, emparejado con ofertas, entrevistas asistidas	Menos tiempo de contratación

5.1 La IA en la vida cotidiana

Muchas tecnologías que usas a diario son IA, aunque no lo parezca (Parlamento Europeo):

Uso cotidiano	Cómo interviene la IA
Compras online y publicidad	Recomendaciones personalizadas según búsquedas y compras; optimización de inventario y logística
Buscadores	Aprenden de los datos para devolver resultados relevantes
Asistentes de voz	Responden, recomiendan y organizan rutinas
Traducción automática	Traduce texto y voz; subtítulos automáticos
Hogar y ciudad inteligente	Termostatos que aprenden del comportamiento; regulación del tráfico
Coches	Asistentes de seguridad, navegación
Ciberseguridad	Detecta y combate ciberataques reconociendo patrones
Desinformación	Detecta noticias falsas analizando fuentes y lenguaje
Agricultura	Monitoriza animales, riego y fertilizantes para reducir costes e impacto ambiental
Administración pública	Alertas tempranas de catástrofes, trámites automatizados

5.2 Casos de estudio con beneficio medible

Caso 1 · Detección de fraude en banca (RA1-b, RA1-c, RA1-d) Un banco entrena un modelo de ML con **datos históricos etiquetados** (transacción fraudulenta o no). El modelo analiza en tiempo real importe, ubicación, horario y hábitos de cada cliente. *Antes:* revisión manual posterior a la pérdida. *Después:* alerta inmediata y bloqueo preventivo. **Mejora operativa:** menos pérdidas económicas y menor fricción para clientes legítimos.

Caso 2 · Mantenimiento predictivo en industria (RA1-b, RA1-c) Sensores en máquinas envían datos (temperatura, vibración, horas de uso). Un modelo de ML predice cuándo fallará un equipo antes de que ocurra. *Antes:* averías inesperadas y paradas de línea. *Después:* sustitución programada. **Mejora operativa:** menos paradas, menos costes de reparación, mayor disponibilidad.

Caso 3 · Atención al cliente con PLN (RA1-c, RA1-d) Un **chatbot** con PLN responde consultas frecuentes (estado de pedido, devoluciones) y deriva las complejas a personas. *Antes:* colas y horario limitado. *Después:* atención 24/7 e inmediata. **Mejora operativa:** menor coste por consulta, más satisfacción, agentes dedicados a casos difíciles.

Caso 4 · Salud: detección precoz (RA1-b) Un programa con IA analiza llamadas de emergencia para **reconocer un paro cardíaco** durante la llamada, más rápido que un operador humano; también se usa visión para detectar infecciones en TAC pulmonar. **Mejora operativa:** menor tiempo de respuesta y mayor precisión en el diagnóstico.

6. Nuevas formas de interacción y eficiencia operativa (RA1-d)

6.1 Nuevas interacciones

Interacción	Qué es	Ejemplo
Asistente virtual	Responde por voz o texto a peticiones del usuario	Siri, Alexa, asistentes corporativos
Chatbot	Conversación automatizada en web/mensajería	Chat de soporte de una tienda online

Interacción	Qué es	Ejemplo
Interacción por voz	Transcribe y analiza audio	Transcripción de reuniones, análisis de llamadas
Interacción por visión	Lee e interpreta imágenes/vídeo	Lectura de documentos, control de accesos, inspección visual
Agentes autónomos	Ejecutan tareas y usan herramientas por sí solos	Reservar vuelo, tramitar una incidencia

EL PLN DETRÁS DE LAS INTERACCIONES POR TEXTO Y VOZ

El **procesamiento del lenguaje natural (PLN)** es la técnica de IA que permite entender y generar lenguaje humano — se estudia a fondo en la UD03. Detrás de un chatbot, un asistente de voz o un análisis de opiniones hay varias tareas de PLN:

Tarea de PLN	Qué hace	Ejemplo
Análisis de sentimiento	Determina si un texto es positivo, negativo o neutro	Valorar opiniones de clientes
Reconocimiento de entidades (NER)	Identifica nombres, lugares, fechas	Extraer <code>Maria</code> , <code>Madrid</code> , <code>05/05</code> de un texto
Clasificación de texto	Asigna un texto a una categoría	Spam, tipo de incidencia, idioma
Traducción automática	Convierte texto de un idioma a otro	Documentos, subtítulos
Resumen de texto	Condensa documentos largos	Resumir informes y artículos
Reconocimiento de voz	Convierte audio en texto	Transcripción de llamadas

Un *pipeline* de PLN típico es: **preprocesado** (tokenización, minúsculas, eliminación de palabras vacías, lematización) → **extracción de características** (bolsa de palabras, TF-IDF, embeddings) → **modelo** (ML o redes profundas, p. ej. *transformers* como BERT) → **salida** (clasificación, traducción, respuesta). En el notebook de la unidad verás un ejemplo de sentimiento con `scikit-learn`.

6.2 Eficiencia operativa

La IA mejora la **eficiencia operativa** cuando reduce **costes, tiempos o errores** en un proceso. Indicadores que suelen mejorar:

Caso	Antes (sin IA)	Después (con IA)
Atención al cliente	Colas, horario limitado	Chatbot 24/7, respuesta inmediata
Inspección de calidad	Revisión manual, errores	Visión artificial: más rápida y constante
Previsión de demanda	Roturas de stock o excedentes	Predicción con ML: menos pérdidas
Detección de fraude	Revisión posterior	Detección en tiempo real

6.3 CÓMO MEDIR LA MEJORA (KPIs)

Para saber si una solución de IA merece la pena, se compara el proceso **antes y después** con indicadores objetivos:

Indicador	Qué mide
Tiempo de ciclo	Cuánto tarda una operación (respuesta, inspección, trámite)
Coste unitario	Coste por consulta, por pedido, por transacción
Tasa de error	Errores humanos o del proceso por cada N operaciones
Disponibilidad	Horas al día en que el servicio está operativo
Rendimiento	Operaciones por unidad de tiempo o por empleado

Según el tipo de proceso, se usan KPIs específicos que conviene conocer por su nombre, porque aparecen en informes y ofertas de las empresas de IA:

Ámbito	KPI	Qué mide
Atención al cliente	FCR (<i>first contact resolution</i>)	% de consultas resueltas en el primer contacto
Atención al cliente	AHT (<i>average handling time</i>)	Tiempo medio de gestión de una consulta
Atención al cliente	Containment rate	% de consultas cerradas por la IA sin intervención humana
Industria	OEE (<i>overall equipment effectiveness</i>)	Disponibilidad × rendimiento × calidad de una máquina
Industria	MTBF (<i>mean time between failures</i>)	Tiempo medio entre dos averías
Operaciones	Cycle time	Tiempo total de una operación de principio a fin
Operaciones	Coste por documento	Coste de procesar cada documento (contrato, factura, email)



KPI ≠ mejora anecdótica

Decir "el chatbot funciona muy bien" no sirve para justificar una inversión. Lo que vale es un KPI comparado: "la tasa de resolución en el primer contacto pasó de X a Y" o "el tiempo medio de gestión bajó de 5 minutos a 2". En el ejemplo siguiente se hace exactamente eso.

Ejemplo resuelto: un centro de atención recibe 1.000 consultas/día, cada una cuesta 3 € y tarda 5 min. Un chatbot resuelve el 60 % de las consultas en 10 segundos y a un coste de 0,10 €.

- Coste diario antes: $1.000 \times 3 \text{ €} = \mathbf{3.000 \text{ €}}$.
- Coste diario después: $600 \times 0,10 \text{ €} + 400 \times 3 \text{ €} = 60 \text{ €} + 1.200 \text{ €} = \mathbf{1.260 \text{ €}}$.
- **Reducción de coste** $\approx 58 \%$ y de tiempo medio de atención de 5 min a ~ 2 min.

Este tipo de análisis (identificar indicador → estimar antes/después → decidir) es exactamente lo que se practica en el taller 3.

6.4 De la técnica al beneficio

Técnica de IA	Ejemplo de uso	Mejora operativa típica
Clasificación (supervisado)	Priorizar incidencias, cribar CV	Menos tiempo y errores de clasificación
Regresión (supervisado)	Prever demanda, precios	Menos stock roto, mejor fijación de precios
Clustering (no supervisado)	Segmentar clientes	Campañas más rentables
Detección de anomalías	Fraude, averías	Menos pérdidas y paradas
PLN	Chatbots, sentimiento, resúmenes	Atención 24/7, menos coste por consulta
Visión artificial	Inspección, lectura de documentos	Menos errores y tiempo de inspección
Robótica	Almacén, fabricación	Mayor velocidad y seguridad
Sistemas expertos	Diagnóstico técnico, rutas	Conocimiento experto disponible 24/7
IA generativa / agentes	Redacción, tramitación	Automatización de tareas repetitivas



De identificar a implementar

En esta unidad aprenderás a **identificar** oportunidades (qué técnica encaja y qué mejora aporta). **Implementar** modelos y sistemas es el trabajo de las unidades siguientes.

6.5 Ejemplo guiado: de un problema de negocio a una solución de IA

Para cerrar el bloque, recorreremos juntos los pasos que repetirás en el taller 3. El caso es completamente ficticio para no depender de datos externos.

Problema: una tienda online recibe 200 reclamaciones de devolución al día por email. Hoy, dos personas leen cada correo, lo clasifican a mano y lo enrutan. Tardan unos 8 minutos por correo y el error de clasificación ronda el 15 %.

Paso 1 · Elegir el KPI. Medimos el **tiempo de ciclo** (8 min/reclamación), la **tasa de error** (15 %) y el **coste** (2 personas × jornada completa).

Paso 2 · Identificar la técnica. Clasificar texto en categorías (devolución, cambio, defecto, consulta) es una tarea de **PLN + clasificación supervisada** (p. ej. naive Bayes o un árbol de decisión sobre textos vectorizados).

Paso 3 · Estimar el antes/después. Con un clasificador que resuelve el 70 % de los correos en 30 segundos y el resto lo revisa una persona:

- Tiempo medio por correo: $0,30 \times 0,5 \text{ min} + 0,70 \times 8 \text{ min} \approx \mathbf{5,8 \text{ min}}$ (frente a 8).
- Error de clasificación objetivo: **< 5 %** (frente al 15 %).
- Las dos personas pasan de clasificar a **tratar solo los casos complejos**.

Paso 4 · Decidir y comunicar. El responsable presenta la mejora con los KPIs antes/después y señala los riesgos (correos ambiguos, privacidad de los datos de los clientes según el RGPD).



El papel del ejemplo guiado

Este mismo esquema —*problema* → *KPI* → *técnica* → *antes/después* → *decisión*— es el que define qué es **caracterizar un sistema de IA** (RA1): no hace falta entrenar todavía el modelo, solo identificar correctamente qué técnica encaja y qué mejora operativa aporta.

7. Beneficios, riesgos y ética (visión general)

- **Beneficios** (para valorar la eficiencia operativa): automatización de tareas repetitivas, más y más rápida información de los datos, mejor toma de decisiones, menos errores humanos, disponibilidad 24×7, menor riesgo físico.
- **Riesgos** (se estudian a fondo en UD06): datos con sesgos o manipulación, modelos robados o alterados, fallos operativos (*model drift*), problemas de privacidad y ética. Un ejemplo relevante: la formación de los modelos puede reforzar sesgos de género si los datos de entrenamiento los contienen.
- **Marco normativo:** el **AI Act** (Reglamento UE 2024/1689) clasifica la IA por riesgo y entra en vigor por fases: las prácticas prohibidas desde **2/2/2025**, las sanciones desde **2/8/2025** y la aplicación general desde **2/8/2026**. Un modelo de IA de uso general con **riesgo sistémico** se presume cuando el cómputo acumulado de su entrenamiento supera los **10²⁵ FLOPS** (art. 51). El **RGPD** limita además el uso de datos personales, lo que obliga a equilibrar el entrenamiento masivo con la **minimización de datos**. Todo esto se trabaja en la UD06.
- **IA responsable:** explicabilidad, equidad, robustez, rendición de cuentas, privacidad y cumplimiento normativo (RGPD, AI Act).

8. Puntos clave de la unidad

- Un sistema inteligente **percibe, razona y actúa**, y se caracteriza por autonomía, adaptación y toma de decisiones.
- La IA se clasifica por **tarea** (débil/fuerte), por **escuela** (convencional/computacional) y por **capacidades** (Russell-Norvig, Hintze) — son lentes complementarias, no excluyentes.
- **IA > ML > DL > IA generativa:** todo ML es IA, pero no toda IA es ML.
- Aprendizaje **supervisado** (con etiquetas: clasificación/regresión), **no supervisado** (patrones sin etiquetas: clustering), **por refuerzo** (recompensas), **semi** y **auto-supervisado**.
- El **deep learning** (redes profundas) domina visión, lenguaje y generación, pero requiere datos y cómputo.
- La IA ya está en la vida cotidiana (compras, buscadores, asistentes, traducción, coches, ciberseguridad).
- Las **nuevas interacciones** (chatbots, voz, visión, agentes) mejoran la **eficiencia operativa** reduciendo coste, tiempo o error.

9. Glosario

Término	Definición
IA (inteligencia artificial)	Tecnología que simula capacidades humanas (aprender, razonar, decidir)
Sistema inteligente	Sistema que percibe, razona y actúa para lograr un objetivo
IA débil / fuerte	Orientada a una tarea concreta (toda la actual) frente a IA general al nivel humano (teórica)
IA convencional / computacional	Escuela simbólico-deductiva (reglas explícitas) frente a subsimbólica-inductiva (aprende de datos)
Teoría de la mente / autoconciencia	Niveles teóricos de la clasificación de Hintze, aún sin sistemas reales
ML (machine learning)	Técnicas que aprenden patrones de los datos sin programación explícita
Deep learning	Subconjunto del ML basado en redes neuronales profundas
IA generativa	IA que crea contenido nuevo (texto, imagen, audio)
LLM	Modelo de lenguaje grande (p. ej. ChatGPT) entrenado con enormes volúmenes de texto
Modelo	Resultado del entrenamiento de un algoritmo sobre datos
Entrenamiento	Proceso de ajustar los parámetros del modelo con datos
Generalización	Capacidad del modelo de acertar con datos nuevos
Supervisado	Aprendizaje con ejemplos etiquetados
No supervisado	Aprendizaje que encuentra estructura sin etiquetas
Refuerzo (RL)	Aprendizaje por prueba-error con recompensas
Clasificación	Tarea supervisada de predecir categorías
Regresión	Tarea supervisada de predecir valores continuos
Clustering	Agrupación no supervisada de datos similares
PLN	Procesamiento del lenguaje natural (texto y voz)
Visión por computador	IA que interpreta imágenes y vídeo
Robot	Sistema físico que percibe y actúa en su entorno
Sistema experto	IA basada en reglas de un experto humano
Agente de IA	Programa autónomo que usa herramientas para lograr objetivos
Prompt	Instrucción de texto que se da a un modelo generativo
Eficiencia operativa	Reducción de costes, tiempos o errores en un proceso

10. FAQ

¿Todo lo que usa datos es machine learning? ▾

No. Hay IA que no es ML (sistemas basados en reglas, búsquedas heurísticas). El ML es la familia de técnicas que aprende patrones de los datos; todo ML es IA, pero no toda IA es ML.

? ¿En qué se diferencian IA débil e IA fuerte, y por qué toda la IA actual es débil? ▾

La IA débil resuelve una tarea concreta y no se adapta más allá de lo programado; la fuerte igualaría a una persona en cualquier tarea. La fuerte es un objetivo teórico: ningún sistema actual generaliza tan ampliamente como para considerarse IA general.

? ¿Qué diferencia hay entre clasificación y regresión? ▾

La **clasificación** predice una categoría (spam/no spam); la **regresión** predice un valor numérico continuo (precio de una vivienda).

? ¿Cuándo conviene usar aprendizaje profundo y cuándo no? ▾

Conviene con **muchos datos y tareas complejas** (imágenes, audio, texto). Con datos pequeños o problemas sencillos, un modelo clásico (árbol, KNN, regresión logística) suele bastar y es más explicable.

? ¿Un chatbot entiende de verdad lo que le digo? ▾

Los chatbots modernos con PLN/LLM **procesan estadísticamente el lenguaje** y generan respuestas plausibles, pero no tienen comprensión ni intenciones humanas; pueden equivocarse y conviene verificar sus respuestas.

? ¿La IA va a sustituir a las personas? ▾

La IA **automatiza tareas** concretas, no profesiones enteras. Lo habitual es que cambie el trabajo: las personas se centran en casos complejos y la IA en tareas repetitivas. La pregunta de fondo es de organización y ética, no solo técnica.

? ¿Qué es la IA generativa y en qué se diferencia de la IA predictiva? ▾

La IA **predictiva** estima un resultado (probabilidad, categoría, valor). La IA **generativa** crea contenido nuevo (texto, imagen, audio) a partir de un prompt, usando modelos como los LLM o los modelos de difusión.

11. Planificación sesión a sesión

Semana	Horas	Contenido	CE	Evidencia / actividad
3	3	Fundamentos; escuelas de pensamiento y clasificaciones (débil/fuerte, Russell-Norvig, Hintze)	RA1-a	Ejercicios bloque A, Taller 1
4	3	Campos de aplicación de la IA	RA1-b	Ejercicios bloque C
5	3	Técnicas de la IA (ML, PLN, visión, robótica, sistemas expertos)	RA1-c	Ejercicios bloques B y D, Taller 2
6	3	Nuevas interacciones, eficiencia operativa y KPIs; entrega	RA1-d	Ejercicios bloques E y F, Talleres 3 y 4, notebook demo

12. Tabla final RA/CE

CE	Dónde se trabaja	Con qué se evalúa
RA1-a	\$3	Ejercicios bloque A, Taller 1, Taller 4, prueba del RA1
RA1-b	\$5	Ejercicios bloque C, Taller 1, prueba del RA1

CE	Dónde se trabaja	Con qué se evalúa
RA1-c	\$4, \$6.1	Ejercicios bloques B y D, Taller 2, prueba del RA1
RA1-d	\$6	Ejercicios bloques E y F, Taller 3, prueba del RA1

13. Recursos

- [Diapositivas](#)
- [Ejercicios de la unidad](#)
- Talleres: [T01](#) · [mapa de sistemas inteligentes](#) · [T02](#) · [técnicas en casos reales](#) · [T03](#) · [nuevas interacciones](#) · [T04](#) · [línea del tiempo](#)
- [Actividades guiadas](#) — notebook demo del profesor
- **Notebooks** — el notebook guiado, con descarga y apertura en Colab, en el menú «Notebooks»

 **Referencias de la unidad** ▾

- [IBM](#) · [¿Qué es la IA?](#)
- [IBM](#) · [¿Qué es el machine learning?](#)
- [IBM](#) · [¿Qué es el PLN?](#)
- [scikit-learn](#) · [Aprendizaje supervisado](#)
- [Parlamento Europeo](#) · [¿Qué es la IA y cómo se usa?](#)
- [AI Act \(UE 2024/1689\)](#)
- [Russell y Norvig](#) · [AIMA \(sitio oficial\)](#)
- [Arend Hintze](#) · [«Understanding the four types of AI» \(The Conversation, 2016\)](#)
- [Vaswani et al.](#) · [Attention Is All You Need \(2017\)](#)

14. Evaluación

Peso	Instrumento
40 % actividades	Media de los 4 talleres, con su rúbrica
60 % prueba escrita	Prueba del RA1 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «*en función de la consecución de los RA*»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.

15. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 Orden 8/2025): repetir el análisis del caso real con un caso distinto y las pruebas de autoevaluación de esta unidad.

[Volver al índice](#)

 24 de agosto de 2026

3.2 Diapositivas de la UD01



Diapositivas PDF



Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

🕒 24 de agosto de 2026

3.3 UD01 — Actividades guiadas

Notebook que se trabaja **en clase**, con el profesor. No es un entregable calificable: es la demostración en código de las técnicas de IA vistas en la teoría.

Notebook 1 · Técnicas de IA en acción

Ejemplo de **aprendizaje supervisado, no supervisado** y de cómo medir la mejora operativa con `numpy`, `pandas` y `scikit-learn` — las mismas técnicas del §4 y §6 de la teoría, aplicadas.

Recurso	Enlace
Notebook	UD01_N01_tecnicas_ia.ipynb

[Volver a la UD01 · Ejercicios](#)

 24 de agosto de 2026

4. UD02

4.1 UD02 — Modelos de IA y resolución de problemas

Unidad 2 · 12 h · semanas 7-10 (9 de noviembre al 3 de diciembre)

1. Introducción

En la UD01 aprendiste a **identificar** sistemas de IA y a relacionarlos con la eficiencia operativa. En esta unidad damos un paso más: no solo reconocer la IA, sino **formalizar y resolver problemas con modelos de IA**. Para ello recorremos un camino progresivo:

1. Primero, **qué debe cumplir un sistema** que resuelve problemas con IA, y **cómo se representa un problema** para que una máquina pueda resolverlo (espacio de estados y algoritmos de búsqueda).
2. Después, **cómo se clasifican los modelos de IA** (por aprendizaje, por análisis, por conocimiento frente a datos).
3. A continuación, **qué se puede automatizar** y con qué tecnología (RPA frente a IA, agentes).
4. Luego, las dos familias de razonamiento que tocarás en el laboratorio: **lógica difusa y sistemas basados en reglas**.
5. Por último, **cómo elegir el modelo adecuado** para cada problema (CE f).

El núcleo práctico usa dos librerías de Python que ya están en el stack del curso: `scikit-fuzzy` (lógica difusa) y `experta` (sistemas basados en reglas). La unidad se cierra con **Robocode Tank Royale**: programar un bot que compite en un campo de batalla es, literalmente, implementar un sistema de resolución de problemas de principio a fin.

” Hilo conductor de la unidad

Antes de programar IA hay que saber formalizar un problema y elegir bien el modelo. Lo haremos paso a paso: modelar → clasificar → automatizar → razonar (difuso y por reglas) → decidir → implementar.

2. Resultado de aprendizaje y criterios de evaluación

RA2 — Utiliza modelos de sistemas de Inteligencia Artificial implementando sistemas de resolución de problemas.

CE	Criterio de evaluación
RA2-a	Se han determinado los requisitos básicos a implementar en un sistema de resolución de problemas.
RA2-b	Se han clasificado modelos de Inteligencia Artificial.
RA2-c	Se han caracterizado los modelos de automatización de tareas.
RA2-d	Se han caracterizado los modelos de razonamiento impreciso.
RA2-e	Se han caracterizado los modelos de sistemas basados en reglas.
RA2-f	Se ha valorado la adecuación de los modelos a la implementación del sistema de resolución de problemas.

Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo para este RA dice textualmente:

- *Utilización de modelos de Inteligencia Artificial:*
- *- Requisitos básicos de un sistema de resolución de problemas.*
- *- Modelos de sistemas de Inteligencia Artificial: Automatización de tareas. Sistemas de razonamiento impreciso. Sistemas basados en reglas.*

3. Objetivos de la unidad

Objetivo	Descripción
O1	Enumerar los requisitos básicos de un sistema de resolución de problemas de IA.
O2	Formalizar un problema como espacio de estados (estado inicial, acciones, transición, objetivo).
O3	Aplicar búsqueda en anchura, en profundidad y A* y saber cuándo usar cada una.
O4	Clasificar modelos de IA por paradigma de aprendizaje, por tipo de análisis y por base de conocimiento/datos.
O5	Diferenciar RPA de IA y describir la automatización inteligente y los agentes software.
O6	Explicar la lógica difusa: conjuntos difusos, funciones de pertenencia, reglas, Mamdani/Sugeno y defuzzificación.
O7	Implementar un control difuso sencillo con <code>scikit-fuzzy</code> .
O8	Describir el ciclo reconocer-actuar de un sistema basado en reglas y el papel de CLIPS y de <code>experta</code> .
O9	Implementar un sistema basado en reglas con <code>experta</code> (con el parche de compatibilidad).
O10	Elegir el modelo adecuado a un problema según datos, explicabilidad, coste y tiempo real.
O11	Implementar un sistema de resolución de problemas completo (Robocode) dotando de comportamiento a un bot.

4. Sistema de resolución de problemas (RA2-a)

4.1 Cinco requisitos de un sistema de resolución de problemas

Antes de formalizar un problema concreto, conviene fijar **qué debe cumplir** cualquier sistema de IA que pretenda resolver problemas de forma efectiva y útil:

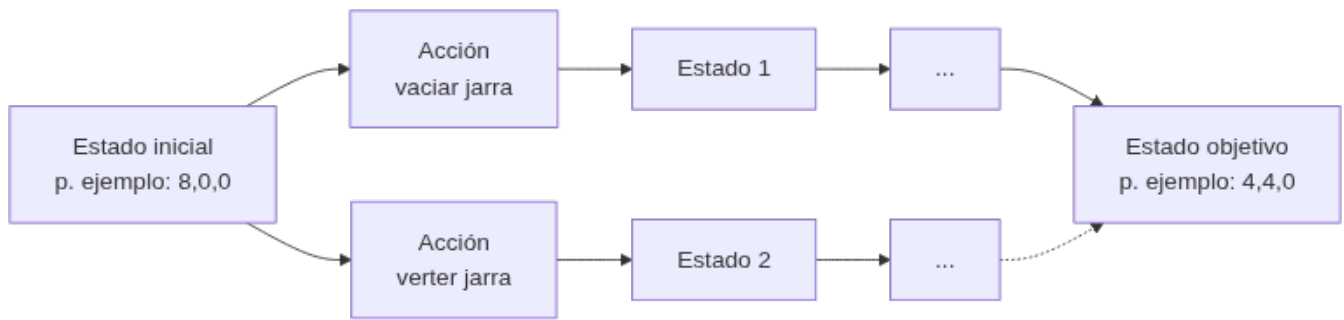
1. **Representación del problema:** elegir una estructura de datos y un modelo que reflejen fielmente el dominio. En PLN, por ejemplo, convertir texto a vectores numéricos; en un puzzle, una matriz de fichas (§4.2).
2. **Razonamiento y toma de decisiones:** aplicar técnicas lógicas, de aprendizaje o de búsqueda heurística para llegar a una solución a partir de la información disponible.
3. **Aprendizaje y adaptabilidad:** mejorar el rendimiento con la experiencia — aprendizaje automático o por refuerzo cuando el problema lo permite.
4. **Eficiencia computacional:** usar algoritmos y estructuras que den respuestas rápidas y escalables (por eso importa tanto BFS/DFS/A*, §4.3).
5. **Interacción con las personas usuarias:** una interfaz comprensible que facilite la comunicación entre el sistema y quien lo usa.

**No son independientes**

Un sistema puede cumplir muy bien 4 requisitos y fallar estrepitosamente por el quinto: un algoritmo de búsqueda óptimo (requisito 4) que nadie sabe usar porque no tiene interfaz (requisito 5) no resuelve el problema en la práctica.

4.2 El espacio de estados

Muchos problemas de IA se resuelven convirtiéndolos en una **búsqueda**: existe un conjunto de configuraciones posibles (los **estados**) y unas **acciones** que pasan de un estado a otro. Ese grafo de estados conectados por acciones es el **espacio de estados**.



Un **sistema de resolución de problemas** necesita, formalmente (base AIMA):

1. **Estado inicial:** la configuración de partida.
2. **Acciones aplicables:** qué movimientos se pueden hacer en cada estado.
3. **Modelo de transición:** función que, dado un estado y una acción, produce el siguiente estado.
4. **Test de objetivo:** cómo saber que hemos llegado a la solución.
5. **Coste de camino:** el coste de cada acción (para encontrar la solución más barata).

Además, se elige la **representación** (cómo codificar cada estado) y la **estrategia de búsqueda** (completa, óptima, memoria).



Regla para recordar

Un problema de IA bien planteado es aquel del que sabes responder: *¿cuál es el estado inicial?, ¿qué acciones puedo aplicar?, ¿cuándo he terminado?* Si no sabes responderlas, el problema no está bien formalizado.

4.3 Ejemplos clásicos de representación

Problema	Representación	Detalle
Jarras 8-5-3	Triple $[jarra8, jarra5, jarra3]$	De $[8,0,0]$ a $[4,4,0]$ en 7 pasos; las acciones son "verter hasta vaciar o llenar"
Misioneros y caníbales	Vector $\langle m, c, barca \rangle$	De $\langle 3,3,1 \rangle$ a $\langle 0,0,0 \rangle$; se descartan los estados donde los caníbales superan a los misioneros
8-reinas	Una columna por reina (permutación de 1..8)	Sin restricciones hay 4.426.165.368 arreglos; con restricciones $8! = 40.320$; solo 92 soluciones
15-puzzle	Matriz 4×4 con las fichas	$16!/2 \approx 10,46 \cdot 10^{12}$ estados (la mitad alcanzable); soluciones óptimas de hasta 80 movimientos



La explosión combinatoria

Estos problemas "de juguete" sirven para medir algo esencial: el número de estados crece **exponencialmente** con el tamaño (factor de ramificación b elevado a profundidad d). Un espacio de estados real (rutas de reparto, planificación de tareas) es enorme: por eso hace falta una **estrategia de búsqueda** y, cuando se puede, una **heurística**.

4.4 Algoritmos de búsqueda: BFS, DFS y A*

Criterio	Búsqueda en anchura (BFS)	Búsqueda en profundidad (DFS)	A*
Estructura	Cola (FIFO)	Pila (LIFO)	Cola de prioridad por $f = g + h$
Completa	Sí	No (en espacios infinitos)	Sí
Óptima	Sí (coste unitario)	No	Sí (heurística admisible)
Memoria	$O(b^d)$	$O(b \cdot d)$	$O(b^d)$

Criterio	Búsqueda en anchura (BFS)	Búsqueda en profundidad (DFS)	A*
Cuándo usar	Camino más corto en número de pasos	Exploración exhaustiva, backtracking	Ruta óptima con costes variados y buena heurística

- **BFS** explora por niveles: encuentra el camino más corto en pasos, pero consume mucha memoria (guarda todos los estados del nivel).
- **DFS** baja hasta el fondo: usa poca memoria, pero puede no encontrar la solución óptima (o no terminar en espacios infinitos). Es la base del *backtracking* (8-reinas).
- **A*** combina el coste acumulado $g(n)$ con una **heurística** $h(n)$ (una estimación de lo que falta): $f(n) = g(n) + h(n)$. Con una heurística admisible (que no sobreestime), A* es completo y óptimo. Se usa mucho en videojuegos, mapas y planificación de rutas.



Ejemplo de heurística

En un puzzle 8-puzzle (3×3), la heurística "número de fichas mal colocadas" o la "distancia de Manhattan" (suma de pasos horizontales+verticales de cada ficha hasta su posición correcta) estiman cuánto queda. Con esa h , A* guía la búsqueda hacia el objetivo en lugar de explorar a ciegas.



Regla práctica (Red Blob Games)

"Usa el algoritmo más simple que puedas": si todos los costes son iguales y solo quieres el camino más corto en pasos, BFS basta. Si los costes varían, Dijkstra. Si apuntas a un único objetivo, prefiere A* con la heurística más simple posible (p. ej. Manhattan en rejillas).

5. Clasificación de modelos de IA (RA2-b)

5.1 Por paradigma de aprendizaje

Criterio	Supervisado	No supervisado	Refuerzo
Datos	Etiquetados (ground truth)	Sin etiquetas	Estados, acciones, recompensas
Objetivo	Predecir la salida correcta	Descubrir patrones	Maximizar recompensa acumulada
Tareas	Clasificación, regresión	Clustering, asociación, reducción de dim.	Control, juegos, robótica
Ejemplos	Spam, precio de un coche	Segmentación de clientes (k-means)	Robot que aprende a andar, AlphaGo
Algoritmos (scikit-learn)	Regresión logística, SVM, árboles	k-means, DBSCAN, PCA	(fuera del stack estándar)



Recordatorio de la UD01

Todo ML es IA, pero no toda IA es ML. Los sistemas basados en reglas o en lógica difusa son IA, pero no aprenden de datos. En esta unidad estudiamos precisamente esos modelos "no-ML".

5.2 Por tipo de análisis

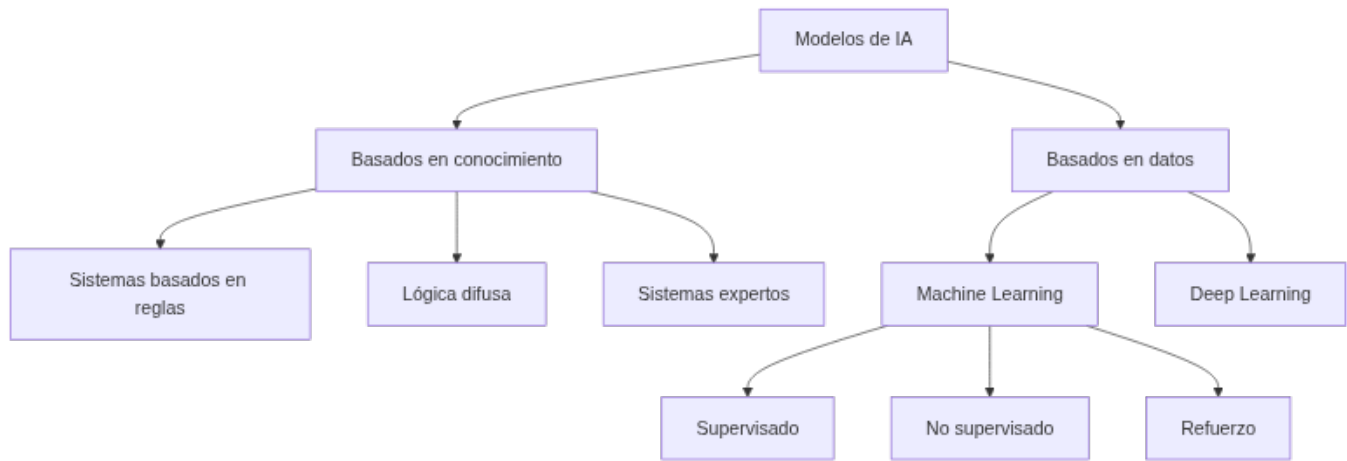
Fase	Pregunta	Ejemplo
Descriptivo	¿Qué pasó?	Informe de ventas
Predictivo	¿Qué pasará?	Previsión de demanda

Fase	Pregunta	Ejemplo
Prescriptivo	¿Qué hacer?	Qué precio fijar y qué ocurre si lo cambio

El **prescriptivo** es el que aporta más valor: no solo predice, **recomienda o ejecuta** la decisión óptima (Gartner lo llama "la última frontera" de la analítica).

5.3 Basados en conocimiento frente a basados en datos

Criterio	Basados en conocimiento	Basados en datos (ML/DL)
Conocimiento	Explícito (reglas IF-THEN)	Implícito (aprendido de los datos)
Ejemplos	Sistemas expertos, lógica difusa, termostato	Árboles, regresión, redes neuronales
Ventajas	Interpretable, fácil de mantener, explica el razonamiento	Flexible, escala a tareas complejas
Limitaciones	Frágil en tareas complejas; adquisición de conocimiento difícil	Requiere datos y cómputo; caja negra
Requisito	Experto + ingeniero del conocimiento	Datos etiquetados suficientes y representativos



🔥 Dos formas de resolver el mismo problema

Un termostato con reglas `SI temperatura < 18 ENTONCES calefacción ON` es IA basada en conocimiento (reglas). Un sistema que aprende de datos históricos qué temperatura poner según la hora y la estación es ML. La elección depende del problema (lo veremos en el CE f).

6. Modelos de automatización de tareas (RA2-c)

6.1 RPA frente a IA

Criterio	RPA	IA
Qué hace	Hace: replica tareas humanas repetitivas en la interfaz (GUI, formularios)	Piensa/aprende: reconoce patrones, decide, mejora con los datos
Base	Procesos (<i>process-driven</i>), reglas predefinidas	Datos (<i>data-driven</i>), modelos
Se adapta	No: los flujos hay que mantenerlos si cambia el sistema	Sí: aprende de la experiencia
Ejemplo	Un robot que copia datos de un email a un ERP	Un modelo que clasifica si el email es urgente o no



RPA no es IA

La distinción de IBM es clara: RPA **automatiza procesos** replicando acciones en pantalla; la IA **razona sobre datos**. Se complementan: la IA da decisión y el RPA ejecuta. Cuando se combinan IA + BPM (orquestración de flujos) + RPA (ejecución) hablamos de **automatización inteligente (IA)**.

6.2 Agentes software

Un **agente de IA** es un programa que percibe su entorno y actúa para lograr un objetivo. Se ordenan de más simple a más avanzado:

Tipo de agente	Cómo decide	Ejemplo
Reflejo simple	Reglas si... entonces... directas	Termostato
Reflejo con modelo	Mantiene un estado interno del mundo	Robot aspiradora que "recuerda" la habitación
Basado en objetivos	Busca y planifica para alcanzar una meta	Navegador que elige una ruta
Basado en utilidad	Maximiza una utilidad (coste, tiempo, riesgo)	Ruta que minimiza peajes y tiempo
Con aprendizaje	Aprende de la experiencia	Motor de recomendación

6.3 Tareas cognitivas automatizables

La IA automatiza **tareas cognitivas** que antes requerían juicio humano:

- **Extracción:** leer documentos (OCR), extraer entidades (NER), capturar datos de facturas.
- **Clasificación:** spam, incidencias, prioridades. Los **sistemas de reconocimiento de voz** (Google Speech-to-Text, Microsoft Azure Speech Service) son otro caso de automatización de tareas: convierten habla en texto y automatizan la transcripción o los comandos de voz.
- **Generación:** redactar respuestas, resúmenes o informes (IA generativa).



Caso con cifra (ilustrativo)

En automatización de back-office, un sistema que extrae datos de emails y mensajería puede reducir el cierre contable mensual de varios días a uno, con menor tasa de error. Las cifras concretas dependen de cada despliegue; en la práctica se mide con KPIs (tiempo de ciclo, coste por documento, tasa de error) como vimos en la UD01. Otro ejemplo habitual es la **detección de fraude**: algoritmos de aprendizaje automático que analizan patrones en transacciones financieras para señalar operaciones sospechosas, reduciendo la carga de revisión manual.

7. Razonamiento impreciso: lógica difusa (RA2-d)

7.1 De lo booleano a lo difuso

La lógica clásica solo admite **verdadero/falso** (0/1). La **lógica difusa** (Zadeh, 1965) permite grados de verdad **entre 0 y 1**: algo puede ser *parcialmente verdadero*. Modela la **vaguedad** del lenguaje humano (la probabilidad, en cambio, modela la incertidumbre). Es la familia de modelos que se usa cuando los datos son **incompletos o inciertos** y aun así hay que decidir algo razonable.

Variable	Valor clásico	Valor difuso
Temperatura	"18 °C es frío" (sí/no)	"18 °C es frío con 0,7"
Velocidad	Rápida o no	"Rápida con 0,6, media con 0,4"

7.2 Conjuntos difusos y funciones de pertenencia

Un **conjunto difuso** asigna a cada valor un **grado de pertenencia** $\mu(x) \in [0,1]$ mediante una **función de pertenencia**. Las más usadas:

Función	Forma	Uso típico
Triangular (<code>trimf</code>)	Pico en un punto	Variables simples (p. ej. "media")
Trapezoidal (<code>trapmf</code>)	Meseta central	Intervalos ("normal")
Gaussiana (<code>gaussmf</code>)	Campana suave	Variables continuas suaves
Sigmoidal (<code>sigmf</code>)	Escalón suave	Extremos ("frío", "caliente")



Diseño de variables

No importa tanto la forma exacta como el **número y la posición** de las funciones: se recomiendan entre **3 y 7 curvas** por variable, solapadas para que no existan "huecos".

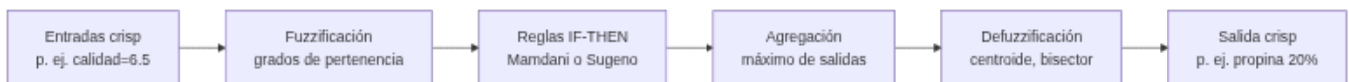
7.3 Operaciones difusas

Zadeh definió las operaciones básicas sobre grados de pertenencia:

- **AND (intersección)**: mínimo de los grados ($\mu_A \wedge \mu_B = \min(\mu_A, \mu_B)$).
- **OR (unión)**: máximo ($\mu_A \vee \mu_B = \max(\mu_A, \mu_B)$).
- **NOT (complemento)**: $1 - \mu$.

En `scikit-fuzzy`, las reglas usan por defecto `fmin / fmax` para AND/OR (se pueden cambiar a producto/suma difusa).

7.4 El sistema de inferencia difuso (FIS)



1. **Fuzzificar**: convertir cada entrada numérica en grados de pertenencia.
2. **Aplicar reglas** SI calidad es mala O servicio es pobre ENTONCES propina es baja.
3. **Agregar**: combinar la salida de todas las reglas.
4. **Defuzzificar**: obtener un número nítido. Métodos típicos: **centroide** (centro de gravedad), **bisector**, **MOM/SOM/LOM**.

Hay dos familias de sistemas: **Mamdani** (la salida es un conjunto difuso que luego se defuzzifica; la implementa `scikit-fuzzy`) y **Sugeno/TSK** (la salida de cada regla es una función $z = f(x, y)$, más eficiente para control).

7.5 Aplicaciones reales

- **Metro de Sendai (Japón, 1987)**: control difuso de aceleración y frenado.
- **Autofocus de Canon**: 12 entradas (claridad y velocidad del objetivo), **13 reglas**, solo **1,1 KB** de memoria.
- **ABS** (frenos): estima el agarre con lógica difusa sobre la temperatura del freno.
- **Electrodomésticos**: lavadoras Hitachi (peso/suciedad → programa), vacuolas Matsushita.
- **Control industrial**: hornos de cemento (Dinamarca, 1976), aires acondicionados Mitsubishi (calienta/enfría 5× más rápido con un 24 % menos de consumo).
- **Control de tráfico**: ajuste de los tiempos de los semáforos en función del flujo vehicular en tiempo real, para reducir la espera de los vehículos.
- **Apoyo al diagnóstico médico**: evaluación de síntomas cuando los resultados de las pruebas no son concluyentes, para dar una valoración preliminar que considere varios factores a la vez.

7.6 Práctica con `scikit-fuzzy`

`pip install -U scikit-fuzzy` (versión 0.5.0, 2024). Ejemplo clásico: el **problema de la propina** — la propina depende de la calidad del servicio y de la comida.

```

1  import numpy as np
2  import skfuzzy as fuzz
3  from skfuzzy import control as ctrl
4
5  # 1. Variables (universo de discurso)
6  calidad = ctrl.Antecedent(np.arange(0, 11, 1), 'calidad')
7  servicio = ctrl.Antecedent(np.arange(0, 11, 1), 'servicio')
8  propina = ctrl.Consequent(np.arange(0, 26, 1), 'propina')
9
10 # 2. Funciones de pertenencia
11 calidad.automf(3, names=['mala', 'acceptable', 'excelente'])
12 servicio.automf(3, names=['pobre', 'acceptable', 'bueno'])
13 propina['baja'] = fuzz.trimf(propina.universe, [0, 5, 10])
14 propina['media'] = fuzz.trimf(propina.universe, [10, 15, 20])
15 propina['alta'] = fuzz.trimf(propina.universe, [15, 20, 25])
16
17 # 3. Reglas
18 regla1 = ctrl.Rule(calidad['mala'] | servicio['pobre'], propina['baja'])
19 regla2 = ctrl.Rule(servicio['acceptable'], propina['media'])
20 regla3 = ctrl.Rule(servicio['bueno'] | calidad['excelente'], propina['alta'])
21
22 # 4. Sistema y simulación
23 sistema = ctrl.ControlSystem([regla1, regla2, regla3])
24 sim = ctrl.ControlSystemSimulation(sistema)
25 sim.input['calidad'] = 6.5
26 sim.input['servicio'] = 9.8
27 sim.compute()
28 print(f"Propina sugerida: {sim.output['propina']:.2f}%")
29 # Salida esperada: Propina sugerida: ~20% (el valor exacto depende del universo)

```

⚠ Compatibilidad de la librería

`scikit-fuzzy` es una librería **semi-mantenida** (última versión 2024). Para los ejercicios del curso funciona sin problemas, pero conviene fijar la versión (`scikit-fuzzy==0.5.0`) en el entorno reproducible del contenedor.

8. Sistemas basados en reglas (RA2-e)

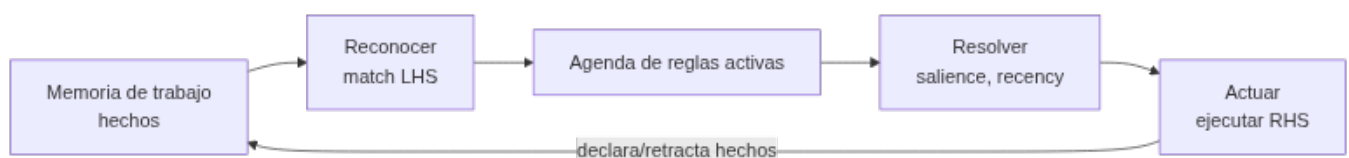
8.1 El ciclo reconocer-actuar

Un **sistema de producción** (o sistema basado en reglas) tiene:

- **Memoria de trabajo:** los hechos conocidos en cada momento.
- **Base de conocimiento:** las reglas SI <condiciones> ENTONCES <acciones>.
- **Motor de inferencia:** ejecuta el ciclo **reconocer-actuar**.

El ciclo se repite hasta que no queden reglas aplicables:

1. **Reconocer (match):** se comparan los hechos de la memoria de trabajo con las condiciones de las reglas; las reglas cuyas condiciones se cumplen van a la **agenda**.
2. **Resolver (resolve):** si hay varias reglas activas, se elige una según prioridad (*salience*), frescura de los hechos o especificidad.
3. **Actuar (act):** se ejecuta la parte ENTONCES, que modifica la memoria de trabajo (declarar/retractar hechos) y el ciclo se repite.



Los motores industriales usan el **algoritmo RETE** (Forgy, 1974) para no reevaluar todas las reglas cada vez: construye una red de filtrado en memoria.

8.2 Encadenamiento hacia delante y hacia atrás

- **Hacia delante (forward chaining, data-driven):** parte de los hechos y deduce hechos nuevos. Ideal para monitorización, planificación y entornos dinámicos. Es el que usan CLIPS y `experta`.
- **Hacia atrás (backward chaining, goal-driven):** parte de una **meta** y busca qué hechos la sustentan (preguntando al usuario solo lo necesario). Ideal para diagnóstico (MYCIN, Prolog).

8.3 CLIPS y experta

CLIPS (*C Language Integrated Production System*) es el motor de reglas de referencia, desarrollado por la NASA (1985-1996) y de dominio público desde 1996. `experta` es una librería de Python **inspirada en CLIPS** (fork de `pyknow`) que implementa un motor RETE con sintaxis nativa de Python.

8.4 Ejemplos cotidianos de sistemas basados en reglas

Antes de los grandes sistemas expertos históricos (§8.6), los sistemas basados en reglas están en aplicaciones que usas a diario:

- **Sistemas de recomendación:** plataformas de comercio electrónico como Amazon sugieren productos relacionados aplicando reglas sobre el historial de compras y el comportamiento del usuario.
- **Soporte al diagnóstico médico:** en asistencia remota, reglas basadas en los síntomas declarados dan una recomendación preliminar antes de que la persona sea atendida por un profesional.

Se usan mucho porque el razonamiento es **transparente**: las reglas son explícitas y una persona puede leer exactamente por qué el sistema tomó esa decisión — al contrario que una red neuronal.

8.5 Práctica con experta

⚠ Compatibilidad con Python 3.10+ (obligatorio)

`experta` (1.9.4, de 2019) fija `frozendict==1.2`, que usa `collections.Mapping`, eliminado en Python 3.10. Sin parche, `from experta import *` falla. Dos soluciones: - **En el código** (más didáctico): añadir el monkey-patch antes del import. - **En el entorno** (más limpio): `pip install "frozendict>=2.3" + pip install --no-deps experta + pip install schema==0.6.7`.

```
1 # PARCHE para Python 3.10+ (issue #34 de experta)
2 import collections
3 import collections.abc
4 if not hasattr(collections, 'Mapping'):
5     collections.Mapping = collections.abc.Mapping
6     collections.Iterable = collections.abc.Iterable
7     collections.MutableMapping = collections.abc.MutableMapping
8
9 from experta import *
10
11 class DiagnosticoVehiculo(KnowledgeEngine):
12     @DefFacts()
13     def inicio(self):
14         yield Fact(accion="diagnosticar")
15
16     @Rule(Fact(accion="diagnosticar"), salience=10)
17     def arrancar(self):
18         print("Iniciando diagnóstico del vehículo...")
19         self.declare(Fact(bateria="descargada"))
20         self.declare(Fact(luces="no_encienden"))
21
22     @Rule(Fact(bateria="descargada"), Fact(luces="no_encienden"))
23     def bateria(self):
24         self.declare(Fact(causa="bateria_descargada"))
25
26     @Rule(Fact(causa="bateria_descargada"))
27     def resultado(self):
28         print("CAUSA PROBABLE: Batería descargada")
29
30 engine = DiagnosticoVehiculo()
31 engine.reset() # activa DefFacts e InitialFact
32 engine.run() # ejecuta el ciclo reconocer-actuar
```

Salida esperada:

```
1 Iniciando diagnóstico del vehículo...
2 CAUSA PROBABLE: Batería descargada
```

8.6 Sistemas expertos reales

Sistema	Origen	Qué hacía	Dato relevante
MYCIN	Stanford (1972)	Diagnóstico de infecciones y antibióticos (backward chaining + factores de certeza)	~500-600 reglas; acierto del 65-70 % frente a facultativos

Sistema	Origen	Qué hacía	Dato relevante
XCON/R1	CMU/DEC (1978)	Configuración de ordenadores VAX	Ahorró a DEC del orden de 25 M\$/año
SID	DEC (años 80)	Diseño de puertas de la CPU del VAX 9000	Generó el 93 % de las puertas lógicas
Dendral	Stanford (años 70)	Identificación de moléculas orgánicas	Primer sistema experto "completo"



Cuándo usar reglas y cuándo ML

Usa **reglas** cuando el dominio esté acotado, la explicabilidad sea exigible (auditoría, normativa) o no haya datos. Usa **ML** cuando haya datos abundantes y patrones complejos. Una práctica habitual es el **híbrido**: reglas como guardarraíl/validación y ML para la decisión fina.

9. Adecuación del modelo (RA2-f)

9.1 Criterios para elegir modelo

Criterio	Pregunta guía
Datos disponibles	¿Hay datos? ¿Etiquetados? ¿Representativos?
Explicabilidad	¿Necesito justificar la decisión (auditoría, normativa)?
Coste	¿Cuánto cuesta el cómputo, el etiquetado y el mantenimiento?
Precisión requerida	¿Cuánto mejora frente a una solución no-ML (baseline)?
Tiempo real / latencia	¿La decisión debe ser instantánea (RPA/reglas) o admite espera?



La regla de oro: empieza simple

- **Google (problem framing)**: "El ML es una herramienta especializada; no quieras una solución compleja cuando una más simple funciona". Primero optimiza la solución **no-ML** y úsala como *benchmark*.
- **scikit-learn**: su mapa de selección te hace probar primero estimadores simples y avanzar ("Try next") solo si no alcanzas el objetivo.
- **Azure ML**: "No tengas miedo a una competición en paralelo entre varios algoritmos".
- **No Free Lunch** (Wolpert y Macready, 1997): ningún algoritmo es mejor en promedio sobre todos los problemas → la elección depende de las suposiciones sobre tu problema.

9.2 Secuencia práctica recomendada

1. **Reglas o heurística** (explicable, sin datos).
2. **Árbol de decisión / regresión logística** (datos tabulares pequeños).
3. **Ensamble** (Random Forest / XGBoost) si hace falta precisión.
4. **Deep learning** solo si hay datos masivos y el problema lo exige.



Evidencia para «empezar simple»

Un estudio con 45 datasets tabulares de tamaño medio (~10.000 muestras) mostró que los modelos basados en árboles siguen siendo estado del arte, además de más rápidos de entrenar que el deep learning. No siempre hace falta una red neuronal.

10. Caso de estudio: Robocode como sistema de resolución de problemas completo

Los cinco requisitos del §4.1 y la elección de modelo del §9 dejan de ser teoría en cuanto se programa un bot de **Robocode Tank Royale** (la [actividad entregable](#) de la unidad):

- **Representación:** el estado del bot (posición, energía, rumbo del radar) y del campo de batalla se traduce a variables que el programa puede leer en cada turno.
- **Razonamiento:** decidir hacia dónde disparar o moverse es exactamente el problema de elegir modelo del §9 — puedes usar reglas fijas («si el enemigo está a menos de 100 px, dispara»), lógica difusa sobre la distancia y el ángulo, o una combinación.
- **Eficiencia:** el bot decide en tiempo real, turno a turno; una lógica demasiado costosa computacionalmente pierde el combate por lentitud, no por mala estrategia.
- **Adecuación del modelo (CE f):** no hay datos de entrenamiento previos sobre "cómo gana este bot en concreto", así que el punto de partida natural es la regla o heurística (§9.2, paso 1), no el aprendizaje automático.

11. Puntos clave de la unidad

- Un sistema de resolución de problemas de IA debe cumplir **5 requisitos**: representación, razonamiento, aprendizaje/adaptabilidad, eficiencia computacional e interacción con el usuario.
- Un problema se formaliza con **estado inicial, acciones, transición, objetivo y coste**; la búsqueda recorre el **espacio de estados**.
- **BFS** es completa y óptima en pasos pero usa mucha memoria; **DFS** usa poca memoria pero no es óptima; **A*** es óptimo con heurística admisible.
- Los modelos de IA se clasifican por **aprendizaje** (supervisado/no supervisado/refuerzo), por **análisis** (descriptivo/predictivo/prescriptivo) y por **base** (conocimiento vs datos).
- **RPA "hace" y la IA "piensa"**: la automatización inteligente combina IA + BPM + RPA.
- La **lógica difusa** modela la vaguedad con grados de pertenencia $[0,1]$, funciones de pertenencia, reglas Mamdani/Sugeno y defuzzificación por centroide.
- Los **sistemas basados en reglas** ejecutan el ciclo **reconocer-actuar** con un motor RETE; `experta` es su port Python (requiere parche en Py3.10+).
- Para elegir modelo (CE f): valora **datos, explicabilidad, coste, precisión y tiempo real** y **empieza por el modelo más simple**.

12. Glosario

Término	Definición
Espacio de estados	Grafo de configuraciones posibles conectadas por acciones
Estado	Configuración concreta de un problema en un momento dado
Búsqueda en anchura (BFS)	Algoritmo que explora el espacio por niveles (cola)
Búsqueda en profundidad (DFS)	Algoritmo que explora hasta el fondo (pila)
A*	Búsqueda con heurística: $f = g + h$ (coste acumulado + estimación)
Heurística	Estimación del coste restante hasta el objetivo
Backtracking	Exploración con retroceso (base de DFS)
RPA	Automatización robótica de procesos: replica tareas en la interfaz
Automatización inteligente	Combinación de IA (decisión) + BPM (orquestración) + RPA (ejecución)
Agente de IA	Programa que percibe y actúa para lograr un objetivo
Lógica difusa	Lógica con grados de verdad en $[0,1]$ (Zadeh, 1965)
Función de pertenencia	Asigna a cada valor su grado de pertenencia $\mu(x)$
Variable lingüística	Variable con términos difusos (frío, templado, caliente)
FIS	Sistema de inferencia difuso (fuzzificar $\rightarrow$ reglas $\rightarrow$ defuzzificar)
Mamdani	Inferencia difusa con salida difusa defuzzificada
Sugeno/TSK	Inferencia con salida funcional, más eficiente para control
Defuzzificación	Conversión del resultado difuso a número nítido (centroide...)

Término	Definición
Sistema de producción	Sistema basado en reglas con memoria de trabajo y motor de inferencia
Ciclo reconocer-actuar	match → resolve → act hasta agotar la agenda
RETE	Algoritmo de emparejamiento eficiente de reglas
Salience	Prioridad numérica de una regla
Forward chaining	Encadenamiento hacia delante (data-driven)
Backward chaining	Encadenamiento hacia atrás (goal-driven), propio del diagnóstico
CLIPS	Motor de reglas de la NASA (1985-1996), referencia histórica
No Free Lunch	Teorema: ningún algoritmo es mejor en promedio sobre todos los problemas

13. FAQ

? ¿Un problema «de juguete» como el de las jarras sirve para algo real? ▾

Sí: es la misma técnica que usan los planificadores de rutas, los motores de búsqueda de rutas en logística o los sistemas de planificación de tareas. Si aprendes a representar estados y acciones, sabes formalizar problemas reales.

? ¿A* siempre encuentra la solución más corta? ▾

Con una heurística **admisible** (que nunca sobreestime el coste restante), sí. Si la heurística sobreestima, A* pierde la garantía de optimalidad (pero suele ser más rápido).

? ¿La lógica difusa es lo mismo que la probabilidad? ▾

No. La difusa modela la **vaguedad** (pertenencia imprecisa: "hace calor") y la probabilidad modela la **incertidumbre** ("hay un 60 % de que llueva"). Son matemáticas distintas.

? ¿experta no funciona en Python moderno? ▾

La librería (2019) tiene un problema de dependencias con Python 3.10+. Con el parche `collections.Mapping = collections.abc.Mapping` (o instalando `frozendict>=2.3` y `--no-deps`) funciona bien en el entorno del curso.

? ¿Un sistema basado en reglas aprende de los datos? ▾

No: las reglas las escribe un experto. No aprende solo, pero es **totalmente explicable** (puede justificar cada decisión), algo que el ML no siempre puede.

? ¿Cuándo conviene usar RPA en lugar de un modelo de ML? ▾

Si la tarea es **repetitiva y con reglas claras** (copiar datos entre sistemas, rellenar formularios), RPA. Si exige **juicio, patrones o lenguaje**, IA/ML. Y a menudo se combinan.

? ¿Por qué el entregable de la unidad es un juego (Robocode) y no un dataset? ▾

Porque un bot de combate obliga a decidir en tiempo real con información incompleta — es un sistema de resolución de problemas completo y compacto, más cercano a control que a clasificación, y permite comparar en la práctica una solución por reglas con una por lógica difusa (\$10).

14. Planificación sesión a sesión

Semana	Horas	Contenido	CE	Evidencia / actividad
7	3	Requisitos de un SRP; espacio de estados y representación; BFS/DFS/A*	RA2-a	Ejercicios bloque A
8	3	Clasificación de modelos; automatización de tareas	RA2-b, RA2-c	Ejercicios bloques B y C
9	3	Lógica difusa (teoría + Taller 1 scikit-fuzzy)	RA2-d	Taller 1
9-10	3	Sistemas basados en reglas (teoría + Taller 2 experta); CE f	RA2-e, RA2-f	Taller 2
9-10	—	Talleres 3-5: preparar el entorno, GitHub y Markdown para Robocode	—	Talleres 3-5
10	3	Robocode Tank Royale: entrega y evaluación	RA2-a, RA2-f	Actividad entregable

15. Tabla final RA/CE

CE	Dónde se trabaja	Con qué se evalúa
RA2-a	\$4	Ejercicios bloque A, Robocode
RA2-b	\$5	Ejercicios bloque B
RA2-c	\$6	Ejercicios bloque C
RA2-d	\$7	Ejercicios bloque D, Taller 1
RA2-e	\$8	Ejercicios bloque E, Taller 2
RA2-f	\$9, \$10	Ejercicios bloque F, Robocode

16. Recursos

- [Diapositivas](#)
- [Ejercicios de la unidad](#)
- Talleres: [T01](#) · [control difuso](#) · [T02](#) · [sistema de reglas](#) · [T03](#) · [preparar el entorno](#) · [T04](#) · [GitHub](#) · [T05](#) · [Markdown](#)
- Actividad entregable: [Robocode Tank Royale](#) · [comparativa Java/Python](#) · [tutorial Java](#) · [tutorial Python](#)
- **Notebooks** — Talleres 1 y 2, con descarga y apertura en Colab, en el menú «Notebooks»



Referencias de la unidad ▼

- [Red Blob Games](#) · [A*](#)
- [Wikipedia](#) · [A*](#)
- [scikit-fuzzy](#) (pythonhosted)
- [experta](#) (readthedocs)
- [CLIPS](#)
- [IBM](#) · [RPA](#)
- [IBM](#) · [Automatización inteligente](#)
- [scikit-learn](#) · [Elegir el estimador](#)
- [arXiv](#) · [Tree-based vs DL en tabulares](#)
- [Robocode Tank Royale](#) · [documentación oficial](#)

17. Evaluación

Peso	Instrumento
40 % actividades	Media de los 5 talleres y la actividad entregable (Robocode), con su rúbrica
60 % prueba escrita	Prueba del RA2 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «*en función de la consecución de los RA*»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.

18. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 Orden 8/2025): repetir la resolución de un problema de búsqueda con un caso distinto y las pruebas de autoevaluación de la unidad.

[Volver al índice](#)

 24 de agosto de 2026

4.2 Diapositivas de la UD02



Diapositivas PDF



Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

🕒 24 de agosto de 2026

4.3 Actividades Entregables

UD02 — Actividad entregable: Robocode Tank Royale



Cierre de la unidad

Robocode es la práctica de cierre del RA2, del **23 de noviembre al 3 de diciembre**. Cuenta dentro del 40 % de actividades de la unidad, junto con los 5 talleres.

Introducción

Robocode es un juego de programación donde el objetivo es codificar un bot en forma de tanque virtual para competir contra otros bots en un campo de batalla. Quien juega es quien programa el bot: no hay control directo durante la partida. El programa dice cómo debe comportarse y reaccionar el bot ante los eventos del campo de batalla — es, en la práctica, el sistema de resolución de problemas del §10 de la teoría hecho código.

El nombre viene de una versión anterior del juego ("Código de robot"); esta versión usa "bot" en vez de "robot". Las batallas tienen lugar en un campo donde varios bots luchan hasta que solo queda uno, como en un *Battle Royale* — de ahí Tank **Royale**.

- Documentación del juego: <https://robocode-dev.github.io/tank-royale/>
- Repositorio en GitHub: <https://github.com/robocode-dev/tank-royale>
- API Java: [overview](#) · [Bot API](#)



Tank Royale no es el Robocode clásico

RCTR se basa en una versión más antigua de Robocode, pero con un API distinto: los bots antiguos y el funcionamiento del campo de batalla han cambiado sustancialmente. Mucha documentación que encontrarás en internet es del sistema antiguo — la estrategia sigue siendo válida, pero el código no. Hay un puente entre las dos versiones en [robocode-api-bridge](#), la documentación y estrategias de la API antigua siguen en [robocode.sourceforge.io](#), y un archivo con mucha información histórica sobre estrategias y robots está cacheado en [Wayback Machine](#).

Antes de empezar

1. Elige lenguaje con la [comparativa Java vs Python](#).
2. Completa el [Taller 3](#) para dejar listo tu entorno.
3. Sigue el tutorial completo: [Robocode en Java](#) o [Robocode en Python](#), según el lenguaje elegido.

Objetivo de la práctica

Usando Robocode Tank Royale, te pones en el papel de los primeros programadores que dotaban de «inteligencia» a los primeros sistemas: reaccionan solo ante estímulos que su programador previó, no tienen intuición y el resultado es tan bueno como lo sea su programación — el mismo debate del §9 de la teoría sobre cuándo usar reglas y cuándo aprendizaje.

El profesor establecerá unos bots simples que tu bot deberá derrotar para superar la práctica: no sirve ganar por azar, tiene que estar razonado y documentado. Después habrá una competición entre todo el alumnado.

Pasos a seguir

1. **Preparación del entorno** — en una sesión conjunta, instalación y primer combate de pruebas.
2. **Mi primer bot** — con la guía de la documentación y el profesor, generar el primer bot, añadirlo a la batalla y depurar su funcionamiento.
3. **¿Cómo mejoro mi bot?** — estudiar estrategias y mejoras para dotarlo de comportamiento inteligente (radar, movimiento, disparo — ver los tutoriales).
4. **Investigación y desarrollo propio** — trabajo individual: prever a los adversarios (los conocidos y los de tus compañeros) y aplicar las técnicas que consideres más útiles para subir en la clasificación.

Qué debes entregar

A través de Moodle, un archivo **ZIP** que contenga el código fuente de tu bot (nombre: tu nombre más los 4 últimos dígitos de tu DNI/NIE, sin letras) y la memoria justificativa en PDF.

- `TuNombreNNNN.java` (Java) o `TuNombreNNNN.py` (Python) — clase del bot
- `TuNombreNNNN.json` — información del autor
- `TuNombreNNNN.cmd` y `TuNombreNNNN.sh` — arranque en Windows y Linux

La memoria en PDF debe incluir, al menos: datos del alumnado, descripción del funcionamiento (estructura, sistema basado en reglas/casos, análisis y evolución de la solución...), descripción detallada de los métodos definidos o usados, conclusiones y webgrafía/bibliografía.

Requisitos mínimos de la competición

- **Versión 0.34.0 de la API** (multi-idioma y con escalado)
- Modo **classic**, 10 asaltos, tamaño de campo 2000×2000
- Adversarios de referencia: `RamFire`, `Walls`, `SpinBot`
- Gana quien acumule más puntuación; velocidad de enfriamiento 0.1; tiempo máximo de inactividad 450
- No se permite llamar a métodos prohibidos ni ganar por azar: **hay que demostrar IA**
- Los combates se graban y se devuelven como feedback

Rúbrica

Rúbrica real de la tarea en Moodle (`assign_8288613`, método `rubric`). Suma **10 puntos**: el peso del resultado en combate (criterios 3 y 4) es de **8 sobre 10** — se premia el comportamiento del bot, no solo la documentación.

CRITERIO 1 · ENTREGA DE FICHEROS (0-1 PUNTO)

Puntos	Nivel
0	No entregados
0,2	Faltan archivos o es imposible hacer funcionar el bot
0,4	Se entregan todos o casi todos los ficheros, pero el docente no consigue hacer funcionar el bot
0,6	Ficheros con el formato correcto; el bot funciona con grandes correcciones del docente
0,8	Ficheros con el formato correcto; el bot funciona con pequeñas correcciones del docente
1	Se entregan únicamente los ficheros solicitados, con el formato correcto, y el bot funciona sin que el docente deba modificarlos

CRITERIO 2 · MEMORIA (0-1 PUNTO)

Puntos	Nivel
0	No entregada
0,25	Insuficiente
0,5	Suficiente
0,75	Bien

Puntos	Nivel
1	Muy bien

CRITERIO 3 · RESULTADO DE LA MELÉ FINAL, TODOS CONTRA TODOS (0-3 PUNTOS)

Puntos	Nivel
0	No entregado
0,5	Ni gana ni se acerca en ninguna de las rondas
1	No gana ninguna ronda, pero queda relativamente cerca de la cabeza en las 3
2	Gana al menos una de las 3 rondas
3	Gana las 3 rondas

CRITERIO 4 · RESULTADO CONTRA RAMFIRE, WALLS Y SPINBOT (0-5 PUNTOS)

Configuración: 3 rondas × 10 asaltos.

Puntos	Nivel
0	No entregado
1	Queda al final de la clasificación y muy alejado en puntos
2	Queda al final, pero relativamente cerca en puntuación
3	No gana ninguna ronda, pero queda cerca de la cabeza en alguna
4	Gana al menos una de las 3 rondas
5	Gana las 3 rondas

**Soluciones**

Las soluciones no se publican: se corrigen en clase y en la competición final.

[Volver a la UD02](#) · [Comparativa Java/Python](#)

24 de agosto de 2026

UD02 · Comparativa Java vs Python para Robocode

Robocode Tank Royale (RCTR) ofrece API oficial para múltiples lenguajes. En esta unidad podéis elegir entre **Java (JVM)** y **Python**. A continuación se detallan las diferencias, ventajas e inconvenientes de cada opción.

¿Por qué dos lenguajes?

RCTR v0.34.0 introdujo la API multilingüe. El proyecto Robocode nació en Java, pero la comunidad y la evolución del software han traído soporte para Python, .NET y TypeScript. Esto permite elegir el lenguaje que mejor se adapte a vuestro perfil y a los objetivos del curso.

Tabla comparativa

Aspecto	Java	Python
Entorno requerido	JDK 11-23 + IntelliJ IDEA + Maven	Python 3.10+ + VS Code + pip
Dependencias	<code>pom.xml</code> (Maven) o JAR manual	<code>pip install robocode-tank-royale</code>
Sintaxis API	<code>camelCase()</code> — <code>setTurnLeft()</code> , <code>onScannedBot()</code>	<code>snake_case()</code> — <code>set_turn_left()</code> , <code>on_scanned_bot()</code>
Tipado	Estático (obligatorio)	Dinámico (opcional, con type hints)
Proyecto starter	ZIP preempaquetado (<code>IABDBotMaven.zip</code>)	Manual: 4 ficheros (<code>.py</code> + <code>.json</code> + <code>.cmd</code> + <code>.sh</code>)
Conf. servidor	Variables de entorno en configuración de ejecución de IntelliJ	Variables de entorno <code>SERVER_SECRET</code> y <code>SERVER_URL</code> en terminal
Curva aprendizaje	Más pronunciada (JDK, Maven, IDE pesado)	Más suave (pip, editor ligero)
Rendimiento batalla	Compilado JIT (mayor velocidad)	Interpretado (suficiente para RCTR, pero menor rendimiento)
Ecosistema IA/ML	Limitado para IA aplicada	Rico: scikit-learn, PyTorch, TensorFlow, etc.
Documentación comunitaria	RoboWiki mayoritariamente en Java	RoboWiki en Java (traducción a Python necesaria)
Uso en CE IA&BD	Transversal (programación genérica)	Alineado (Python es el lenguaje de referencia en IA)
API RCTR	Madura (existe desde el origen)	Estable (v0.34.0+, en evolución)

Ejemplos de código comparativos

CLASE BOT E IMPORTS

Java

Python

```

1  import dev.robocode.tankroyale.botapi.*;
2  import dev.robocode.tankroyale.botapi.events.*;
3
4  public class MiBot extends Bot {
5      MiBot() {
6          super(BotInfo.fromFile(getConfigPath()));
7      }
8
9      private static String getConfigPath() {
10         String configPath = "src/main/java/MiBot.json";
11         java.io.File configFile = new java.io.File(configPath);
12         if (!configFile.exists()) {
13             configPath = "MiBot.json";
14         }
15         return configPath;
16     }
17 }

```

```

1  from robocode_tank_royale.bot_api import Bot, BotInfo
2
3  class MiBot(Bot):
4      def __init__(self):
5          super().__init__(BotInfo.from_file("MiBot.json"))

```

MÉTODO run() Y BUCLE PRINCIPAL

Java

Python

```

1  public void run() {
2      setAdjustRadarForBodyTurn(true);
3      setAdjustGunForBodyTurn(true);
4      setAdjustRadarForGunTurn(true);
5
6      while (isRunning()) {
7          setTurnRadarLeft(Double.POSITIVE_INFINITY);
8          forward(100);
9          turnGunLeft(360);
10         back(100);
11         turnGunLeft(360);
12     }
13 }

```

```

1  def run(self):
2      self.set_adjust_radar_for_body_turn(True)
3      self.set_adjust_gun_for_body_turn(True)
4      self.set_adjust_radar_for_gun_turn(True)
5
6      while self.is_running: # type: ignore[attr-defined] # pylint: disable=no-member
7          self.set_turn_radar_left(float("inf"))
8          self.forward(100)
9          self.turn_gun_left(360)
10         self.back(100)
11         self.turn_gun_left(360)

```

EVENT HANDLER onScannedBot**Java****Python**

```
1 public void onScannedBot(ScannedBotEvent e) {
2     fire(1);
3 }
```

```
1 def on_scanned_bot(self, scanned_bot_event: ScannedBotEvent):
2     del scanned_bot_event
3     self.fire(1)
```

DISPARO PREDICTIVO (TRIGONOMÉTRICO)**Java****Python**

```
1 public void onScannedBot(ScannedBotEvent e) {
2     double enemyDistance = distanceTo(e.getX(), e.getY());
3     double firePower = Math.min(500 / enemyDistance, 3);
4     double bulletSpeed = 20 - firePower * 3;
5     long time = (long) (enemyDistance / bulletSpeed);
6
7     double enemyVelocity = e.getSpeed();
8     double enemyDirection = e.getDirection();
9     double deltaX = enemyVelocity * Math.cos(Math.toRadians(enemyDirection));
10    double deltaY = enemyVelocity * Math.sin(Math.toRadians(enemyDirection));
11    double futureX = e.getX() + (deltaX * time);
12    double futureY = e.getY() + (deltaY * time);
13
14    setTurnGunLeft(gunBearingTo(futureX, futureY));
15    if (getGunTurnRemaining() <= 0 && getGunHeat() == 0) {
16        fire(firePower);
17    }
18 }
```

```
1 def on_scanned_bot(self, e: ScannedBotEvent):
2     enemy_distance = self.distance_to(e.x, e.y)
3     fire_power = min(500 / enemy_distance, 3)
4     bullet_speed = 20 - fire_power * 3
5     time = int(enemy_distance / bullet_speed)
6
7     enemy_velocity = e.speed
8     enemy_direction = e.direction
9     delta_x = enemy_velocity * math.cos(math.radians(enemy_direction))
10    delta_y = enemy_velocity * math.sin(math.radians(enemy_direction))
11    future_x = e.x + (delta_x * time)
12    future_y = e.y + (delta_y * time)
13
14    self.set_turn_gun_left(self.gun_bearing_to(future_x, future_y))
15    if self.gun_turn_remaining <= 0 and self.gun_heat == 0:
16        self.fire(fire_power)
```

Ventajas e inconvenientes**JAVA****Ventajas:**

- Documentación histórica amplia (RoboWiki, ejemplos clásicos)
- Proyecto starter preempaquetado (descomprimir y abrir en IntelliJ)
- Rendimiento compilado mayor en cálculos intensivos
- API muy madura y estable

Inconvenientes:

- Requerimientos de entorno pesados (JDK + IntelliJ)
- Curva de aprendizaje mayor si no se conoce Java/Maven
- Menos alineado con el ecosistema de IA y Big Data del curso
- Configuración de ejecución dependiente del IDE

PYTHON

Ventajas:

- Instalación sencilla (`pip install`)
- Editor ligero (VS Code) y rápido de configurar
- Sintaxis limpia y legible
- Alineado con el resto del curso (ecosistema IA/BD)
- Mejor integración con librerías de IA si se quiere ampliar el bot
- Variables de entorno simples (terminal o script)

Inconvenientes:

- Documentación histórica en Java (hay que traducir conceptualmente)
- Proyecto starter manual (4 ficheros) en lugar de ZIP preempaquetado
- Rendimiento interpretado (normalmente no perceptible en RCTR)

Recomendación

Para la naturaleza de este curso de especialización en **Inteligencia Artificial y Big Data**, se recomienda el uso de **Python** como lenguaje principal. Python es la herramienta estándar de la industria en IA/ML, y su sintaxis clara permite centrarse en las estrategias del bot en lugar de la infraestructura del lenguaje.

La opción Java queda disponible para alumnado con experiencia previa en Java o interés particular, pero la vía Python será la que reciba soporte preferente por parte del profesorado.

**Elegid vuestro camino**

- **Noveles en programación o provenientes de Python** → seguid la guía Python
- **Experiencia previa en Java o interés en JVM** → seguid la guía Java
- **Dudas** → consultad al profesorado

🕒 15 de julio de 2026

UD02 · Robocode Tank Royale en Java

Preparación del entorno

- Al menos java 11 (Yo estoy usando Java 23 (OpenJDK) y la versión 0.34.0 de Tank Royale)

```
1 java -version
```

- Descargar la última versión desde releases: <https://github.com/robocode-dev/tank-royale/releases>
- Ejecutar el `jar` descargado:

```
1 java -jar robocode-tankroyale-gui-x.y.z.jar
```

- Descarga y descomprime los Bots de ejemplo: <https://github.com/robocode-dev/tank-royale/releases>
- Configura la gui para acceder a la carpeta de robots: Config -> Bot Root Directories (del menú)
- [opcional] Añadir sonidos al gui. Descarga y descomprime la carpeta `sounds.zip` de: <https://github.com/robocode-dev/sounds/releases>

```
1 [directorio padre]
2 └─ robocode-tankroyale-gui-x.y.z.jar
3 └─ sounds/ <-- carpeta de sonidos
4     └─ bots_collision.wav
5     ...
6     └─ wall_collision.wav
```

- Necesitaras IntelliJ para poner todo en funcionamiento y para programar tu Bot.



Consejo

Juega un poco por tu cuenta con el entorno GUI, explora las opciones, tipos de batallas, crea alguna ronda de las mismas, inicializa Bots, añádelos, juega con la velocidad de juego, etc.

Robocode con Maven

Robocode tankroyale está disponible como artefacto del repositorio de maven, si estas familiarizado con el uso de maven puede simplificar bastante el trabajo y actualizaciones de dependencias, te dejo el enlace por si quieres investigar por tu cuenta, ya que escapa del objetivo de esta asignatura: <https://central.sonatype.com/search?q=g:dev.robocode.tankroyale&smo=true>

Mi primer Bot

PROYECTO INTELLIJ

Para acelerar el proceso, he preparado un proyecto en IntelliJ con toda la arquitectura básica que necesitará tu Bot. Descarga y descomprime [este paquete](#) y abre el proyecto con el IDE IntelliJ.



Sin maven

Si no quieres usar maven y deseas montar el proyecto por tu cuenta sigue estas indicaciones:

También necesitaras la librería (API) de Robocode Tank Royale que puedes descargar desde aquí: <https://github.com/robocode-dev/tank-royale/releases>. Descarga el archivo: `robocode-tankroyale-bot-api-x.y.z.jar`

La estructura debería quedar de la siguiente manera:

```
1 [directorio padre]
2 └─ lib
3     └─ robocode-tankroyale-bot-api-x.y.z.jar
4 └─ IABDBot <-- Proyecto de IntelliJ
5     └─ src
6     ...
7     └─ IABDBot.iml
```

 Atención

Asegúrate de entender el funcionamiento del Bot IABDBot, tiene comentarios donde se explican partes importante del Bot y que daremos por conocidas en los siguientes puntos.

Es muy importante que entiendas la diferencia entre movimientos bloqueantes y simultáneos, así como las condiciones y los eventos disponibles.

 Ojo!

Gracias a este fragmento, tu bot funcionará correctamente desde IntelliJ y desde el gui de RoboCode:

```

1 // Constructor, que carga el fichero de configuración
2 IABDBot() {
3     super(BotInfo.fromFile(getConfigPath()));
4 }
5
6 // Este método obtiene la ruta del fichero de configuración y se asegura que funcione desde IntelliJ y desde la gui de Robocode
7 private static String getConfigPath() {
8     String miBot = "IABDBot";
9     String configPath = "src/main/java/" + miBot + ".json";
10    java.io.File configFile = new java.io.File(configPath);
11    if (!configFile.exists()) {
12        configPath = miBot + ".json";
13    }
14    return configPath;
15 }
```

ÚLTIMAS NOVEDADES DEL PROYECTO

- Se pueden grabar los combates en un fichero dentro de la carpeta `recordings` con la extensión `battle.gz` que luego podemos reproducir en diferido. Esto nos permite revisar situaciones y estudiar estrategias de mejora.

CONFIGURACIÓN DEL SERVIDOR (GUI) Y EL PROYECTO INTELIJ PARA EJECUTAR EN LOCAL O REMOTO

El servidor permite conexiones en remoto/local a través de un password que se genera automáticamente la primera vez que lanzas la GUI. Este password lo puedes encontrar/modificar en el archivo `server.properties`, en el ejemplo siguiente la clave de acceso es **CEIABDEPM2025**

```

1 bots-secrets=CEIABDEPM2025
2 [...]
```

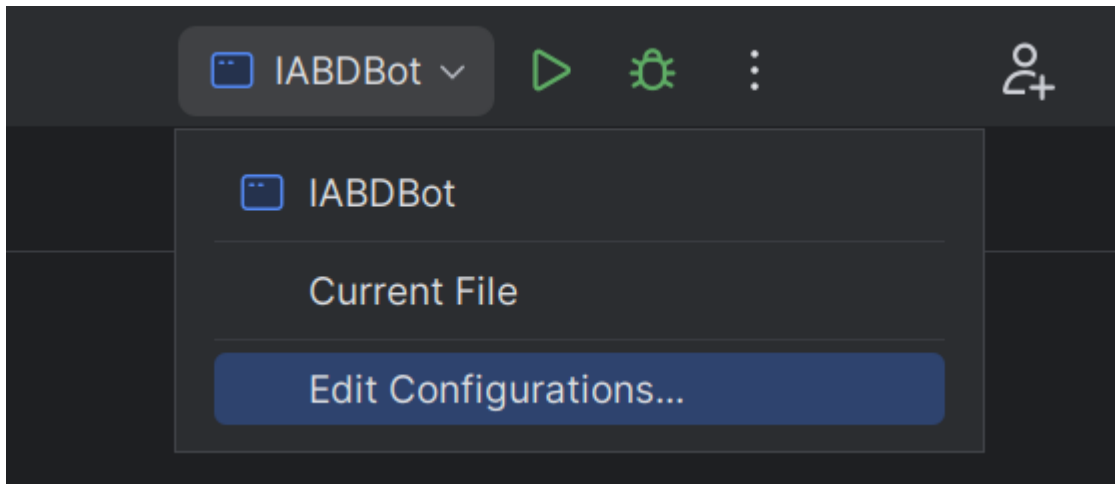
 Ojo

Con las últimas versiones de la gui el uso de contraseñas en el servidor está inicialmente deshabilitado, si queremos habilitarlo lo podemos hacer desde el propio interfaz: `Config/Server Options/Enable server secrets` o bien desde el fichero de configuración `server.properties` añadiendo:

```
1 server-secrets-enabled=true
```

Otra información que necesitaras será la IP del servidor al que quieres conectar, normalmente será `localhost` (tu host local), y cuando sea necesario hacer pruebas el profesor te facilitará su IP.

Ahora solo queda configurar nuestro proyecto de IntelliJ para configurar estas variables en el momento de ejecutar el Bot. Accede a la configuración de las ejecuciones en la parte superior derecha (una vez abierta la clase `IABDBot.java`):



Puedes ver que en el apartado `Environment Variables` tienes el siguiente valor:

```
1 SERVER_SECRET=CEIABDEPM2025;SERVER_URL=ws://localhost:7654
```

Como puedes imaginar **SERVER_SECRET** hace referencia a la clave del servidor, y **SERVER_URL** a la dirección del servidor y es donde deberás reemplazar (según el caso) `localhost` por la IP que te indique el profesor. Así podrás ejecutar tu Bot directamente desde IntelliJ y aparecerá en el Servidor para poder añadirlo a las batallas.

⚡ Atención

Ojo, el servidor debe estar inicializado antes de lanzar el bot de lo contrario obtendrás un error similar a este:

```
1 Exception in thread "main" dev.robocode.tankroyale.botapi.BotException: Could not create web socket for URL: ws://localhost:7654
2   at dev.robocode.tankroyale.botapi.internal.BaseBotInternals.connect(BaseBotInternals.java:268)
3   at dev.robocode.tankroyale.botapi.internal.BaseBotInternals.start(BaseBotInternals.java:254)
4   at dev.robocode.tankroyale.botapi.BaseBot.start(BaseBot.java:114)
5   at IABDBot.main(IABDBot.java:11)
6
7 Process finished with exit code 1
```

Si el Bot encuentra el servidor y la contraseña es correcta debería aparecer esto al inicializarlo en IntelliJ:

```
1 Connected to: ws://localhost:7654
```

🔥 Consejo

Prueba por tu cuenta mezclando en las batallas Bots de ejemplo, con tu Bot o incluso el de algún compañero/a.

¿Cómo mejoro mi Bot?

Dividiremos todo lo que debemos conocer en 4 apartados:

CONOCIENDO EL CAMPO DE BATALLA

Sigue atentamente la documentación de RCTR sobre la [anatomía de los Bots](#)

Puntos importantes:

- Si el radar no se mueve, no detecta
- Inicialmente el robot, el cañón y el radar se mueven conjuntamente (pero se puede cambiar)
- El Bot se simplifica a un cuadrado de 36x36 unidades.
- Para las colisiones (con Bots o paredes) se simula como un círculo de 18 unidades de radio.

A continuación revisa las [coordenadas y ángulos](#)

Puntos importantes:

- Este apartado es totalmente diferente al de Robocode Original.

Estudia también las **físicas**

Puntos importantes:

- Se frena más rápido que se acelera.
- Cuanto más rápido vas, más lento giras.
- El cañón gira máximo 20º por turno.
- El radar gira máximo 45º por turno.
- La potencia de tiro influye en el daño provocado y los puntos conseguidos, pero también en el calentamiento del cañón.
- Inicialmente el cañón está caliente (3 unidades), al principio has de esperar a que se enfíe para disparar.
- El atropello da más puntos que los disparos (pero te resta energía)
- Si chocas con una pared, pierdes puntos.
- La energía que te sobra en una ronda no da puntos (modo kamikaze con el último robot?)

Por último te en cuenta las **puntuaciones**

Puntos importantes:

- Consigues puntos si:
- tu disparo golpea a otro Bot (sino, te resta)
- eres el que mata al Bot
- cada vez que otro Bot muere, pero tu sigues vivo
- eres el último robot vivo
- atropellas a otro Bot
- si matas por atropello a otro Bot
- El último Bot que quede vivo no tiene porqué ser el ganador.

Eventos propios

Definimos una condición, y cuando esta se cumple se dispara un evento:

```

1  // Esta condición se dispara cuando el bot completa su giro
2  public static class TurnCompleteCondition extends Condition {
3
4      private final IBot bot;
5
6      public TurnCompleteCondition(IBot bot) {
7          this.bot = bot;
8      }
9
10     @Override
11     public boolean test() {
12         // El método test() es el que debemos reescribir para definir el resultado de nuestra condición.
13         return bot.getTurnRemaining() == 0;
14     }
15 }
```

Podríamos usar esta condición en un fragmento similar a este:

```

1  waitFor(new TurnCompleteCondition());
```

Podríamos usar funciones Lambda de la siguiente manera:

```

1  waitFor(new Condition() -> getTurnRemaining() == 0);
```

El resultado de los dos fragmentos sería el mismo.

PERSONALIZAR MI BOT

Podemos establecer colores para el cuerpo, torreta de cañón, el radar, el arco de escaneo y de la bala:

```

1  Color verde = Color.fromRgb(0,255,0);
2  setBodyColor(Color.KHAKI);
3  setTurretColor(Color.KHAKI);
4  setRadarColor(Color.KHAKI);
5  setScanColor(Color.KHAKI);
6  setBulletColor(Color.KHAKI);

```

Puedes encontrar la lista completa de colores disponibles en la API de Robocode TankRoyale.

TÉCNICAS DE ESCANEO (RADAR)

Sentidos de nuestro Bot:

Sentido del **tacto**, tu Bot sabe cuando:

- golpea una pared (`onHitWall`),
- es alcanzado por un disparo (`onHitByBullet`),
- es alcanzado por otro Bot (`onHitBot`).

Sentido de la **vista**, tu Bot sabe cuándo ha visto otro robot, pero sólo si lo escanea (`onScannedBot`)

Otros sentidos, tu robot también sabe cuándo ha muerto (`onDeath`), cuándo ha muerto otro robot (`onRobotDeath`).

Además también es consciente de sus balas y sabe cuando una bala ha sido disparada (`onBulletFired`) ha alcanzado a un oponente (`onBulletHit`), cuando una bala golpea una pared (`onBulletWall`) o cuando una bala golpea a otra bala (`onBulletHitBullet`).

Configurar correctamente tu Bot con:

```

1  setAdjustRadarForBodyTurn(true); //true if the radar must adjust/compensate for the body's turn;
2                                     //false if the radar must turn with the body turning (default).
3  setAdjustGunForBodyTurn(true);    //true if the gun must adjust/compensate for the bot's turn;
4                                     //false if the gun must turn with the bot's turn.
5  setAdjustRadarForGunTurn(true);   //true if the radar must adjust/compensate for the gun's turn;
6                                     //false if the radar must turn with the gun's turn.

```

Que el radar siempre de vueltas:

```

1  setTurnRadarLeft(Double.POSITIVE_INFINITY); //degrees - is the amount of degrees to turn left. If negative, the radar will turn right.

```

Revisa el API (`rescan()` i `setRescan()`)

Fijar el radar en un enemigo (one on one radar):

- Escaneo con multiplicador

```

1  public void onScannedBot(ScannedBotEvent e) {
2      double factor=1.9;
3      setTurnRadarLeft(factor * radarBearingTo(e.getX(), e.getY()));
4  }

```

Segun el factor:

- 1.0 - Bloqueo de radar delgado. Debe llamar a `scan()` para evitar perder el bloqueo. Que Dios te ayude si alguna vez te saltas un turno.
- 1.9 - El arco del radar comienza amplio y se estrecha lentamente tanto como sea posible mientras se mantiene en el objetivo.
- 2.0 : el arco del radar recorre un ángulo fijo. El ángulo exacto elegido depende de las posiciones del enemigo y del radar cuando se detecta al enemigo por primera vez. El ángulo se incrementará si es necesario para mantener el bloqueo.
- Arco de radar Ancho

```

1  public void onScannedBot(ScannedBotEvent e) {
2      //set the wide to add to the scan.
3      double wide=36.0;
4      // Absolute angle towards target
5      double angleToEnemy = radarBearingTo(e.getX(), e.getY());
6      // Distance we want to scan from middle of enemy to either side
7      // The 36.0 is how many units from the center of the enemy robot it scans.
8      double extraTurn = Math.min(Math.toDegrees(Math.atan(36.0 / distanceTo(e.getX(), e.getY()))), getMaxRadarTurnRate());
9
10     // Adjust the radar turn so it goes that much further in the direction it is going to turn
11     // Basically if we were going to turn it left, turn it even more left, if right, turn more right.
12     // This allows us to overshoot our enemy so that we get a good sweep that will not slip.
13     if (angleToEnemy < 0)
14         angleToEnemy -= extraTurn;
15     else
16         angleToEnemy += extraTurn;
17
18     //Turn the radar
19     setTurnRadarLeft(angleToEnemy);
20 }

```

- Radar Oscilante (variable global que sepa la dirección y nos ayude a cambiarla de vez en cuando)
- Escaneo más inteligente (seguir al cercano? según la situación?) ...
- Nos interesa mantener una lista de enemigos? `e.getScannedBotId()` ? y por ejemplo fijar el radar en el más débil?...

TÉCNICAS DE DESPLAZAMIENTO (MOVIMIENTO)

Pensamiento lateral

Si has jugado baloncesto antes, sabes que si quieres defender a alguien que sostiene el balón, debes maximizar tu movimiento lateral enfrentándote siempre a él (de frente). Lo mismo ocurre con tu robot.

Para conseguir esta posición debemos hacer algo similar a esto:

```

1  turnLeft(bearingTo(e.getX(), e.getY()) + 90);

```

que siempre colocará tu robot perpendicular (90 grados) a tu enemigo.

¿Hacia adelante o hacia atrás?

Cuando te enfrentas a un oponente, la idea de "adelante" y "atrás" (del primer Bot) se vuelven algo obsoletas. Probablemente estés pensando más en términos de "atacar a la izquierda" o "atacar a la derecha". Para realizar un seguimiento de la dirección del movimiento, simplemente declare una variable como hicimos para oscilar el radar.

```

1  class MyRobot extends Bot {
2      private byte moveDirection = 1;

```

luego, cuando quieras mover tu robot, simplemente puedes decir:

```

1  setForward(100 * moveDirection);

```

Puedes cambiar de dirección cambiando el valor de `moveDirection` de 1 a -1 así:

```

1  moveDirection *= -1;

```

Cambiar de dirección

El enfoque más intuitivo para cambiar de dirección es simplemente cambiar la dirección del movimiento cada vez que golpeas una pared o golpeas a otro robot de esta manera:

```

1  public void onHitWall(HitWallEvent e) { moveDirection *= -1; }
2  public void onHitBot(HitBotEvent e) { moveDirection *= -1; }

```

Sin embargo, descubrirás que si haces eso, terminarás presionando obstinadamente contra un robot que te embiste desde un costado (como un perro en celo). Esto se debe a que se llama a `onHitRobot()` tantas veces que `moveDirection` sigue cambiando y nunca te alejas.

Un mejor enfoque es simplemente probar para ver si su robot se ha detenido. Si es así, probablemente significa que has golpeado algo y querrás cambiar de dirección. Puedes hacerlo con el código:

```

1  if (getSpeed() == 0)
2      moveDirection *= -1;

```

Ponlo en tu método `doMove()` (o en cualquier otro lugar donde estés manejando el movimiento) y podrás manejar todos los eventos de impacto con las paredes u otros Bots.

¿Bailamos?

Dando vueltas (Círculos)

Puedes rodear a tu enemigo simplemente usando las técnicas anteriores:

```

1  public class IABDBot extends Bot {
2      double moveDirection=1;
3      private double ultimoEnemigoVisto;
4      ...
5      @Override
6      public void run() {
7          while (isRunning()) {
8              doMove();
9              go();
10         }
11
12         public void onScannedBot(ScannedBotEvent e) {
13             ...
14             ultimoEnemigoVisto=bearingTo(e.getX(), e.getY());
15             ...
16         }
17
18         public void doMove() {
19             // switch directions if we've stopped
20             if (getSpeed() == 0)
21                 moveDirection *= -1;
22             // circle our enemy
23             setTurnLeft(ultimoEnemigoVisto+90);
24             setForward(1000 * moveDirection);
25         }

```

Objetivo: rodea a tu enemigo usando el código de movimiento anterior, como un tiburón rodeando a su presa en el agua.

Evita Ametrallamiento

Un problema que puedes notar con el anterior tipo de movimiento es que es presa fácil para los objetivos predictivos porque sus movimientos son tan... predecibles.

Para evadir las balas de manera más efectiva, debes moverte de lado a lado o "atacar". Una buena forma de hacerlo es cambiar de dirección después de un cierto número de "tics", así:

```

1  public void doMove() {
2      // switch directions if we've stopped
3      if (getSpeed() == 0)
4          moveDirection *= -1;
5      // circle our enemy
6      setTurnLeft(ultimoEnemigoVisto+90);
7      if (getTurnNumber() % 20 == 0) {
8          moveDirection *= -1;
9          setForward(1000 * moveDirection);
10     }
11 }

```

Mira como se balancea hacia adelante y hacia atrás usando el código de movimiento anterior. Observa lo bien que esquivas las balas. Pero ojo, porque puede afectar a tu precisión de disparo.

Cada vez más cerca

Notarás que tanto el primero como este último tipo de movimientos tienen otro problema: se atascan fácilmente en las esquinas y terminan golpeándose contra las paredes. Un problema adicional es que si su enemigo está lejos, disparan mucho pero no aciertan mucho.

Para hacer que tu robot se acerque a tu enemigo, simplemente modifica el código para que se gire ligeramente hacia su enemigo, así:

```

1  setTurnLeft(lastScannedBearing + (90 - (15 * moveDirection)));

```

15 es un factor en grados que puedes modificar y ajustar. Prueba!

Modificando el primero conseguimos una variación que utiliza el código anterior para lanzarse en espiral hacia su enemigo.

Mientras que con el segundo que utiliza el código anterior para acercarse cada vez más. También es bastante bueno para evadir balas.

Fíjate que ninguno de los Bots anteriores queda atrapado en una esquina por mucho tiempo.

Evitando las paredes

Un problema con todos los Bots anteriores es que golpean mucho las paredes y golpearlas agota tu energía. Una mejor estrategia sería detenerse antes de chocar contra las paredes. ¿Pero cómo?

Agregar un evento personalizado

Lo primero que debemos hacer es decidir qué tan cerca permitiremos que nuestro robot llegue a las paredes:

```
1 public class WallAvoider extends Bot {
2     ...
3     private int wallMargin = 60;
```

A continuación, agregamos un evento personalizado que se activará cuando se cumpla una determinada condición dentro del método `run()`:

```
1 public void run() {
2     ...
3     // No te acerques mucho a las paredes
4     addCustomEvent(new Condition("TooCloseToWalls") {
5         public boolean test() {
6             // El método test() es el que debemos reescribir para definir el resultado de nuestra condición.
7             return (
8                 // cerca de la pared izquierda
9                 (getX() <= wallMargin ||
10                // cerca de la pared derecha
11                getX() >= getArenaWidth() - wallMargin ||
12                // cerca de la pared inferior
13                getY() <= wallMargin ||
14                // cerca de la pared superior
15                getY() >= getArenaHeight() - wallMargin)
16            );
17         }
18     });
19     ...
20 }
```

Ten en cuenta que estamos creando una clase interna anónima con esta llamada. Necesitamos hacer override del método `test()` para devolver un valor booleano cuando ocurra nuestro evento personalizado.

Gestionar el evento personalizado

Lo siguiente que debemos hacer es manejar el evento, que se puede hacer así:

```
1 public void onCustomEvent(CustomEvent e) {
2     if (e.getCondition().getName().equals("TooCloseToWalls")) {
3         // switch directions and move away
4         moveDirection *= -1;
5         forward(100 * moveDirection);
6     }
7 }
```

Sin embargo, el problema con ese enfoque es que este evento podría dispararse una y otra vez, provocando que cambiemos rápidamente de un lado a otro, sin alejarnos nunca. O si ya aparecemos cerca de la pared quedamos atrapados.

Para evitar este "sacudida de la muerte" deberíamos tener una variable que indique que estamos manejando el evento. Podemos declarar otro así:

```
1 public class WallAvoider extends Bot {
2     ...
3     private int tooCloseToWall = 0;
```

Luego maneje el evento de manera un poco más inteligente:


```

1 public void onCustomEvent(CustomEvent e) {
2     if (e.getCondition().getName().equals("TooCloseToWalls")) {
3         if (tooCloseToWall <= 0) {
4             // Si no estábamos ya cerca de las paredes, ahora lo estamos
5             tooCloseToWall += wallMargin;
6             setMaxSpeed(0); // Para!!!
7         }
8     }
9 }

```

Manejo de los dos modos

Hay dos últimos problemas que debemos resolver. En primer lugar, tenemos un método `doMove()` donde colocamos todo nuestro código de movimiento normal. Si estamos tratando de alejarnos de una pared, no queremos que se llame a nuestro código de movimiento normal, creando (una vez más) la "sacudida de la muerte". En segundo lugar, queremos volver eventualmente al movimiento "normal", por lo que eventualmente deberíamos tener el "tiempo de espera" de la variable `tooCloseToWall`.

Podemos resolver ambos problemas con la siguiente implementación de `doMove()`:

```

1 public void doMove() {
2     ...
3
4     // if we're close to the wall, eventually, we'll move away
5     if (tooCloseToWall > 0) tooCloseToWall--;
6
7     // switch directions if we've stopped
8     if (getSpeed() == 0) {
9         setMaxSpeed(8);
10        moveDirection *= -1;
11        setForward(1000 * moveDirection);
12    }
13 }

```

Con todo el código anterior evitamos chocar contra las paredes. Observa cómo se desliza suavemente hacia los lados pero nunca (bueno, rara vez) los golpea.

Bot multimodo

Además de los colores que elijas, la mayor parte de la personalidad de tu robot está en su código de movimiento. Por otra parte, situaciones diferentes requieren tácticas diferentes. Usando el ejemplo de evitar paredes, es posible que desees codificar tu bot para que cambie los "modos" según ciertos criterios. Podrías pensar en algo como esto:

```

1 public class MultiModeBot extends Bot {
2     private final static int MOD0_RONDAR=0;
3     private final static int MOD0_ESQUIVAR=1;
4     private final static int MOD0_ASESINO=2;
5     private int modoRobot=MOD0_RONDAR;
6     ...
7 }
8
9 public void onBotDeath(BotDeathEvent e) {
10    ...
11    if (getEnemyCount() > 10) {
12        // Un gran número de enemigos sugiere movimientos fluidos
13        modoRobot=MOD0_RONDAR;
14    } else if (getEnemyCount() > 1) {
15        // esquivar es la mejor táctica para grupos pequeños
16        modoRobot=MOD0_ESQUIVAR;
17    } else if (getEnemyCount() == 1) {
18        // Si solo queda un robot, persíguelo
19        modoRobot=MOD0_ASESINO;
20    }
21    ...
22 }
23
24 public void doMove() {
25     switch (modoRobot){
26         case MOD0_RONDAR:
27             //ronda
28             break;
29         case MOD0_ESQUIVAR:
30             //esquiva
31             break;
32         case MOD0_ASESINO:
33             //asesina
34             break;
35     }
36     ...
37 }

```

Los detalles se dejan (como siempre) como ejercicio para el alumnado.

TÉCNICAS DE ATAQUE (DISPARO)

Técnicas básicas

- Separa el movimiento del cañón del movimiento del Bot (`setAdjustGunForBodyTurn`)
- Técnica de apuntado básica: `setTurnGunLeft` o `setTurnGunRight` junto con `gunBearingTo`

Fórmula de cálculo de potencia de fuego

Otro aspecto importante al disparar es calcular la potencia de fuego de tu bala. La documentación del método `fire()` explica que puedes disparar una bala en el rango de `0.1` a `3.0`. Como probablemente ya habrás concluido, es una buena idea disparar balas de baja potencia cuando tu enemigo está lejos y balas de alta resistencia cuando está cerca.

```
1 public void smartFire(double distance){
2     if ((distance >200) || (getEnergy() <15)){
3         fire(1);
4     } else if (distance > 50){
5         fire(2);
6     } else {
7         fire(3);
8     }
9 }
```

Podrías usar una serie de declaraciones if-else-if-else para determinar la potencia de fuego, en función de si el enemigo está a 50 unidades, a 200, etc (como en el fragmento anterior). Pero tales construcciones son demasiado rígidas. Después de todo, el rango de posibles valores de potencia de fuego cae a lo largo de un continuo, no de bloques discretos. Un mejor enfoque es utilizar una fórmula. He aquí un ejemplo:

```
1 setFire(400/distanceTo(e.getX(), e.getY()));
```

Con esta fórmula, a medida que aumenta la distancia del enemigo, la potencia de fuego disminuye. Asimismo, a medida que el enemigo se acerca, la potencia de fuego aumenta. Los valores superiores a 3 se reducen a 3, por lo que nunca dispararemos una bala mayor que 3, pero probablemente deberíamos reducir el valor de todos modos (solo para estar seguros) de esta manera:

```
1 setFire(Math.min(500/distanceTo(e.getX(), e.getY()), 3));
```

Evitar disparos prematuros

Una situación que encontraras es que tu Bot dispare antes de haber girado el arma hacia el objetivo. Para evitar disparos prematuros, usa el método `getGunTurnRemaining()` para ver qué tan lejos está tu cañón del objetivo y no dispare hasta que esté cerca.

Además, no puedes disparar si el arma está "caliente" desde el último disparo y llamar a `fire()` o `setFire()` solo desperdiciará un turno. Podemos probar si el arma está fría llamando a `getGunHeat()`.

Apuntando de manera predictiva

O... "Usar la trigonometría para impresionar a tus amigos y destruir a tus enemigos".

Si quisiéramos poder golpear a un robot que recorre las paredes siempre fallaríamos, necesitamos poder predecir dónde estará en el futuro, pero ¿cómo podemos hacerlo?

\$\$ Distancia = Velocidad * Tiempo \$\$ Usando la anterior formula podemos calcular cuánto tiempo tardará una bala en llegar allí.

- **Distancia:** se puede encontrar llamando a `distanceTo(e.getX(), e.getY())`
- **Velocidad:** según la documentación de RCTR, una bala viaja a una velocidad de: `20 - potenciaDeFuego * 3`
- **Tiempo:** podemos calcular el tiempo resolviendo: `Tiempo = {Distancia \over Velocidad}`

El siguiente código lo hace:

```
1 // calcular la potencia basado en la distancia
2 double enemyDistance = distanceTo(e.getX(), e.getY());
3 double firePower = Math.min(500 / enemyDistance, 3);
4 // calcular la velocidad de la bala
5 double bulletSpeed = 20 - firePower * 3;
6 // calculamos el tiempo que necesita la bala para impactar al enemigo
7 long time = (long) (enemyDistance / bulletSpeed);
```

Obteniendo futuras coordenadas X,Y

A continuación, debemos calcular la posición futura de nuestro enemigo:

```

1 // Calcular la velocidad actual de tu enemigo
2 double enemyVelocity = e.getSpeed();
3 // Calcular la dirección actual del enemigo
4 double enemyDirection = e.getDirection();
5 // Descomponer el desplazamiento en las coordenadas X e Y
6 // Componente X del desplazamiento COSENO
7 double deltaX = enemyVelocity * Math.cos(Math.toRadians(enemyDirection));
8 // Componente Y del desplazamiento SENO
9 double deltaY = enemyVelocity * Math.sin(Math.toRadians(enemyDirection));
10
11 // Calcular las coordenadas futuras
12 double futureX = e.getX() + (deltaX * time);
13 double futureY = e.getY() + (deltaY * time);

```

Girando el arma al punto previsto

Ahora debemos apuntar nuestro cañon al lugar previsto de impacto:

```

1 setTurnGunLeft(gunBearingTo(futureX, futureY));

```

Disparando!

Por último disparamos nuestro cañon

```

1 if (getGunTurnRemaining() <= 0 && getGunHeat() == 0) {
2     fire(firePower);
3 }

```

Mejor aún...!

Aquí te dejo algunas mejoras para que las valores:

- ¿Debemos esperar siempre a que el arma esté completamente en la dirección calculada?
- Cuando nuestro enemigo se acerca a una pared, nuestros disparos impactan en ella.
- ¿Puedo saber a quien disparo si mi previsión de impacto falla mucho?

Investigación y desarrollo propio

A partir de aquí el trabajo será individual de cada alumno (puede solaparse con el paso 3). Deberéis investigar/prever a vuestros adversarios (los conocidos, y los de vuestros compañeros). Aplicar las técnicas que consideréis más útiles para intentar quedar lo más arriba posible en la tabla de clasificación.

🕒 19 de octubre de 2025

UD02 · Robocode Tank Royale en Python

Antes de empezar

Leed primero la [Comparativa Java vs Python](#) para entender las diferencias y elegir vuestro camino.

Esta guía es la versión **Python** de la actividad Robocode. Si buscáis la versión Java, id a [Robocode en Java](#).

Preparación del entorno

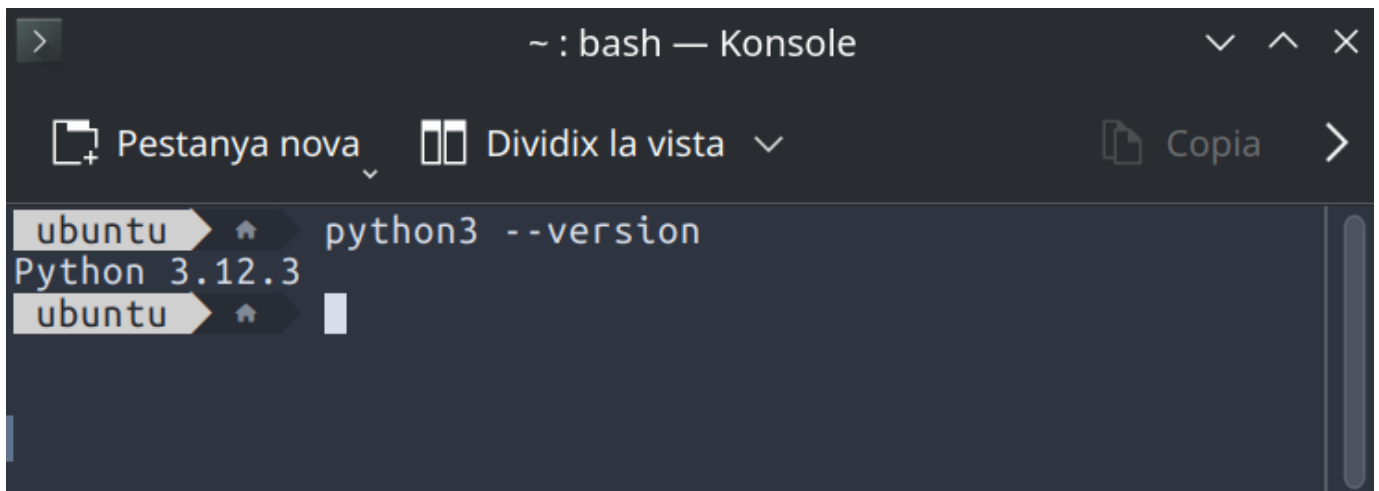
REQUISITOS PREVIOS

- **Python 3.10 o superior** instalado en el sistema
- **VS Code** (Visual Studio Code) u otro editor de código
- Conexión a Internet para instalar dependencias

COMPROBACIÓN DE PYTHON

Abrid un terminal y ejecutad:

```
1 python3 --version
```



The screenshot shows a terminal window titled '~ : bash — Konsole'. The terminal has a dark background with light-colored text. At the top, there are window controls and a menu bar with options like 'Pestanya nova', 'Dividix la vista', and 'Copia'. The terminal content shows a prompt 'ubuntu' followed by the command 'python3 --version' and the output 'Python 3.12.3'. A second prompt 'ubuntu' is visible below the output.

Windows

Si usas Windows, el comando es `python --version` en lugar de `python3 --version`.

Si no tenéis Python instalado, descargadlo desde <https://www.python.org/downloads/> o usad el gestor de paquetes de vuestro sistema:

Linux (Debian/Ubuntu)

Linux (Fedora)

macOS

Windows

```
1 sudo apt install python3 python3-pip python3-venv
```

```
1 sudo dnf install python3 python3-pip
```

```
1 brew install python3
```

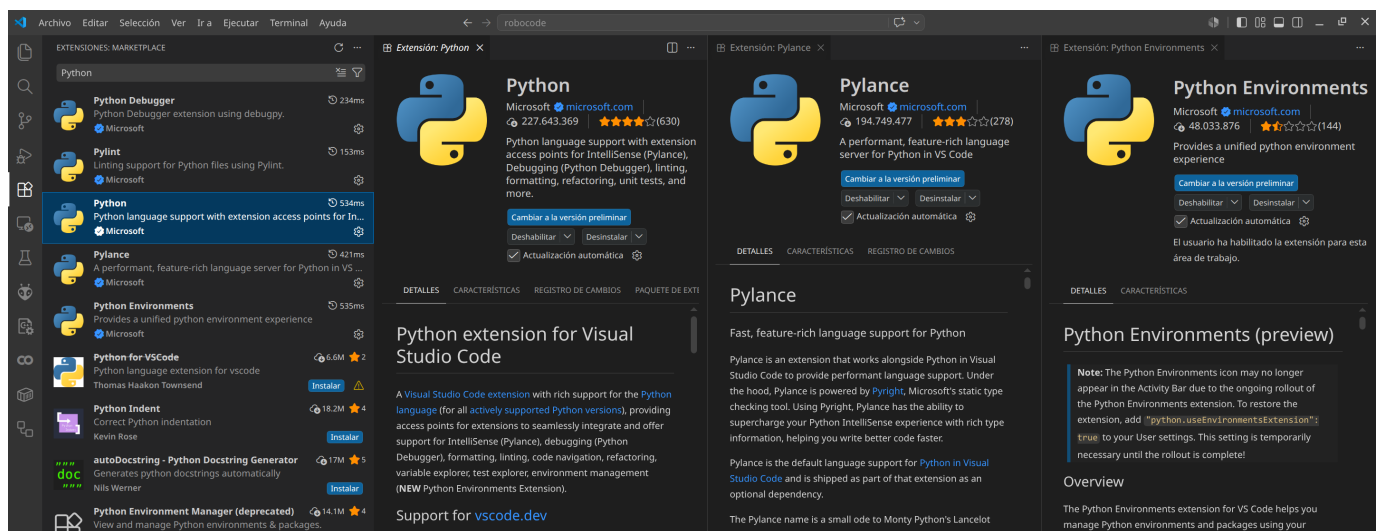
Descargad el instalador desde <https://www.python.org/downloads/> y marcad la opción "Add Python to PATH"

VS CODE Y EXTENSIONES

Si aún no tenéis VS Code, descargadlo desde <https://code.visualstudio.com/>

Una vez instalado, añadid las siguientes extensiones:

1. **Python** (de Microsoft) — todo el soporte para Python: linting, debugging, IntelliSense
2. **Pylance** (de Microsoft) — completado de código y tipado avanzado
3. *Opcional:* **Python Environments** (de Microsoft)



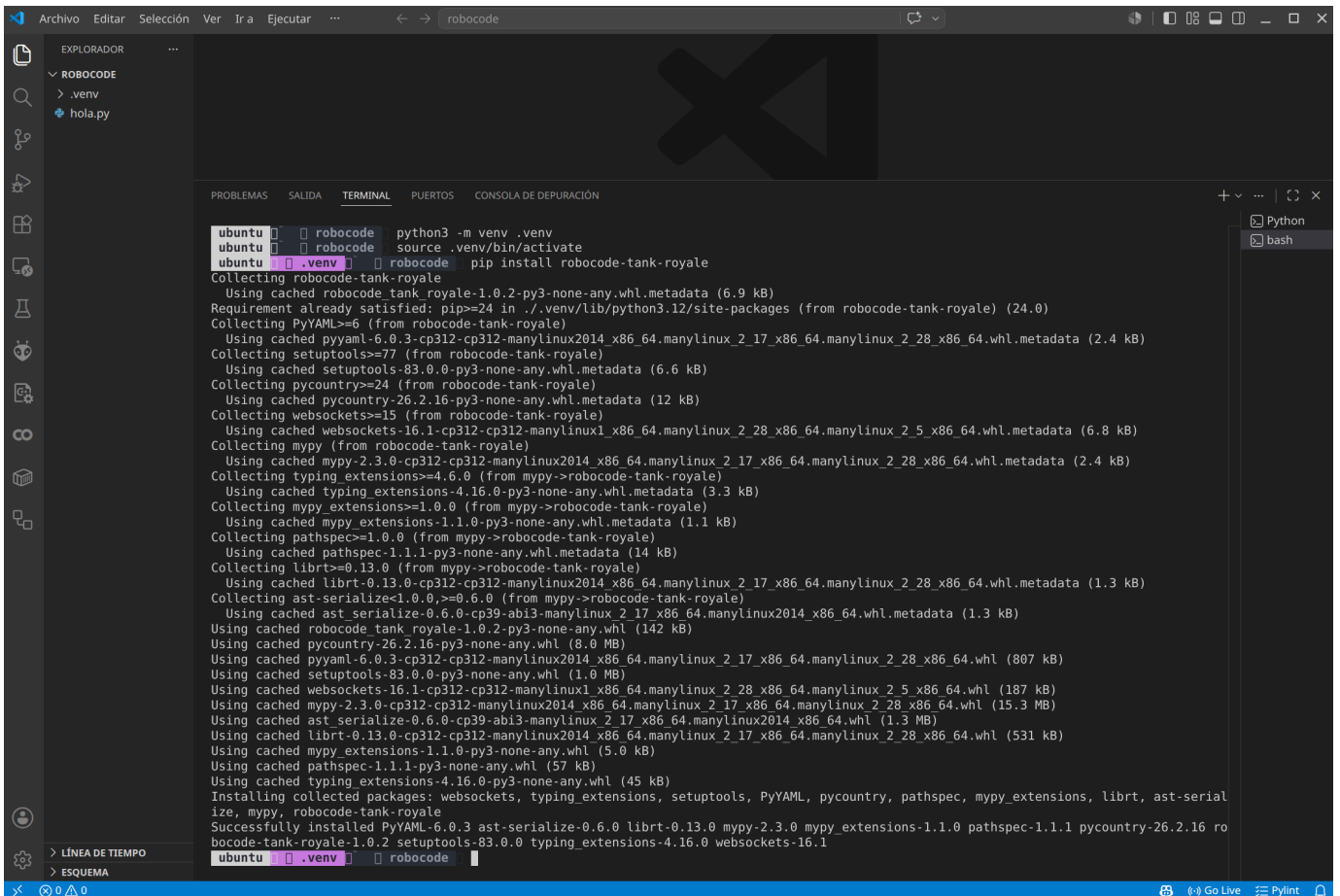
INSTALACIÓN DE LA API DE ROBOCODE

La API de Python para Robocode Tank Royale se distribuye como paquete pip:

```
1 pip install robocode-tank-royale
```

O si trabajáis con entornos virtuales:

```
1 python3 -m venv .venv
2 # Linux/macOS:
3 source .venv/bin/activate
4 # Windows PowerShell:
5 # .venv\Scripts\Activate.ps1
6 # Windows CMD:
7 # .venv\Scripts\activate.bat
8
9 pip install robocode-tank-royale
```



```

ubuntu | robocode | python3 -m venv .venv
ubuntu | robocode | source .venv/bin/activate
ubuntu | .venv | pip install robocode-tank-royale
Collecting robocode-tank-royale
  Using cached robocode_tank_royale-1.0.2-py3-none-any.whl.metadata (6.9 kB)
Requirement already satisfied: pip>=24 in ./.venv/lib/python3.12/site-packages (from robocode-tank-royale) (24.0)
Collecting PyYAML>=6 (from robocode-tank-royale)
  Using cached pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.4 kB)
Collecting setuptools>=77 (from robocode-tank-royale)
  Using cached setuptools-83.0.0-py3-none-any.whl.metadata (6.6 kB)
Collecting pycountry>=24 (from robocode-tank-royale)
  Using cached pycountry-26.2.16-py3-none-any.whl.metadata (12 kB)
Collecting websockets>=15 (from robocode-tank-royale)
  Using cached websockets-16.1-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (6.8 kB)
Collecting mypy (from robocode-tank-royale)
  Using cached mypy-2.3.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.4 kB)
Collecting typing_extensions>=4.6.0 (from mpy->robocode-tank-royale)
  Using cached typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting mpy_extensions>=1.0.0 (from mpy->robocode-tank-royale)
  Using cached mpy_extensions-1.1.0-py3-none-any.whl.metadata (1.1 kB)
Collecting pathspec>=1.0.0 (from mpy->robocode-tank-royale)
  Using cached pathspec-1.1.1-py3-none-any.whl.metadata (14 kB)
Collecting librt>=0.13.0 (from mpy->robocode-tank-royale)
  Using cached librt-0.13.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (1.3 kB)
Collecting ast_serialize>=0.6.0 (from mpy->robocode-tank-royale)
  Using cached ast_serialize-0.6.0-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (1.3 kB)
Using cached robocode_tank_royale-1.0.2-py3-none-any.whl (142 kB)
Using cached pycountry-26.2.16-py3-none-any.whl (8.0 MB)
Using cached pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (807 kB)
Using cached setuptools-83.0.0-py3-none-any.whl (1.0 MB)
Using cached websockets-16.1-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (187 kB)
Using cached mpy-2.3.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (15.3 MB)
Using cached ast_serialize-0.6.0-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.3 MB)
Using cached librt-0.13.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (531 kB)
Using cached mpy_extensions-1.1.0-py3-none-any.whl (5.0 kB)
Using cached pathspec-1.1.1-py3-none-any.whl (57 kB)
Using cached typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Installing collected packages: websockets, typing_extensions, setuptools, PyYAML, pycountry, pathspec, mpy_extensions, librt, ast-serial
ize, mpy, robocode-tank-royale
Successfully installed PyYAML-6.0.3 ast-serialize-0.6.0 librt-0.13.0 mpy-2.3.0 mpy_extensions-1.1.0 pathspec-1.1.1 pycountry-26.2.16 ro
bocode-tank-royale-1.0.2 setuptools-83.0.0 typing_extensions-4.16.0 websockets-16.1
ubuntu | .venv | robocode |

```

La GUI de Robocode Tank Royale

DESCARGA Y EJECUCIÓN DE LA GUI

La GUI (interfaz gráfica) de Robocode es **independiente del lenguaje**. El proceso es idéntico tanto si usáis Java como Python:

1. Id a <https://github.com/robocode-dev/tank-royale/releases>
2. Descargad la última versión del fichero `robocode-tankroyale-gui-x.y.z.jar`
3. Ejecutadlo:

```
1 java -jar robocode-tankroyale-gui-x.y.z.jar
```

⚠ Java necesario para la GUI

La GUI requiere Java 11 o superior para ejecutarse, independientemente de que vuestro bot esté escrito en Python. Si no tenéis Java, ved el [Taller 3: preparar el entorno](#) o instalad OpenJDK:

```
1 sudo apt install default-jdk # Linux
```

CONFIGURACIÓN INICIAL DE LA GUI

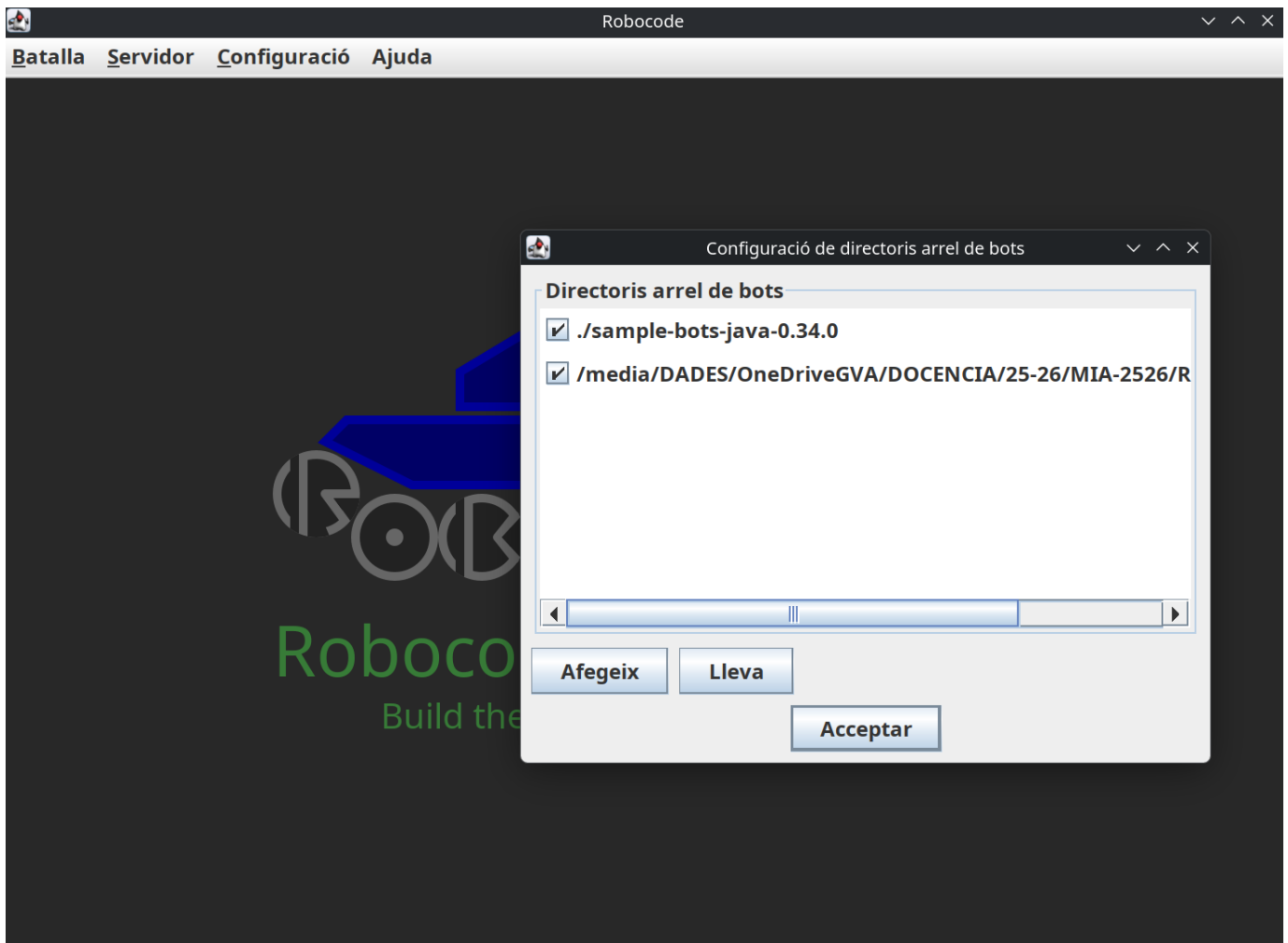
Una vez abierta la GUI:

1. Descargad y descomprimid los **Bots de ejemplo** desde la misma página de releases
2. Id a Config → Bot Root Directories y añadid la carpeta donde habéis descomprimido los bots de ejemplo
3. *Opcional*: descargad y descomprimid `sounds.zip` de los [releases de sonidos](#) en el mismo directorio que el JAR

```

1 [directorio padre]
2 |─ robocode-tankroyale-gui-x.y.z.jar
3 |─ sounds/
4 |   └─ bots_collision.wav
5 |   ...
6 |   └─ wall_collision.wav
7 |─ bots/      <-- carpeta de bots de ejemplo

```



Consejo

Jugad un poco con el entorno GUI, explorad las opciones, tipos de batallas, cread alguna ronda, inicializad bots, añadidlos, ajustad la velocidad de juego, etc.

Vuestro primer Bot en Python

ESTRUCTURA DEL PROYECTO

Cread una carpeta para vuestro bot (por ejemplo, `MiBot`) con la siguiente estructura:

```

1 MiBot/
2 |─ MiBot.py
3 |─ MiBot.json
4 |─ MiBot.cmd      (Windows)
5 |─ MiBot.sh       (Linux/macOS)

```

FICHERO DE CONFIGURACIÓN JSON

El bot necesita un fichero `.json` con la información del bot. Cread `MiBot.json`:

```

1  {
2      "name": "MiBot",
3      "version": "1.0",
4      "authors": ["Vuestro Nombre"],
5      "description": "Mi primer bot con Python",
6      "homepage": "",
7      "countryCodes": ["es"],
8      "platform": "Python",
9      "programmingLang": "Python 3.x"
10 }

```

CÓDIGO BÁSICO DEL BOT

Cread `MiBot.py` con el siguiente contenido:

```

1  import os
2  from robocode_tank_royale.bot_api import Bot, BotInfo
3  from robocode_tank_royale.bot_api.events import ScannedBotEvent, HitByBulletEvent
4
5  class MiBot(Bot):
6      def __init__(self):
7          config_path = os.path.join(os.path.dirname(__file__), "MiBot.json")
8          super().__init__(BotInfo.from_file(config_path))
9
10
11     def run(self):
12         while self.is_running: # type: ignore[attr-defined] # pylint: disable=no-member
13             self.forward(100)
14             self.turn_gun_left(360)
15             self.back(100)
16             self.turn_gun_left(360)
17
18     def on_scanned_bot(self, scanned_bot_event: ScannedBotEvent):
19         del scanned_bot_event
20         self.fire(1)
21
22     def on_hit_by_bullet(self, hit_by_bullet_event: HitByBulletEvent):
23         bearing = self.calc_bearing(hit_by_bullet_event.bullet.direction)
24         self.turn_right(90 - bearing)
25
26
27     def main():
28         bot = MiBot()
29         bot.start()
30
31
32     if __name__ == "__main__":
33         main()

```



VS Code: Pylint/Pylance no encuentran el paquete

Si después de seleccionar el intérprete del venv (`Ctrl+Shift+P` → "Python: Select Interpreter" → `./venv/bin/python`) los errores persisten, es porque Pylint usa su propio intérprete y Pylance necesita el fichero `.vscode/settings.json`.

Cread la carpeta `.vscode/` dentro de la carpeta de vuestro bot (`/home/ubuntu/robocode/MiBot/`) y añadid el siguiente fichero:

```

1  {
2      "python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python",
3      "python.linting.pylintEnabled": false,
4      "python.linting.enabled": false,
5      "python.analysis.autoImportCompletions": true
6  }

```

Si no tenéis Pylint instalado en el venv (solo lo tiene el sistema), VS Code puede ejecutar el Pylint del sistema que no ve los paquetes instalados en el venv. Opciones: (1) desactivar Pylint (`"python.linting.enabled": false`) y confiar solo en Pylance, o (2) instalar Pylint en el venv: `pip install pylint`.

Los subrayados rojos desaparecerán al recargar la ventana (`Ctrl+Shift+P` → "Developer: Reload Window").

SCRIPTS DE INICIO

Windows (`MiBot.cmd`)

```

1  @echo off
2  python MiBot.py %*

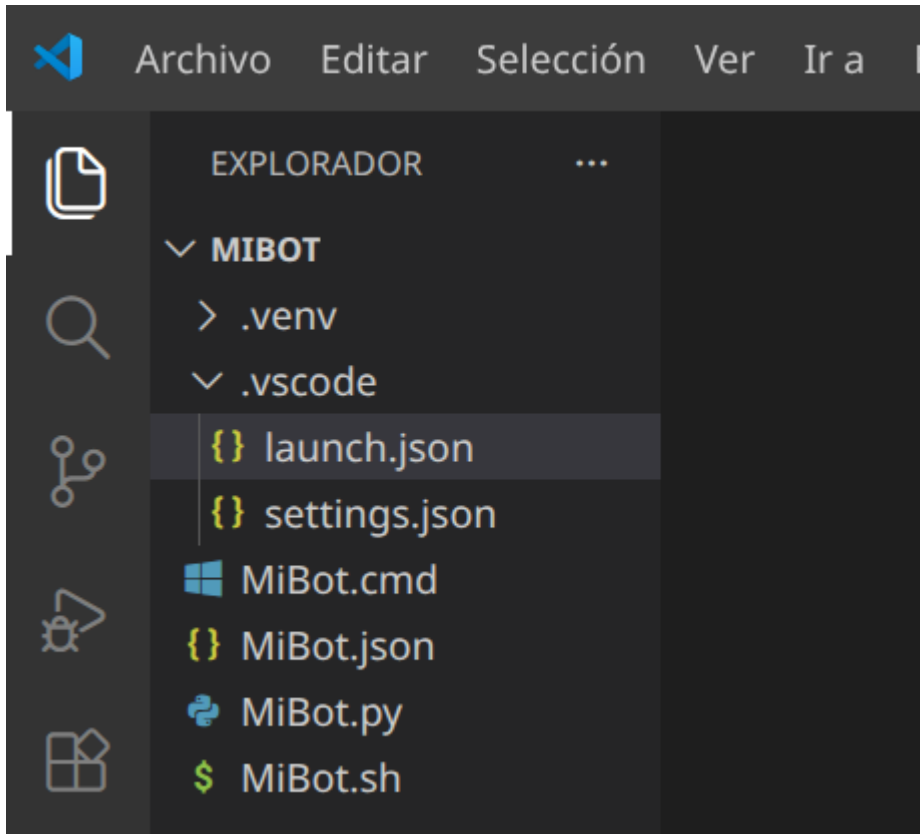
```


Linux/macOS (MiBot.sh)

```
1 #!/usr/bin/env sh
2 python3 "$(dirname "$0")/MiBot.py" "$@"
```

Hacedlo ejecutable:

```
1 chmod 755 MiBot.sh
```



PRUEBA DEL BOT

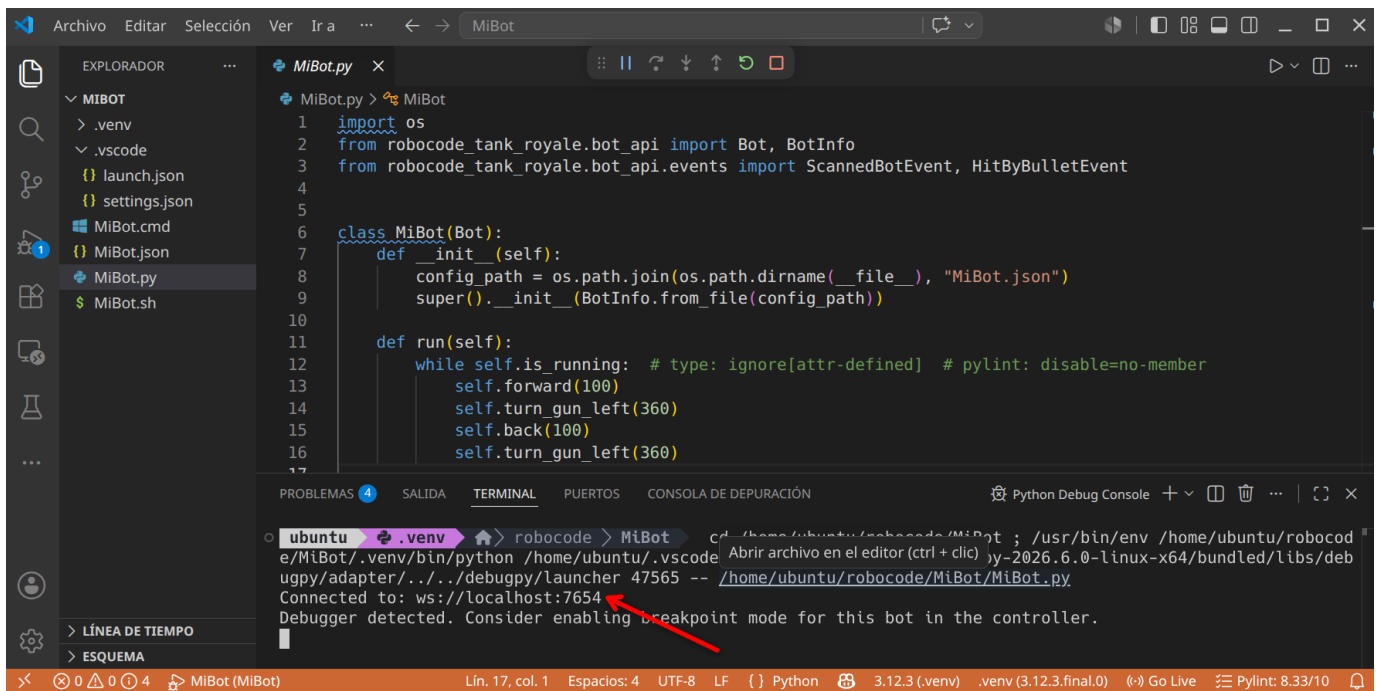
1. Iniciad la GUI de Robocode (`java -jar robocode-tankroyale-gui-x.y.z.jar`)
2. Abrid un terminal en el directorio de vuestro bot
3. Ejecutad:

```
1 python3 MiBot.py
```

Si todo va bien, veréis un mensaje similar a:

```
1 Connected to: ws://localhost:7654
```

Y vuestro bot aparecerá en la lista de bots disponibles en la GUI.



Configuración del servidor y secrets

SECRETS DEL SERVIDOR

El servidor de Robocode puede requerir una contraseña (secret) para conectarse. Esta contraseña se genera automáticamente la primera vez que lanzáis la GUI.

Para habilitar los secrets:

1. Id a Config → Server Options → Enable server secrets en la GUI
2. O editad `server.properties` y añadid:

```
1 server-secrets-enabled=true
2 bots-secrets=CEIABDEPM2025
```

CONECTAR EL BOT CON SECRET

Estableced la variable de entorno `SERVER_SECRET` antes de ejecutar el bot:

Linux/macOS

Windows CMD

Windows PowerShell

```
1 export SERVER_SECRET=CEIABDEPM2025
2 python3 MiBot.py
```

```
1 set SERVER_SECRET=CEIABDEPM2025
2 python MiBot.py
```

```
1 $Env:SERVER_SECRET = "CEIABDEPM2025"
2 python MiBot.py
```

CONECTARSE A UN SERVIDOR REMOTO

Si el profesor indica una IP remota, podéis especificarla con `SERVER_URL` :

Linux/macOS

Windows CMD

```
1 export SERVER_URL=ws://192.168.1.100:7654
2 export SERVER_SECRET=CEIABDEPM2025
3 python3 MiBot.py
```

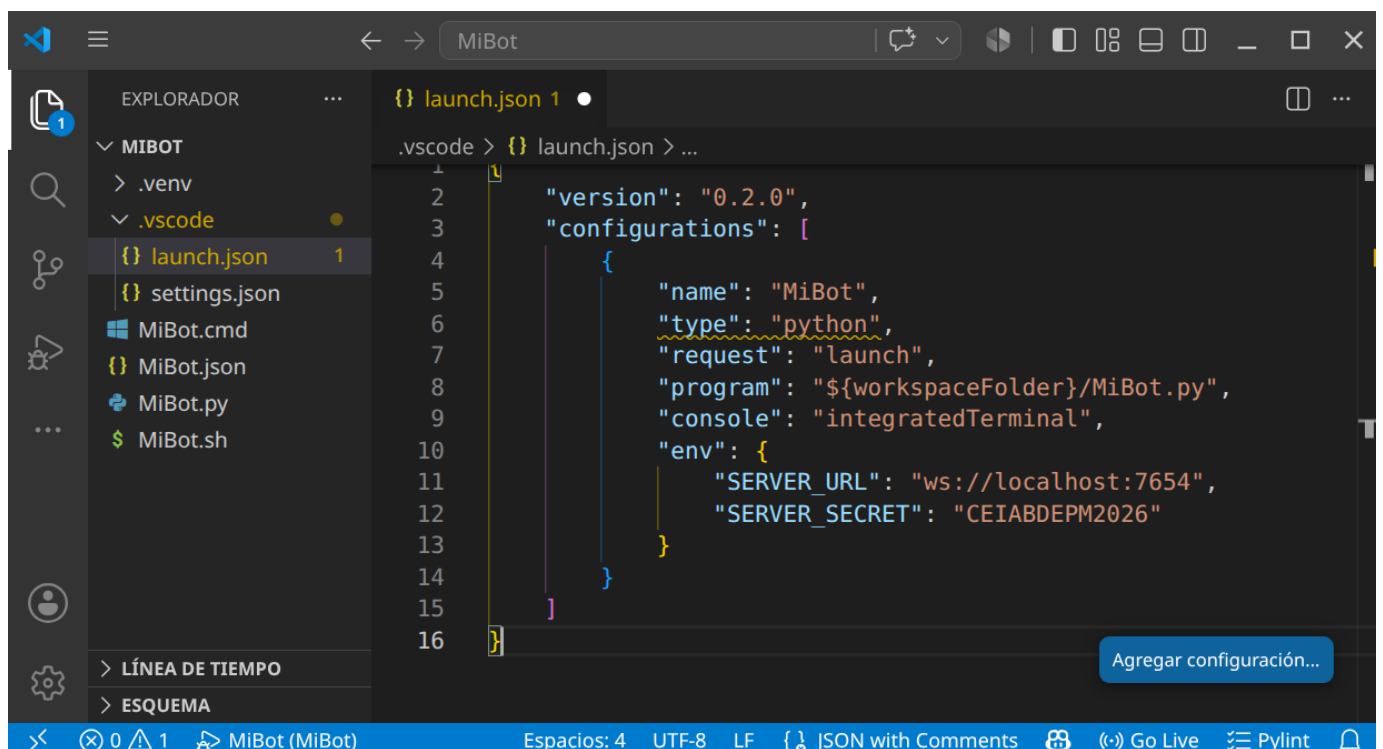
```
1 set SERVER_URL=ws://192.168.1.100:7654
2 set SERVER_SECRET=CEIABDEPM2025
3 python MiBot.py
```

USO CON VS CODE (LAUNCH.JSON)

Para mayor comodidad, podéis configurar VS Code para ejecutar el bot con las variables de entorno definidas en el fichero `.vscode/launch.json` :

Cread la carpeta `.vscode/` dentro del proyecto y el fichero `launch.json` :

```
1 {
2   "version": "0.2.0",
3   "configurations": [
4     {
5       "name": "MiBot",
6       "type": "python",
7       "request": "launch",
8       "program": "${workspaceFolder}/MiBot.py",
9       "console": "integratedTerminal",
10      "env": {
11        "SERVER_URL": "ws://localhost:7654",
12        "SERVER_SECRET": "CEIABDEPM2025"
13      }
14    }
15  ]
16 }
```



Ahora podéis ejecutar el bot directamente desde VS Code pulsando `F5` o yendo a `Run` → `Start Debugging` .

**El servidor debe estar en marcha**

El servidor (la GUI) debe estar inicializado **antes** de lanzar el bot. Si no, obtendréis un error:

```
1 Could not create web socket for URL: ws://localhost:7654
```

API de Python: diferencias clave con Java

La API de Python utiliza **snake_case** en lugar de **camelCase**. Aquí tenéis una tabla de correspondencia:

Java	Python
<code>setTurnLeft(degrees)</code>	<code>set_turn_left(degrees)</code>
<code>setForward(dist)</code>	<code>set_forward(dist)</code>
<code>setTurnRadarLeft(degrees)</code>	<code>set_turn_radar_left(degrees)</code>
<code>setAdjustRadarForBodyTurn(bool)</code>	<code>set_adjust_radar_for_body_turn(bool)</code>
<code>distanceTo(x, y)</code>	<code>distance_to(x, y)</code>
<code>bearingTo(x, y)</code>	<code>bearing_to(x, y)</code>
<code>gunBearingTo(x, y)</code>	<code>gun_bearing_to(x, y)</code>
<code>getX()</code>	<code>self.x</code> (propiedad)
<code>getY()</code>	<code>self.y</code> (propiedad)
<code>getSpeed()</code>	<code>self.speed</code> (propiedad)
<code>getEnergy()</code>	<code>self.energy</code> (propiedad)
<code>getTurnRemaining()</code>	<code>self.turn_remaining</code> (propiedad)
<code>getGunHeat()</code>	<code>self.gun_heat</code> (propiedad)
<code>isRunning()</code>	<code>self.is_running</code> (propiedad)
<code>onScannedBot(e)</code>	<code>on_scanned_bot(self, e)</code>
<code>e.getX()</code>	<code>e.x</code> (propiedad)
<code>e.getY()</code>	<code>e.y</code> (propiedad)
<code>e.getSpeed()</code>	<code>e.speed</code> (propiedad)
<code>e.getDirection()</code>	<code>e.direction</code> (propiedad)
<code>Double.POSITIVE_INFINITY</code>	<code>float("inf")</code>
<code>Math.min(a, b)</code>	<code>min(a, b)</code>
<code>Math.cos(angle)</code>	<code>math.cos(angle)</code>
<code>Math.toRadians(degrees)</code>	<code>math.radians(degrees)</code>

¿Cómo mejoro mi Bot?

Dividiremos todo lo que debemos conocer en 4 apartados:

CONOCIENDO EL CAMPO DE BATALLA

Sigue atentamente la documentación de RCTR sobre la [anatomía de los Bots](#)

Puntos importantes:

- Si el radar no se mueve, no detecta
- Inicialmente el robot, el cañón y el radar se mueven conjuntamente (pero se puede cambiar)

- El Bot se simplifica a un cuadrado de 36x36 unidades.
- Para las colisiones (con Bots o paredes) se simula como un círculo de 18 unidades de radio.

A continuación revisa las **coordenadas y ángulos**

Puntos importantes:

- Este apartado es totalmente diferente al de Robocode Original.

Estudia también las **físicas**

Puntos importantes:

- Se frena más rápido que se acelera.
- Cuanto más rápido vas, más lento giras.
- El cañón gira máximo 20º por turno.
- El radar gira máximo 45º por turno.
- La potencia de tiro influye en el daño provocado y los puntos conseguidos, pero también en el calentamiento del cañón.
- Inicialmente el cañón está caliente (3 unidades), al principio has de esperar a que se enfríe para disparar.
- El atropello da más puntos que los disparos (pero te resta energía)
- Si chocas con una pared, pierdes puntos.
- La energía que te sobra en una ronda no da puntos (modo kamikaze con el último robot?)

Por último ten en cuenta las **puntuaciones**

Puntos importantes:

- Consigues puntos si:
- tu disparo golpea a otro Bot (sino, te resta)
- eres el que mata al Bot
- cada vez que otro Bot muere, pero tu sigues vivo
- eres el último robot vivo
- atropellas a otro Bot
- si matas por atropello a otro Bot
- El último Bot que quede vivo no tiene porqué ser el ganador.

Eventos propios

Definimos una condición, y cuando esta se cumple se dispara un evento:

```
1 from robocode_tank_royale.bot_api.events import Condition
2
3 class TurnCompleteCondition(Condition):
4     def __init__(self, bot):
5         super().__init__()
6         self.bot = bot
7
8     def test(self):
9         return self.bot.turn_remaining == 0
```

Podríamos usar esta condición en un fragmento similar a este:

```
1 self.wait_for(TurnCompleteCondition(self))
```

Podríamos usar funciones lambda de la siguiente manera:

```
1 self.wait_for(Condition(lambda: self.turn_remaining == 0))
```

El resultado de los dos fragmentos sería el mismo.

PERSONALIZAR MI BOT

Podemos establecer colores para el cuerpo, torreta de cañón, el radar, el arco de escaneo y de la bala:

```

1 verde = Color.from_rgb(0, 255, 0)
2 self.set_body_color(Color.from_rgb(0, 0, 0))
3 self.set_turret_color(Color.from_rgb(255, 0, 0))
4 self.set_radar_color(Color.from_rgb(255, 0, 0))
5 self.set_scan_color(Color.from_rgb(255, 255, 0))
6 self.set_bullet_color(Color.from_rgb(255, 255, 255))

```

Puedes encontrar la lista completa de colores disponibles en la API de Robocode TankRoyale.

TÉCNICAS DE ESCANEO (RADAR)

Sentidos de nuestro Bot:

Sentido del **tacto**, tu Bot sabe cuando:

- golpea una pared (`on_hit_wall`),
- es alcanzado por un disparo (`on_hit_by_bullet`),
- es alcanzado por otro Bot (`on_hit_bot`).

Sentido de la **vista**, tu Bot sabe cuándo ha visto otro robot, pero sólo si lo escanea (`on_scanned_bot`)

Otros sentidos, tu robot también sabe cuándo ha muerto (`on_death`), cuándo ha muerto otro robot (`on_bot_death`).

Además también es consciente de sus balas y sabe cuando una bala ha sido disparada (`on_bullet_fired`) ha alcanzado a un oponente (`on_bullet_hit`), cuando una bala golpea una pared (`on_bullet_wall_hit`) o cuando una bala golpea a otra bala (`on_bullet_hit_bullet`).

Configurar correctamente tu Bot con:

```

1 self.set_adjust_radar_for_body_turn(True) # True si el radar debe ajustar/compensar el giro del cuerpo;
2                                           # False si el radar gira con el cuerpo (por defecto).
3 self.set_adjust_gun_for_body_turn(True)   # True si el cañón debe ajustar/compensar el giro del cuerpo;
4                                           # False si el cañón gira con el cuerpo (por defecto).
5 self.set_adjust_radar_for_gun_turn(True)  # True si el radar debe ajustar/compensar el giro del cañón;
6                                           # False si el radar gira con el cañón (por defecto).

```

Que el radar siempre de vueltas:

```

1 self.set_turn_radar_left(float("inf")) # grados - cantidad de grados a girar a la izquierda. Si es negativo, gira a la derecha.

```

Revisa el API (`rescan()` y `set_rescan()`)

Fijar el radar en un enemigo (one on one radar):

- Escaneo con multiplicador

```

1 def on_scanned_bot(self, e: ScannedBotEvent):
2     factor = 1.9
3     self.set_turn_radar_left(factor * self.radar_bearing_to(e.x, e.y))

```

Según el factor:

- 1.0 - Bloqueo de radar delgado. Debe llamar a `scan()` para evitar perder el bloqueo. Que Dios te ayude si alguna vez te saltas un turno.
- 1.9 - El arco del radar comienza amplio y se estrecha lentamente tanto como sea posible mientras se mantiene en el objetivo.
- 2.0: el arco del radar recorre un ángulo fijo. El ángulo exacto elegido depende de las posiciones del enemigo y del radar cuando se detecta al enemigo por primera vez. El ángulo se incrementará si es necesario para mantener el bloqueo.
- Arco de radar Ancho

```

1  def on_scanned_bot(self, e: ScannedBotEvent):
2      wide = 36.0
3      # Ángulo absoluto hacia el objetivo
4      angle_to_enemy = self.radar_bearing_to(e.x, e.y)
5      # Distancia que queremos escanear desde el centro del enemigo a cada lado
6      extra_turn = min(
7          math.degrees(math.atan(wide / self.distance_to(e.x, e.y))),
8          self.max_radar_turn_rate
9      )
10     # Ajustamos el giro del radar para que vaya más allá en la dirección que ya gira
11     if angle_to_enemy < 0:
12         angle_to_enemy -= extra_turn
13     else:
14         angle_to_enemy += extra_turn
15     # Giramos el radar
16     self.set_turn_radar_left(angle_to_enemy)

```

- Radar Oscilante (variable global que sepa la dirección y nos ayude a cambiarla de vez en cuando)
- Escaneo más inteligente (seguir al cercano? según la situación?)
- Nos interesa mantener una lista de enemigos? `e.scanned_bot_id`? y por ejemplo fijar el radar en el más débil?

TÉCNICAS DE DESPLAZAMIENTO (MOVIMIENTO)

Pensamiento lateral

Si has jugado baloncesto antes, sabes que si quieres defender a alguien que sostiene el balón, debes maximizar tu movimiento lateral enfrentándote siempre a él (de frente). Lo mismo ocurre con tu robot.

Para conseguir esta posición debemos hacer algo similar a esto:

```

1  self.turn_left(self.bearing_to(e.x, e.y) + 90)

```

que siempre colocará tu robot perpendicular (90 grados) a tu enemigo.

¿Hacia adelante o hacia atrás?

Cuando te enfrentas a un oponente, la idea de "adelante" y "atrás" (del primer Bot) se vuelven algo obsoletas. Probablemente estés pensando más en términos de "atacar a la izquierda" o "atacar a la derecha". Para realizar un seguimiento de la dirección del movimiento, simplemente declara una variable como hicimos para oscilar el radar.

```

1  class MiBot(Bot):
2      def __init__(self):
3          super().__init__(BotInfo.from_file("MiBot.json"))
4          self.move_direction = 1

```

luego, cuando quieras mover tu robot, simplemente puedes decir:

```

1  self.set_forward(100 * self.move_direction)

```

Puedes cambiar de dirección cambiando el valor de `move_direction` de 1 a -1 así:

```

1  self.move_direction *= -1

```

Cambiar de dirección

El enfoque más intuitivo para cambiar de dirección es simplemente cambiar la dirección del movimiento cada vez que golpeas una pared o golpeas a otro robot de esta manera:

```

1  def on_hit_wall(self, e: HitWallEvent):
2      self.move_direction *= -1
3
4  def on_hit_bot(self, e: HitBotEvent):
5      self.move_direction *= -1

```

Sin embargo, descubrirás que si haces eso, terminarás presionando obstinadamente contra un robot que te embiste desde un costado (como un perro en celo). Esto se debe a que se llama a `on_hit_bot` tantas veces que `move_direction` sigue cambiando y nunca te alejas.

Un mejor enfoque es simplemente probar para ver si tu robot se ha detenido. Si es así, probablemente significa que has golpeado algo y querrás cambiar de dirección. Puedes hacerlo con el código:

```

1  if self.speed == 0:
2      self.move_direction *= -1

```

Ponlo en tu método que maneje el movimiento y podrás manejar todos los eventos de impacto con las paredes u otros Bots.

¿Bailamos?

Dando vueltas (Círculos)

Puedes rodear a tu enemigo simplemente usando las técnicas anteriores:

```

1  class MiBot(Bot):
2      def __init__(self):
3          super().__init__(BotInfo.from_file("MiBot.json"))
4          self.move_direction = 1
5          self.ultimo_enemigo_visto = 0
6
7      def run(self):
8          while self.is_running: # type: ignore[attr-defined] # pylint: disable=no-member
9              self.do_move()
10             self.go()
11
12     def on_scanned_bot(self, e: ScannedBotEvent):
13         self.ultimo_enemigo_visto = self.bearing_to(e.x, e.y)
14
15     def do_move(self):
16         if self.speed == 0:
17             self.move_direction *= -1
18         self.set_turn_left(self.ultimo_enemigo_visto + 90)
19         self.set_forward(1000 * self.move_direction)

```

Objetivo: rodea a tu enemigo usando el código de movimiento anterior, como un tiburón rodeando a su presa en el agua.

Evita Ametrallamiento

Un problema que puedes notar con el anterior tipo de movimiento es que es presa fácil para los objetivos predictivos porque sus movimientos son tan... predecibles.

Para evadir las balas de manera más efectiva, debes moverte de lado a lado o "atacar". Una buena forma de hacerlo es cambiar de dirección después de un cierto número de "tics", así:

```

1  def do_move(self):
2      if self.speed == 0:
3          self.move_direction *= -1
4          self.set_turn_left(self.ultimo_enemigo_visto + 90)
5      if self.turn_number % 20 == 0:
6          self.move_direction *= -1
7          self.set_forward(1000 * self.move_direction)

```

Mira como se balancea hacia adelante y hacia atrás usando el código de movimiento anterior. Observa lo bien que esquivo las balas. Pero ojo, porque puede afectar a tu precisión de disparo.

Cada vez más cerca

Notarás que tanto el primero como este último tipo de movimientos tienen otro problema: se atascan fácilmente en las esquinas y terminan golpeándose contra las paredes. Un problema adicional es que si tu enemigo está lejos, disparan mucho pero no aciertan mucho.

Para hacer que tu robot se acerque a tu enemigo, simplemente modifica el código para que se gire ligeramente hacia su enemigo, así:

```

1  self.set_turn_left(self.ultimo_enemigo_visto + (90 - (15 * self.move_direction)))

```

15 es un factor en grados que puedes modificar y ajustar. ¡Prueba!

Modificando el primero conseguimos una variación que utiliza el código anterior para lanzarse en espiral hacia su enemigo.

Mientras que con el segundo que utiliza el código anterior para acercarse cada vez más. También es bastante bueno para evadir balas.

Fíjate que ninguno de los Bots anteriores queda atrapado en una esquina por mucho tiempo.

Evitando las paredes

Un problema con todos los Bots anteriores es que golpean mucho las paredes y golpearlas agota tu energía. Una mejor estrategia sería detenerse antes de chocar contra las paredes. ¿Pero cómo?

Agregar un evento personalizado

Lo primero que debemos hacer es decidir qué tan cerca permitiremos que nuestro robot llegue a las paredes:

```
1 class WallAvoider(Bot):
2     def __init__(self):
3         super().__init__(BotInfo.from_file("WallAvoider.json"))
4         self.wall_margin = 60
5         self.too_close_to_wall = 0
6         self.move_direction = 1
```

A continuación, agregamos un evento personalizado que se activará cuando se cumpla una determinada condición dentro del método `run()`:

```
1 def run(self):
2     # ... inicialización ...
3     # No te acerques mucho a las paredes
4     self.add_custom_event(Condition("TooCloseToWalls", lambda: (
5         self.x <= self.wall_margin or
6         self.x >= self.arena_width - self.wall_margin or
7         self.y <= self.wall_margin or
8         self.y >= self.arena_height - self.wall_margin
9     )))
10    # ... resto del bucle ...
```

Necesitamos hacer override del método `test()` para devolver un valor booleano cuando ocurra nuestro evento personalizado.

Gestionar el evento personalizado

Lo siguiente que debemos hacer es manejar el evento, que se puede hacer así:

```
1 def on_custom_event(self, e: CustomEvent):
2     if e.condition.name == "TooCloseToWalls":
3         self.move_direction *= -1
4         self.set_forward(100 * self.move_direction)
```

Sin embargo, el problema con ese enfoque es que este evento podría dispararse una y otra vez, provocando que cambiemos rápidamente de un lado a otro, sin alejarnos nunca. O si ya aparecemos cerca de la pared quedamos atrapados.

Para evitar este "sacudida de la muerte" deberíamos tener una variable que indique que estamos manejando el evento:

```
1 def on_custom_event(self, e: CustomEvent):
2     if e.condition.name == "TooCloseToWalls":
3         if self.too_close_to_wall <= 0:
4             self.too_close_to_wall += self.wall_margin
5             self.set_max_speed(0)
```

Manejo de los dos modos

Hay dos últimos problemas que debemos resolver. En primer lugar, tenemos un método `do_move()` donde colocamos todo nuestro código de movimiento normal. Si estamos tratando de alejarnos de una pared, no queremos que se llame a nuestro código de movimiento normal, creando (una vez más) la "sacudida de la muerte". En segundo lugar, queremos volver eventualmente al movimiento "normal", por lo que eventualmente deberíamos tener el "tiempo de espera" de la variable `too_close_to_wall`.

Podemos resolver ambos problemas con la siguiente implementación de `do_move()`:

```
1 def do_move(self):
2     if self.too_close_to_wall > 0:
3         self.too_close_to_wall -= 1
4     if self.speed == 0:
5         self.set_max_speed(8)
6         self.move_direction *= -1
7         self.set_forward(1000 * self.move_direction)
```

Con todo el código anterior evitamos chocar contra las paredes. Observa cómo se desliza suavemente hacia los lados pero nunca (bueno, rara vez) los golpea.

Bot multimodo

Además de los colores que elijas, la mayor parte de la personalidad de tu robot está en su código de movimiento. Por otra parte, situaciones diferentes requieren tácticas diferentes. Usando el ejemplo de evitar paredes, es posible que desees codificar tu bot para que cambie los "modos" según ciertos criterios. Podrías pensar en algo como esto:

```

1  MODO Rondar = 0
2  MODO Esquivar = 1
3  MODO Asesino = 2
4
5  class MultiModeBot(Bot):
6      def __init__(self):
7          super().__init__(BotInfo.from_file("MultiModeBot.json"))
8          self.modo_robot = MODO Rondar
9
10     def on_bot_death(self, e: BotDeathEvent):
11         if self.enemy_count > 10:
12             self.modo_robot = MODO Rondar
13         elif self.enemy_count > 1:
14             self.modo_robot = MODO Esquivar
15         elif self.enemy_count == 1:
16             self.modo_robot = MODO Asesino
17
18     def do_move(self):
19         if self.modo_robot == MODO Rondar:
20             # ronda
21             pass
22         elif self.modo_robot == MODO Esquivar:
23             # esquivo
24             pass
25         elif self.modo_robot == MODO Asesino:
26             # asesina
27             pass

```

Los detalles se dejan (como siempre) como ejercicio para el alumnado.

TÉCNICAS DE ATAQUE (DISPARO)

Técnicas básicas

- Separa el movimiento del cañón del movimiento del Bot (`set_adjust_gun_for_body_turn`)
- Técnica de apuntado básica: `set_turn_gun_left` o `set_turn_gun_right` junto con `gun_bearing_to`

Fórmula de cálculo de potencia de fuego

Otro aspecto importante al disparar es calcular la potencia de fuego de tu bala. La documentación del método `fire()` explica que puedes disparar una bala en el rango de 0.1 a 3.0. Como probablemente ya habrás concluido, es una buena idea disparar balas de baja potencia cuando tu enemigo está lejos y balas de alta resistencia cuando está cerca.

```

1  def smart_fire(self, distance):
2      if distance > 200 or self.energy < 15:
3          self.fire(1)
4      elif distance > 50:
5          self.fire(2)
6      else:
7          self.fire(3)

```

Podrías usar una serie de declaraciones if-elif-else para determinar la potencia de fuego, en función de si el enemigo está a 50 unidades, a 200, etc (como en el fragmento anterior). Pero tales construcciones son demasiado rígidas. Después de todo, el rango de posibles valores de potencia de fuego cae a lo largo de un continuo, no de bloques discretos. Un mejor enfoque es utilizar una fórmula. He aquí un ejemplo:

```

1  self.set_fire(400 / self.distance_to(e.x, e.y))

```

Con esta fórmula, a medida que aumenta la distancia del enemigo, la potencia de fuego disminuye. Asimismo, a medida que el enemigo se acerca, la potencia de fuego aumenta. Los valores superiores a 3 se reducen a 3, por lo que nunca dispararemos una bala mayor que 3, pero probablemente deberíamos reducir el valor de todos modos (solo para estar seguros) de esta manera:

```

1  self.set_fire(min(500 / self.distance_to(e.x, e.y), 3))

```

Evitar disparos prematuros

Una situación que encontraras es que tu Bot dispare antes de haber girado el arma hacia el objetivo. Para evitar disparos prematuros, usa la propiedad `gun_turn_remaining` para ver qué tan lejos está tu cañón del objetivo y no dispare hasta que esté cerca.

Además, no puedes disparar si el arma está "caliente" desde el último disparo y llamar a `fire()` o `set_fire()` solo desperdiciará un turno. Podemos probar si el arma está fría llamando a `gun_heat`.

Apuntando de manera predictiva

O... "Usar la trigonometría para impresionar a tus amigos y destruir a tus enemigos".

Si quisiéramos poder golpear a un robot que recorre las paredes siempre fallaríamos, necesitamos poder predecir dónde estará en el futuro, pero ¿cómo podemos hacerlo?

$\backslash$ [Distancia = Velocidad $\times$ Tiempo]

Usando la anterior fórmula podemos calcular cuánto tiempo tardará una bala en llegar allí.

- **Distancia:** se puede encontrar llamando a `distance_to(e.x, e.y)`
- **Velocidad:** según la documentación de RCTR, una bala viaja a una velocidad de: $\$ \$ 20 - potenciaDeFuego \times 3 \$ \$$
- **Tiempo:** podemos calcular el tiempo resolviendo: $\$ \$ Tiempo = \{Distancia \over Velocidad\} \$ \$$

El siguiente código lo hace:

```
1 # calcular la potencia basado en la distancia
2 enemy_distance = self.distance_to(e.x, e.y)
3 fire_power = min(500 / enemy_distance, 3)
4 # calcular la velocidad de la bala
5 bullet_speed = 20 - fire_power * 3
6 # calculamos el tiempo que necesita la bala para impactar al enemigo
7 time = int(enemy_distance / bullet_speed)
```

Obteniendo futuras coordenadas X, Y

A continuación, debemos calcular la posición futura de nuestro enemigo:

```
1 import math
2
3 # Calcular la velocidad actual de tu enemigo
4 enemy_velocity = e.speed
5 # Calcular la dirección actual del enemigo
6 enemy_direction = e.direction
7 # Descomponer el desplazamiento en las coordenadas X e Y
8 # Componente X del desplazamiento COSENO
9 delta_x = enemy_velocity * math.cos(math.radians(enemy_direction))
10 # Componente Y del desplazamiento SENO
11 delta_y = enemy_velocity * math.sin(math.radians(enemy_direction))
12
13 # Calcular las coordenadas futuras
14 future_x = e.x + (delta_x * time)
15 future_y = e.y + (delta_y * time)
```

Girando el arma al punto previsto

Ahora debemos apuntar nuestro cañón al lugar previsto de impacto:

```
1 self.set_turn_gun_left(self.gun_bearing_to(future_x, future_y))
```

¡Disparando!

Por último disparamos nuestro cañón:

```
1 if self.gun_turn_remaining <= 0 and self.gun_heat == 0:
2     self.fire(fire_power)
```

Mejor aún...!

Aquí te dejo algunas mejoras para que las valores:

- ¿Debemos esperar siempre a que el arma esté completamente en la dirección calculada?
- Cuando nuestro enemigo se acerca a una pared, nuestros disparos impactan en ella.
- ¿Puedo saber a quién disparo si mi previsión de impacto falla mucho?

Investigación y desarrollo propio

A partir de aquí el trabajo será individual de cada alumno (puede solaparse con el paso 3). Deberéis investigar/prever a vuestros adversarios (los conocidos, y los de vuestros compañeros). Aplicar las técnicas que consideréis más útiles para intentar quedar lo más arriba posible en la tabla de clasificación.

Algunos recursos adicionales:

- [Documentación oficial de la API Python](#)
- [Tutorial oficial "My First Bot for Python"](#)
- [The Book of Robocode](#) — estrategias avanzadas
- [RoboWiki](#) — conocimiento comunitario (código en Java, conceptos universales)

🕒 15 de julio de 2026

5. UD05

5.1 UD05 — Sistemas expertos y controladores inteligentes

Unidad 5 · 12 h · semanas 11-15 (7 de diciembre al 14 de enero)

Continúa las reglas y la lógica difusa de RA2, justo antes de que la fragmentación de Navidad parta el curso. Se evalúa con **cinco entregables prácticos** y la prueba escrita del RA5.

1. Introducción

Todo el curso ha sido **cómo construir** sistemas de IA. Esta unidad da un paso más: los **sistemas expertos**, que codifican el conocimiento de un experto humano en reglas y lo utilizan para **diagnosticar, asesorar y controlar**. Y lo conectamos con el **control**, porque un sistema experto puede actuar como **controlador inteligente** de un proceso físico, compitiendo con los controladores clásicos (PID).

Antes de construir nada hay que situar **qué es el conocimiento** y **cómo se representa** para que una máquina pueda usarlo — esa es la primera pregunta de la inteligencia artificial simbólica, y la respondemos con la jerarquía DIKW y el continuo de representaciones del §5.

El recorrido de la unidad:

1. **Arquitectura y dinámica** de los sistemas expertos (base de conocimiento, motor de inferencia, ciclo reconocer-actuar).
2. **Estructuras elementales** de representación del conocimiento (reglas, marcos, lógica, ontologías, redes semánticas).
3. **Representar y simular comportamientos** con la librería `experta`, en dominios muy diversos: diagnóstico médico, clasificación, control de procesos.
4. **Sistemas híbridos reglas/datos**: cuando las reglas se combinan o se extraen del aprendizaje automático.
5. **Sistemas de razonamiento impreciso**: la lógica difusa, para trabajar con lo que no es blanco o negro.
6. **Cómo influye la variación de las características** en la dinámica del sistema (sensibilidad y robustez).
7. **Estrategias de control** de un sistema experto.
8. **Controladores inteligentes** (difuso, por reglas, redes neuronales, MPC) y su influencia en el comportamiento del sistema.
9. **Aplicaciones y tendencias** (MYCIN, XCON, BRMS, neuro-simbólico, IA explicable).

Hilo conductor de la unidad

Un sistema experto convierte el conocimiento de un experto en reglas que pueden diagnosticar, asesorar y controlar. Primero situamos qué es ese conocimiento y cómo se representa; después lo construimos y simulamos, con reglas puras, híbridas o difusas; luego estudiamos por qué a veces falla; y por último lo usamos como controlador, comparándolo con el PID clásico.

2. Resultado de aprendizaje y criterios de evaluación

RA5 — Aplica sistemas expertos evaluando la influencia de los controladores inteligentes en el comportamiento del sistema.

CE	Criterio de evaluación	Bloque
RA5-a	Se ha descrito la dinámica y las estructuras elementales de los sistemas expertos.	\$4-5
RA5-b	Se han determinado las destrezas necesarias para representar y simular comportamientos básicos de sistemas de muy diversos ámbitos.	\$6-8
RA5-c	Se ha razonado cómo influye la variación de las características de los sistemas en su dinámica de actuación.	\$9
RA5-d	Se han desarrollado estrategias de control definiendo los objetivos y las especificaciones de la respuesta del sistema.	\$10
RA5-e		\$11

CE	Criterio de evaluación	Bloque
	Se han relacionado los controladores inteligentes con el comportamiento del sistema.	



Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo llama a este bloque, textualmente, «*Sistemas Expertos*», y le asigna estos contenidos:

- *Dinámica de los sistemas expertos.*
- *Estructuras elementales de los sistemas expertos.*
- *Representar y simular comportamientos básicos.*
- *Estrategias de control de un sistema experto.*
- *Aplicaciones de sistemas expertos.*
- *Tendencias en sistemas expertos.*

Son **seis contenidos para cinco criterios de evaluación**, y no encajan uno a uno: los dos últimos (aplicaciones y tendencias) no tienen CE propio y se reparten entre todos, mientras que RA5-c (variación de las características) **no tiene contenido explícito** en el anexo y lo desarrolla el centro (art. 3.3 del Decreto 95/2026).

3. Objetivos de la unidad

Objetivo	Al terminar la unidad serás capaz de...
O1	Situ ar el conocimiento en la jerarquía DIKW y descri bir la arquitectura de un sistema experto.
O2	Diferenciar los modos de representación del conocimiento (reglas, frames, lógica, ontologías, redes semánticas).
O3	Representar y simular un comportamiento básico con <i>experta</i> , en un dominio real.
O4	Combinar reglas con datos cuando el conocimiento experto no basta por sí solo.
O5	Aplicar lógica difusa a un problema con incertidumbre.
O6	Explicar cómo influye la variación de reglas, hechos y umbrales en la dinámica (sensibilidad/robustez).
O7	Desarrollar estrategias de control (salience, control de meta) con sus especificaciones.
O8	Relacionar los controladores inteligentes (difuso, por reglas, ANN, MPC) con el comportamiento del sistema y compararlos con un PID.
O9	Conocer aplicaciones reales y tendencias de los sistemas expertos.

4. Arquitectura y dinámica de los sistemas expertos (RA5-a)

4.1 Del dato al conocimiento: la jerarquía DIKW

Antes de representar el conocimiento hay que distinguirlo de sus vecinos. La **jerarquía DIKW** (*Data, Information, Knowledge, Wisdom*) los ordena:



Nivel	Qué es	Ejemplo
Datos	Hechos o valores registrados, independientes de quien los lee	«Un reloj inteligente registra la temperatura corporal»
Información	Los datos interpretados por un agente; subjetiva	«La temperatura corporal es 37 °C»
Conocimiento	Información integrada en un modelo del mundo	«Si la temperatura supera 37 °C, la persona tiene fiebre»
Sabiduría	Meta-conocimiento: cuándo y cómo aplicar el conocimiento	«Si tiene fiebre, debe tomar paracetamol»

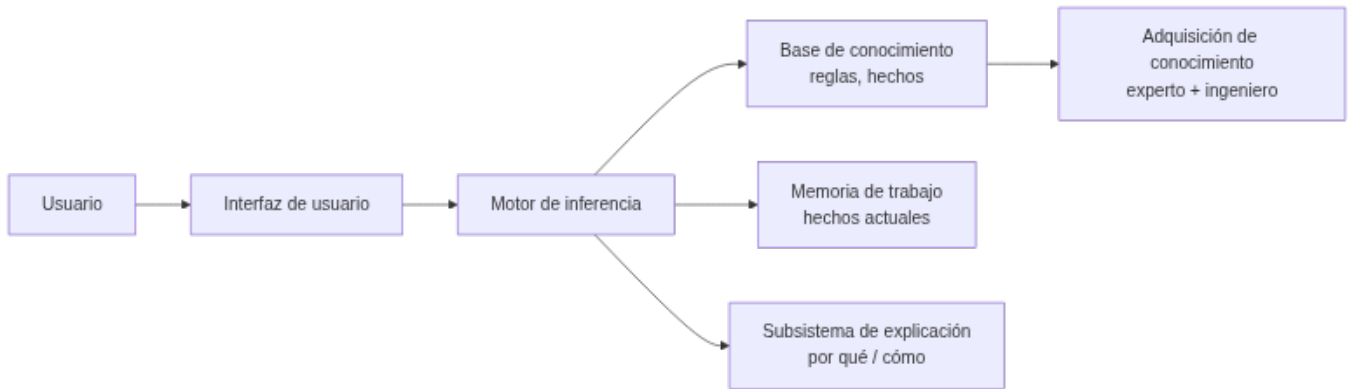


Por qué importa esta distinción

Un sistema experto no almacena datos ni información: almacena **conocimiento** (reglas) y, en los más avanzados, un poco de **sabiduría** (metarreglas que deciden cuándo aplicar otras reglas — el «control de meta» del §10.1). Confundir los niveles es el error más común al empezar a diseñar uno: se acumulan datos en vez de codificar reglas.

4.2 ¿Qué es un sistema experto?

Un **sistema experto** es un programa de IA que **emula el razonamiento de un experto humano** en un dominio concreto: codifica su conocimiento en una **base de conocimiento** y utiliza un **motor de inferencia** para aplicar ese conocimiento a los hechos del problema. Comenzaron su desarrollo en los años 70 y fueron muy populares esa década y la siguiente: se consideran los primeros sistemas de IA capaces de obtener resultados con utilidad práctica real.



4.3 Componentes

Componente	Función
Interfaz de usuario	Vía de las consultas: pregunta datos, muestra resultados y alerta de datos erróneos. Incluye un módulo de comunicaciones (con otros sistemas, útil en automatización industrial)
Base de conocimiento (KB)	Reglas y hechos del dominio, formalizados por el ingeniero de conocimiento junto al experto. Puede organizarse como listas, objetos relacionados, cálculo de predicados o redes semánticas
Memoria de trabajo (base de hechos)	Hechos actuales: los que aporta el usuario o los sensores, más los deducidos durante el razonamiento. Guarda el histórico de estados de la consulta
Motor de inferencia	Evalúa qué reglas se cumplen, resuelve conflictos y ejecuta las acciones. Determina el orden de las acciones y la interacción entre las partes del sistema
Subsistema de explicación	Justifica el razonamiento («por qué» y «cómo» llegó a esa conclusión), mostrando las reglas usadas. Ayuda también al ingeniero de conocimiento a depurar el motor y al experto a verificar la coherencia de la base
Adquisición de conocimiento	Herramientas para incorporar nuevo conocimiento sin perfil técnico (el famoso <i>bottleneck</i> : conseguir que el experto vuelque lo que sabe es la parte más lenta de construir el sistema)



La explicación marca la diferencia

La capacidad de **explicar** su razonamiento es lo que distingue a un sistema experto de un modelo de ML: en dominios regulados (medicina, finanzas) la justificación es obligatoria. MYCIN fue el pionero de esta *explicabilidad*, mostrando las reglas de inferencia que empleó — aunque, para consultas complejas, enumerar todas las reglas puede resultar tedioso para el usuario.

4.4 El ciclo de ejecución (dinámica)

El motor de inferencia opera en un **bucle reconocer-actuar**:

1. **Reconocer (match)**: se comparan los hechos de la memoria de trabajo con las condiciones (LHS) de las reglas; las reglas activas van a la **agenda**.
2. **Resolver (resolve)**: si hay varias reglas activas, se elige una según la estrategia de control (saliencia, recency, especificidad — ver §10.1).
3. **Actuar (act)**: se ejecuta el consecuente (RHS), que declara/modifica/retira hechos, y el ciclo se repite hasta agotar la agenda.

El objetivo de fondo es hacer que la lógica sea **explícita** en vez de estar enterrada en código que solo revisa un informático: con reglas legibles, el propio experto del dominio puede revisar y mantener el sistema.

4.5 Encadenamiento

- **Hacia delante (forward, data-driven)**: parte de los hechos y deduce conclusiones. Ideal para monitorización, control de procesos en tiempo real y planificación.

- **Hacia atrás (backward, goal-driven)**: parte de una **meta** y busca qué hechos la sustentan, preguntando al usuario solo lo necesario. Ideal para diagnóstico (MYCIN, Prolog).
- **Encadenamiento mixto**: combina los dos anteriores.
- Otros mecanismos de razonamiento: **búsqueda heurística** (el proceso de inferencia recorre un árbol) y **herencia** (en entornos orientados a objetos, un objeto hijo hereda propiedades del padre).



Las dos reglas de inferencia clásicas

Antes de forward/backward chaining está la lógica que los sustenta: **Modus Ponens** («si P implica Q , y P es verdad, entonces Q es verdad») y **Modus Tollens** («si P implica Q , y Q no es cierto, entonces P no es cierto»). El encadenamiento hacia delante aplica Modus Ponens repetidamente; el encadenamiento hacia atrás busca qué P sustentaría un Q dado.

4.6 Incertidumbre: factores de certeza

MYCIN introdujo los **factores de certeza (CF)** para manejar la incertidumbre: cada regla tiene un CF y se combinan al encadenar. Alternativas modernas: lógica difusa (§8), teoría de Dempster-Shafer o redes bayesianas (Heckerman & Shortliffe, 1992).



Combinar factores de certeza

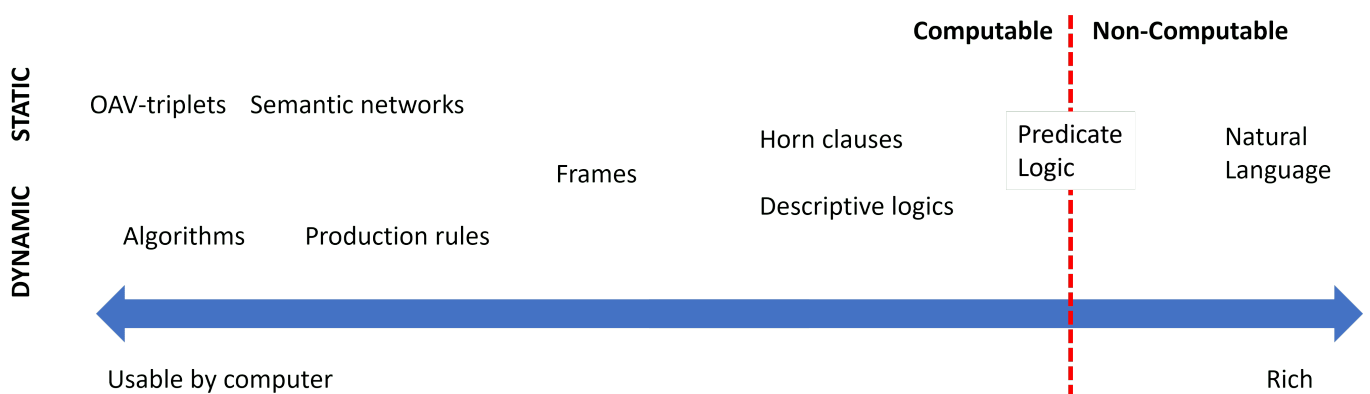
Una regla dice «SI hay fiebre alta ENTONCES sospecha de meningitis **con CF = 0,6**». Otra regla independiente aporta «SI rigidez de nuca ENTONCES sospecha de meningitis **con CF = 0,4**». MYCIN combina ambos apoyos para dar una confianza conjunta mayor que cada una por separado (combinación de evidencia). Si en cambio hubiera evidencia en contra, los CF se descuentan. De ahí nace el concepto de **acumulación de evidencia**, que luego se formalizó con redes bayesianas.

Esta gestión explícita de la **incertidumbre** es una diferencia clave frente a un sistema de reglas «duro» (todo verdadero o falso): permite razonar con conocimiento parcial y explicar el nivel de confianza de una conclusión. La lógica difusa del §8 resuelve el mismo problema de otra forma: en vez de un factor de certeza sobre una regla booleana, deja que el propio hecho sea **parcialmente verdadero**.

5. Estructuras elementales de representación (RA5-a/b)

5.1 Un continuo, no una lista cerrada

Representar el conocimiento es uno de los problemas fundamentales de la IA: hay que hacerlo de forma **entendible** para las máquinas, **útil** para resolver problemas y **eficiente** de procesar. Las representaciones forman un **continuo**:



En un extremo están las representaciones **simples** (algoritmos): eficientes para el ordenador pero poco flexibles. En el otro, las **flexibles** (texto natural): muy expresivas pero no directamente utilizables por una máquina. Entre medias viven las que usamos en esta unidad.

5.2 Las representaciones, con sus ventajas y límites

Representación	Estructura	Inferencia	Ventajas	Limitaciones
Pares atributo-valor (tripletes objeto-atributo-valor)	Lista de nodos y aristas de un grafo: «el perro es un animal, tiene cuatro patas...»	Recorrido y coincidencia de atributos	Muy simple de construir a partir de descripciones	Poco expresiva para relaciones complejas
Reglas de producción	SI ... ENTONCES ...	Forward/backward + resolución de conflictos	Modular, legible, explicable	Difícil modelar jerarquías; lenta con bases grandes
Representaciones jerárquicas	Árbol: nodos = conceptos, aristas = relaciones (Animales → Vertebrados → Mamíferos → Perros)	Recorrido del árbol, herencia	Natural para taxonomías	Rígida si un concepto pertenece a varias ramas
Marcos (frames)	Registros con ranuras (slots) y valores por defecto	Herencia de clases, disparadores	Taxonomías y conocimiento estructurado	Conflicto con herencia múltiple
Lógica formal	Predicados de primer orden (propuesta por Aristóteles hace más de 2000 años)	Resolución, unificación, deducción	Rigor matemático	Explosión combinatoria; poco utilizable directamente por máquinas (salvo un subconjunto, como en Prolog)
Redes semánticas	Grafos dirigidos con etiquetas	Búsqueda en grafos, propagación	Intuitiva para relaciones	Semántica ambigua
Ontologías	Clases, propiedades, axiomas (OWL)	Razonadores (Pellet, HermiT)	Interoperabilidad, reutilización	Curva de aprendizaje alta



Lo mismo en cuatro formas distintas

El hecho «si la temperatura supera 37 °C, la persona tiene fiebre» se puede escribir como **regla de producción** (SI temperatura > 37 ENTONCES fiebre), como **lógica proposicional** ($(p \rightarrow q)$, con (p) : «tiene fiebre», (q) : «debe tomar paracetamol»), como **jerarquía** (Síntomas → Fiebre → Paracetamol) o como **par atributo-valor** (persona.temperatura = 38.2, evaluado por una regla externa). Elegir la representación es elegir **qué es fácil de hacer** con ese conocimiento después.



Regla práctica de esta unidad

Usaremos sobre todo **reglas de producción** (con `experta`) y, en el §8, **lógica difusa** (con `scikit-fuzzy`). Las ontologías y las redes semánticas se mencionan como alternativas; los **frames** y la **lógica formal** se tratan a nivel conceptual.

6. Representar y simular comportamientos con experta (RA5-b)

Para **simular un comportamiento** se escribe la base de conocimiento en `experta` y se ejecuta el ciclo de inferencia. Recordamos el parche obligatorio para Python 3.10+ (la librería es de 2019 y `collections.Mapping` desapareció en esa versión):

```
1 import collections, collections.abc
2 if not hasattr(collections, 'Mapping'):
3     collections.Mapping = collections.abc.Mapping
4     collections.Iterable = collections.abc.Iterable
5     collections.MutableMapping = collections.abc.MutableMapping
6
7 from experta import *
8
9 class DiagnosticoPc(KnowledgeEngine):
10     @DefFacts()
11     def inicio(self):
12         yield Fact(accion="diagnosticar")
13
14     @Rule(Fact(accion="diagnosticar"), salience=10)
15     def arrancar(self):
16         print("Diagnóstico del PC...")
17         self.declare(Fact(luz_encendida=True))
18         self.declare(Fact(sonido="pitidos_cortos"))
19
20     @Rule(Fact(luz_encendida=True), Fact(sonido="pitidos_cortos"))
21     def ram(self):
22         self.declare(Fact(causa="problema_ram"))
23
24     @Rule(Fact(causa="problema_ram"))
25     def resultado(self):
26         print("DIAGNÓSTICO: Fallo de memoria RAM.")
27
28 engine = DiagnosticoPc()
29 engine.reset()
30 engine.run()
```

Salida esperada:

```
1 Diagnóstico del PC...
2 DIAGNÓSTICO: Fallo de memoria RAM.
```

 **Otro ámbito: clasificación de animales (sistema de producción clásico)**

El mismo motor `experta` simula un sistema de clasificación con reglas de un zoólogo:

```
1 class Animales(KnowledgeEngine):
2     @DefFacts()
3     def inicio(self):
4         yield Fact(analizar=True)
5
6     @Rule(Fact(analizar=True), salience=10)
7     def datos(self):
8         self.declare(Fact(pelo=True), Fact(carnivoro=True), Fact(color="leonado"))
9         self.declare(Fact(manchas="oscuras"))
10
11     @Rule(Fact(pelo=True))
12     def mamifero(self):
13         self.declare(Fact(mamifero=True))
14
15     @Rule(Fact(mamifero=True), Fact(carnivoro=True), Fact(color="leonado"),
16           Fact(manchas="oscuras"))
17     def guepardo(self):
18         print("IDENTIFICADO: guepardo")
```

Así se comprueba que un sistema experto **representa y simula el comportamiento** de un dominio sin programar la lógica imperativamente: el motor aplica las reglas hasta llegar a la conclusión.

6.1 «Muy diversos ámbitos»: los notebooks de la unidad

El CE b exige simular comportamientos en **dominios distintos**, no repetir el mismo ejemplo. Esta unidad trae varios notebooks ejecutados de verdad, en dominios reales (la lista completa está en las [actividades guiadas](#) y las [entregables](#)):

Notebook	Dominio	Qué simula
UD05_N01_experta_primeros_pasos.ipynb	Introducción	Primeros pasos con <code>experta</code> : hechos, reglas, <code>DefFacts</code>
UD05_N02_piedra_papel_tijera.ipynb	Juego	Piedra, papel o tijera decidido por reglas
UD05_N03_clasificacion_animales.ipynb	Zoología	El ejemplo de arriba, completo y ejecutado
EX1 · lesión de rodilla	Medicina	Diagnóstico de una lesión a partir de síntomas, con <code>experta</code>

Notebook	Dominio	Qué simula
UD05_N04_reglas_desde_datos_titanic.ipynb	Datos históricos	Reglas extraídas de datos, no escritas a mano (§7)
EX2 · valor de mercado	Deporte	Estimación híbrida reglas + aprendizaje automático (§7)
EX3 · centrocampistas · EX4 · quemador de gas	Deporte · industria	Lógica difusa (§8) y control de un proceso real (§10-11)



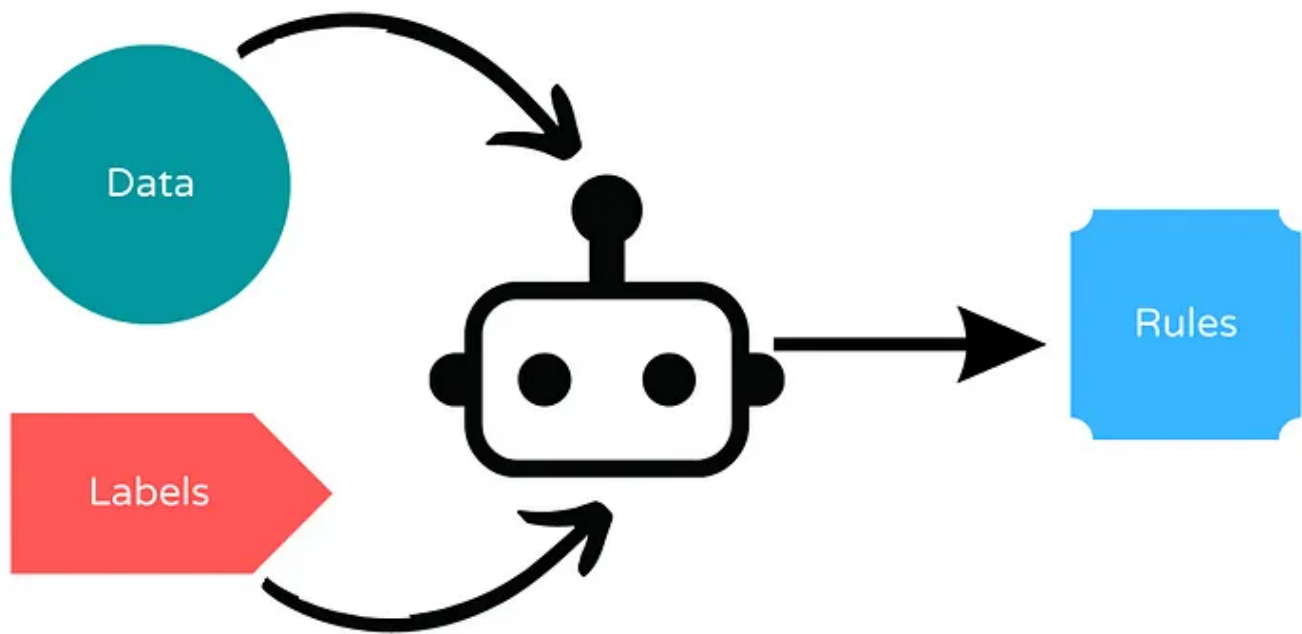
Por qué importa la diversidad

Un sistema experto no es una técnica de un solo dominio: la **misma arquitectura** (base de conocimiento + motor de inferencia) sirve para diagnosticar una rodilla, valorar a un futbolista o regular un quemador de gas. Lo que cambia es el conocimiento que se codifica, no el motor que lo aplica.

7. Sistemas híbridos reglas/datos (RA5-b)

Un sistema experto puro necesita que **alguien escriba las reglas a mano**. Pero a veces el conocimiento del dominio ya está en los datos, o conviene mezclar las dos fuentes. Hay dos enfoques:

- **Deducir reglas a partir de datos:** un algoritmo genera reglas legibles a partir de un conjunto de entrenamiento, en vez de que las escriba una persona. Facilita la **interpretación** del razonamiento, porque el resultado sigue siendo `SI ... ENTONCES ...` en vez de una caja negra.
- **Integrar reglas definidas por el usuario con aprendizaje automático:** el experto pone las reglas de partida y el aprendizaje automático las **mejora** con datos.



7.1 Bibliotecas del segundo enfoque

Biblioteca	Qué hace
Human-Learn	Permite definir y dibujar reglas propias como si fueran un clasificador de scikit-learn (<code>FunctionClassifier</code>), y combinarlas con aprendizaje automático

Biblioteca	Qué hace
skope-rules	Analiza los datos y deduce reglas de clasificación; permite auditarlas e interpretarlas
FIGS (<code>imodels</code>)	Genera reglas fáciles de interpretar combinando varios árboles pequeños («sumas de árboles»), en vez de un único árbol grande difícil de leer
spaCy	Reglas para extraer información de texto : útil cuando no hay suficientes datos etiquetados o para casos muy específicos

UD05_N04_reglas_desde_datos_titanic.ipynb : reglas que salen de los datos, no de un experto

En vez de que alguien escriba a mano «si viajas en primera clase y eres mujer, sobrevives», **FIGS** analiza los datos históricos del Titanic y **genera esa regla por sí solo**, junto con otras. El resultado sigue siendo legible —un árbol de reglas pequeño—, pero nadie lo escribió a mano: se dedujo de los datos. Es el primer enfoque de la tabla de arriba.

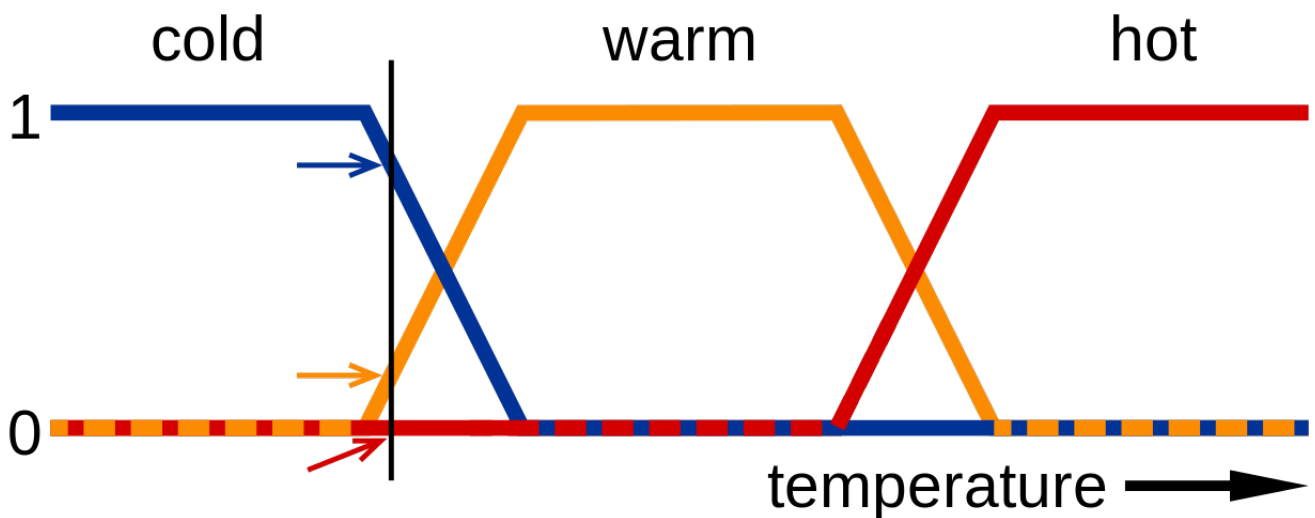
EX2 : valor de mercado de futbolistas con Human-Learn + FIGS

El entregable **EX2** combina las dos ideas sobre los mismos datos (FIFA 22): una `FunctionClassifier` de Human-Learn (reglas que **tú** defines sobre atributos del jugador) y un `FIGSClassifier` que **deduce** sus propias reglas del conjunto de entrenamiento. Comparar los dos resultados sobre el mismo problema es la mejor forma de sentir la diferencia entre los dos enfoques híbridos.

🔥 Esto no es una técnica aislada: es una tendencia

Los sistemas híbridos reglas/datos son la puerta de entrada a los **sistemas neuro-simbólicos** y a las **reglas como guardarraíl del ML** que se tratan en el §12.2. No son un tema aparte de los sistemas expertos: son su evolución natural cuando el conocimiento no cabe entero en la cabeza de un experto.

8. Sistemas de razonamiento impreciso: lógica difusa (RA5-b/d)



8.1 De lo binario a lo continuo

La lógica proposicional clásica es **binaria**: un enunciado es cierto o falso. Pero los humanos no razonamos así — «hace frío» no es verdadero o falso de forma tajante, es una cuestión de grado. La **lógica difusa** (o borrosa) extiende la lógica proposicional para trabajar con esa incertidumbre:

- Los valores de verdad son **números reales en el intervalo $[0, 1]$** : 0 es falso, 1 es cierto, 0,5 es «cierto al 50 %».
- La pertenencia de un elemento a un conjunto viene dada por una **función de pertenencia**, $\mu_A(x)$: el grado de pertenencia de (x) al conjunto (A) .

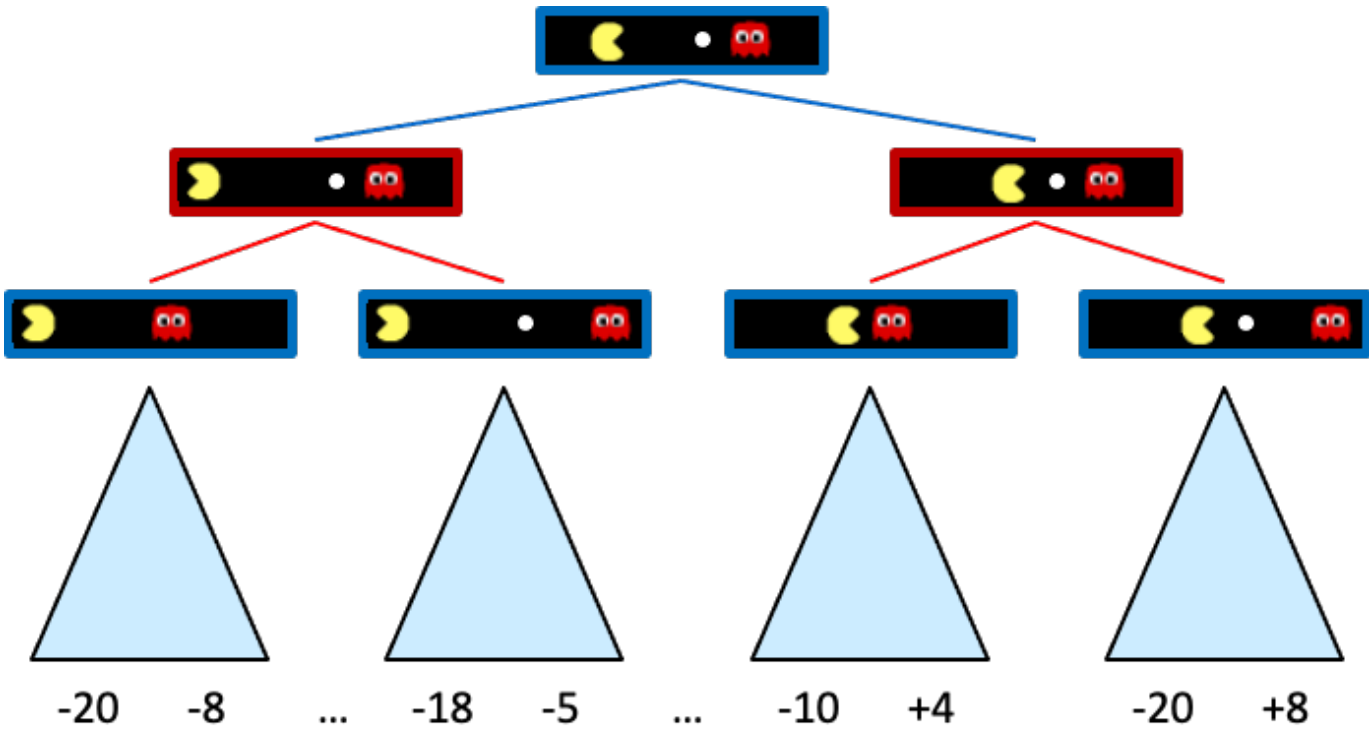
- Conceptos como *húmedo* o *frío* son difíciles de definir con precisión, pero la lógica difusa los define con funciones de pertenencia — lo que facilita crear dispositivos como termostatos: «*si la temperatura es fría, entonces enciende la calefacción*».
- Un **sistema de razonamiento impreciso** es un sistema basado en reglas que usa lógica difusa. Permite trabajar con valores **continuos**, modela mejor el conocimiento humano y es muy apropiado para **sistemas de control** — por eso enlaza directamente con el §10 y el §11.

8.2 Conceptos básicos

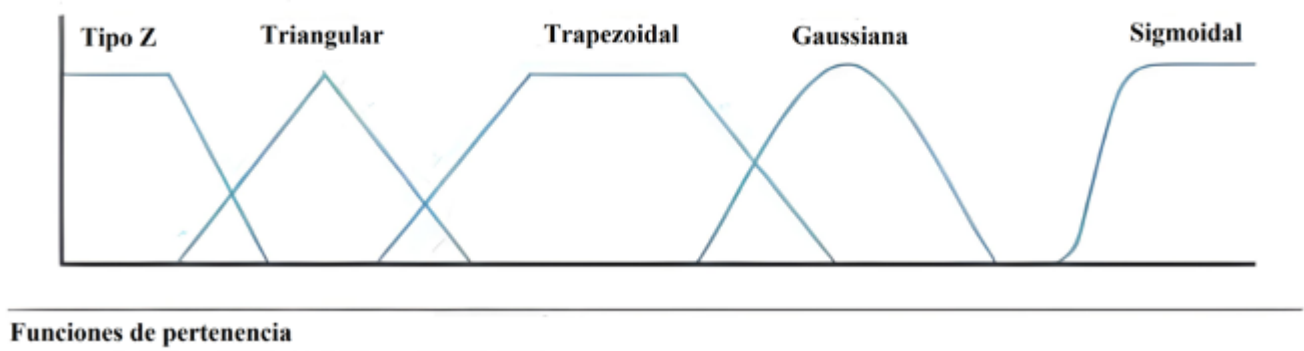
Concepto	Qué es	Ejemplo
Variable lingüística	Variable que toma valores lingüísticos	Temperatura
Valores lingüísticos	Los valores que puede tomar esa variable	Frío, Calor
Función de pertenencia	Asigna a cada valor un grado de pertenencia a un valor lingüístico	$\backslash(27\text{,}^{\circ}\text{C} \rightarrow \text{Calor} = 0\text{,}\text{,}8\backslash)$
Regla difusa	Regla que usa valores difusos	«Si la temperatura es fría , entonces calefacción alta »
Función de agregación	Combina los valores difusos de varias reglas en una conclusión	$\backslash(\text{Calor}=0\text{,}\text{,}8\backslash \text{H}\acute{\text{u}}\text{medo}=0\text{,}\text{,}7 \rightarrow \text{Sensaci}\acute{\text{o}}\text{n}\text{ desagradoable}=0\text{,}\text{,}8\backslash)$

8.3 El funcionamiento en tres pasos

1. **Fuzzificación:** convierte los datos de entrada precisos en valores difusos, usando las funciones de pertenencia. $\backslash(27\text{,}^{\circ}\text{C} \rightarrow \text{Calor}=0\text{,}\text{,}8\backslash \text{MuchoCalor}=0\text{,}\text{,}2\backslash)$.
2. **Evaluación de las reglas:** se aplican las reglas del sistema, combinando las funciones de pertenencia de las variables de entrada para deducir la relevancia de la salida. Por ejemplo: «si la temperatura es **alta** y la humedad es **baja**, la velocidad del ventilador debe ser **alta**».
3. **Desfuzzificación:** convierte la conclusión difusa de vuelta en un valor preciso, con una función de agregación (normalmente **centro de gravedad** o **máximo**).



Las funciones de pertenencia más usadas son **trapezoidales** y **triangulares**; las sinusoidales sirven para representar periodos, y las sigmoidales, probabilidades.

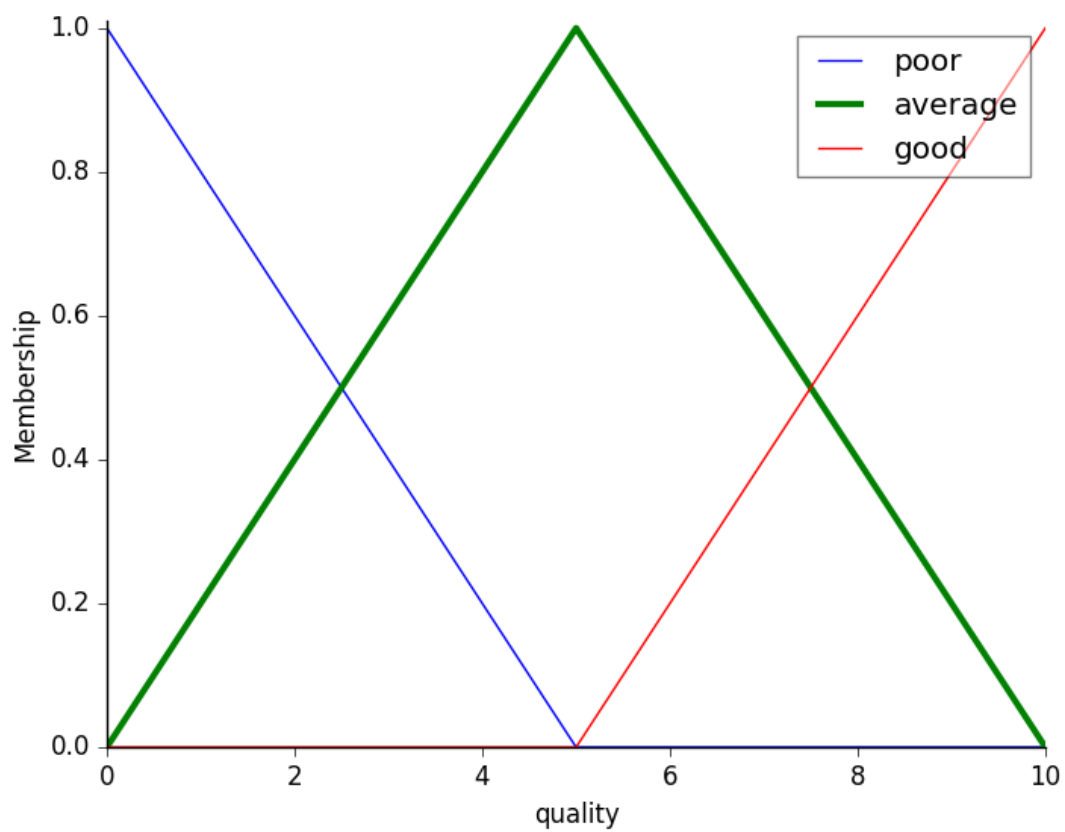
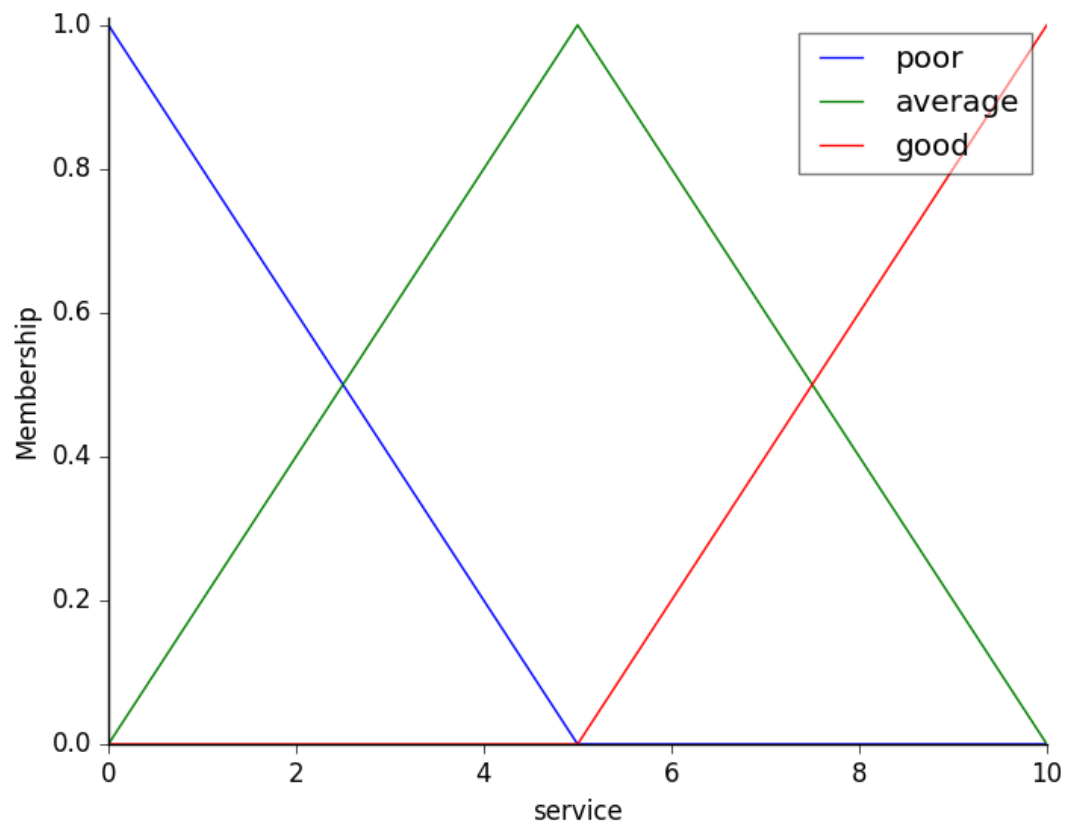


8.4 Ejemplo trabajado: la propina del restaurante

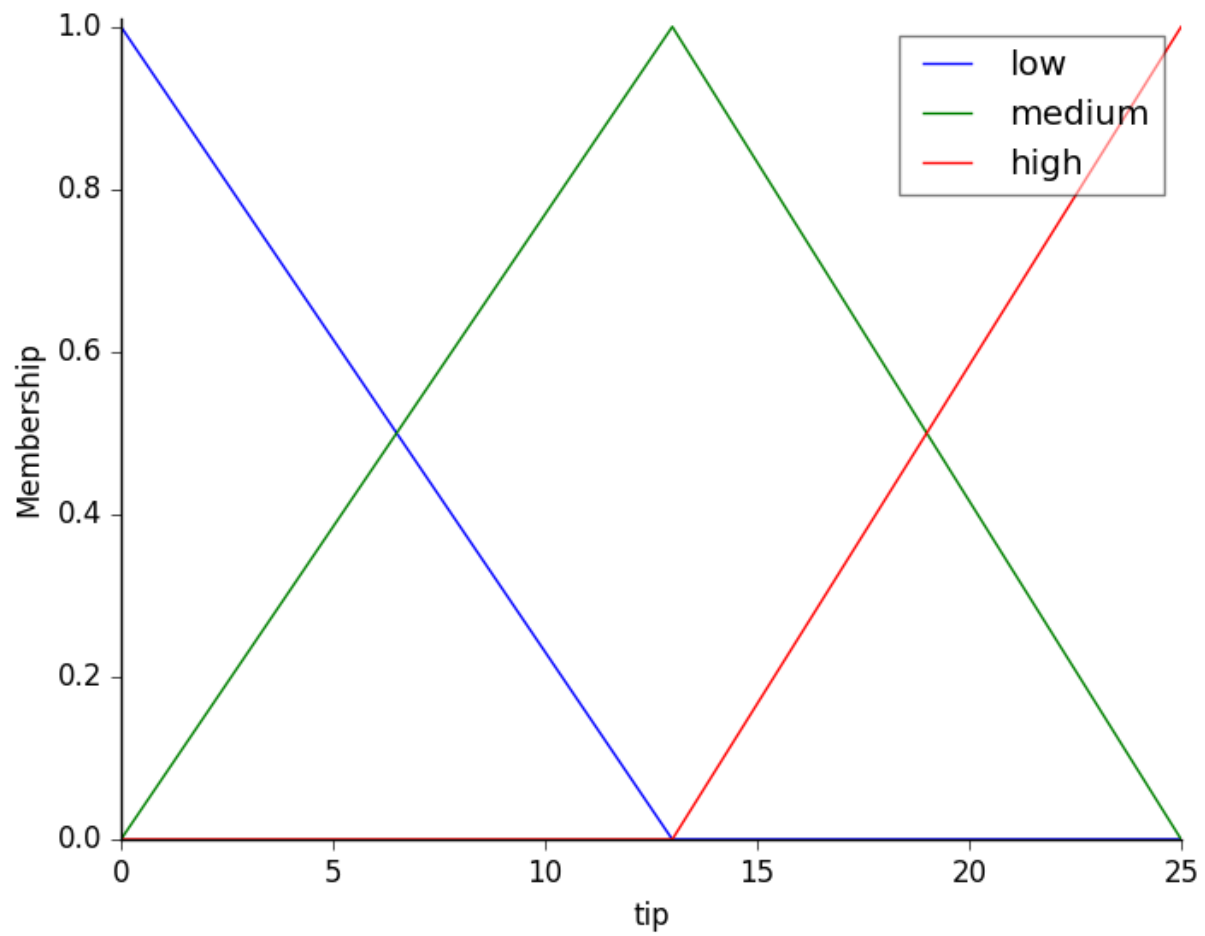
Es el ejemplo canónico de `scikit-fuzzy`, y el que verás resuelto paso a paso en el [Taller 2](#).

Variables de entrada (funciones triangulares):

- **Calidad del servicio:** baja $\setminus([0,5])$ · media $\setminus([0,10])$ · alta $\setminus([5,10])$
- **Calidad de la comida:** mala $\setminus([0,5])$ · media $\setminus([0,10])$ · buena $\setminus([5,10])$

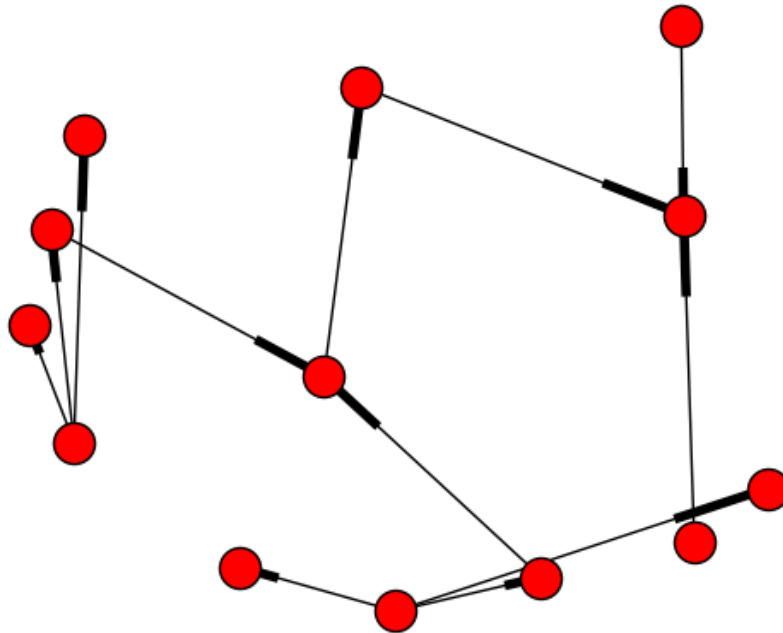


Variable de salida: propina — baja $\setminus([0,13])$ · media $\setminus([0,25])$ · alta $\setminus([13,25])$

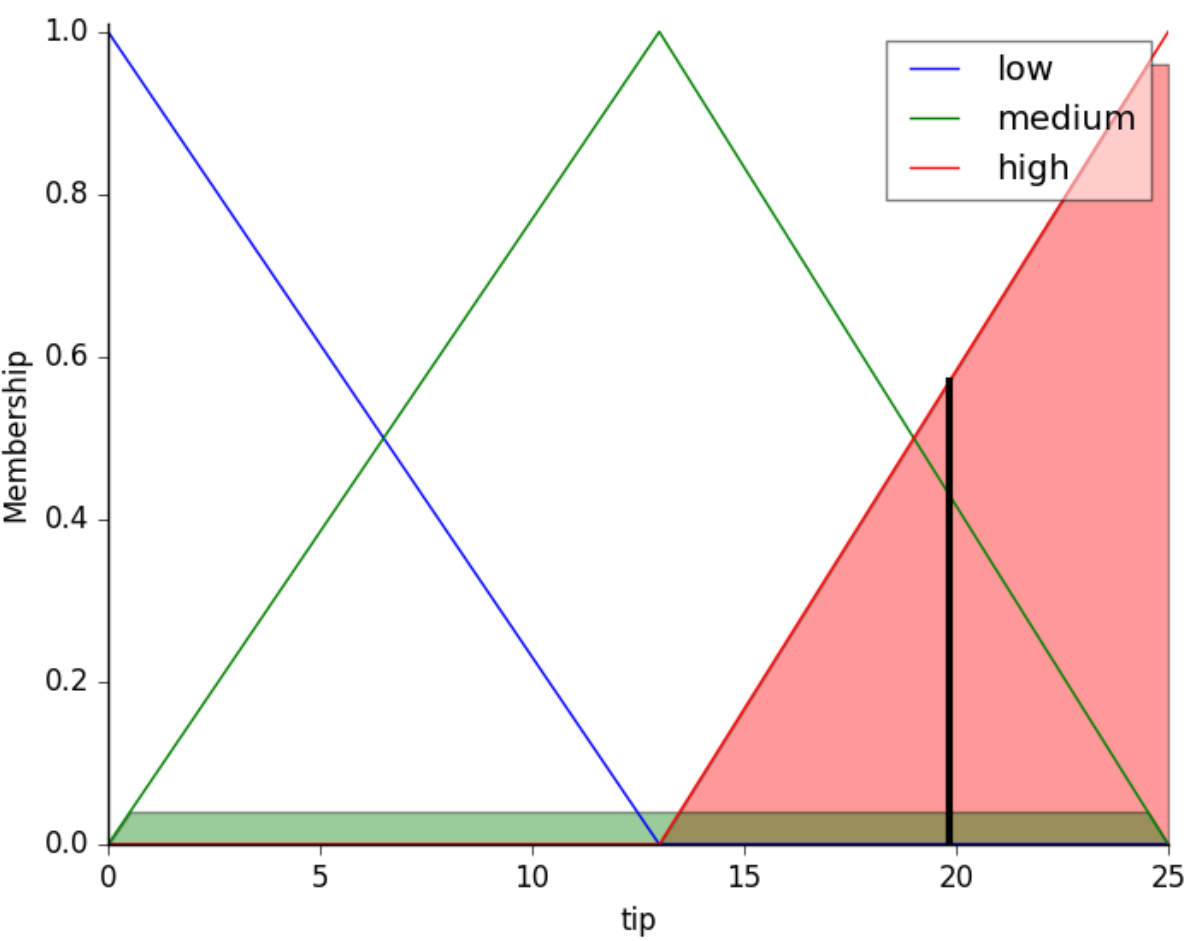


Reglas:

- SI (servicio es **baja** O comida es **mala**) ENTONCES propina es **baja**
- SI (servicio es **media**) ENTONCES propina es **media**
- SI (servicio es **alta** O comida es **buena**) ENTONCES propina es **alta**



Inferencia: con un servicio de **9,8** y una comida de **6,5**, el sistema desfuzzifica una propina de **19,24 €** — en la banda alta, coherente con un servicio casi perfecto y una comida notable.



De la propina al quemador de gas

Este ejemplo es idéntico en estructura al **Taller 3** (control de un quemador de gas) y al entregable **EX4**: variables de entrada difusas, reglas lingüísticas, una salida desfuzzificada. La diferencia es que ahí la salida no es una propina, es la **potencia de un actuador real** — es el paso del §8 al §11.

9. Variación de características y dinámica (RA5-c)

9.1 Sensibilidad y robustez

- **Sensibilidad:** cómo cambian las conclusiones ante pequeñas desviaciones en los parámetros o datos de entrada. Un sistema muy sensible detecta pronto anomalías, pero **falsas alarmas** ante ruido.
- **Robustez:** capacidad de mantener conclusiones estables y seguras ante perturbaciones, ruido o fallos parciales.

9.2 Tres mecanismos de variación

Qué varía	Efecto en la dinámica
Las reglas (estructura lógica)	Añadir condiciones al antecedente → más inercia (la regla se dispara menos); simplificar → sobreactivación (reglas se disparan ante transitorios sin riesgo)
Los hechos (perturbaciones)	Ruido en un sensor (p. ej. CO en un túnel oscilando alrededor del umbral) → el sistema conmuta actuadores sin parar
Los umbrales de certeza	Subir el umbral de un diagnóstico → menos falsos positivos pero puede ignorar alertas tempranas



El problema de la histéresis

Un sensor de CO en un túnel oscila entre 28 y 32 ppm con umbral en 30. Sin histéresis, los extractores se encienden y apagan en cada ciclo. Soluciones: una **regla de histéresis temporal** (la alerta debe mantenerse 30 s) o un **controlador difuso** que suavice la transición (enlaza con el §8).

10. Estrategias de control de un sistema experto (RA5-d)

10.1 Control de la agenda (resolución de conflictos)

Estrategia	Qué hace
Salience	Prioridad numérica explícita de cada regla (las de emergencia van antes)
Recency	Prefiere las reglas con hechos más recientes (time-tags)
Specificity	Prefiere la regla con más condiciones en el antecedente
Control de meta (metarrazonamiento)	Reglas de nivel superior que modifican prioridades, activan grupos de reglas según el modo (arranque, operación, parada) o cambian la estrategia. Es la sabiduría de la jerarquía DIKW del §4.1, aplicada al propio motor

10.2 Especificaciones de respuesta

Para controlar un proceso, el sistema experto debe cumplir **especificaciones** sobre la respuesta de la variable controlada:

Especificación	Qué mide
Precisión	Error en régimen permanente (diferencia entre la variable y el setpoint al estabilizarse)
Tiempo de respuesta	Tiempo de subida (t_r) y de asentamiento (t_s)
Estabilidad	Ausencia de oscilaciones; sobreimpulso (overshoot) máximo aceptable
Alcance	Rango de operación de la planta



Nivel FP

En este módulo se trabajan estas especificaciones de forma **conceptual** (qué miden y por qué importan), sin las fórmulas de la teoría de control de ingeniería. El foco está en *definir el objetivo y saber interpretar si la respuesta lo cumple*.

10.3 Ejemplo guiado: definir las especificaciones de un controlador de climatización

Recorremos juntos el razonamiento que repetirás en el Taller 3, antes de aplicarlo al quemador de gas real de [EX4](#).

Problema: queremos controlar la temperatura de una sala para que llegue a 21 °C.

Paso 1 · Setpoint y precisión. La variable controlada (temperatura) debe estabilizarse en 21 °C con un **error en régimen permanente** de $\pm 0,5$ °C como máximo.

Paso 2 · Tiempo de respuesta. La sala debe alcanzar la banda aceptable (20,5-21,5 °C) en menos de **15 minutos** desde el arranque (tiempo de asentamiento).

Paso 3 · Estabilidad y sobreimpulso. Se tolera un sobreimpulso máximo de **1 °C** por encima del setpoint (sin oscilaciones que dañen el equipo).

Paso 4 · Elegir el controlador. Con un PID bien sintonizado se cumple en un sistema lineal, pero la sala tiene inercia térmica y perturbaciones (puertas, sol). Un **controlador experto o difuso** con reglas del operador («si la diferencia es grande → máxima potencia, y al acercarse → reducir para no sobrepasar») consigue la respuesta con **menos sobreimpulso y más robustez** ante las perturbaciones.

Paso 5 · Comprobar y ajustar. Se simula la respuesta y se mide t_s y el sobreimpulso; si no se cumple la especificación, se ajustan reglas o ganancias (análisis de sensibilidad, §9).



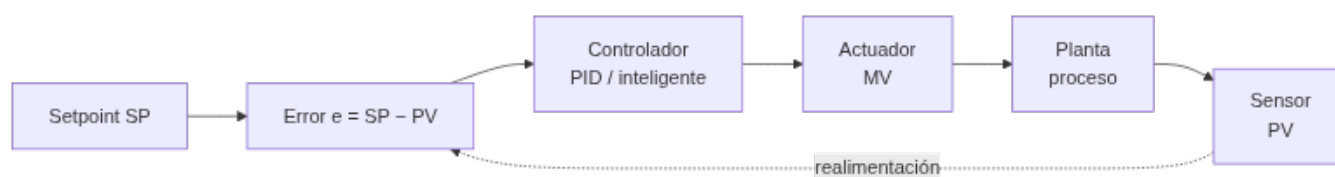
Conclusión del ejemplo guiado

Controlar no es «conectar un motor»: es **definir el objetivo** (precisión), **acotar el tiempo**, **limitar el sobreimpulso** y **elegir el controlador** que lo cumpla, verificándolo con una simulación. Ese es el sentido del CE d y del CE e — y exactamente lo que se te pide en EX4, con un quemador de gas real en vez de una sala.

11. Controladores inteligentes (RA5-e)

11.1 Del control clásico al control inteligente

Un **lazo de control** clásico compara el setpoint (SP) con la variable medida (PV), calcula el error y aplica una acción (MV) con un **controlador PID** (proporcional-integral-derivativo). El PID es eficaz en sistemas lineales, pero sufre con **retrasos**, **no linealidades** y **perturbaciones multivariable**.



La acción del PID es $u = K_p \cdot e + K_i \cdot \int e + K_d \cdot de/dt$: la parte proporcional responde al error; la integral elimina el error residual y la derivada reduce el sobreimpulso. Sus límites: no predice el futuro, se degrada con retrasos y no linealidades, y hay que **re-sintonizarlo** cuando la planta envejece (ganancias fijas).

Los **controladores inteligentes** sustituyen o complementan al PID usando técnicas de IA:

Controlador	Cómo funciona	Ventaja frente al PID
Control difuso	Reglas lingüísticas + funciones de pertenencia (Mamdani/Sugeno) — el §8 aplicado al control	Robusto ante ruido y no linealidad; no necesita modelo
Control por reglas (experto)	Motor de inferencia con reglas del operador experto	Captura heurísticas de operadores; fácil de entender
Redes neuronales	Aprenden la dinámica inversa de la planta	Adaptación a sistemas no lineales
Control predictivo (MPC)	Predice la trayectoria futura y optimiza la acción	Anticipa restricciones; proactivo

11.2 Ejemplos con datos

Caso	Mejora frente a PID
Control térmico difuso	Tiempo de asentamiento –76 % (416 s → 100 s); sobreimpulso 5,64 % → 0 %
Horno de fundición (ANN)	MSE 132,75 frente a 134,13 del PID (ligera mejora)
Mitsubishi (aire acondicionado)	Calienta/enfría 5× más rápido, –24 % consumo
DeepMind centros de datos	–40 % en energía de refrigeración (–15 % PUE overhead)

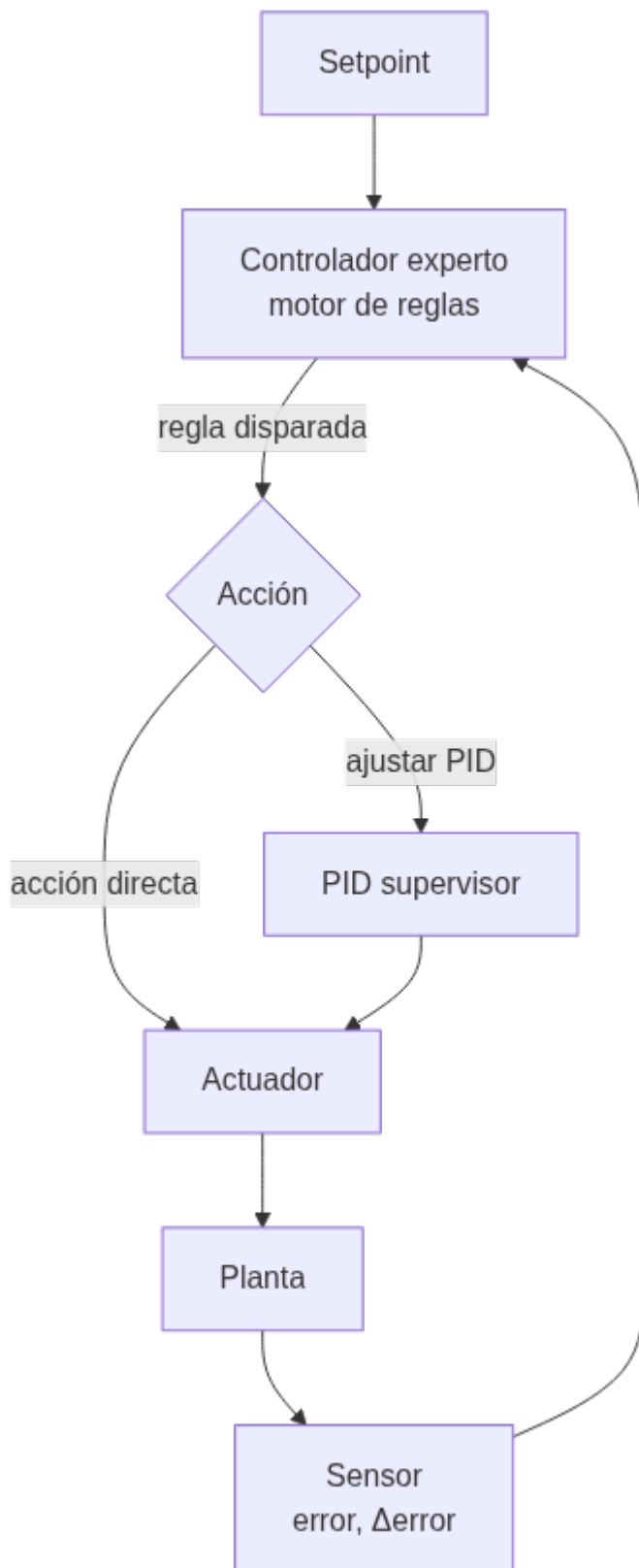


¿Controlador inteligente siempre gana?

No siempre. El PID es simple, barato y fiable en sistemas bien modelados. El controlador inteligente aporta cuando hay **no linealidad**, **retraso o ruido fuerte**. La decisión es *empezar simple* y añadir inteligencia solo si se justifica — es lo que comprobarás tú mismo en EX4, que pide **dos enfoques distintos** para el mismo quemador.

11.3 El sistema experto como controlador

El patrón del **control experto** (Åström, Anton y Årén, 1986) usa un motor de inferencia dentro del lazo: el sistema evalúa el error y su variación, decide la acción por reglas (supervisando y ajustando incluso un PID) y explica la decisión. Es la base de sistemas industriales como **Gensym G2** (refinerías, energía, farmacia) desde los años 80.



El controlador experto puede actuar de dos formas: **directamente** (decide la acción por reglas, por ejemplo «si el error es negativo grande y aumenta → máxima potencia») o **supervisando** un PID (ajusta sus ganancias cuando detecta que la respuesta se degrada). En ambos casos, la decisión es **explicable**: se puede consultar qué regla se disparó y por qué.



Regla de un controlador experto de climatización

1 SI error de temperatura es NEGATIVO_GRANDE
2 Y variación del error es POSITIVA_PEQUENA
3 ENTONCES potencia del quemador es MEDIA_ALTA

Estas reglas capturan la heurística de un operador humano y son fáciles de auditar, algo que un PID o una red neuronal no ofrecen de forma directa.

12. Aplicaciones y tendencias (contenidos del RA5)

12.1 Aplicaciones por sector

Sector	Uso
Industria	Control de procesos, diagnóstico de máquinas, mantenimiento
Salud	Diagnóstico asistido (MYCIN, GARVAN-ES1), dosificación
Finanzas	Asesoramiento, detección de fraude por reglas
Telecomunicaciones	Gestión de redes, diagnóstico de averías



Ejemplos clásicos con datos

- **MYCIN** (Stanford, 1972): ~500-600 reglas de diagnóstico clínico; precisión 69-70 % (equiparable a especialistas). Pionero de la explicación y los factores de certeza (§4.6).
- **XCON/R1** (CMU/DEC, 1978): configuración de ordenadores VAX; pasó de 250 a más de 6.200 reglas en 1986; precisión 95-98 % y ahorro de ~25 M\$/año para DEC.
- **DENDRAL** (1965): primer sistema experto; acotaba millones de isómeros moleculares a un conjunto manejable.
- **SID** (DEC): generó el 93 % de las puertas lógicas del VAX 9000.
- **Cyc** (1984): proyecto de enciclopedia el sentido común (millones de hechos y reglas).

12.2 Tendencias

- **BRMS** (*Business Rule Management Systems*): motores de reglas empresariales como **Drools** (Apache KIE) con estándares DMN/JSR-94. Permiten a las empresas gestionar miles de reglas de negocio separadas del código, con motores optimizados (PHREAK) que escalan linealmente.
- **Sistemas neuro-simbólicos** (3.ª ola de la IA): combinan redes neuronales (aprendizaje) con reglas y razonamiento simbólico (lógica), reduciendo alucinaciones y aportando explicabilidad — es la generalización de los **sistemas híbridos reglas/datos** del §7. Ejemplo: Amazon utiliza arquitecturas neuro-simbólicas en sus motores de compra para contrastar las salidas generativas con hechos y reglas.
- **IA explicable (XAI)**: MYCIN fue el origen de la explicación; hoy se usan SHAP/LIME para explicar modelos y la normativa exige el **derecho a explicación** del RGPD (enlaza UD06).
- **Agentes y razonamiento declarativo**: agentes que planifican con reglas y guardas lógicas de seguridad, garantizando que las decisiones distribuidas cumplen restricciones.
- **Reglas como guardarraíl del ML**: usar reglas para validar y acotar las salidas de los modelos (p. ej. un LLM no puede emitir una orden fuera de los rangos seguros definidos por reglas). Es la misma idea del §7, llevada a los modelos generativos.



Del pasado al presente

Los sistemas expertos «clásicos» (MYCIN, XCON) demostraron que el conocimiento declarativo y la explicación importan. Hoy esa idea **no ha muerto**: vive en los BRMS, en los sistemas neuro-simbólicos y en los guardarraíles que aseguran las salidas del ML. Saber construir y auditar sistemas basados en reglas —puras, híbridas o difusas— sigue siendo una competencia valiosa.

13. Puntos clave de la unidad

- La jerarquía **DIKW** distingue dato, información, conocimiento y sabiduría: un sistema experto almacena conocimiento (reglas) y, en su forma más avanzada, sabiduría (metarreglas).
- Un **sistema experto** codifica el conocimiento de un experto en una **base de conocimiento** y lo aplica con un **motor de inferencia** (ciclo reconocer-actuar, forward/backward).
- La **explicación** del razonamiento es su gran ventaja frente al ML (obligatoria en dominios regulados).
- La **representación del conocimiento** forma un continuo: pares atributo-valor, reglas, jerarquías, frames, lógica, redes semánticas u ontologías.
- Con `experta` (parche Py3.10+) se **simulan comportamientos** de ámbitos muy diversos: medicina, zoología, deporte, industria.
- Los **sistemas híbridos reglas/datos** (Human-Learn, FIGS, skope-rules) combinan el conocimiento experto con lo que aprenden los datos, cuando las reglas escritas a mano no bastan.
- La **lógica difusa** extiende la lógica a valores continuos $\in [0,1]$, con fuzzificación, evaluación de reglas y desfuzzificación — clave para el control de procesos reales.
- **Sensibilidad vs robustez**: variar reglas, hechos o umbrales cambia la dinámica (inanición, sobreactivación, falsas alarmas); la **histéresis** o el control difuso la estabilizan.
- Las **estrategias de control** (salience, control de meta) y las **especificaciones** (precisión, tiempo, estabilidad) definen la calidad de la respuesta.
- Los **controladores inteligentes** (difuso, por reglas, ANN, MPC) superan al PID en sistemas no lineales, con datos como -76% de asentamiento o 0% de sobreimpulso.
- **Tendencias**: BRMS/Drools, neuro-simbólico, XAI, reglas como guardarraíl del ML.

14. Glosario

Término	Definición
Dato	Hecho o valor registrado, independiente de quien lo interpreta
Información	Un dato interpretado por un agente
Conocimiento	Información integrada en un modelo del mundo
Sabiduría	Meta-conocimiento: cuándo y cómo aplicar el conocimiento
Sistema experto	Programa de IA que emula el razonamiento de un experto en un dominio
Base de conocimiento	Reglas y hechos del dominio
Memoria de trabajo	Hechos actuales del problema
Motor de inferencia	Ejecuta el ciclo reconocer-actuar
Ingeniero de conocimiento	Profesional que formaliza el conocimiento del experto
Adquisición de conocimiento	Proceso de capturar el conocimiento (bottleneck)
Subsistema de explicación	Justifica el razonamiento del sistema
Forward chaining	Encadenamiento hacia delante (data-driven)
Backward chaining	Encadenamiento hacia atrás (goal-driven)
Modus Ponens	Si P implica Q y P es verdad, Q es verdad
Modus Tollens	Si P implica Q y Q es falso, P es falso
Factor de certeza	Medida de confianza de una conclusión (MYCIN)
Frame	Estructura con ranuras y valores por defecto
Red semántica	Grafo de conceptos y relaciones
Ontología	Esquema formal de clases y propiedades (OWL)
Sistema híbrido reglas/datos	Combina reglas escritas o deducidas con aprendizaje automático

Término	Definición
FIGS	Algoritmo que genera reglas interpretables como suma de árboles pequeños
Lógica difusa	Extensión de la lógica a valores continuos en $\mathbb{[0,1]}$
Variable lingüística	Variable que toma valores lingüísticos (p. ej. «temperatura»)
Función de pertenencia	Grado de pertenencia de un valor a un conjunto difuso
Fuzzificación	Conversión de un valor preciso a valores difusos
Desfuzzificación	Conversión de una conclusión difusa a un valor preciso
Salience	Prioridad numérica de una regla
Control de meta	Metarreglas que gestionan la propia inferencia
Sensibilidad	Variación de las conclusiones ante perturbaciones pequeñas
Robustez	Estabilidad de las conclusiones ante ruido o fallos
Histéresis	Retardo en el cambio de estado para evitar conmutaciones
PID	Controlador proporcional-integral-derivativo
Lazo de control	SP → error → controlador → planta → PV
Error en régimen permanente	Diferencia residual entre variable y setpoint
Sobreimpulso (overshoot)	Exceso máximo sobre el setpoint durante el transitorio
Tiempo de asentamiento	Tiempo hasta que la respuesta entra en la banda aceptable
Control difuso	Controlador basado en reglas lingüísticas y pertenencia
Control predictivo (MPC)	Optimiza la acción prediciendo la trayectoria futura
BRMS	Sistema de gestión de reglas de negocio (Drools)
Neuro-simbólico	Combinación de redes neuronales y razonamiento simbólico
XAI	IA explicable

15. FAQ

? ¿Un sistema experto puede aprender de los datos? ∨

No por sí mismo: las reglas las escribe un experto. Por eso es totalmente **explicable**, pero requiere que el conocimiento esté disponible y formalizado (el famoso *bottleneck*). Los **sistemas híbridos** del §7 son justo la respuesta a esta limitación: dejan que el aprendizaje automático deduzca o mejore las reglas.

? ¿Los sistemas expertos están anticuados? ∨

No: los motores de reglas siguen en **BRMS empresariales** (Drools, SAP) y resurgen en los **sistemas neuro-simbólicos** y como **guardarrail** del ML. MYCIN/XCON fueron los pioneros, pero la tecnología evolucionó (RETE → PHREAK, estándares DMN).

? ¿Por qué un sistema experto puede dar falsas alarmas? ∨

Por **sensibilidad excesiva**: si un sensor tiene ruido y el umbral de la regla es muy justo, el sistema conmuta sin parar. Se corrige con **histéresis**, suavizado o un controlador difuso.

? ¿Qué es mejor, un PID o un controlador inteligente? ▾

Depende del sistema. El PID es simple y fiable en sistemas lineales; el controlador inteligente (difuso, ANN, MPC) gana en no linealidades, retrasos o ruido. Se empieza simple y se añade inteligencia si hace falta — es lo que comprobarás con las dos versiones que pide [EX4](#).

? ¿`experta` funciona en Python moderno? ▾

La librería (2019) falla en Python 3.10+ por `frozendict`. Con el parche `collections.Mapping = collections.abc.Mapping` funciona igual de bien. Existe también un fork (`om-experta`) que lo resuelve de otra forma, pero está archivado desde 2023: en este módulo usamos siempre el parche, para no tener dos soluciones al mismo problema.

? ¿Cuándo uso reglas puras y cuándo un sistema híbrido? ▾

Si el experto puede formalizar el conocimiento completo, con reglas puras basta y son más explicables. Si el conocimiento es parcial, cambia con el tiempo, o hay muchos datos disponibles, un sistema híbrido (§7) aprovecha lo mejor de las dos fuentes.

? ¿Los sistemas expertos pueden explicar sus decisiones? ▾

Sí, y es su gran ventaja: el subsistema de **explicación** puede responder «¿por qué?» y «¿cómo?» mostrando las reglas usadas. MYCIN fue el primero. Hoy esto se llama **IA explicable (XAI)** y es obligatorio en algunos dominios (derecho a explicación del RGPD).

16. Sesiones

Semana	Horas	Contenido	CE
11	3	DIKW, arquitectura y dinámica; estructuras de representación	RA5-a
12	3	Taller 1 (<code>experta</code>) + guiadas; EX1 ; sistemas híbridos reglas/datos (EX2)	RA5-b
13	3	Lógica difusa: Taller 2 (propinas); variación y dinámica	RA5-b, RA5-c
14	3	Estrategias de control; controladores inteligentes; Taller 3	RA5-d
15	3	EX3 , EX4 ; aplicaciones y tendencias; evaluación	RA5-d, RA5-e

17. Recursos

- [Diapositivas](#)
- [Ejercicios de la unidad](#)
- Talleres: [T01](#) · [simular un sistema experto](#) · [T02](#) · [lógica difusa](#) · [T03](#) · [controlador experto](#)
- [Actividades guiadas](#) — 6 notebooks de introducción y práctica
- [Actividades entregables](#) — 5 notebooks evaluables ([EX0](#) - [EX4](#)), con sus rúbricas
- **Notebooks** — todos los de la unidad, con descarga y apertura en Colab, en el menú «Notebooks»

**Referencias de la unidad** ▾

- [Wikipedia · Expert system](#)
- [Wikipedia · MYCIN](#)
- [Wikipedia · XCON](#)
- [experta · readthedocs](#)
- [CLIPS · clipsrules.net](#)
- [Drools / Apache KIE](#)
- [Human-Learn](#)
- [imodels \(FIGS\)](#)
- [skope-rules](#)
- [Wikipedia · Lógica difusa](#)
- [Wikipedia · Fuzzy control system](#)
- [Wikipedia · PID controller](#)
- [Modus Ponens · Modus Tollens](#)

18. Evaluación

Peso	Instrumento
40 % actividades	Media de los cinco entregables (EX0 - EX4); cada uno con su rúbrica sobre 10
60 % prueba escrita	Prueba del RA5 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «*en función de la consecución de los RA*»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.
- Los entregables EX1 - EX4 cubren un dominio fijo cada uno (medicina, deporte × 2, industria); EX0 es de libre elección y premia la originalidad.

19. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 Orden 8/2025): repetir la construcción de un sistema experto con un problema distinto y las pruebas de autoevaluación de la unidad.

23 de agosto de 2026

5.2 Diapositivas de la UD05



Diapositivas PDF



Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

🕒 23 de agosto de 2026

5.3 UD05 · Actividades guiadas

Se trabajan en clase

Seis notebooks de introducción y práctica guiada, en dominios distintos (§6.1 de la teoría). No son evaluables por sí solos, pero preparan los cinco entregables de la [página siguiente](#).

Notebook	Dominio	Descargar	Ejecutar
0 · Sistema experto con Python y Experta	Introducción a experta	 Descargar .ipynb	 Open in Colab
0 · Piedra, papel o tijera	Juego decidido por reglas	 Descargar .ipynb	 Open in Colab
1 · Sistema experto: clasificación de animales	Zoología	 Descargar .ipynb	 Open in Colab
2 · Sistema basado en reglas: Titanic	Reglas extraídas de datos (§7)	 Descargar .ipynb	 Open in Colab
3 · Sistema de control: regar el césped	Control difuso (§8, §10)	 Descargar .ipynb	 Open in Colab
4 · Lógica difusa: quién jugará	El clásico «jugar al tenis» (§8)	 Descargar .ipynb	 Open in Colab

Si Google Colab da un error de versión

Los notebooks 3 y 4 necesitan una versión concreta del entorno de ejecución de Colab ([Editar] → [Configuración del cuaderno] → **2025.7** en vez de «Última (recomendada)»), o puedes usar el `docker-compose.yml` de la unidad para ejecutarlos en local.

Referencia: un sistema experto grande, resuelto

[EX0 · Tasación de vehículos usados](#) ([descargar](#) · [Colab](#)) es un ejemplo **extenso ya resuelto**, portado de una práctica original en CLIPS: no es un enunciado que se te pida entregar, es una referencia para ver cómo se estructura un sistema experto grande de principio a fin, con más reglas y más profundidad que los ejemplos guiados de arriba.

[Volver a la UD05](#) · [Ejercicios](#) · [Taller 1](#) · [Taller 2](#) · [Taller 3](#) · [Actividades entregables](#)

 23 de agosto de 2026

5.4 UD05 · Actividades entregables



Cinco entregables, promediados, 40 % de la nota de RA5

Los cinco cuentan para la nota (decisión del profesor, 2026-08-22): se promedian sobre 10. Cuatro tienen dominio fijo (EX1 - EX4); EX0 es de libre elección.

Notebook	Dominio	Descargar	Ejecutar
EX0 · Sistema experto de libre elección	El que tú elijas	 Descargar .ipynb	 Open in Colab
EX1 · Sistema experto: detectar lesiones de rodilla	Medicina	 Descargar .ipynb	 Open in Colab
EX2 · Sistema basado en reglas: previsión del valor de mercado	Deporte · sistema híbrido (\$7)	 Descargar .ipynb	 Open in Colab
CSV de jugadores del FIFA 22 (para EX2 y EX3)	—	 CSV players_22.csv	—
EX3 · Lógica difusa: centrocampistas con potencial	Deporte · lógica difusa (\$8)	 Descargar .ipynb	 Open in Colab
EX4 · Sistema de control: simulador de quemador de gas	Industria · control real (\$10-11)	 Descargar .ipynb	 Open in Colab



EX2 y EX3 comparten el mismo fichero de datos

Descarga EX2.-players_22.csv una vez y súbelo a la carpeta de archivos de Colab (o colócalo junto al notebook si trabajas en local) antes de ejecutar cualquiera de los dos.

Rúbricas

Las cinco rúbricas son las mismas que ya se usaban en Moodle. Cada una puntúa sobre el máximo de su tabla, y esa suma se escala a la nota final de la tarea sobre 10.

EX0 · Sistema experto de libre elección

Criterio	Mínimo (0)	Niveles intermedios	Máximo
Variabilidad de las reglas	No entregado	No ha usado reglas (2) · fuera de plazo (2,5) · todas iguales (3) · dos tipos (4)	Tres o más tipos de reglas (5)
Memoria	No entregada	Insuficiente (2) · fuera de plazo (2,5) · suficiente (3) · bien (4)	Muy bien (5)
Hechos iniciales (@DefFacts)	No entregado	Ninguno (2) · fuera de plazo (2,5) · uno (3) · dos (4)	Tres o más (5)
Reglas (@Rule)	No entregado	Ninguna (2) · fuera de plazo (2,5) · una (3) · dos (4)	Tres o más (5)
Originalidad	No entregado	Nada original (2) · fuera de plazo (2,5) · poco original (3) · original (4)	Muy original (5)
Funciones extra (preguntas, respuestas, matemáticas...)	No entregado	Ninguna (2) · fuera de plazo (2,5) · una (3) · dos (4)	Tres o más (5)

EX1 · Detectar lesiones de rodilla

Criterio	Mínimo (0)	Niveles intermedios	Máximo
Regla de diagnóstico	No entregada	No la ha definido, pero muestra algo (1)	Correctamente definida (2)
Método para añadir hechos	No entregado o con <code>declare</code>	—	Definido correctamente (1)
Pruebas	No entregado	Fallan algunas celdas (1)	Todas las celdas de prueba funcionan (2)
Reglas (<code>@Rule</code>)	No entregado	Sin reglas (1) · casi todas (8)	Todas las reglas correctamente definidas (10)

EX2 · Previsión del valor de mercado

Criterio	Mínimo (0)	Niveles intermedios	Máximo
<code>FunctionClassifier</code> de Human-Learn (<code>score</code>)	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
<code>FIGSClassifier</code> (<code>fit</code> , predicción, <code>score</code>)	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
<code>FIGS</code> sin campos evidentes	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)

EX3 · Centrocampistas con potencial

Criterio	Mínimo (0)	Niveles intermedios	Máximo
Antecedentes y consecuentes	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
Funciones de pertenencia (×3)	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
Reglas + <code>ControlSystem</code> + <code>view</code>	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
Pruebas del potencial (×4)	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
Columna de potencial en el <code>DataFrame</code>	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
Conclusiones	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)

EX4 · Simulador de quemador de gas



Pide dos enfoques, no uno

Esta rúbrica evalúa **dos soluciones distintas** para el mismo quemador: un controlador simple y un enfoque alternativo. No es un error del enunciado entregar dos veces «lo mismo» con matices: es lo que se pide.

Criterio	Mínimo (0)	Niveles intermedios	Máximo
Controlador simple	No entregado	Muy mal (1) · mal (2) · neutral (3) · bien (4)	Muy bien (5)
Antecedentes, consecuente y conjuntos difusos	No entregado	Muy mal (1) · mal (2) · neutral (3) · bien (4)	Muy bien (5)
4 reglas, controlador y <code>view</code>	No entregado		Muy bien (5)

Criterio	Mínimo (0)	Niveles intermedios Muy mal (1) · mal (2) · neutral (3) · bien (4)	Máximo
Enfoque alternativo: antecedentes, consecuente, conjuntos difusos	No entregado	Muy mal (1) · mal (2) · neutral (3) · bien (4)	Muy bien (5)
3 reglas, controlador y <code>view</code> (del enfoque alternativo)	No entregado	Muy mal (1) · mal (2) · neutral (3) · bien (4)	Muy bien (5)

[Volver a la UD05](#) · [Ejercicios](#) · [Taller 1](#) · [Taller 2](#) · [Taller 3](#) · [Actividades guiadas](#)

 23 de agosto de 2026

6. UD03

6.1 UD03 — Procesamiento del Lenguaje Natural



Unidad 3 · 12 h · semanas 16-19

Es el RA con **más criterios de evaluación** de todo el módulo: **siete**. Se evalúa con **cuatro entregables prácticos** y la prueba escrita del RA3.

1. Introducción

De todas las formas de inteligencia, el **lenguaje** es la que mejor nos permite comunicar ideas, pedir ayuda, decidir y persuadir. Conseguir que las máquinas entiendan y generen lenguaje humano es el objetivo del **procesamiento del lenguaje natural (PLN)**: el campo de la IA que está detrás de los buscadores, los traductores, los asistentes de voz, los correctores y los chatbots.

Esta unidad tiene una particularidad que conviene saber desde el principio: de sus **siete criterios**, tres no se demuestran escribiendo código. Hablan del **papel del lingüista**, del **trabajo cooperativo** entre perfiles y de la **formación** que necesita quien investiga en PLN. No son relleno: son la mitad del RA, y en esta unidad se evalúan con una parte escrita dentro de los entregables.

El recorrido:

1. **Qué es el PLN** y qué tareas comprende, del análisis de una palabra a la generación de texto.
2. **Qué se consigue hoy** — el potencial, con cifras.
3. **La ambigüedad**: por qué es *el* problema del lenguaje, en sus seis formas.
4. **Desambiguación y etiquetado morfológico (POS)**: cómo se ataca ese problema.
5. **Las demás limitaciones**: falta de comprensión, sesgos, lenguas con pocos recursos, ironía.
6. **Cuándo es factible** aplicar PLN a un problema, con la lupa del AI Act.
7. **Quién participa**: el lingüista, la cooperación con informáticos y la formación necesaria.
8. **Las herramientas** en Python: `nlk`, `spaCy`, `scikit-learn` y `transformers`.
9. **Cómo construir un sistema** de PLN orientado a una tarea concreta.



Hilo conductor de la unidad

El lenguaje es la interfaz más natural entre personas y máquinas, y también la más ambigua. Primero vemos qué es el PLN y hasta dónde llega; luego por qué falla —la ambigüedad— y cómo se ataca; después quién hace falta en un proyecto y cuándo merece la pena; y por último construimos un sistema que resuelva una tarea concreta.



De dónde sale el material de esta unidad

Los bloques de **ambigüedad**, **desambiguación y etiquetado morfológico**, con sus ejemplos en español, y el itinerario de formación provienen del material del profesor, adaptados aquí. Las prácticas usan `nlk`, `spaCy`, `PyTorch` y modelos de **Hugging Face**. La referencia teórica de fondo es *Speech and Language Processing* de **Jurafsky y Martin**, de acceso libre.

2. Resultado de aprendizaje y criterios de evaluación

RA3 — Relaciona el procesamiento de lenguaje natural con sus aplicaciones determinando su potencial e identificando sus limitaciones.

CE	Criterio de evaluación	Bloque
RA3-a	Se ha caracterizado el procesamiento de lenguaje natural.	\$4, \$7
RA3-b	Se ha justificado el papel del lingüista en un proyecto de inteligencia artificial.	\$9

CE	Criterio de evaluación	Bloque
RA3-c	Se ha determinado el potencial de las técnicas existentes de procesamiento de lenguaje, así como sus limitaciones.	\$5-8
RA3-d	Se ha considerado en qué casos es factible aplicar estas técnicas en la resolución de un problema.	\$9
RA3-e	Se ha evaluado el trabajo cooperativo entre lingüistas e informáticos en el campo del procesamiento del lenguaje natural.	\$10
RA3-f	Se ha descrito la formación teórica que precisa el investigador en procesamiento del lenguaje natural.	\$10
RA3-g	Se ha elaborado un sistema de procesamiento de lenguaje orientado a una tarea específica.	\$11-12



Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo llama a este bloque, textualmente, «*Procesamiento del Lenguaje Natural*», y le asigna estos contenidos:

- *Procesamiento del lenguaje natural: Potencial y limitaciones.*
- *Aplicaciones del procesamiento del lenguaje natural.*

Son **dos contenidos para siete criterios de evaluación**: el desajuste **más grande de todo el módulo**. Ni el papel del lingüista, ni el trabajo cooperativo, ni la formación del investigador —tres de los siete CE— tienen contenido propio en el anexo. Los desarrolla el centro (art. 3.3 del Decreto 95/2026), y es lo que hace esta unidad.

3. Objetivos de la unidad

Objetivo	Al terminar la unidad serás capaz de...
O1	Explicar qué es el PLN y situarlo entre la lingüística y la informática.
O2	Describir las tareas del PLN y el <i>pipeline</i> típico de un sistema.
O3	Valorar el potencial de las técnicas actuales con ejemplos y cifras.
O4	Distinguir los seis tipos de ambigüedad y reconocerlos en frases reales.
O5	Explicar cómo el etiquetado morfológico (POS) desambigua, y qué cuesta elegir el conjunto de etiquetas.
O6	Identificar las demás limitaciones: falta de comprensión, sesgos, lenguas con pocos recursos, ironía.
O7	Determinar cuándo es factible aplicar PLN, incluidas las restricciones del AI Act.
O8	Justificar el papel del lingüista, evaluar la cooperación con informáticos y describir la formación necesaria.
O9	Usar <code>nlTK</code> y <code>spaCy</code> para tareas básicas de PLN en español.
O10	Construir y evaluar un sistema de PLN orientado a una tarea específica.

4. Qué es el PLN (RA3-a)

4.1 Definición

El **procesamiento del lenguaje natural** es el subcampo de la informática y de la IA que busca que las máquinas **comprendan y se comuniquen con el lenguaje humano**. Combina tres disciplinas:

Disciplina	Qué aporta
Lingüística computacional	Los modelos del lenguaje: morfología, sintaxis, semántica, pragmática
Estadística y aprendizaje automático	Aprender los patrones de los textos, en vez de programar todas las reglas a mano
Aprendizaje profundo	Redes que aprenden representaciones del lenguaje (<i>transformers</i>)



No es magia, y la distinción importa

Un buscador o un traductor no «entienden» como una persona: **procesan estadísticamente** el texto. Esa diferencia no es filosófica, es operativa: explica por qué fallan donde fallan, y es el hilo de todo el §6.

4.2 Los niveles del lenguaje

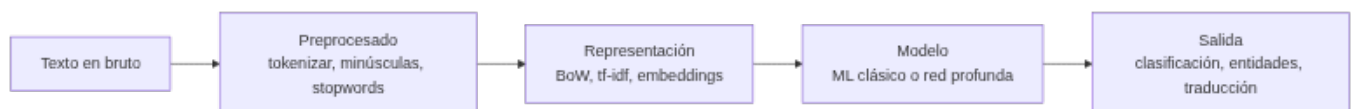
El lenguaje se analiza por capas, y cada una tiene sus tareas:

Nivel	Qué analiza	Tarea de PLN
Fonética y fonología	Los sonidos	Reconocimiento de voz (ASR), síntesis (TTS)
Morfología	La estructura de las palabras	<i>Stemming</i> , lematización, etiquetado POS
Sintaxis	La estructura de las oraciones	Análisis sintáctico, dependencias
Semántica	El significado de palabras y frases	Desambiguación (WSD), entidades (NER)
Pragmática y discurso	El significado según el contexto	Correferencia, actos de habla, sentimiento

Esta tabla vale también como mapa de la ambigüedad: **hay ambigüedad en todos los niveles**, y el §6 recorre uno a uno.

4.3 Las tareas del PLN

Tarea	Qué hace	Ejemplo
Tokenización	Divide el texto en unidades	«Hola mundo» → [«Hola», «mundo»]
Etiquetado POS	Asigna categoría gramatical	«correr» → verbo
Entidades (NER)	Identifica nombres, lugares, fechas	«María vive en Madrid» → María: PER, Madrid: LOC
Análisis de sentimiento	Positivo, negativo o neutro	«¡Genial!» → positivo
Traducción automática	Traduce entre idiomas	ES ↔ EN
Resumen	Condensa documentos	Resumir un informe
Generación	Produce texto nuevo	Chatbots, redacción asistida
Respuesta a preguntas	Responde a partir de un texto	Asistentes, buscadores



4.4 El pipeline, paso a paso

1. **Preprocesado:** tokenizar, pasar a minúsculas, quitar palabras vacías (*stopwords*), lematizar.
2. **Representación:** convertir el texto en números — bolsa de palabras, tf-idf, *embeddings*.
3. **Modelo:** aplicar un modelo clásico o una red neuronal.
4. **Salida:** la tarea concreta.



Esto lo usas todos los días

El corrector del móvil usa morfología y un modelo de lenguaje. El buscador clasifica tu consulta y le extrae entidades. El asistente de voz primero **transcribe** (ASR), luego interpreta y responde. Los tres son *pipelines* de PLN, con los mismos cuatro pasos.

5. El potencial (RA3-c)

5.1 Qué se consigue hoy

Aplicación	Estado actual
Buscadores	Entienden la consulta, extraen entidades y responden directamente
Asistentes de voz	Transcriben, interpretan y ejecutan órdenes
Traducción automática	Documentos y conversación en tiempo real, con calidad funcional
Análisis de opinión	Valorar miles de reseñas o publicaciones automáticamente
Extracción de información	Leer facturas, contratos e historiales y sacar los campos (OCR + NER)
Chatbots	Resolver consultas frecuentes sin horario
Modelos generativos	Redactar, resumir, traducir y generar contenido



Tres cifras para calibrar

- En **SQuAD 2.0** (respuesta a preguntas), los mejores sistemas **superan la precisión humana**: EM ~91 frente a ~87.
- El análisis de sentimiento en reseñas de cine (IMDb) pasó de ~89 % en 2011 a **95-97 %** con *transformers*.
- El ecosistema **Hugging Face** supera los **2 millones de modelos** y 500.000 *datasets* públicos. Es el «almacén» del que salen los modelos de esta unidad.

5.2 Los grandes modelos de lenguaje

El **AI Act** (Reglamento UE 2024/1689) define los *modelos de IA de uso general*: modelos con al menos **mil millones de parámetros**, entrenados con autosupervisión a gran escala, que realizan con competencia una amplia variedad de tareas. Los **LLM** son el ejemplo típico.



Modelos de gran impacto

El AI Act presume que un modelo de uso general tiene **capacidades de gran impacto** cuando el cómputo acumulado de su entrenamiento supera los **10²⁵ FLOPS** (art. 51.2), y entonces le exige documentación y evaluación adicionales. Se trata a fondo en la UD06.

6. La ambigüedad, el problema de fondo (RA3-c)

Si hay que quedarse con una sola limitación del PLN, es esta: **el lenguaje es ambiguo en todos sus niveles**. Múltiples interpretaciones de una misma palabra pueden arruinar la capacidad de un sistema para hacer su trabajo. Y no es un caso raro de laboratorio: aparece en frases que decimos sin darnos cuenta.

6.1 Los seis tipos de ambigüedad

Tipo	Cuándo aparece
Sintáctica	La oración admite más de un análisis sintáctico : más de una regla gramatical la representa
Léxica y morfológica	La léxica, cuando un término aislado admite varias interpretaciones; la morfológica, cuando una palabra puede tener más de un rol o categoría dentro de la frase
Semántica	Un elemento de la frase se puede interpretar de varios modos
Pragmática	El sentido depende del contexto y de quién habla, en ese momento
Fonológica	Una cadena de sonidos resulta confusa
Funcional	Un término se usa con doble función gramatical



Los seis, con frases reales

- **Sintáctica** — «*Los perros y los gatos enfermos son recogidos por el servicio municipal.*» ¿Se recogen todos los perros y solo los gatos enfermos, o solo los enfermos de las dos especies? Y «*Compro los libros baratos.*»: ¿compro **los baratos** (adjetivo) o compro libros, **que además son baratos** (complemento predicativo)?
- **Léxica y morfológica** — «*Usted aquí no pinta nada.*» ¿No tiene mando o no pinta paredes? Y «*Pedro y yo escribimos un cuento.*»: ¿ya lo escribimos o lo estamos escribiendo? La forma conjugada vale para presente y para pretérito.
- **Semántica** — «*Pedro quiere pelearse con un italiano.*» ¿Con cualquier italiano o con uno concreto?
- **Pragmática** — «*Golpeó el armario con el bastón y lo rompió.*» ¿Se rompió el bastón o el armario?
- **Fonológica** — «*es-conde*»: ¿el verbo *esconder* conjugado, o *es + conde*?
- **Funcional** — «*He vuelto a oler.*» ¿He recuperado el olfato, o he regresado a un sitio para oler algo?

6.2 Un banco de frases para probar cualquier sistema

Estas frases son un buen test rápido: si un sistema las resuelve, es que usa el contexto de verdad.

- Vino de la Rioja.
- Compré unos zapatos de piel de señora.
- La policía observó al sospechoso con unos prismáticos.
- El pescado está listo para comer.
- El cura recibió una cura en su habitación.
- Juan vio a Pedro enfurecido.
- Antonio no nada nada.
- No puedo ir a la fiesta porque no traje traje.
- Estaré de vacaciones solo unos días.
- El Villarreal le ganó al Valencia en su campo.
- Me quedé esperándote en el banco.



Fíjate en el patrón

Casi todas se resuelven **con el contexto**, no con la frase. «*Vino de la Rioja*» es un sustantivo o un verbo según lo que venga antes; «*en su campo*» depende de quién sea «su». Esa es exactamente la información que un modelo estadístico intenta capturar — y la razón de que el tamaño del contexto sea tan determinante en los *transformers*.

6.3 Por qué la ambigüedad no se «arregla»

La ambigüedad no es un error del lenguaje que se pueda corregir: es una **propiedad** suya, y además útil —permite ser breve, irónico o cortés—. Lo que hace un sistema de PLN es **elegir la interpretación más probable** dado el contexto, y por eso siempre puede equivocarse.



Forma frente a significado

Emily Bender y Alexander Koller argumentaron que un modelo entrenado **solo con la forma** del texto no tiene manera *a priori* de aprender el **significado**: aprende correlaciones estadísticas, no comprensión. De ahí que un modelo generativo pueda «sonar» impecable y estar equivocado. Es la limitación que hay que tener presente al leer las cifras del \$5.

7. Desambiguación y etiquetado morfológico (RA3-a, RA3-c)

7.1 Qué es el POS *tagging*

El **etiquetado morfológico** —*POS tagging*, etiquetado léxico, desambiguación morfosintáctica— es el proceso de asignar a **cada palabra de un texto su categoría gramatical**, sin entrar en las relaciones sintácticas.

Y aquí está la clave: **una palabra aislada suele ser ambigua respecto a su categoría; dentro de un contexto, casi siempre se puede desambiguar.**



La misma palabra, tres categorías

- «*Mételo en ese **sobre***» → **nombre**
- «*Déjalo **sobre** la mesa*» → **preposición**
- «*Dame lo que te **sobre***» → **verbo**

Tres frases, la misma forma, tres categorías. Ningún diccionario resuelve esto: hace falta el contexto. Etiquetar bien la categoría es el **primer paso** de casi todo lo demás — de ahí que aparezca tan pronto en el *pipeline* del §4.4.

7.2 Elegir el conjunto de etiquetas: un compromiso

El **conjunto de etiquetas** (*tagset*) que se elige cambia la dificultad del problema, y hay que equilibrar dos cosas que tiran en direcciones opuestas:

- **Más información:** etiquetas específicas — distinguir tiempo, persona, número, género.
- **Menos trabajo de desambiguación:** etiquetas generales — solo «verbo», «nombre», «adjetivo».

Cuantas más etiquetas, más informativo el resultado y **más difícil acertar**.

Conjunto de etiquetas	Número de etiquetas
Penn Treebank	45
Brown Corpus	87
LexEsp (PAROLE)	~250
Susanne	350

Las **categorías principales** son las de siempre: nombres (comunes y propios), pronombres, determinantes, adjetivos, verbos, adverbios, preposiciones y conjunciones. Se dividen en clases **abiertas** —admiten palabras nuevas: nombres, verbos, adjetivos— y **cerradas** —no: preposiciones, determinantes, conjunciones—.



Por qué esto es asunto del lingüista

Decidir el *tagset*, escribir la guía de anotación y resolver los casos dudosos **no es una tarea de programación**: es lingüística aplicada, y condiciona todo lo que el modelo podrá aprender después. Es el ejemplo más concreto del **RA3-b**, y por eso este bloque va justo antes del §9.

7.3 Cómo se etiqueta en la práctica

Enfoque	Cómo funciona	Dónde encaja
Basado en reglas	Reglas escritas a mano sobre el contexto	Dominios muy acotados; didáctico
Modelos de Markov (HMM, TnT)	Probabilidad de una secuencia de etiquetas dado el texto	El clásico; es lo que se practica con el corpus <code>cess_esp</code> de <code>nlTK</code>
Modelos neuronales	La categoría sale del modelo del lenguaje, con el contexto completo	<code>spaCy</code> y <i>transformers</i> : lo que se usa hoy

En la práctica de la unidad se hacen los tres: reglas y HMM con `nlTK`, y el enfoque neuronal con `spaCy`.

8. Las demás limitaciones (RA3-c)

8.1 Falta de comprensión real

Los modelos **generan respuestas plausibles sin comprender**: no verifican hechos, no razonan sobre el mundo y pueden **alucinar** — inventar datos con total seguridad. En cualquier tarea con consecuencias, la salida **se verifica**.

8.2 Sesgos

Los modelos aprenden de los textos que hay, y esos textos llevan **sesgos históricos y sociales**:

- Sesgo de **género** en los *embeddings*: «médico → hombre», «enfermera → mujer».
- Sesgos **raciales, étnicos y culturales** en los datos de entrenamiento.
- El AI Act advierte de que los sesgos pueden **perpetuarse y amplificarse** en bucles de retroalimentación.

 **Inferir emociones está prohibido en dos contextos**

El AI Act **prohíbe** los sistemas que infieren el estado emocional **en el trabajo y en centros educativos**: la base científica es débil —la expresión de las emociones varía entre culturas— y su uso ahí es discriminatorio. Es un límite **legal**, no solo técnico, y afecta directamente a lo que se puede construir con análisis de sentimiento.

8.3 Lenguas con pocos recursos

Casi todos los modelos se entrenan con textos mayoritariamente en inglés. En lenguas con menos datos, las técnicas **estadísticas clásicas** pueden aún superar a las neuronales, que solo dan buenos resultados a partir de decenas de miles de *tokens*. No es un detalle menor: decide si un proyecto en una lengua minoritaria es viable.

8.4 Ironía y sarcasmo

«¡Qué buena idea, se me ha caído el móvil al agua!» es literalmente positivo y pragmáticamente negativo. Es ambigüedad **pragmática** pura, y los modelos fallan a menudo.

9. Cuándo es factible aplicar PLN (RA3-d)

9.1 Cinco criterios de decisión

Criterio	Pregunta guía
Tarea clara y acotada	¿Qué salida exacta quiero: clasificar, extraer, traducir, resumir?
Datos disponibles	¿Hay textos suficientes y representativos? ¿Están anotados?
Dominio con vocabulario conocido	¿El texto es de un ámbito concreto — médico, legal, atención al cliente?
Calidad esperada realista	¿Acepto un 5-10 % de error? ¿Cuánto cuesta cada error?
Regulación aplicable	¿Está el uso prohibido o restringido por el AI Act?

9.2 Factible y no factible

Factible hoy (bajo riesgo)	No factible o prohibido
Convertir datos no estructurados en estructurados	Inferir emociones en el trabajo o la educación
Clasificar documentos y detectar duplicados	Extracción no selectiva de rostros de internet
Mejorar el registro o el lenguaje de un documento	Manipulación subliminal del comportamiento
Traducción funcional de documentos	Comprensión profunda con inferencia abierta
Buscar y vincular datos en archivos	Respuesta fiable en dominios de alto riesgo sin verificación
Análisis de sentimiento en un dominio acotado	Detección de sarcasmo con alta precisión

 **El AI Act como herramienta de diseño**

El **considerando 53** enumera tareas de PLN de **bajo riesgo** —procesamiento delimitado: transformar datos, clasificar, detectar duplicados, mejorar el lenguaje, traducción funcional— y el **artículo 5** prohíbe prácticas concretas. Leídos juntos, son una lista de comprobación excelente para decidir qué construir **antes** de empezar. Eso es el CE d.

10. Quién hace un proyecto de PLN (RA3-b, RA3-e, RA3-f)

Aquí están **tres de los siete criterios** de este RA, y son los que un material puramente técnico suele resolver en un párrafo. No se demuestran con código, pero deciden si un proyecto de PLN sale bien o sale caro.

10.1 Qué aporta el lingüista (RA3-b)

Aportación	En qué consiste
Anotación de datos	Etiquetar entidades, sentimiento o categorías en los textos de entrenamiento
Diseño del conjunto de etiquetas y de la guía	Definir el <i>tagset</i> del §7.2 y las reglas para aplicarlo de forma coherente
Recursos lingüísticos	Construir corpus y árboles sintácticos (<i>treebanks</i>), un trabajo que puede llevar años
Gramáticas y reglas	Las reglas morfológicas y sintácticas de la rama simbólica del PLN
Evaluación y análisis de errores	Explicar por qué falla un modelo: ambigüedad, registro, dominio



Las anotaciones deciden las predicciones

La calidad del etiquetado condiciona directamente lo que el modelo puede aprender: con etiquetas malas o **inconsistentes entre anotadores**, el modelo aprende esa inconsistencia. Y esto no se arregla con más datos ni con un modelo mejor.

Es la razón de fondo del CE b: **el lingüista no es un apoyo del proyecto, es parte de la tubería de datos**. Volviendo al §7.2: quien decide si el *tagset* tiene 45 etiquetas o 250 está decidiendo el techo de precisión del sistema.

10.2 El trabajo cooperativo (RA3-e)

El PLN es **interdisciplinar por definición**: lingüistas, informáticos y, según el proyecto, psicólogos o expertos en lógica. Dos casos reales que muestran cómo se hace bien:

- **Meta NLLB** — traducción entre 200 idiomas. Trabajaron con **hablantes nativos** para anotar y evaluar, y consiguieron **+44 % de BLEU medio** además de controlar la toxicidad de las traducciones. Sin hablantes nativos no había forma de evaluar la mayoría de esos idiomas.
- **Masakhane** — PLN para lenguas africanas. Más de **1.000 participantes de 30 países**, y una regla explícita: los lingüistas locales **no son «solo anotadores», son investigadores**. Rechazan la investigación *paracaidista*: entrar, tomar los datos y marcharse sin dejar capacidad local.

Lo que se gana cooperando	Lo que cuesta
Datos y evaluación de mucha más calidad	Vocabularios y métodos distintos: hay que traducirse
Cobertura de lenguas minoritarias	La anotación es lenta y cara
Mejor ciencia y sistemas más justos	Riesgo de usar a las comunidades como proveedores de datos



El riesgo tiene nombre

La cooperación puede degenerar en extracción: una empresa del norte global obtiene datos anotados baratos y publica; la comunidad que los produjo no queda con capacidad ni con crédito. Es el mismo problema que en la UD06 aparece como **el trabajo invisible detrás de la IA**. Por eso el CE e dice «**evaluar**» el trabajo cooperativo, no solo describirlo: hay formas mejores y peores de hacerlo.

10.3 La formación del investigador (RA3-f)

El perfil combina tres patas, y **ninguna sobra**:

Pata	Contenidos
Lingüística	Morfología, sintaxis, semántica, pragmática y discurso
Estadística y aprendizaje automático	Probabilidad, modelos, métricas y evaluación
Informática y métodos formales	Algoritmos, estructuras de datos, programación

El itinerario recomendado, en este orden:

1. **Jurafsky y Martin**, *Speech and Language Processing* — la referencia clásica, de acceso libre. Da la base teórica y el vocabulario.
2. **El manual de NLTK** — el mismo camino, pero con las manos: cada paso del *pipeline*, a mano.
3. **spaCy**, en inglés y en español — el salto a herramientas de producción.
4. **Modelos neuronales y transformers** — Hugging Face, y de ahí a plataformas de inferencia si hace falta desplegar.



Perfiles que no existían hace cinco años

Junto al lingüista computacional clásico han aparecido el **anotador o evaluador especializado** —con formación en derecho, medicina o ciencia, porque hay que entender el texto para etiquetarlo bien—, el perfil de **prompt engineer**, en evolución hacia LLMOps, y el **evaluador de sesgos y ética** de modelos. Los tres parten de la misma base: **lingüística más técnica**.



Cómo se evalúan estos tres criterios en esta unidad

No con un examen aparte: con la **parte escrita de los entregables EX2 y EX5**. En cada uno se pide justificar qué decisiones tomaría un lingüista en ese problema concreto, qué aporta cada perfil y qué formación haría falta. Está en la rúbrica, y cuenta.

11. Las herramientas en Python (RA3-c, RA3-g)



Dos entornos, y conviene saber cuál toca

Esta unidad usa las librerías más pesadas del módulo. La regla:

- **En el contenedor de la unidad** (`Dockerfile` y `docker-compose.yml` de la carpeta): `nlTK`, `spaCy`, `scikit-learn`, `gensim`, `textblob`. Todo lo que no entrena redes grandes.
- **En Colab**: todo lo que use `transformers`, entrenamiento de modelos o **audio**. Ahí hay GPU, y afinar un DistilBERT en CPU es cuestión de horas.

Cada notebook lo indica en su primera celda.

11.1 NLTK, para entender

NLTK es una plataforma de enseñanza e investigación con más de 50 corpus y recursos léxicos, como WordNet. Su virtud es que es **transparente**: cada paso del procesamiento se hace a mano, y por eso es la herramienta con la que se **aprende**.

```
1 pip install nltk
2 python -c "import nltk; nltk.download(['punkt_tab', 'stopwords', 'wordnet', 'cess_esp'])"
```

```
1 import nltk
2 from nltk.tokenize import word_tokenize
3 from nltk.corpus import stopwords
4 from nltk.stem.snowball import SnowballStemmer
5
6 texto = "La inteligencia artificial está transformando la atención al cliente."
7 tokens = word_tokenize(texto, language='spanish')
8 sin_vacias = [t for t in tokens if t.lower() not in stopwords.words('spanish')]
9 raiz = SnowballStemmer('spanish').stem
10
11 print("Tokens:", tokens)
12 print("Sin stopwords:", sin_vacias)
13 print("Stemming:", [raiz(t) for t in sin_vacias])
```

**El pipeline de NLTK, paso a paso**

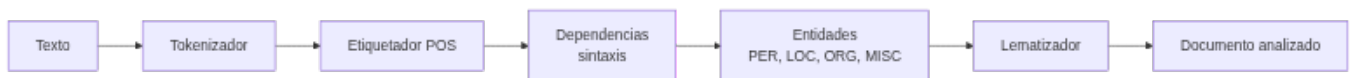
1. **Tokenizar:** `word_tokenize` separa palabras y signos.
2. **Limpiar:** se descartan artículos, preposiciones y palabras sin contenido.
3. **Normalizar:** el *stemming* recorta a una raíz aproximada («transformando» → «transform»); la **lematización** da el lema real («transformando» → «transformar»). El primero es rápido y tosco; la segunda necesita un diccionario.
4. **Enriquecer:** etiquetar POS (`pos_tag`), detectar entidades (`ne_chunk`), contar frecuencias (`FreqDist`) o buscar concordancias (`concordance`).

11.2 spaCy, para producir

```
1 pip install spacy
2 python -m spacy download es_core_news_sm
```

```
1 import spacy
2
3 nlp = spacy.load("es_core_news_sm")
4 doc = nlp("Maria trabaja en Madrid para una empresa de inteligencia artificial.")
5
6 for token in doc:
7     print(f"{token.text:<12} {token.pos_:<8} {token.lemma_:<12}")
8
9 print("\nEntidades:")
10 for ent in doc.ents:
11     print(f" {ent.text:<12} → {ent.label_}")
```

El pipeline de spaCy ejecuta toda la cadena **de una pasada**:



En una frase corriente, el analizador sintáctico puede encontrar **miles de análisis posibles**, y spaCy elige el más probable con su modelo estadístico. Es el \$6 en acción: la ambigüedad no se elimina, **se decide**.

**NLTK o spaCy: no compiten**

NLTK para **aprender** — cada paso a mano, todo visible. **spaCy** para **producir** — un objeto `nlp`, modelos preentrenados, mucho más rápida y con 25 idiomas. En los talleres se empieza con NLTK y se pasa a spaCy, en ese orden y por ese motivo.

**Por qué las entidades dan dinero**

La extracción de entidades es la base de leer facturas, contratos o historiales: el sistema encuentra fechas, importes, nombres y organizaciones **sin que nadie los marque a mano**. Es una de las aplicaciones más rentables del PLN, y de las menos vistas.

11.3 Análisis de sentimiento

Con **scikit-learn**, que funciona en español y sin GPU:

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.pipeline import make_pipeline
4 from nltk.corpus import stopwords
5
6 reseñas = ["me encantó la comida", "pésimo servicio", "todo perfecto", "no volveré"]
7 etiquetas = [1, 0, 1, 0]
8
9 pipe = make_pipeline(
10     TfidfVectorizer(stop_words=stopwords.words('spanish')),
11     LogisticRegression()
12 )
13 pipe.fit(reseñas, etiquetas)
14 print(pipe.predict(["la comida estaba buenisima"]))
```

**El detalle que rompe el ejemplo en español**

`TfidfVectorizer(stop_words=...)` **solo acepta la cadena 'english'**. Para español hay que pasar una **lista** — `stopwords.words('spanish')` — o no pasar nada y usar `max_df`. Si se escribe `stop_words='spanish'`, el error es poco claro.

11.4 Transformers y Hugging Face

Los modelos **Transformer** se usan con la librería `transformers` y sus *pipelines*. Funcionan en CPU, pero **entrenarlos o afinarlos, no**: eso va a Colab.

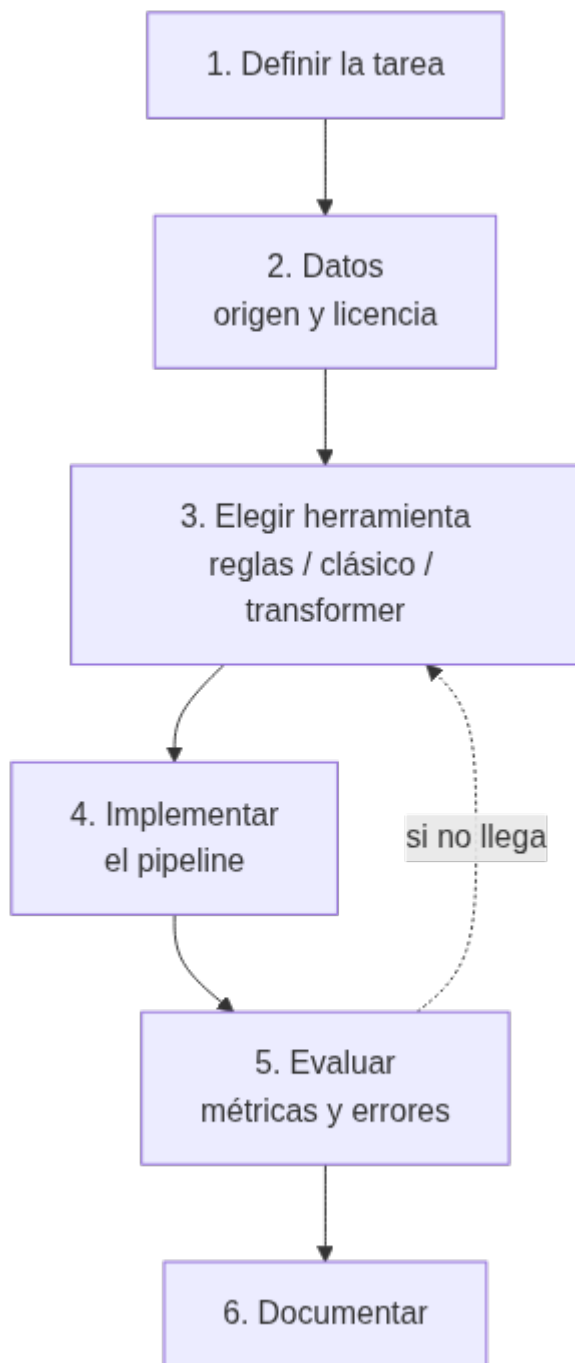
```
1 from transformers import pipeline
2
3 clasificador = pipeline("sentiment-analysis",
4                         model="pytorch/sentiment-analysis-roberta-base")
5 print(clasificador("El servicio fue excelente, volveré seguro."))
```

**DistilBERT, y por qué se usa aquí**

DistilBERT tiene un **40 % menos de parámetros** y es un **60 % más rápido** que BERT, conservando el **97 %** de su capacidad. Por eso es el modelo de los notebooks de la unidad: cabe en una sesión de clase. El entregable **EX4** lo **afina** para reseñas de cine, que es *transfer learning* de manual.

12. Construir un sistema orientado a una tarea (RA3-g)**12.1 La metodología, en seis pasos**

1. **Definir la tarea**: qué entra, qué sale y para quién.
2. **Conseguir o anotar los datos**: origen, formato, tamaño y **licencia**.
3. **Elegir la herramienta**: reglas, modelo clásico o *transformer*, según datos y recursos.
4. **Implementar** el *pipeline*: preprocesado → representación → modelo → salida.
5. **Evaluar** sobre datos **no vistos**, y **analizar los errores** — no solo mirar la métrica.
6. **Documentar**: origen de los datos, licencia, decisiones y limitaciones.



El paso 5 es el que más se salta

Medir la exactitud es fácil; **mirar los errores** es lo que enseña. Casi siempre se agrupan: un registro que no estaba en el entrenamiento, un tipo de ambigüedad concreta, un vocabulario de dominio. Y eso apunta a qué hacer después — que es volver al paso 3, no al paso 4.

12.2 La progresión de los notebooks

Los notebooks de la unidad recorren esa metodología de menos a más, y conviene verlos como una escalera, no como piezas sueltas:

Notebook	Qué construye	Representación
N01 · introducción al PLN	Tokenizar, <i>stopwords</i> , BoW y tf-idf con <code>nltk</code>	Cuenta de palabras
N02 · clasificación con PyTorch	Un clasificador de texto entrenado desde cero	De BoW a <i>word embeddings</i>
N03 · modelos de lenguaje	Afinar DistilBERT para sentimiento	<i>Embeddings</i> contextuales
N04 · spaCy	El <i>pipeline</i> completo: POS, dependencias, entidades	Modelo preentrenado
N05 · <code>nltk</code> y Python	POS <i>tagging</i> con el corpus <code>cess_esp</code> y validación cruzada	Etiquetado estadístico

Y los cuatro entregables aplican cada nivel a un problema propio:

Entregable	Tarea	Qué demuestra
EX1	Representación de texto: tokenizar, <i>stopwords</i> , BoW y tf-idf	Los fundamentos, por dos caminos (NLTK y TextBlob)
EX2	Clasificar preguntas repitiendo el proceso de N02	Construir un clasificador propio
EX3	Etiquetado morfosintáctico del corpus <code>cess_esp</code> con NLTK	Trabajar con anotación real en español
EX4	Afinar DistilBERT para reseñas de cine	<i>Transfer learning</i> a una tarea nueva
EX5	Asistente virtual por voz	Un sistema de punta a punta, con audio



Ejemplo guiado: un clasificador de reseñas en seis pasos

1 · Tarea: clasificar una reseña de restaurante como positiva o negativa. **2 · Datos:** 60 reseñas anotadas a mano, 40 para entrenar y 20 para probar; licencia propia. **3 · Herramienta:** scikit-learn con tf-idf, porque con 60 ejemplos un *transformer* no aporta nada. **4 · Implementar:** `TfidfVectorizer` → `LogisticRegression`. **5 · Evaluar:** exactitud en las 20 de prueba y **leer los dos errores** — casi siempre son ironía o una negación. **6 · Documentar:** origen, licencia y el límite obvio (60 ejemplos de un solo dominio).

Fíjate en el paso 3: **la herramienta se elige por los datos que hay**, no por lo moderna que sea. Con 60 ejemplos, tf-idf gana.

13. Puntos clave de la unidad

- El **PLN** busca que las máquinas comprendan y se comuniquen con el lenguaje humano, combinando lingüística computacional, estadística y aprendizaje profundo.
- Sus tareas van de la **tokenización** al etiquetado POS, entidades, sentimiento, traducción, resumen y generación; el *pipeline* típico es **preprocesado** → **representación** → **modelo** → **salida**.
- El **potencial** es alto y medible: en respuesta a preguntas los sistemas **superan la precisión humana** (SQuAD 2.0), y el sentimiento en reseñas ronda el **95-97 %**.
- La **ambigüedad** es el problema de fondo, y tiene **seis formas**: sintáctica, léxica y morfológica, semántica, pragmática, fonológica y funcional. No se elimina: **se decide** con el contexto.
- El **etiquetado POS** es la herramienta central de desambiguación. Elegir el *tagset* es un compromiso: **más etiquetas = más información y más difícil acertar** (45 en Penn Treebank, 350 en Susanne).
- Las otras limitaciones: **forma ≠ significado** (Bender y Koller), alucinaciones, **sesgos**, lenguas con pocos recursos, ironía y sarcasmo.
- El **AI Act** sirve de herramienta de diseño: su considerando 53 lista PLN de bajo riesgo y su artículo 5 **prohíbe** prácticas concretas, como **inferir emociones en el trabajo o la educación**.
- El **lingüista** no es un apoyo: diseña el *tagset*, la guía de anotación y los recursos. **Las anotaciones deciden las predicciones**, y eso no se arregla con más datos.
- La **cooperación** entre perfiles se **evalúa**, no solo se describe: NLLB con hablantes nativos (+44 % BLEU) y Masakhane con 1.000 participantes son ejemplos de hacerlo bien; la investigación *paracaidista*, de hacerlo mal.
- La **formación** combina lingüística, estadística e informática, con un itinerario claro: Jurafsky → NLTK → spaCy → *transformers*.

- **NLTK para aprender, spaCy para producir**; scikit-learn cuando hay pocos datos y *transformers* cuando hay muchos y GPU. **La herramienta se elige por los datos que hay.**
- Construir un sistema son **seis pasos**, y el que más se salta es el quinto: **mirar los errores**, no solo la métrica.

14. Glosario

Término	Definición
PLN	Procesamiento del lenguaje natural
Token	Unidad mínima en que se divide un texto: palabra, signo, subpalabra
Tokenización	Dividir el texto en <i>tokens</i>
Stopwords	Palabras vacías (artículos, preposiciones) que se descartan
Stemming	Recortar la palabra a una raíz aproximada
Lematización	Reducir la palabra a su lema real, con diccionario
POS tagging	Asignar a cada palabra su categoría gramatical
Tagset	El conjunto de etiquetas gramaticales que se usa
Penn Treebank	<i>Tagset</i> de referencia en inglés: 45 etiquetas
PAROLE / LexEsp	<i>Tagset</i> de referencia en español: ~250 etiquetas
Categoría abierta / cerrada	Admite palabras nuevas (nombres, verbos) o no (preposiciones)
Ambigüedad sintáctica	La oración admite más de un análisis sintáctico
Ambigüedad léxica	Un término aislado admite varias interpretaciones
Ambigüedad morfológica	Una palabra puede tener más de una categoría en la frase
Ambigüedad semántica	Un elemento se puede interpretar de varios modos
Ambigüedad pragmática	El sentido depende del contexto y del hablante
Ambigüedad fonológica	Una cadena de sonidos resulta confusa
Ambigüedad funcional	Un término con doble función gramatical
Desambiguación	Elegir la interpretación correcta según el contexto
WSD	Desambiguación del sentido de las palabras
NER	Reconocimiento de entidades nombradas
Análisis de dependencias	Determinar las relaciones sintácticas entre palabras
Correferencia	Saber a qué se refiere un pronombre
Bolsa de palabras (BoW)	Representar un texto por la cuenta de sus palabras, sin orden
tf-idf	Pesar cada palabra por su frecuencia relativa en el documento y en el corpus
Word embedding	Representar palabras como vectores densos con significado
Embedding contextual	El vector de una palabra cambia según su contexto (BERT)
Transformer	Arquitectura basada en atención, base de los modelos actuales
BERT / DistilBERT	Modelo de lenguaje bidireccional, y su versión reducida
Transfer learning	Adaptar un modelo ya entrenado a una tarea nueva
Fine-tuning	Afinar los pesos de un modelo preentrenado con datos propios
LLM	Modelo grande de lenguaje
Modelo de uso general	En el AI Act, modelo con $\geq$ mil millones de parámetros y autosupervisión a gran escala

Término	Definición
Alucinación	Que el modelo genere información falsa con apariencia de certeza
NLTK	Biblioteca de PLN orientada a la enseñanza y la investigación
spaCy	Biblioteca de PLN orientada a producción
WordNet	Base de datos léxica de relaciones entre significados
cess_esp	Corpus del español anotado, incluido en NLTK
HMM / TnT	Modelos de Markov para etiquetado morfológico
Hugging Face	Repositorio de modelos y <i>datasets</i> de PLN
SQuAD	Conjunto de referencia para respuesta a preguntas
BLEU	Métrica de calidad de traducción automática
Corpus	Colección de textos usada para entrenar o evaluar
Treebank	Corpus anotado con árboles sintácticos
Guía de anotación	Documento que fija cómo etiquetar de forma coherente
Lengua con pocos recursos	Idioma con pocos datos disponibles para entrenar

15. FAQ

? ¿El PLN «entiende» de verdad el lenguaje? ▾

No. Procesa estadísticamente: aprende qué palabras y estructuras aparecen juntas. Bender y Koller lo argumentaron con precisión: un modelo entrenado **solo con la forma** del texto no tiene manera *a priori* de aprender el **significado**. Por eso puede escribir un párrafo impecable y equivocarse en el dato.

? Si la ambigüedad no se puede eliminar, ¿cómo funciona nada? ▾

Porque el sistema **no necesita acertar siempre**: necesita acertar lo suficiente para la tarea. Elige la interpretación más probable según el contexto, y en la mayoría de los casos es la buena. El problema aparece cuando el coste de equivocarse es alto — y ahí entra el \$9.

? ¿Cuántas etiquetas POS conviene usar? ▾

Las mínimas que resuelvan tu problema. Con 45 (Penn Treebank) se acierta más; con 250 (PAROLE) se sabe mucho más de cada palabra pero se falla más. Si solo necesitas distinguir nombres de verbos, usar 250 etiquetas es tirar precisión a la basura.

? ¿Para qué necesito un lingüista si tengo un modelo preentrenado? ▾

Para tres cosas que el modelo no hace: **decidir qué etiquetar y cómo** (el *tagset* y la guía), **anotar de forma coherente** los datos de tu dominio, y **explicar por qué falla** cuando falla. Un modelo preentrenado te ahorra entrenar, no te ahorra entender tu problema.

? ¿NLTK o spaCy? ▾

Las dos, en este orden. **NLTK** para aprender, porque cada paso es visible y se hace a mano. **spaCy** para trabajar, porque un solo objeto ejecuta todo el *pipeline* con modelos preentrenados y es mucho más rápida.

? ¿Puedo hacer análisis de sentimiento para detectar el ánimo de mis empleados? ▾

No. El AI Act **prohíbe** los sistemas que inferen el estado emocional en el **trabajo** y en **centros educativos**: la base científica es débil y el uso es discriminatorio. Es un límite legal, y es exactamente el tipo de decisión que pide el CE d.

? ¿Hace falta GPU para esta unidad? ▾

Para la mitad, no: `nlTK`, `spaCy`, `scikit-learn` y las representaciones de texto van en el contenedor. Para **afinar DistilBERT** y para el notebook de audio, sí — y por eso esos van en **Colab**, que la da gratis.

? ¿Por qué mi modelo va peor en español que en inglés? ▾

Porque casi todos se entrenan con textos mayoritariamente en inglés. En español hay recursos buenos, pero menos; en lenguas minoritarias, muy pocos. En esos casos, las técnicas estadísticas clásicas pueden **superar** a las neuronales, que necesitan decenas de miles de *tokens* para despegar.

? ¿Qué diferencia hay entre *stemming* y lematización? ▾

El *stemming* recorta a una raíz aproximada sin diccionario: rápido y tosco («transformando» → «transform», que no es una palabra). La lematización devuelve el **lema real** usando un diccionario: más lento y correcto («transformando» → «transformar»). Para buscar, suele bastar el primero; para analizar, hace falta el segundo.

16. Sesiones

Semana	Horas	Contenido	CE
16	3	Qué es el PLN, tareas y <i>pipeline</i> ; el potencial con cifras. <code>N01</code> (<code>nlTK</code>) y <code>EX1</code>	RA3-a, RA3-c
17	3	La ambigüedad en sus seis formas ; desambiguación y POS <i>tagging</i> . <code>N04</code> (<code>spaCy</code>) y <code>N05</code> (<code>cess_esp</code>); Taller 1	RA3-a, RA3-c
18	3	Las demás limitaciones; cuándo es factible con la lupa del AI Act. <code>N02</code> y <code>EX2</code> ; Taller 2	RA3-c, RA3-d, RA3-g
19	3	El lingüista, la cooperación y la formación; sistemas orientados a tarea. <code>N03</code> , <code>EX4</code> y <code>EX5</code> ; evaluación	RA3-b, RA3-e, RA3-f, RA3-g

Sobre el reparto

Los tres CE «humanos» se tratan en la **semana 19**, cuando el alumnado ya ha peleado con los datos y con la anotación: así la discusión sobre el papel del lingüista deja de ser abstracta. Y el material de **ampliación** (el clasificador de géneros musicales) **no cuenta horas**.

17. Recursos

- [Diapositivas](#)
- [Ejercicios de la unidad](#)
- Talleres: `T01` · [del texto al vector](#) · `T02` · [un sistema de PLN de punta a punta](#)
- [Actividades guiadas](#) — 5 notebooks
- [Actividades entregables](#) — 5 notebooks evaluables, con sus rúbricas
- **Notebooks** — todos los de la unidad, con descarga y apertura en Colab, en el menú «Notebooks»



Referencias de la unidad ▾

- [Speech and Language Processing](#) — Jurafsky y Martin (acceso libre)
- [Introduction to Natural Language Processing](#) — Eisenstein
- [NLTK Book](#) · [spaCy](#) · [Usage](#) · [Hugging Face](#)
- [Bender y Koller \(2020\), Climbing towards NLU](#)
- [Meta NLLB](#) · [Masakhane](#)
- [Reglamento \(UE\) 2024/1689 · AI Act](#) — considerando 53 y art. 5
- [SQuAD](#) · [Penn Treebank POS tags](#)

18. Evaluación

Peso	Instrumento
40 % actividades	Media de los cinco entregables (EX1 - EX5); cada uno con su rúbrica sobre 10
60 % prueba escrita	Prueba del RA3 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «*en función de la consecución de los RA*»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.
- **EX2 y EX5 llevan una parte escrita** que cubre los criterios b, e y f: el papel del lingüista, la cooperación entre perfiles y la formación necesaria. Está en su rúbrica y **cuenta**.

CE	Dónde se trabaja	Con qué se evalúa
RA3-a	§4, §7	Taller 1, EX1 , prueba del RA3
RA3-b	§10.1	Parte escrita de EX2 y EX5 , prueba del RA3
RA3-c	§5-8	EX1 , EX4 , Talleres 1 y 2, prueba del RA3
RA3-d	§9	Taller 2, prueba del RA3
RA3-e	§10.2	Parte escrita de EX2 y EX5 , prueba del RA3
RA3-f	§10.3	Parte escrita de EX2 y EX5 , prueba del RA3
RA3-g	§11-12	EX2 , EX4 , EX5 , Taller 2

19. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 de la Orden 8/2025): construir un sistema de PLN orientado a una tarea distinta —otro dominio, otro tipo de salida— y las pruebas de autoevaluación de la unidad.

.....

6.2 Diapositivas de la UD03



Diapositivas PDF



Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

🕒 23 de agosto de 2026

6.3 UD03 · Actividades guiadas

Cómo se trabajan

Estos cinco notebooks **no se entregan**: se hacen en clase, con el profesor, y son la preparación de los **cinco entregables**. Están ejecutados: puedes leerlos antes de tocar nada para ver qué hace cada celda.

Dos entornos

- **N01 y N04**: van en el **contenedor de la unidad** (`docker compose up -d` en esta carpeta) o en cualquier Python 3.12+ con `nltk` y `spaCy`.
- **N02, N03 y N05**: usan `transformers`, entrenan modelos o procesan audio. Van en **Colab**, que tiene GPU — en local, sin GPU, tardan horas.

N01 · Introducción al PLN

Los fundamentos con `nltk` y `TextBlob`: tokenizar, quitar *stopwords*, etiquetar categorías gramaticales, extraer frases nominales y construir representaciones bolsa de palabras, `tf-idf` y *word embeddings*.

Recurso	Enlace
Notebook	UD03_N01_introduccion_pln.ipynb

`gensim` no instala en Python 3.14

La celda de *Word2Vec* usa `gensim`, que **todavía no publica rueda para Python 3.14** (su última versión, 4.4.0, llega hasta 3.13). En el contenedor de la unidad no pasa nada —lleva Python 3.12—, pero si trabajas con un intérprete más nuevo, esa celda concreta fallará al instalar. El resto del notebook no depende de `gensim`.

N02 · Clasificación de texto con PyTorch

Construye un clasificador de texto **desde cero**: de la bolsa de palabras a los *word embeddings*, con un modelo de PyTorch entrenado y evaluado. Es el notebook que **EX2** repite sobre datos propios.

Recurso	Enlace
Notebook	UD03_N02_clasificacion_texto_torch.ipynb

N03 · Modelos de lenguaje: afinar DistilBERT

Transfer learning de manual: parte de un **DistilBERT** preentrenado y lo afina para análisis de sentimiento con una base de datos de tuits. **EX4** repite el proceso sobre reseñas de cine.

Recurso	Enlace
Notebook	UD03_N03_modelos_lenguaje_distilbert.ipynb

N04 · spaCy, primeros pasos

El *pipeline* de spaCy de una sola pasada: tokenización, POS, dependencias y entidades, sobre texto en español.

Recurso	Enlace
Notebook	UD03_N04_spacy_primeros_pasos.ipynb

N05 · Ampliación: clasificador de géneros musicales

Un sistema de PLN aplicado a **audio**: clasificar el género de una canción combinando `librosa` y `transformers`. Es el notebook más largo del módulo (240 celdas) y **no cuenta horas de la unidad**: queda como material de ampliación para quien quiera ir más allá de EX5.

Recurso	Enlace
Notebook	UD03_N05_ampliacion_clasificador_generos_musicales.ipynb

[Volver a la UD03](#) · [Entregables](#)

 24 de agosto de 2026

6.4 UD03 · Actividades entregables



Los cinco cuentan para la nota del RA3

Los cinco notebooks forman el **40 % de actividades** del RA3. Cada uno se califica **sobre 10** con su [rúbrica](#), que tienes más abajo. EX2 y EX5 llevan además una **parte escrita** que cubre los criterios sobre el papel del lingüista, el trabajo cooperativo y la formación del investigador (RA3-b, RA3-e, RA3-f) —léela antes de empezar, porque cuenta en la rúbrica.



Dos entornos

EX1 y EX3 van en el **contenedor** de la unidad. EX2, EX4 y EX5 usan `transformers` o audio: van en **Colab**.



Identificadores de *dataset* que ya no funcionan

Los nombres clásicos de algunos conjuntos de datos de Hugging Face **ya no resuelven**: `datasets` exige el formato `namespace/name`. EX2 usa `SetFit/TREC-QC` en vez de `trec`, y EX4 usa `stanfordnlp/imdb` en vez de `imdb`.



EX5 : tres trampas verificadas

`pipeline("translation", ...)` ya no existe como tarea genérica (usa `AutoTokenizer` + `AutoModelForSeq2SeqLM`), cargar el audio con la ruta directa puede pedir `ffmpeg` (carga con `librosa` en su lugar) y el modelo de traducción necesita `sentencepiece`. Todo explicado en el propio notebook.

EX1 · Representación de texto, por dos caminos

Tokenizar, quitar *stopwords* y construir una bolsa de palabras y un vector tf-idf, resuelto **con NLTK y con TextBlob** — el mismo ejercicio, dos librerías.

Recurso	Enlace
Notebook	UD03_EX1_representacion_texto.ipynb

Se entrega: el notebook con los cuatro ejercicios resueltos por **uno** de los dos caminos —tú eliges— y una comparación breve con el otro.

EX2 · Clasificador de preguntas

Repite el proceso de N02 —de la bolsa de palabras a un clasificador entrenado— sobre un conjunto de **preguntas**, en vez de noticias.

Recurso	Enlace
Notebook	UD03_EX2_clasificador_preguntas.ipynb

Se entrega: el notebook con el clasificador entrenado y evaluado, **y una sección de conclusiones** que responda: ¿qué decisiones de anotación tomaría un lingüista con este conjunto de preguntas? ¿Qué perfiles harían falta si el proyecto creciera? ¿Qué formación necesitarías tú para mejorar este sistema?

EX3 · NLTK y Python: etiquetado con `cess_esp`

Procesa el corpus español anotado `cess_esp`: separa entrenamiento y prueba, reduce el conjunto de etiquetas morfosintácticas de 289 a un conjunto manejable, y valida con validación cruzada.

Recurso	Enlace
Notebook	UD03_EX3_nltk_python.ipynb

Se entrega: el notebook con los seis apartados resueltos.

EX4 · Análisis de sentimiento en reseñas de cine

Transfer learning: parte del DistilBERT de N03 —afinado para tuits— y adaptación a reseñas de IMDb, un dominio distinto.

Recurso	Enlace
Notebook	UD03_EX4_sentimiento_imdb.ipynb

Se entrega: el notebook con el modelo afinado y evaluado sobre las reseñas.

EX5 · Asistente virtual por voz

Encadena tres modelos —voz a texto, traducción y texto a voz— en **una sola función** que reciba un audio y devuelva otro.

Recurso	Enlace
Notebook	UD03_EX5_asistente_virtual.ipynb
Audio de prueba	OpenTheDoor.wav

Se entrega: el notebook con la función conjunta funcionando de punta a punta, **y una sección final** que responda: ¿qué papel tendría un lingüista revisando las transcripciones y traducciones de este asistente? ¿Qué formación necesitaría el equipo si el asistente tuviera que funcionar en varios idiomas?

Rúbricas

Son las rúbricas reales del curso. Las cinco tareas se califican **sobre 10**; los puntos de cada rúbrica se **escalan** a esa nota.

EX1 · Representación de texto (8 puntos)

Cuatro criterios de 2 puntos, uno por ejercicio (correcta 2 · parcial 1 · incorrecta 0,5).

EX2 · Clasificador de preguntas (18 puntos)

Criterio	Puntos
Preparación del entorno	2
Cargar el <i>dataset</i>	2
Tokenización	2
Bolsa de palabras	2
<i>Word embeddings</i>	2
Crear el modelo neuronal	2
Entrenar	2
Evaluación	2
Conclusiones o comentarios	2

El último criterio es donde se puntúa la **parte escrita** sobre el lingüista y la cooperación.

EX3 · NLTK y `cess_esp` (10 puntos)

Cinco criterios de 2 puntos, apartados a) a e).

EX4 · Sentimiento en IMDb (10 puntos)

Criterio	Puntos
Cargar el <i>dataset</i>	2
Evaluación previa	2
Definir las etiquetas	2
Afinar el modelo (<i>fine-tuning</i>)	2
Inferencia	2

EX5 · Asistente virtual (8 puntos)

Criterio	Puntos
Modelo voz a texto	2
Modelo de traducción	2
Modelo texto a voz	2
Función conjunta con todos los pasos	2

El cuarto criterio es el que demuestra RA3-g: encadenar los tres modelos en un sistema, no usarlos por separado.

[Volver a la UD03](#) · [Actividades guiadas](#) · Talleres: [T01](#) · [T02](#)

 24 de agosto de 2026

7. UD04

7.1 UD04 — Análisis de sistemas robotizados

Unidad 4 · 12 h · semanas 15-18

Cierra el bloque de aplicaciones de la IA. Se evalúa con **seis entregables prácticos** y la prueba escrita del RA4.

1. Introducción

Los robots ya no son una promesa de futuro: en 2024 se instalaron **más de 540.000 robots industriales** en el mundo y España fue el **tercer mercado europeo**, impulsado por la automoción. Entender cómo funciona un robot, cómo se modela su movimiento y cómo se diseña un sistema robotizado es competencia directa de un especialista en IA, porque un robot es un **agente encarnado**: el único sistema de IA de este curso que puede cambiar el estado del mundo físico.

Eso lo hace distinto de todo lo anterior. Un clasificador que se equivoca produce una etiqueta errónea; un robot que se equivoca rompe una pieza, o algo peor. Y su entorno es **parcialmente observable, estocástico y con más agentes dentro**: las cámaras no ven tras las esquinas, los engranajes patinan y las personas que comparten el espacio son impredecibles.

El recorrido de la unidad:

1. **Métodos y aplicaciones de la robótica**: qué es un robot, su hardware (sensores, actuadores, pinzas) y dónde se usa, con datos del sector.
2. **Qué clase de problema resuelve la robótica** y la jerarquía tarea → movimiento → control.
3. **Modelado y control cinemático** de manipuladores: grados de libertad, parámetros DH, cinemática directa e inversa.
4. **Los problemas típicos** (singularidades, redundancia) y sus **soluciones**: espacio de configuración, planificación de movimiento y seguimiento de trayectoria.
5. **Percepción robótica**: localización, mapeo y SLAM.
6. **Planificación con incertidumbre y aprendizaje** en robótica.
7. **Las técnicas de programación** de robots, del *teach pendant* al ROS 2 — y comparadas en la práctica sobre un mismo problema.
8. **Humanos y robots**: coordinación y aprender lo que la persona quiere.
9. **El diseño e implementación** de un sistema robotizado: selección, célula, seguridad y normativa.

Hilo conductor de la unidad

Un robot percibe, razona y actúa sobre el mundo físico. Primero vemos con qué lo hace (hardware); luego cómo se describe y controla su movimiento (cinemática); después cómo decide por dónde ir (planificación) y cómo sabe dónde está (percepción); y por último cómo se programa y cómo se integra en un sistema seguro.

De dónde sale el material de esta unidad

La parte de **percepción, planificación de movimiento, aprendizaje por refuerzo e interacción humano-robot** está construida a partir del capítulo 26 (*Robotics*) de *Artificial Intelligence: A Modern Approach*, 4.^a edición, de **Stuart Russell y Peter Norvig**, adaptada al nivel y a los criterios de evaluación de este módulo. Las prácticas con el simulador AITK provienen del curso *Models d'IA* de **Carles Gonzalez**, con licencia CC BY-NC-SA 4.0.

2. Resultado de aprendizaje y criterios de evaluación

RA4 — Analiza sistemas robotizados, evaluando opciones de diseño e implementación.

CE	Criterio de evaluación	Bloque
RA4-a		\$4-6

CE	Criterio de evaluación	Bloque
	Se han recopilado los problemas del modelado y control cinemático en robots manipuladores.	
RA4-b	Se han buscado soluciones a los problemas de los robots.	\$7-9
RA4-c	Se han valorado las características diferenciadoras de las técnicas de programación de robots y de sistemas robotizados.	\$10-11
RA4-d	Se han evaluado diferentes opciones en el diseño e implementación de sistemas robotizados.	\$12



Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo llama a este bloque, textualmente, «*Análisis de sistemas robotizados*», y le asigna estos contenidos:

- *Métodos y aplicaciones de la robótica.*
- *Modelado y control de robots.*
- *Programación de robots y aplicaciones.*
- *Sistemas robotizados. Diseño e implementación.*

Son **cuatro contenidos para cuatro criterios de evaluación** y, a diferencia del resto de los RA de este módulo, aquí **encajan casi uno a uno**: es la excepción. El único matiz es que «*modelado y control*» alimenta a la vez RA4-a (los problemas) y RA4-b (las soluciones).

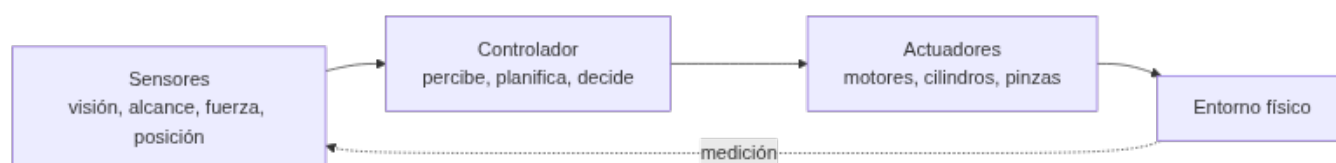
3. Objetivos de la unidad

Objetivo	Al terminar la unidad serás capaz de...
O1	Clasificar los robots por tipo y describir sus aplicaciones reales con datos del sector.
O2	Distintuir los sensores por lo que miden (entorno, ubicación, configuración interna) y elegir el adecuado para una tarea.
O3	Explicar grados de libertad, articulaciones y espacio articular frente a cartesiano.
O4	Resolver la cinemática directa de un manipulador con parámetros DH y entender por qué la inversa es más difícil.
O5	Identificar los problemas típicos (singularidades, redundancia, límites) y sus soluciones.
O6	Plantear un problema de planificación de movimiento en el espacio de configuración y elegir un método.
O7	Explicar cómo un robot se localiza y construye un mapa (SLAM) a partir de sensores con ruido.
O8	Comparar las técnicas de programación de robots resolviendo un mismo problema por reglas, lógica difusa, red neuronal y neuroevolución.
O9	Aplicar criterios de selección y diseño de un sistema robotizado (payload, alcance, sensores, seguridad).
O10	Valorar la normativa de seguridad (ISO 10218:2025) y la integración en la Industria 4.0.

4. Métodos y aplicaciones de la robótica (RA4-a)

4.1 ¿Qué es un robot?

Un **robot** es una máquina programable que **percibe su entorno**, **procesa información** y **actúa físicamente** sobre él. Tiene tres partes que se realimentan:



Los **efectores** son las piezas que ejercen fuerza sobre el entorno: ruedas, patas, articulaciones, pinzas. Cuando actúan, pueden cambiar tres cosas: el **estado del robot** (un coche gira las ruedas y avanza), el **estado del entorno** (un brazo empuja una taza) e incluso el **estado de las personas** alrededor (un exoesqueleto mueve una pierna; un robot se acerca al ascensor y alguien se aparta).

4.2 Tipos de robot, desde el hardware

La imagen popular del robot con cabeza y dos brazos —el **antropomórfico** de la ficción— es la menos frecuente en la realidad:

Tipo	Qué es	Ejemplo
Manipulador	Un brazo, no necesariamente unido a un cuerpo: se atornilla a una mesa o al suelo	Brazos de una línea de montaje; brazos montados en sillas de ruedas para asistencia
Robot móvil con ruedas	Se desplaza sobre ruedas, dentro o fuera	Aspiradora, AGV/AMR de almacén, coche autónomo, rover de Marte
Robot con patas	Atraviesa terreno accidentado donde la rueda no llega	Cuadrúpedos de inspección
Aéreo (UAV) y submarino (AUV)	Rotores o propulsión sumergida	Cuadricópteros, vehículos de exploración oceánica
Cobot	Manipulador que comparte espacio con personas, con potencia y fuerza limitadas	Montaje asistido, inspección
Otros	Prótesis, exoesqueletos, robots con alas, enjambres y entornos inteligentes , donde el robot es la habitación entera	Rehabilitación, agricultura de precisión

La diferencia entre un manipulador de una tonelada que ensambla coches y un brazo de asistencia no es solo el tamaño: el primero mueve mucha carga, el segundo mueve poca pero es **seguro entre personas**. Payload y seguridad son criterios distintos, y a menudo opuestos.

4.3 Sensores: qué se mide y con qué

Los sensores son la interfaz perceptiva del robot. Se clasifican de dos formas a la vez.

Por cómo obtienen la señal: los **pasivos** (cámaras) captan lo que el entorno emite; los **activos** (sonar, lidar) emiten energía y miden lo que vuelve. Los activos dan más información, pero consumen más y **se interfieren entre sí** cuando hay varios trabajando a la vez.

Por lo que miden:

Clase	Qué informa	Sensores típicos
Del entorno	Distancia y forma de lo que hay alrededor	Sonar, visión estéreo, luz estructurada (Kinect), cámara de tiempo de vuelo, lidar , radar, táctiles
De la ubicación	Dónde está el robot	GPS/GLONASS, balizas de interior, análisis de señal wifi, balizas de sonar bajo el agua
De la configuración interna (propioceptivos)	Cómo está el propio robot	Encoders de eje, odometría de rueda, giroscopios, sensores de fuerza y par

Las cifras ayudan a elegir:

Sensor	Alcance y precisión	Dónde encaja
Lidar de barrido	Precisión de ~1 cm a 100 m	El sensor de referencia en coches autónomos
Cámara de tiempo de vuelo	Imágenes de rango a hasta 60 fps	Interiores y distancias cortas; peor que el lidar a plena luz del día
Radar	Hasta kilómetros , y ve a través de la niebla	Vehículos aéreos y automoción

Sensor	Alcance y precisión	Dónde encaja
GPS	Unos metros; con GPS diferencial, precisión milimétrica en condiciones ideales	Exteriores. No funciona en interiores ni bajo el agua
Sensores de fuerza y par	Fuerzas en 3 traslaciones y 3 rotaciones, cientos de medidas por segundo	Manipular objetos frágiles o de forma desconocida



Por qué la odometría no basta

Contar vueltas de rueda parece una forma barata y exacta de saber cuánto has avanzado, y lo es — durante unos metros. Las ruedas **derrapan y patinan**, y el error se **acumula sin límite**: no hay nada que lo corrija. Por eso la odometría se combina siempre con sensores inerciales (giroscopios) y con alguna referencia externa. Es la razón de ser de la localización probabilística del §8.



El caso de la bombilla

Imagina un manipulador de **una tonelada** enroscando una bombilla. Es facilísimo aplicar demasiada fuerza y romperla. Los **sensores de fuerza** le dicen con qué fuerza agarra y los **de par**, con qué fuerza gira; midiendo cientos de veces por segundo, el robot detecta una fuerza inesperada y corrige **antes** de romper el cristal. La capacidad de manipular con cuidado no viene de tener un brazo más suave, sino de **medir rápido**.

4.4 Actuadores y pinzas

El mecanismo que produce el movimiento es el **actuador**:

Tipo	Cómo funciona	Dónde se usa
Eléctrico	Corriente que hace girar un motor	El más común; articulaciones de brazos robóticos
Hidráulico	Fluido a presión (aceite, agua)	Mucha fuerza: maquinaria pesada
Neumático	Aire comprimido	Movimientos rápidos y simples, pinzas

Los actuadores mueven **articulaciones** entre eslabones rígidos: de **revolución** (un eslabón gira respecto al otro) o **prismáticas** (uno se desliza sobre el otro). Las dos son de un solo eje; las esféricas, cilíndricas y planas son multieje.

Para agarrar, el robot usa **pinzas**, y aquí hay un compromiso claro:

- La **pinza de mordaza paralela** —dos dedos, un actuador— es «amada y odiada por su simplicidad»: hace poco, pero nunca falla y se programa en un minuto.
- Las de **tres dedos** ganan algo de flexibilidad sin complicarse.
- En el otro extremo, una **mano antropomórfica** como la Shadow Dexterous Hand tiene **20 actuadores**. Permite manipulación en la mano (coger el móvil y girarlo para orientarlo), pero **aprender a controlarla es mucho más difícil**.



Más grados de libertad no es mejor

Cada actuador que añades multiplica lo que el robot **puede** hacer y también lo que hay que **decidir** en cada instante. Veinte actuadores es un espacio de acción enorme. Casi toda la robótica industrial funciona con dos dedos y un actuador, porque la tarea no necesita más.

4.5 Aplicaciones, con datos (IFR World Robotics 2025)

Dato	Valor (2024)
Robots industriales instalados en el año	542.000 (cuarto año por encima de 500.000)
Stock operativo mundial	4,66 millones (+9 %)
Densidad robótica media	~132-151 robots por 10.000 empleados
País líder en densidad	Corea del Sur (1.012-1.220)

Dato	Valor (2024)
Líder de mercado	China (295.000 unidades, 54 % de las instalaciones)
España, 3.er mercado europeo	5.100 unidades , impulsado por la automoción
Robótica médica	+91 % (~16.700 unidades)
Robots de servicio profesional	~200.000 unidades (+9 %)



¿Por qué estos datos?

La **IFR (International Federation of Robotics)** publica cada año el informe *World Robotics* con las estadísticas oficiales del sector. Es la referencia para argumentar la importancia económica de la robótica con cifras y no con impresiones.

4.6 Cuatro casos reales

Fabricante o robot	Características	Uso real
KUKA KR AGILUS	6-11 kg de carga, 726-1.101 mm de alcance	Envasa 60 sándwiches por minuto; procesa 3.000 muestras de sangre al día
FANUC (familia)	Más de 100 modelos; cargas de hasta 2,3 t , alcance de 4,7 m	Paletizado (M-410, 700 kg), soldadura, manipulación pesada
Universal Robots e-Series	UR3e (3 kg / 500 mm) ... UR16e (16 kg / 900 mm); repetibilidad ±0,03-0,05 mm	Montaje asistido, inspección, carga y descarga de máquinas
Amazon Robotics	Más de 1 millón de robots de almacén	Logística: la flota reduce el tiempo de desplazamiento de los operarios



Pensar en «robot + tarea», no en «robot»

No se elige el robot más caro: se define primero **la tarea** —qué pieza, qué peso, qué ritmo, qué precisión, qué entorno— y después se busca el modelo que cumple payload, alcance, repetibilidad y seguridad. Es el enfoque del §12 y del Taller 2.

5. Qué clase de problema resuelve la robótica

Antes de elegir algoritmos conviene ver **qué tipo de problema** es. La robótica reúne casi todas las dificultades del curso a la vez: es **no determinista**, **parcialmente observable** y **multiagente**.

Y lo multiagente no es solo competitivo o solo cooperativo, sino los dos: en un pasillo estrecho por el que solo cabe uno, el robot y la persona **colaboran** —ninguno quiere chocar— y a la vez **compiten** un poco por pasar primero. Un robot demasiado educado, que siempre cede, se queda atascado para siempre en un sitio concurrido y nunca llega.

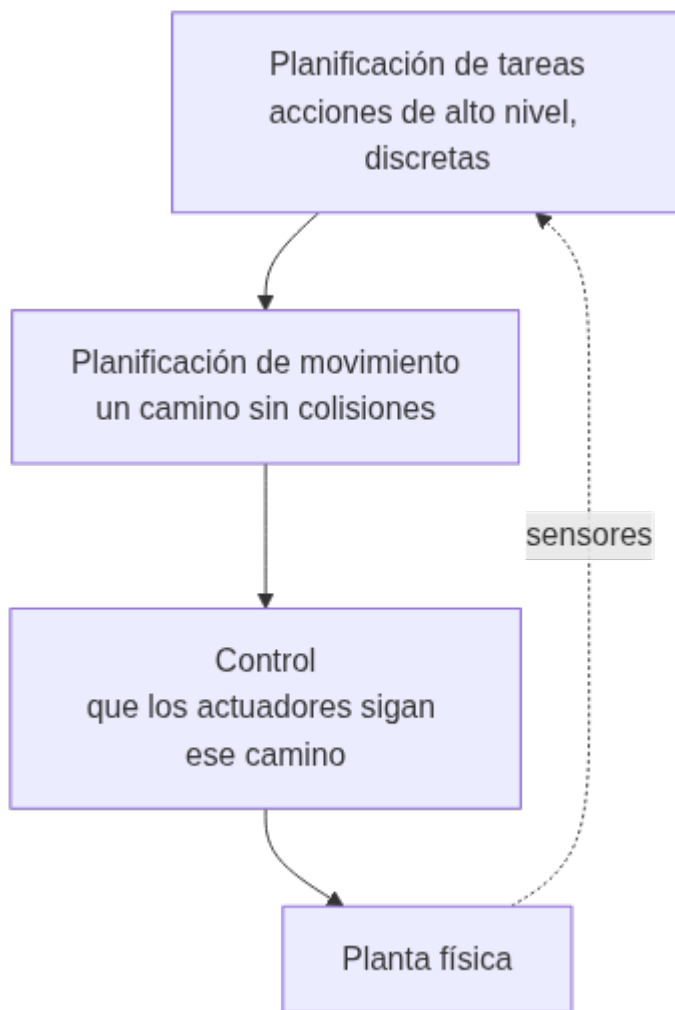


¿De quién es la recompensa?

En casi toda la robótica, el robot actúa **al servicio de una persona**: si lleva la comida a un paciente, la recompensa es del paciente, no suya. Y ahí está el problema: **la función de recompensa verdadera está en la cabeza del usuario**. El robot tiene que deducir lo que la persona quiere, o conformarse con la aproximación que le programó un ingeniero. Es el mismo problema de alineación que se trata en la UD06, aquí con un brazo de una tonelada.

5.1 La jerarquía de tres niveles

En crudo, las observaciones de un robot son señales de sensores (píxeles, impactos de láser) y sus acciones son corrientes eléctricas enviadas a los motores. Entre eso y un plan como «lleva la comida a la habitación 12» hay un abismo. La robótica lo salva **partiendo el problema**:



- **Planificación de tareas:** decide las submetas —ir a la puerta, abrirla, ir al ascensor, pulsar el botón—. Trabaja con estados y acciones **discretos**.
- **Planificación de movimiento:** encuentra un camino que lleva al robot de un punto a otro cumpliendo cada submeta (§7).
- **Control:** consigue que los actuadores ejecuten ese movimiento (§6.4).

⚠ Lo que se pierde al partir el problema

Dividir reduce la complejidad, pero **renuncia a que las partes se ayuden**. La acción puede mejorar la percepción (moverse para ver mejor) y decidir qué percepción hace falta. Un movimiento óptimo sobre el papel puede ser malo de seguir para el controlador. Y un plan de tareas puede ser **imposible de instanciar** a nivel de movimiento. Por eso la robótica actual avanza en cada nivel y, al mismo tiempo, en **volver a integrarlos**: planificar movimiento y control juntos, tareas y movimiento juntos, y cerrar el lazo con la percepción.

6. Modelado y control cinemático (RA4-a)

6.1 Conceptos básicos

Un manipulador se modela como una **cadena cinemática**: eslabones rígidos unidos por articulaciones.

Concepto	Definición
Grado de libertad (DoF)	Número de movimientos independientes. Hacen falta ≥ 6 para colocar el efector en cualquier posición y orientación en 3D
	Gira: su variable es un ángulo θ

Concepto	Definición
Articulación de revolución (R)	
Articulación prismática (P)	Se desliza: su variable es una distancia d
Eslabón	Pieza rígida entre dos articulaciones
Espacio articular	El vector q con la posición de cada articulación
Espacio cartesiano	La pose del efector: posición más orientación

Configuración	Articulaciones	Uso típico
Articulado / antropomórfico	≥ 3R	El más común en industria (6 ejes)
Cartesiano / pórtico	3P	Manipulación simple, gran alcance
SCARA	RRP	Montaje de componentes: rígido en Z, flexible en XY
Delta / paralelo	Paralelas	Empaquetado de alta velocidad

6.2 Cinemática directa (FK)

La **cinemática directa** calcula la pose del efector a partir de los valores articulares: $pose = f(q)$. Se resuelve multiplicando **matrices de transformación homogénea** de 4×4, que combinan rotación y traslación:

$${}^0T_n = {}^0T_1 \cdot {}^1T_2 \cdot \dots \cdot {}^{n-1}T_n$$

Cada transformación se describe con los **parámetros de Denavit-Hartenberg (DH)**: cuatro valores por articulación (θ , d , a , α). Se escribe la tabla DH del robot y la pose sale de encadenar las matrices.



Tabla DH del Puma 560 (fragmento)

El Puma 560, brazo industrial clásico, se describe con una tabla como esta (metros y grados):

Articulación	θ (variable)	d	a	α
1	q_1	0,6718	0	-90°
2	q_2	0	0,4318	0°
3	q_3	0,1500	0,0203	90°
4	q_4	0,4318	0	-90°

Con esos valores, `fkine([0, 0.2, 0.3, 0.4, 0.5, 0.6])` devuelve una matriz 4×4 con la pose del efector: una posición concreta del TCP, del orden de `[0,25; -0,13; 1,15]`. Se comprueba en el notebook de la unidad.

Cinemática directa de un brazo plano 3R, a mano

Para un brazo en el plano con eslabones l_1, l_2, l_3 :

$$\begin{aligned} x &= l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2) + l_3 \cos(\theta_1 + \theta_2 + \theta_3) \\ y &= l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2) + l_3 \sin(\theta_1 + \theta_2 + \theta_3) \\ \varphi &= \theta_1 + \theta_2 + \theta_3 \end{aligned}$$

Con $l_1 = l_2 = l_3 = 1$ y los tres ángulos a 0, el efector está en **(3, 0)** con orientación 0. Poniendo $\theta_1 = 90^\circ$, pasa a **(0, 3)**. El modelo geométrico predice exactamente dónde está la punta del brazo: eso es la cinemática directa, y **siempre tiene una única solución**.

6.3 Cinemática inversa (IK)

La **cinemática inversa** va al revés: dada una pose deseada, calcula los ángulos articulares que la consiguen, $q = f^{-1}(\text{pose})$. Es **mucho más difícil**, y ahí están casi todos los problemas que pide recopilar el criterio RA4-a:

Problema	En qué consiste	Cómo se aborda
Múltiples soluciones	Un 6R general puede tener hasta 16 soluciones ; con muñeca esférica, 8	Elegir por criterio: evitar obstáculos, minimizar el recorrido, codo arriba o abajo
Redundancia	Más DoF de los necesarios → infinitas soluciones	Optimizar en el espacio nulo del jacobiano
Sin solución	El objetivo está fuera del alcance	Detectarlo y avisar, no iterar sin fin
Singularidad	El jacobiano pierde rango → la velocidad articular tiende a infinito	Evitarla al planificar, o cruzarla bajando la velocidad cartesiana

Los métodos se dividen en **analíticos** —desacoplamiento cinemático en muñecas esféricas, teorema de Pieper— y **numéricos** —Newton-Raphson, pseudoinversa del jacobiano, CCD, FABRIK—.

6.4 Control

Estrategia	Qué controla	Cómo
Control de posición	El ángulo de cada articulación	PID por eje: la medida es el encoder, la consigna el ángulo objetivo
Control de velocidad	La velocidad del efector (<i>resolved-rate</i>)	$\dot{q} = J^+ v$, con la pseudoinversa de Moore-Penrose del jacobiano
Control de fuerza / impedancia	La interacción con el entorno	El robot se comporta como un sistema masa-muelle-amortiguador: ensamblaje, pulido

Los controladores de seguimiento van de menos a más:

- **P** — aplica una acción proporcional al error de posición. Simple, pero oscila.
- **PD** — añade un término proporcional a la derivada del error, que **amortigua** la oscilación.
- **PID** — añade la integral del error, que corrige los **errores sistemáticos persistentes**.
- **Par calculado** — usa la **dinámica inversa** del robot para calcular el par que hace falta, y deja al PID solo la corrección de lo que el modelo no acierta. Es lo que usan los robots industriales de verdad.

Los robots industriales trabajan con **servomotores con encoder** en lazo cerrado, no con motores paso a paso en lazo abierto: sin realimentación no hay forma de saber si el eje llegó.



De dónde salen las trayectorias

No se salta de un punto a otro: se **interpola**. `jt看aj` genera la trayectoria en el espacio articular con un polinomio de 5.º grado (aceleración y *jerk* continuos, movimiento suave); `ctr看aj` genera una **línea recta cartesiana**, que es lo que hace falta cuando el efector lleva una herramienta de corte. El controlador va siguiendo esa trayectoria punto a punto.

7. Los problemas y sus soluciones (RA4-b)

7.1 Singularidades cinemáticas

Una **singularidad** es una configuración en la que el jacobiano pierde rango: el robot **pierde capacidad de movimiento** en alguna dirección y los cálculos de velocidad se van a infinito.

Singularidad	Cuándo ocurre
De muñeca	Los ejes 4 y 6 se alinean ($\theta_5 = 0$): las dos articulaciones deberían girar a velocidad infinita
De hombro	El centro de la muñeca corta el eje de la base

Singularidad	Cuándo ocurre
De codo	El brazo queda completamente estirado o plegado
De límite	El efector llega al borde de su volumen de trabajo



Qué pasa de verdad si el robot entra en una singularidad

No es un problema abstracto de álgebra. Las velocidades articulares necesarias **tienden a infinito**, así que lo que se ve en la célula es **sobrecorriente en los servos, vibraciones o una parada de seguridad** en medio del ciclo. Por eso la planificación de trayectorias evita esas configuraciones, o las cruza reduciendo la velocidad cartesiana.

7.2 Soluciones

Problema	Solución
Singularidades	Planificar evitándolas; medir la manipulabilidad (índice de Yoshikawa) para saber cuánto margen queda
IK compleja	Desacoplamiento cinemático : una muñeca esférica separa el problema de posición del de orientación
Múltiples soluciones	Criterios de optimización: evitar obstáculos, minimizar recorrido
Límites articulares	Modelarlos y restringir el <i>solver</i>
Precisión deficiente	Calibrar el robot, que puede mejorar la precisión hasta $\times 10$, y medir la repetibilidad con la ISO 9283



Precisión no es repetibilidad

Un robot puede ser **repetible** y a la vez **impreciso**: vuelve siempre al mismo punto, pero ese punto no es el que se programó. Para trabajar con *teach pendant* basta la repetibilidad — el punto se enseñó ahí—, pero en **programación offline**, donde las coordenadas vienen de un CAD, la precisión es crítica y hay que calibrar (por ejemplo, un ABB IRB 1600 con láser *tracker*). La repetibilidad de los cobots UR e-Series es de $\pm 0,03-0,05$ mm.

7.3 El espacio de configuración

Para planificar el movimiento hay que cambiar de punto de vista. En vez de pensar en el robot moviéndose por una habitación, se piensa en un **punto moviéndose por el espacio de configuración**: el espacio abstracto de **todas las configuraciones posibles** del robot.

Para un brazo de 6 ejes, ese espacio tiene 6 dimensiones —una por articulación—; para un robot móvil en un plano, 3 (x, y, orientación). Los obstáculos del mundo real se convierten en **regiones prohibidas** de ese espacio, y lo que queda es el **espacio libre**. El problema deja de ser geométrico y se vuelve una búsqueda de camino.



El problema de la mudanza del piano

La planificación de movimiento se llama a veces así, por el esfuerzo de una empresa de mudanzas para llevar un piano grande y de forma irregular de una habitación a otra sin golpear nada. Formalmente hacen falta: un espacio de trabajo w , una región de obstáculos $o \subset w$, un robot con su espacio de configuración c , una configuración inicial q_s y una objetivo q_g . Y lo que se busca es un **camino continuo** por el espacio libre entre las dos.

El problema se complica de tres formas típicas: que el objetivo sea un **conjunto** de configuraciones válidas en vez de una sola; que haya que **minimizar un coste** (longitud del camino, energía); o que haya **restricciones** — si el robot lleva una taza de café, tiene que mantenerla vertical todo el trayecto.

7.4 Cómo se planifica el movimiento

Método	Idea	Fuerte en	Flojo en
Grafo de visibilidad	Nodos en los vértices de los obstáculos, aristas donde hay línea de visión clara; después A* o Dijkstra	Da el camino más corto en 2D con obstáculos poligonales	Escala mal con muchos obstáculos; el camino roza las esquinas
Diagrama de Voronoi	Divide el espacio en regiones por cercanía a cada obstáculo; el robot circula por los bordes entre regiones	Maximiza la distancia a los obstáculos: caminos seguros	Caminos más largos de lo necesario
Descomposición celular	Trocea el espacio libre en celdas y busca sobre el grafo de adyacencia	Sencillo y completo en su resolución	El número de celdas explota al subir de dimensión
Muestreo (RRT, PRM)	Va lanzando configuraciones al azar y conectando las válidas hasta unir inicio y meta	Es lo único práctico con muchas dimensiones (brazos de 6-7 ejes)	No garantiza el camino óptimo; el resultado es irregular y hay que suavizarlo

Los dos primeros ilustran bien un compromiso que reaparece siempre: el grafo de visibilidad busca el camino **más corto**, que pasa pegado a las esquinas; el diagrama de Voronoi busca el **más seguro**, que se aleja de todo y por eso es más largo. Cuál conviene depende de si el riesgo es chocar o es tardar.

Cualquiera de los cuatro devuelve un camino que suele necesitar un último paso de **suavizado** o de **replanificación** cuando el entorno cambia.

7.5 Seguimiento de trayectoria: del plan a la política

Aquí hay una distinción que conviene tener clara, porque explica cómo está montada la robótica de verdad.

Un **plan** dice qué camino recorrer. Una **política** dice qué acción tomar **desde cualquier estado** en el que el robot se encuentre. Las leyes de control del §6.4 son políticas, no planes.

Lo que se hace en la práctica es una **componenda en dos pasos**:

1. Se **planifica** en un espacio simplificado — solo el estado cinemático, suponiendo que se puede pasar de una configuración a la vecina sin preocuparse de la dinámica. Sale la **trayectoria de referencia**.
2. Se **convierte en política**: un controlador que intenta seguir ese plan y **vuelve a él** cuando se desvía.

⚠ Esa componenda introduce dos suboptimalidades

La primera, **planificar sin tener en cuenta la dinámica**: el camino más corto en el papel puede ser el que peor sigue el robot real. La segunda, **suponer que si te desvías lo mejor es volver al plan original**, cuando a veces lo óptimo es replanificar desde donde estás. El **control óptimo** ataca las dos a la vez: optimiza directamente sobre las acciones teniendo en cuenta la dinámica, con técnicas de optimización de trayectoria (*multiple shooting*, colocación directa) y, cuando el coste es cuadrático y la dinámica lineal, con el **LQR** —y su variante **iLQR**, que es lo que se usa cuando no se cumplen esas condiciones ideales, que es casi siempre.

8. Percepción robótica (RA4-b)

Percepción es convertir medidas de sensores en una representación interna del entorno. Y el problema de fondo es el del §4.3: **todos los sensores tienen ruido y ninguno lo ve todo**.

8.1 Localización y mapeo

Hay tres problemas, de menos a más difícil:

Problema	Lo que se sabe	Lo que se busca
Localización	El mapa	Dónde está el robot

Problema	Lo que se sabe	Lo que se busca
Mapeo	Dónde está el robot	El mapa
SLAM	Nada de las dos	Las dos a la vez

SLAM (*Simultaneous Localization And Mapping*) es el caso realista y parece un imposible —para saber dónde estás necesitas el mapa, y para hacer el mapa necesitas saber dónde estás—, pero se resuelve **probabilísticamente**: en vez de una respuesta, el robot mantiene una **distribución de probabilidad** sobre dónde puede estar (su *estado de creencia*) y la va afinando con cada medida.

Las herramientas son las mismas que aparecen en toda la IA con incertidumbre: **filtros de Kalman**, **modelos ocultos de Markov** y **redes bayesianas**, que modelan la transición de estado y el sensor.



Localización de Monte Carlo, en tres fotos

La localización con **filtro de partículas** (MCL) representa la creencia con una nube de «partículas», cada una una hipótesis de dónde está el robot:

1. Al arrancar, las partículas están **repartidas por todo el plano**: incertidumbre total.
2. Llega el primer conjunto de medidas y las partículas se **agrupan** en las zonas compatibles. Suelen quedar **varios grupos**: los pasillos de un edificio de oficinas se parecen entre sí, y el robot todavía no puede distinguirlos.
3. Con suficientes medidas, todas las partículas **colapsan en un solo sitio**.

Lo interesante es el paso 2: el robot representa explícitamente que hay **varias respuestas posibles** en vez de escoger una y equivocarse. Solo hacen falta dos modelos: el del movimiento y el del sensor.

8.2 Qué representación conviene

La tendencia en robótica es hacia representaciones con **semántica bien definida** — no solo «aquí hay algo», sino «esto es una puerta». Y las técnicas probabilísticas **ganan** a las alternativas en los problemas difíciles de percepción, como el propio SLAM.



Pero no siempre hace falta la artillería

Las técnicas estadísticas son a veces **demasiado engorrosas** para lo que se necesita, y una solución más simple es igual de efectiva en la práctica. Es el mismo criterio de la UD05 con el PID: empezar simple y añadir complejidad solo cuando se justifica. Los **EX2** y **EX3** de esta unidad navegan con reglas y con lógica difusa sobre los píxeles de una cámara, sin ningún filtro probabilístico, y funcionan.

8.3 Percepción que se adapta

Hay un tipo de aprendizaje que resuelve un problema muy concreto: que las medidas de los sensores cambian **de golpe** cuando cambia el entorno.

Piensa en entrar desde un exterior soleado a una habitación con fluorescentes. No solo hay menos luz: el fluorescente tiene **más componente verde** que el sol, así que todos los colores cambian. Y sin embargo no nos parece que a la gente se le haya puesto la cara verde. Nuestra percepción se **readapta** en segundos y el cerebro ignora la diferencia. Un robot que no haga eso creerá que ha cambiado de mundo.

9. Incertidumbre y aprendizaje (RA4-b)

9.1 Planificar cuando no se sabe el estado exacto

La incertidumbre viene de tres sitios: el entorno es **parcialmente observable**, los efectos de las acciones son **estocásticos** (o simplemente no están modelados) y los propios algoritmos de estimación son **aproximados** — un filtro de partículas no da el estado de creencia exacto ni con un modelo perfecto.

La mayoría de los robots en producción usan, aun así, **algoritmos deterministas**, y se apañan con dos atajos:

1. **Discretizar** el espacio de estados continuo (grafo de visibilidad, descomposición celular).
2. Ante la duda sobre el estado actual, **quedarse con el más probable** de la distribución.

Es más rápido y escala mejor. El precio es que el robot actúa como si estuviera seguro cuando no lo está.



Cuándo ese atajo se rompe

Quedarse con el estado más probable funciona mientras la distribución tenga **un solo pico claro**. Falla justo en los casos del §8.1, paso 2: cuando hay **varias hipótesis igual de buenas** —tres pasillos que se parecen—, elegir la más probable es tirar una moneda, y el robot actúa con plena confianza sobre una respuesta que tiene un 33 % de ser cierta.

9.2 Aprendizaje por refuerzo en robótica

El aprendizaje por refuerzo funciona muy bien en simulación y muy mal en un robot real, y la razón es sencilla: **el mundo real se niega a ir más rápido que el tiempo real**.

	En simulación	En un robot real
Velocidad	Millones de pruebas en unas horas	Las mismas pruebas tardarían años
Riesgo	Ninguno	El robot no puede arriesgarse a una prueba que lo dañe — y por tanto no puede aprender de ella

De ahí que el problema central sea la **transferencia de la simulación a la realidad** (*sim-to-real*), que es un área de investigación activa. Y de ahí también que los sistemas robóticos prácticos incorporen **conocimiento previo** sobre el robot, el entorno y la tarea: es la única forma de aprender rápido y de comportarse con seguridad mientras aprende.



Esto conecta con toda la unidad

EX4, EX5 y EX6 hacen exactamente este recorrido en pequeño y **en simulación**, que es donde se puede: generar datos conduciendo el robot, entrenar una red con ellos, y después dejar que la solución **evolucione** sin datos etiquetados.

10. Técnicas de programación de robots (RA4-c)

10.1 Las cinco formas de programar un robot industrial

Técnica	Cómo funciona	Ventajas	Desventajas	Cuándo
Teach pendant	Guías el brazo con la consola por los puntos de paso y los grabas	Curva de aprendizaje mínima, puesta a punto rápida	Para la producción mientras se programa; poca flexibilidad	Trayectorias básicas, paletizado, soldadura por puntos
Guiado manual	Mueves el efector con la mano y registras posiciones	Muy intuitivo, sin escribir código	Solo en cobots seguros	Cobots, montaje asistido
Programación textual	Código nativo del fabricante: RAPID, KRL, URScript	Determinista, se integra con PLC y seguridad	Sintaxis propietaria, no portable	Células industriales estables de alta cadencia
Programación offline (OLP)	Se programa en simulación sobre modelos CAD	Cero impacto en producción ; permite probar hipótesis	Coste de licencias; exige modelos precisos y robot calibrado	Geometrías complejas, células multimarca
ROS 2 y frameworks	Middleware de nodos, <i>topics</i> y servicios	Estándar abierto, acceso directo a algoritmos de IA	Curva de aprendizaje alta	Investigación, robots móviles, multi-robot



URScript se parece a Python

Los robots de **Universal Robots** se programan en **URScript**, un lenguaje muy parecido a Python que además puede enviarse al robot desde un cliente Python por FTP, SSH o RTDE. Es la puerta de entrada natural a la programación de cobots desde este módulo.